RとSTANで学ぶフリーで楽しい心理統計の世界

心理学データ解析 応用2

データの背後のメカニズムを解析する方法

小杉考司

この本は Creative Commons BY-SA(CC BY-SA) ライセンス Version 4.0 に基づいて提供されています。著者に適切なクレジットを与える限り，この本を再利用，再編集，保持，改訂，再頒布 (商用利用を含む) をすることができます。もし再編集したり，このオープンなテキストを変更したい場合，すべてのバージョンにわたってこれと同じライセンス，CC BY-SA を適用しなければなりません。

https://creativecommons.org/licenses/by-sa/4.0/deed.ja

This book is published under a Creative Commons BY-SA license (CC BY-SA) version 4.0. This means that this book can be reused, remixed, retained, revised and redistributed (including commercially) as long as appropriate credit is given to the authors. If you remix, or modify the original version of this open textbook, you must redistribute all versions of this open textbook under the same license - CC BY-SA.

https://creativecommons.org/licenses/by-sa/4.0/

心理学データ解析応用 2

小杉 考司

Last Compiled on 2026.10.12

昨今はデータサイエンス，情報科学の領域が非常に隆盛で，コンピュータを使ってデータを分析し，経済の動向や購買行動などの予測に用いられることが広く行われている。

人の行動や考え方をどのようにデータにするかについては，当然ながら心理学には一日の長がある。また，人が頭の中でどのような考え方のプロセスをたどるのか，それをどのように検証するのかについても，心理学はその短い歴史の中で徹底的にその技法を洗練させてきた。このような根源的なレベルでの理論や方法論は時代が変わっても色褪せることなく，また今後ますます必要とされてくる時代になっている。

本講ではデータ解析の応用段階として，より実践的なテーマを扱う。すなわち，**心理尺度が作られる理論的背景**と，**データの背後のメカニズムを解析する方法**を知ることである。後期配当の心理学データ解析応用 2(旧カリキュラム名心理学データ解析 2B) では，数値計算を通して実践されている推測統計学的手法の原理・メカニズムを知り，ひいてはデータ生成メカニズムを知る技法を学ぶ。

心理学研究では，調査・実験・観察など様々な手法で得られたデータに対し，統計的な処理で「意味があったかどうか」といった判定を下す。データは知りたい全体 (知的生命) の一部を使って得たものだから，標本の平均値から母集団の平均値を推測する推測統計学の手法が用いられる。推測統計学では確率や微積分の概念が必要であり，高度で抽象的な数学ツールを実感的に理解することは難しい。畢竟，推測統計のメカニズムの理解がおぼつかないまま，ただ機械から出力される数値を確認するだけになりがちである。生兵法は大怪我の基である。そこで，原理の理解度を上げるために確率変数を数値シミュレーションによって具体的な数値にすることで，抽象度を下げて理解することを考えよう。この方法は，ひいては心理的なメカニズムを数式的に表現することで分析する，**数理モデリング**というアプローチにつながる。このモデリングの基礎的な知識，方法論の習得を目指す。

なお，2024 年度から前期と後期の資料を分割して提供するようにした。通年で履修する学生が少ないことと，後期で扱うテーマとして数値シミュレーションを含むように再構成したからである。とくに数値シミュレーションについては拙著「数値シミュレーションで学ぶ統計の仕組み」(小杉他，2023) としてまとめたので，詳細シラバスの第 2 講から第 8 講に該当する箇所では，この外部テキストを参照してほしい。この資料に含まれるのは小杉他 (2023) のエッセンスと，そこから外れる Stan を用いたモデリング部分である。前期後期でテーマが明確になり，またスリムになった分，利用しやすくなったのではないかと思っているが，どうだろうか。

授業のテーマ

データから意味のある情報を取り出すための，さまざまな分析法を習熟するにあたって，その背後にあって語られることのない「発想」の観点から理解する。数値だけに振り回される状態から脱却し，数値を算出する数式に込められた意味について考える視点を持つ。さらにこれらに習熟することで，どのような研究対象に対してどのような心理統計的アプローチができるかを，俯瞰的に見られるようになろう。

一年を通じて伝えたいポイント

乱数生成による確率の理解　ランダムであることを概念的に理解することは難しい。一方で，偶然生じた事象は身の回りに溢れている。確率変数から生じた乱数を用いることで，確率を身近に理解し，推測統計学の理解を深める。

データ生成メカニズム　乱数を用いて，データが生まれてくるメカニズムを考えるトレーニングを行う。データからメカニズムを再現するアプローチは，心理学的事象の深い理解につながる。

統計環境 R と確率的プログラミング言語 Stan による実践　統計環境 R と確率的プログラミング言語 Stan に習熟することで実際に計算し，確認しながら分析を進めることができるようになる。

その他

授業シラバスとこの講義資料を掲載したサイト (https://kosugitti.github.io/psychometrics_syllabus/) で，最新版のシラバスと授業資料，授業で用いるサンプルデータやコードの配布を行なっています。

目次

第 1 章　プログラミングの基礎 9
1.1　パソコンの基礎 . . . 9
1.2　プログラミングの基礎 . . . 11
1.3　いくつかのプログラミング言語 . . . 14
1.4　R 言語の基本的な挙動 . . . 15
1.5　手を動かして確かめる . . . 15

第 2 章　プログラミングの基礎 17
2.1　高級言語の基本的な働き . . . 17
2.2　関数を作る . . . 22
2.3　まとめ . . . 27
2.4　課題 . . . 27

第 3 章　データ生成モデリング 29
3.1　データ生成モデリング . . . 29
3.2　ベイズ推定の基礎 . . . 30
3.3　マルコフ連鎖モンテカルロ法 . . . 32
3.4　乱数によるアプローチの例 . . . 34
3.5　課題 . . . 38

第 4 章　いんたーみっしょん；Stan の概略と環境の準備について 41
4.1　はじめに . . . 41
4.2　Stan の位置付け . . . 42
4.3　導入の概略 . . . 47
4.4　導入方法 3;外部サーバの利用 . . . 48
4.5　Stan を使ってみよう . . . 49

第 5 章　ベイジアンアプローチと確率的プログラミング 1 57
5.1　7 人の科学者 . . . 57
5.2　Stan コードの書き方 . . . 59
5.3　Stan を使った MCMC の実践 . . . 62
5.4　MCMC 結果の診断 . . . 63
5.5　MCMC の結果の解釈 . . . 66
5.6　課題 . . . 67

第 6 章 モデリングの目から見た検定 1；二群の平均値の差 69
6.1 t 検定の過程と実際 . . . 69
6.2 差の分布 . . . 74
6.3 帰無仮説検定を省みる . . . 77
6.4 今回のまとめ . . . 79
6.5 課題 . . . 79

第 7 章 モデリングの目から見た検定 2；パタメータの世界とデータの世界 81
7.1 事後予測分布 . . . 81
7.2 データレベルの仮説 . . . 84
7.3 パラメータ･リカバリ . . . 87
7.4 今回のまとめ . . . 91
7.5 課題 . . . 91

第 8 章 モデリングの目から見た検定 3；多群の平均値差モデル 93
8.1 要因計画モデル . . . 93
8.2 パラメータの変形と制約 . . . 95
8.3 モデルの洗練 . . . 99
8.4 パラメータリカバリ . . . 103
8.5 課題 . . . 104

第 9 章 モデリングの目から見た検定 4；対応のある群の比較 107
9.1 対応のある群 . . . 107
9.2 ID をもったデータ構造 . . . 113
9.3 個人差と変化量のモデルへ . . . 116
9.4 課題 . . . 118

第 10 章 モデリングの目から見た検定 5；カテゴリカル分布をつかって 121
10.1 離散的な分布 . . . 121
10.2 χ^2 検定 . . . 122
10.3 カテゴリカル分布のモデリング . . . 124
10.4 κ 係数の算出 . . . 126
10.5 課題 . . . 129

第 11 章 一般化線形モデル 131
11.1 一般線形モデル . . . 131
11.2 データに合わせた確率分布 . . . 137
11.3 リンク関数とパラメータの解釈 . . . 142
11.4 まとめ . . . 144
11.5 課題 . . . 145

第 12 章 階層線形モデル 147
12.1 一般化線形混合モデル . . . 147
12.2 ネストされたデータ . . . 153

12.3 階層線形モデル . . . 155
12.4 課題 . . . 159

第 13 章 混合分布モデル 163
13.1 混合分布モデル . . . 163
13.2 ターゲット記法と周辺化消去 . . . 167
13.3 ゼロ過剰ポアソン分布モデル . . . 173
13.4 課題 . . . 178

第 14 章 確率的プログラミング；項目反応理論 181
14.1 ロジスティックモデルの復習 . . . 181
14.2 ロジスティックモデルでの実装 . . . 183
14.3 整然データでの分析 . . . 185
14.4 課題 . . . 190

第 15 章 確率的プログラミング；変化点と折線回帰 193
15.1 混合分布モデルの応用 . . . 194
15.2 変化点検出 . . . 196
15.3 折線回帰 . . . 198
15.4 課題 . . . 203

第 16 章 確率的プログラミング；状態空間モデル 205
16.1 時系列データの特徴 . . . 205
16.2 状態空間モデル . . . 206
16.3 欠測値の補間 . . . 210
16.4 状態空間モデルの展開 . . . 220
16.5 課題 . . . 220

第 17 章 モデル比較 223
17.1 ベイジアンモデリング . . . 223
17.2 帰無仮説検定の代案 . . . 225
17.3 モデル比較 . . . 228
17.4 おわりに . . . 229

第 A 章 よくある質問とミスの例 231
A.1 Frequently Miss and Comments . . . 231
A.2 Frequently Asked Questions;よくある質問と答え . . . 235

第 B 章 電子計算機のイロハ 243
B.1 前置き . . . 243
B.2 コンピュータの基礎 . . . 243
B.3 コンピュータの歴史 . . . 244
B.4 情報の単位 . . . 247
B.5 ファイルの種類と拡張子 . . . 248

B.6 クラウドとは 249
B.7 ファイルの位置の指定 251
B.8 ファイルのバージョン管理 253
B.9 おわりに 254

第 C 章 ギリシア文字一覧 257

第 D 章 記号の入力とキーボードの場所 259

引用文献 259

索引 264

第 1 章

プログラミングの基礎

この授業は，R や Stan といったプログラミング言語を扱います。プログラミングはやったことがない，という人でも理解できるように，ゆっくり話を進めていきます。プログラミングのいいところは，人間相手とのやり取りと違って，必ず理屈があって振る舞いが生じている点です。対話を通じて根気よく付き合えば，必ず問題は解決できます。また昨今では，生成 AI がプログラミングをアシストしてくれるようになりました。この授業は TA がついていますから，わからないところがあれば，随時 TA に尋ねてくだされば結構です。それでも人間相手の質問ですから，「こんなことを聞いたらバカだと思われるんじゃないだろうか」「声をかけるのが恥ずかしい」ということがあるかもしれません。でも生成 AI とのやり取りは，いつでも気軽に行えます。また決してこちらを蔑んだり恨んだりすることはありません (というかあっても怖くありません！)。同じ質問を何度しても，生成 AI は一向に意に介さないので，積極的に利用してください。ただし生成 AI は正しい答えを教えてくれるとは限りませんので，**必ず自分が確認できるような質問をしてください**。何かわからないけど，バグが出ないからいいや，と思っていると，自分がなんの成長もしないのはもちろん，大変な問題を引き起こすかもしれませんので。

1.1 パソコンの基礎

21 世紀に生きる私たちは身の回りをコンピュータに囲まれて生きています[*1]。それは携帯電話の形をしていたり，ノートパソコン，デスクトップパソコンの形をしていたりします。また時々テレビなどで報道されますが，気象予報や飛沫がどのように飛び散るかをシミュレーションする大型計算機「富嶽」などもコンピュータですね。

みなさんはスマートフォン，タブレット，ゲーム機には慣れ親しんでいると思います。パソコンも「GIGA たん」などで触れていると思います。パソコンでは文書，プレゼン，図表を作ったり，絵や音楽を作ったりしているでしょう。このようにパソコンを**使う**ことには慣れていても，パソコンに**命令する**という感覚はないかもしれません。

プログラミングとは，計算機に計算をさせる指示書を書くことです。パソコンをはじめとするコンピュータ機器は，その形状はさまざまですが，いずれも電子計算機であり，なんらかの形で計算をさせていることになり

[*1] 私は 1976 年生まれですが，生まれた頃は周りにコンピュータなんかありませんでした。小学生の頃にマイコン (マイクロコンピュータ，小さなコンピュータという意味でもありますが，My Computer，私のコンピュータという意味でもあります。つまり個人単位でコンピュータが使えるようになった，というだけでも大きな出来事だったのです。) という言葉が出てきて，なんかかっこいいなと思った記憶があります。私が 10 歳になったころ，ビデオゲーム (テレビゲームでやるゲームが家庭でできるようになり，それをこのように呼びました。) が身の回りに出てきました。ファミリーコンピュータ，とくに「スーパーマリオブラザーズ」によって日本中の子供たちが熱狂したのが 11 歳の頃，「ドラゴンクエスト」によって社会問題になったのが 12 歳の頃になります。ともかくこの頃は，コンピュータといってもゲーム機のような扱いでした。

ます。ゲームで遊ぶことも，動画を見ることも，文書を作ることも，すべて電子的な計算の結果なのです。プログラミングをする上では，基本的な計算機の仕組みを知っておいても損はないでしょう。

電子計算機には次の 5 つの装置があります。

入力装置　キーボードやマウス，タッチパネルなどを使って情報を取り込むデバイス (装置)

出力装置　モニタやプリンタ，タッチパネルなどを経由して情報を出力するデバイス

演算装置　プログラムの命令に従って計算処理 (四則演算や論理演算) をする装置。一般に電子計算機の中央で一括して処理するので，Central Processing Unit(CPU) と呼ばれるものです。

制御装置　演算結果に従って他の装置に指示を出す装置のこと。演算装置とまとめて CPU に実装されています。

記憶装置　計算結果などの情報をいったん保持しておく装置のこと。コンピュータの内部にあって一時的な計算に使われる一次記憶装置，ハードディスクドライブ (Hard Disk Drive,HDD) やソリッドステートドライブ (Solid State Drive,SSD) などの二次記憶装置など。

最後の記憶装置については，一次記憶装置，二次記憶装置と種類が分かれていますがこの区別は簡単で，電源を落とした時に記憶が消えてしまうのが一次記憶装置，電源を落としても記憶が消えないのが二次記憶装置です。たとえば $12 + 38 =$ という計算をするとき，頭の中で「えーっと一の位が 2 と 8 だから 10 になって繰り上がるから･･･」と考えてから，ノートに $= 50$ という答えを書くと思います。次の問題に進むと，先ほどの「1 繰り上がるから･･･」という情報は忘れてますよね。でもノートに書いた $12 + 38 = 50$ というのは残っています。このノートがいわば二次記憶装置であり，頭の中で一時的に保持していた情報が一次記憶装置ということになります。一次記憶装置は RAM(Random Access Memory) とも呼ばれます[*2]。

ところで，コンピュータがやっているのは計算だけです。私はこの資料を PC に向かって書いており，キーボード (入力装置) を叩きながら，画面 (出力装置) を見て文字を連ねています。これも「キーが押されたら文字を表示させ，その文字列を記録する」という処理を機械が淡々とこなしているに過ぎません。あるいはマウスやトラックパッドで，アイコンを指し示し，カチリと押す[*3]ことで選択し，押し込んだまま移動させ (ドラッグ)，離すことで別の場所に置いたりします (ドロップ)。トラックパッドの場合は 2 本指で同時に押したりしますし，タッチパネルの場合は二本指を広げたり (ピンチアウト)，逆に二本指を狭めたり (ピンチイン)，3 本以上の指でファサーっと触って場所を広げたりします。たとえば「ファイルを掴んでゴミ箱に捨てる (削除する)」というのが我々にとっての操作ですが，コンピュータの内部では実はこんなことをしていません。ファイルは (二次) 記憶装置に書き込まれた情報です。記憶装置は原稿用紙のように小さなマス目がたくさんあって，ファイルとはそのマス目の XXX 番目から YYY 番目までの情報，ということです。このファイルを削除するというのは，記憶装置のある場所 (アドレス) に「削除されたものなので画面に表示しない」という情報を書き込む，という操作をしているだけです。じゃあなぜ私たちは「ゴミ箱にドラッグ＆ドロップ」なんてするのでしょう？ それはそのほうがわかりやすいからですよね。「ファイルを削除する」というのは，「メモリアドレスの XXX 番地に別の情報を書き込む」という操作だと言われてもピンとこないので，コンピュータが人間にとってわかりやすい表現をして見せてくれているのです。このユーザにとってわかりやすい幻を見せてその気にさせてくれるというデザインのことを，ユーザーイリュージョンと言います。ともかくこういう「画面で見ながら操作する」ことをグラフィカル･ユーザ･インターフェイス (Graphical User Interface,GUI) といいますが，これのおかげでコンピュータ

[*2] 実はこのように人間を 1 つのコンピュータに喩えて，そこではどのように計算がされているのか，人間の記憶装置や演算装置，入出力装置の特徴はどうなっているのか，というのを研究するのも心理学の仕事です。記憶や演算，制御については認知心理学や学習心理学，入出力については知覚心理学や生理心理学が専門的に扱っています。そういう意味でもコンピュータの登場は心理学に大きな影響を与えているのですが，それはまた別の講釈で。

[*3] 押すときの音から，この操作をクリック click と言います。2 回続けて押すことをダブルクリックと言います。

の操作は随分楽になっています[*4]。

プログラミングとはこうした装置を使う指示書を書くことでもあります。この授業で扱うのは主に数値計算ですから，逐一「この装置にこの動きを命令」といったところまで書き込む必要はありません。そのあたりのことは，言語環境がやってくれますのできにする必要はないのですが，根本的なところで何が動いているかはイメージしておいた方が良いでしょう。

1.2 プログラミングの基礎

あらためて，プログラミングとは，コンピュータに計算をさせるその仕様書を作ること，だと思ってください。お料理のレシピのように，計算レシピを書くわけですね。ただ相手は計算機ですので，言葉の端々まで正確に伝えないと理解してくれません。よく「思った通りに動いてくれない！」と不満を訴える初心者がいますが，それは当然で，プログラムは**思った通りに動くのではなく，書いた通りに動く**のです。思った通りに動かないのであれば，それは仕様書・レシピの方に間違いがあります。

今の第一原則に加えて，プログラミングを進める上での注意点をあげておきます。簡単なことだと思うかもしれませんが，基本的なルールをしっかり守ることが上達への近道です。

1. プログラムは思った通りには動かない。書いた通りに動く。大文字，小文字，スペルの違いに注意する。
2. 書き間違えないための工夫は美しさ。綺麗に書くことが大事。
3. 一言一句すべてに意味がある。ただの写経ではなく，意味を考えながら書く。
4. 「遊び心」をもって！ここを変えたらどうなるか，を少しずつ「やってみる」が大事
5. 変更は少しずつ。一気に変えるとどこが変わったかわからなくなるから。

1.2.1 綺麗に書こう

綺麗に書く，というのはコードの可読性を上げるために必要です。綺麗な書き方として，松浦 (2016)は次の 5 点をあげています。

- インデントは必ずする
- データを表す変数の先頭の文字は大文字。パラメータを表す変数の先頭の文字は小文字。
- 各ブロックの間は1行あける
- camelCase ではなく snake_case で。
- 「 」や「＝」の前後は 1 スペースあける

1 つめのインデントとは，字下げのことです。行の頭を少し凹ませることで，違う人まとまりであることを明示するのです。たとえば次の 2 つのコード[*5]は同じ働きをしますが，1.2 の方が見やすいと思いませんか？

code : 1.1 インデントのないコード

```
1 data {
2 int<lower=0> N;
3 array[N] real y;}
```

[*4] 実は人間の意識もこのユーザーイリュージョンのようなもので，実際の体の動かし方や感覚情報の受け止め方，処理の仕方は別ですよね。すべての情報を意識に上げるのではなく，「私は XXX をしている」と身体がそれっぽい幻想を投影して見せてくれているのが意識の正体ではないか，という議論があります。興味のある人はノーレットランダーシュ (2002)を読んでみてください。

[*5] このコードは確率的プログラミング言語 Stan の書き方で，この科目の後半で学ぶないようですから，今その中身までは理解しなくても結構です。

```
4  parameters {
5  real mu;
6  real<lower=0> sigma;}
7  model {
8  for(i in 1:N)
9  y[i]~normal(mu, sigma);}
```

code : 1.2　インデントのあるコード

```
1  data {
2    int<lower=0> N;
3    array[N] real y;
4  }
5
6  parameters {
7    real mu;
8    real<lower=0> sigma;
9  }
10
11 model {
12     for( i in 1:N ){
13         y[i] ~ normal(mu, sigma);
14     }
15 }
```

インデントのある 1.2 は，data というブロック ({ と} で囲まれている領域) の中に，2 つの行が入っている，ということが明確にわかります。また model というブロックの中には，for から始まる分がありますが，これも複数行に渡るブロックを構成する文なので，その中身はインデント (字下げ) されています。このように，インデントすることでどこの列がどこまでブロックを組んでいるのかがわかりやすくなるのです。ちなみにこのインデントを作るのは TAB キーを使います。TAB キーは普段，どう使うのかわからないものだと思われがちですが，このインデントをしてくれるためのものです[*6]。コードエディタによっては，カッコで括ったときに自動的に閉じるカッコを用意し，また改行ごとに字下げ位置を合わせてくれるものがあります。これらの機能を使って是非わかりやすいコードを書いてください。

松浦 (2016)の指摘の 2,3 番目についてはお好みで，4 番目もそれほど強い･広く浸透した決まりではありません[*7]。ただし，5 番目のスペースの前後に少し空白を取るのは，見やすさのためにも是非実行してもらいたいところです。

これらの書き方は，**可読性**を上げるためのものです。プログラムは仕様書･レシピですから，あとで読み直した時やほかの人が読むときも，意味がわかるようにしておいた方が良いのです。自分だけわかれば良い，と思うかもしれませんが，その自分ですら何があるのかわからなくなってしまう可能性があります。そのような

[*6] ここには流派があって，TAB でインデントする派とスペースキーを 4 回または 8 回押してインデントする派がいます。どちらでも働きは同じなのですが，個人的には数回の字下げを 1 回のキーで行ってくれる TAB のほうが良いと思っています。

[*7] ちなみにキャメルケース Camel Case，スネークケース Snake Case とは変数名のつけ方のルールです。R などプログラミング言語ではあらゆるものに名前をつけて管理しますが，名前のつけ方は任意です。命名はわかりやすい方がいいですが，長いものになると区切りを入れたくなるかもしれません。さて Camel とはラクダの意味で，ラクダのコブのように区切りのところだけ大文字にする，という記法です。Snake は蛇のことで，区切りのところにアンダースコア_を使うというものです。たとえば野球チーム，という変数名をつけたい時に，キャメルケースのやり方だと BaseballTeam のように書きますし，スネークケースのやり方ですと Baseball_team のような書き方になります。ちなみに R は日本語での命名も許しますが，そのほかの言語では一般的ではありませんし，何より全角と半角を切り替える時のミスやエラーが多くなるので，半角英数字での命名をすべきという点はどちらでも共通です。

読みにくさはすぐにバグにつながりますから，綺麗に書くことを常々心がけておいて欲しいのです。

1.2.2 意味を考えて書こう

プログラミング言語は，ほとんど英単語のようなものです。プログラミング上の都合から，短い言葉に省略されていたり，アンダースコアやピリオドでつながって書いてあったりしますが，比較的わかりやすい言葉であることに違いはありません。

プログラムを習得し上達するためにすべきことは，最初は写経と呼ばれる，テキストや指示にしたがって書き写すことです。もちろんネット上にあるサンプルコードなどをコピー＆ペーストしても良いのですが，自分用の分析コードを書くためには手を入れる必要があったりします。ですから，最初はなるべく自分でキーボードを叩いて，コピー＆ペーストではなく入力する経験を積んでください。もちろん間違えてしまうことが多いでしょうが，転ばずに歩けるようになった人が一人もいないように[*8]，ミスを犯すことでどういうミスだったかを考え，自覚することではじめて，すこしずつですが b ミスが減っていきます。失敗する経験も時には必要なのです。

ということで，時折自分なりにコードを書いてもらうのですが，その時に「ただの記号列」と思わないでください。すでに書いたように，英語あるいはそれを省略した表現になっているので，見慣れないものかもしれませんが必ず意味があります。プログラムを始める最初の頃は，ミススペルが多く，あちこちでエラーが出て気持ちが萎えることもあるかもしれません。エラーが出る時は目を凝らして，どこに問題があるかを考えて修正することになります[*9]。ここで難しいのが，x や s などの大文字と小文字の区別がつきにくいもの，1 と l のように違いがわかりにくいものがあるということです。私は職業柄，多くの人に教え，多くのバグを見つけてきましたが，その中でも特別見つけるのに苦労したのが，`norma1` というミススペルでした。みなさん，これのどこがミススペルかわかりますよね？でもこれ，書く時に「正規分布のことだな」と思っていれば，そもそも入力時にそんな入れ方はしないと思うのです。ただの字だ，と心を無にしてしまうと，後々見つけにくいエラーを作ることにもなりますから，この文字・この名前・この関数はどういう意味だろう，と少し考えながら取り組んでみてください。

1.2.3 遊び心が大事

プログラミングを進める上では，遊び心が大事です。うまく動くコードがかけたら，もうおかしなことになりたくない，と手をつけないままにしてしまう人がいます。

ところで，今まで教えてきた経験からいって，プログラミングが上達する人は，うまくコードが書けたらすぐに「ここをこう変えたらどうなるんだろう」と試してみる人だと思います。最初はプログラムの意味がまったくわからないので，どこをどういじっていいのか見当もつかないかもしれません。しかしたとえば `blue` という文字が出てきたら，これは色の青のことじゃないか，と思いますよね。ではその `blue` を `red` に変えたらどうなるんだろう？やってみよう！となるような，そういう遊び心が大事なのです。

もちろん変えてしまったことでエラーになって，動かなくしてしまうことがあるかもしれません。その場合は，元に戻して元通り動くかどうか，さらに確認すれば良いのです。数字や文字を少しいじっただけで，PC が爆発してしまうようなことはありませんから，ちょっと遊び心を出して，どうなるのかな？と思う気持ちを大事にしてください。

[*8] お釈迦様は除く。

[*9] ちなみにプログラミング歴 30 年以上の私でも，簡単な英単語，たとえば library のスペルを間違えるなんてことはしょっちゅうです。

1.2.4 変える時は少しずつ

プログラムを色々いじって遊ぶことが成長につながる，という話をしました。ただし遊び方には気をつけましょう。まずうまくいくコードができれば，それは保存しておいて，また同じ内容のものを別名で保存し，「遊んでいい方」「壊れてもいい方」を作って，そちらで色々変えて遊ぶといいでしょう。最悪なことがあっても，うまくいくバージョンは常に残っているわけですから。

そして色々試す時のコツは，一箇所ずつ変更していくことです。たとえば `blue` を `red` に変え，`line` を `box` に変え･･･と複数箇所を同時に変更して，うまくいかなかった場合，どちらが原因になっているのか把握できませんよね。これは心理学実験でいうところの交絡です。操作が交絡してしまって，原因が特定できなくなるのです。

また，実行するときも 1 行ずつやりましょう。とくに R は一問一答型，つまり 1 つの命令について 1 つの反応を随時返してきます。複数行をまとめて実行することもできますが，まとめて実行している途中でエラーが出ていて，その後の計算がすべて空回りしているということも少なくありません。エラーがどこで生じているのか，しっかり特定することが重要です。そのためにも焦らず，1 つずつ確実にできることを積み重ねていくという姿勢が重要です。

統計分析も複雑で細部まで設定し尽くしたモデルを作り上げていくことができますが，いきなり全体が完成するのではなく，小さなピースの積み重ねなのです[*10]。

1.3 いくつかのプログラミング言語

前置きが長くなりました。それではプログラミングを始めていきましょう。プログラミングには，プログラムを計算機への命令文として伝達する「言語」を利用することになります。この授業では R という言語を使います。R 言語は統計計算に特化した言語であり，心理学はもちろん統計に関する領域では多岐にわたって利用されています。比較的初心者に優しい言語設計になっており，また導入当初から豊富な関数，可視化 (グラフ化) 機能をもっています。統計解析を含んだ学術論文でも R を使ったものが多く，SPSS や SAS，Stata といった統計アプリケーションよりも広く使われています。その理由の一つは，R がフリーソフトウェアであるということです。マイクロソフトや IBM といった企業が提供するソフトウェア (プロプライエタリといいます) は，企業秘密の名目で，アプリの中で何がどのように計算されているかが隠されています。計算結果に間違いはない，と信じたいところですが，隠れている部分がある以上完全には信用できません。R はオープンソースのフリーソフトウェアです。すなわち，全ての計算過程を公開しており，お金をとって保証やサービスに代えるということをしていません。科学という営みにおいては，公明正大であることが特に重要ですから，この点は強調してもしすぎるということはないでしょう。

ところで，最近流行の AI，人工知能，機械学習といった領域では，R よりも Python の方がよく利用されています。Python も R と同様にオープンソースのフリーソフトウェアですし，優れた計算関数がパッケージとして提供されています。Python でも R と同様のことができますが，R よりもプログラミングにおいて厳格さを要求すること，複数のバージョンが乱立･併用して使われていることなどが理由でこの授業での採用をみおくりました。統計計算という意味では，Julia という言語もありますので，興味がある人はさまざまな言語を使ってみてください。

[*10] R やプログラミングで統計を学ぶ意義はここにあります。GUI で操作できる統計パッケージも少なくありませんが，それらは画面が出てきた時にデフォルトで設定されている値があるのがほとんどで，自分が何をやっているのか細部まで気づかないまま実行できてしまうのです。そうすると当然，エラーが出たり，うまくいっても誤用している，ということにもなりかねません。自分で理解できていない技術に振り回されないようにするために，しっかりとわかるピースを積み重ねていく必要があるのです。

ところで，R，Python，Julia といった言語は**インタプリタ**型の言語です。Interpret(翻訳), という言葉からわかるように, 機械に命令をするときに逐一命令文を翻訳して伝えます。「足し算をしろ (命令)」「したよ (結果)」「掛け算をしろ (命令)」「したよ (結果)」というように，一問一答型でプログラムが進んでいきますから，自分が今何を命令して，どのような答えが出たかが明確です。

これとは別に，**コンパイラ**型言語というのもあります。C，Java などが代表例ですし，この授業の後半で利用する Stan という言語もコンパイラ型です。この方式は，一連の作業工程をかいたプログラムを，まるごと翻訳してから計算を始めます。この翻訳のことを Compile する，といいます。一連の作業工程のどこかにミスがあれば – それがスペルミスのような単純なものであっても – 翻訳の時にエラーが出ます。エラーには「XX 行目でエラーが出たよ」というメッセージが表示されますが，必ずしもその行に問題があるとは限りません。その命令文の前の方で問題が生じていて，発見したのがたまたまそこになっただけという可能性もあります。さらにいえば，コンパイル時はエラーが出ず，計算が実行されてから，計算結果が間違っているとか，変な結果が表示されるということもあります。その時は元の設計書のどこに問題があるか，目を皿のようにして探し回る必要があります。インタプリタに比べて，どうにも不便が多いようにも思えますが，コンパイラの利点はその計算速度です。インタプリタは逐一のやり取りになるので，どうしても計算が遅くなりがちです。一気に計算させたい時，まとまった計算単位がある時はコンパイラ型の方が有利なのです。

1.4 R 言語の基本的な挙動

前置きが長くなりました，と言ってからさらに前置きをしてしまいました。それでは早速 R を使っていきましょう。R はそのままでも使えますが，RStudio を使って利用した方が便利です。すでに一年時に，RStudio の基本的な使い方については触れていると思います。

RStudio をひらいて，この授業用に新しく**プロジェクト**を準備しましょう。プロジェクトを設定することで，ファイルの管理がやりやすくなります。ソースコードに簡単な四則演算を書いて実行し，また簡単な関数 (`sqrt` や `help` など) を実行してみてください。問題なく利用できるでしょうか。以下に第一回目の課題を記しますので，早速やっていきましょう。

もしうまくできない場合は，教員や TA，周りの人を頼ってください。あるいは，生成 AI に質問してもいいでしょう。生成 AI はうまく利用すると，R のコードも書いてくれたりします。簡単な問題であれば，結構正しく動くコードを返してくれます。ただし，生成 AI のいうことを丸呑みにせず，必ず自分で試しながら「意図した通りのことができているか」を確認するようにしましょう。

1.5 手を動かして確かめる

ここまでの内容を，実際に手を動かして確認しておきましょう。提出の必要はありませんが，次章の課題はこれができている前提で進みます。

1. この授業のために，新しいプロジェクトを作成してください。プロジェクトは新しいフォルダでも，既存のフォルダでも構いません。
2. プロジェクトが開いた状態のとき，RStudio のウィンドウ・タブのどこかに「プロジェクト名」が表示されているはずです。確認してください。
3. 新しい R スクリプトファイルを開き，空白のままで結構ですからファイル名をつけて保存してください。
4. RStudio を終了あるいは最小化させ，OS のエクスプローラ/Finder から，プロジェクトフォルダに移動してください。先ほど作ったファイルが保存されていることを確認してください。

5. プロジェクトフォルダには，プロジェクト名 +.proj というファイルが存在するはずです。これを開いて，RStudio のプロジェクトを開いてください。
6. RStudio の File > Close Project からプロジェクトを閉じてください。画面の細部でどこが変わったか，確認してください。
7. RStudio を終了し，再び RStudio を起動してください。起動の方法はプロジェクトファイルからでも，アプリケーションの起動でも構いません。起動後に，プロジェクトを開いてください (あるいはプロジェクトが開かれていることを確認してください。)。

第 2 章

プログラミングの基礎

この授業は，R や Stan といったプログラミング言語を扱います。プログラミングはやったことがない，という人でも理解できるように，ゆっくり話を進めていきます。プログラミングのいいところは，人間相手とのやり取りと違って，必ず理屈があって振る舞いが生じている点です。対話を通じて根気よく付き合えば，必ず問題は解決できます。また昨今では，生成 AI がプログラミングをアシストしてくれるようになりました。この授業は TA がついていますから，わからないところがあれば，随時 TA に尋ねてくだされば結構です。それでも人間相手の質問ですから，「こんなことを聞いたらバカだと思われるんじゃないだろうか」「声をかけるのが恥ずかしい」ということがあるかもしれません。でも生成 AI とのやり取りは，いつでも気軽に行えます。また決してこちらを蔑んだり恨んだりすることはありません (というかあっても怖くありません！)。同じ質問を何度しても，生成 AI は一向に意に介さないので，積極的に利用してください。ただし生成 AI は正しい答えを教えてくれるとは限りませんので，**必ず自分が確認できるような質問をしてください**。何かわからないけど，バグが出ないからいいや，と思っていると，自分がなんの成長もしないのはもちろん，大変な問題を引き起こすかもしれませんので。

2.1 高級言語の基本的な働き

プログラミング言語はさまざまありますが，それらのやっていることは基本的に計算であり，その表現方法が違うだけです。どの言語にも共通する，プログラミング言語の特徴的な働きは「代入」と「反復」と「条件分岐」です。以下，この 3 つの働きについて R を使ってみていきます。

2.1.1 代入

料理でよくあるシーンとして，「野菜を切ってザルにあげておく」など，いったん作業の途中経過を横に退けて別の作業をする，というのがあります。計算プロセスでも，「計算結果をいったん横に置いておく」という操作をしたいことがあります。これが**代入**で，コンピュータとしてはあるメモリ番地に値を書き込んでおく，という操作をすることになります。横に置いておく時に名前をつけることができ，これが R のなかでは**オブジェクト (object)** と呼ばれるものになります。すでにみなさんもやったことがあると思いますが，計算結果をオブジェクトに代入しておくコードの例をみてみましょう (code:2.1)。

code : 2.1　代入操作

```
1  a <- 1
2  b <- 2
3  a + b
4  a <- 3
```

```
5  a + b
```

■コード解説

1 行目　オブジェクト a に 1 を代入

2 行目　オブジェクト b に 2 を代入

3 行目　オブジェクト a と b に $+$ という操作。すなわち $1+2$ の計算を指示していることと同じ。

4 行目　オブジェクト a に 3 を代入。もともと入っていた情報は上書きされる。

5 行目　オブジェクト a と b に $+$ という操作。すなわち $3+2$ の計算を指示していることと同じ。

非常に単純な例で，今更何を，と思った人もいるかもしれませんが，これが基本中の基本なので改めて示しました。R での代入は<-あるいは=を使います。前者は矢印のイメージ，後者は a=3 のようにして，「a は 3 だよ」という意味で代入を表しています。ポイントは 3 行目の「オブジェクト名で値を指定していること」と，4 行目の「値の上書き」にあります。オブジェクト名で計算を表現することで，操作の一般化ができます。すなわち，3 行目と 5 行目が同じ式であるのに結果が違うように，「どんなに値が違っても同じ動作をする」ようにできるわけです。コンピュータがすごいのは，この手の代入がどれほど複雑に，どれほど繰り返されても決して計算ミスをせず，指示通りに動くことができるというところです。人間は「ベクトルの 2 番目の要素と 3 番目の要素を足す」という操作を数回やるだけでも計算ミスをすることがあり得ますが，コンピュータは同じ計算を 100 回やっても 1000 回やってもミスなく動作します！

ただし注意が必要なのは，長いプログラムを書いていて，途中で「さっきの分析をやり直そう」と思って部分的に実行し，さらに続けて分析をする，ということをやっている時に上書きが生じることがあることです。途中の部分的なやり直しでできた計算は，改めて冒頭からやり直すとうまくいかないことになったりします。上書きをする時は注意して，また完成版として清書する時は一度プログラムの冒頭から走らせてみて，処理が間違いなく進んでいるかを確認するようにしましょう。

また，ここでの代入例は 1 つの数字だけでしたが，数字がセットになったベクトルや行列もオブジェクトに代入できます。セットではなく，別々の要素だけどまとめて持っておきたいという時は list 型を使います。さらに一般的なデータセットは行に個体が，列に変数がある矩形行列ですが，R では list 型の特殊ケースである data.frame 型と呼ばれます。このように，形式による違いをデータの**型**といいます。

code : 2.2　さまざまなデータの型

```
1  # 数字をセットで持つvector型，matrix型
2  x <- c(1, 2, 5, 8, 9)
3  A <- matrix(c(1, 2, 3, 4, 5, 6), ncol = 2, nrow = 3)
4  # 型にこだわりなくなんでも収納list型
5  dataSet <- list(name = c("kosugi","koji"),
6                  v1 = c(1,2,4,5,3,5,6),
7                  v2 = matrix(c(1,2,3,4),ncol=2))
8  # 矩形に整えられたlist型であるdata.frame
9  df <- data.frame(list(name=c("kosugi","suzuki"),
10             V1 = c(176,173),
11             V2 = c(83,55)))
```

R の出力 2.1: データの型による持ち方の違い (コード 2.2 の作ったオブジェクト)

```
> x
[1] 1 2 5 8 9
> A
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
> dataSet
$name
[1] "kosugi" "koji"

$v1
[1] 1 2 4 5 3 5 6

$v2
     [,1] [,2]
[1,]    1    3
[2,]    2    4

> df
    name  V1 V2
1 kosugi 176 83
2 suzuki 173 55
```

2.1.2 反復

エビフライをたくさん作ることを考えてみましょう。「えびの殻を剥いて，衣をつけてあげる」という動作を何回も繰り返すことになると思います。このような反復捜査もコンピュータは得意とするところです。なにせ，疲れを知りませんので。反復操作を指示するコードは for 文という書式を使います。反復計算コードの例は次のようなものです (code:2.3)。

code : 2.3 反復操作 1

```
1  a <- 0
2  for (i in 1:4) {
3    a <- a + 1
4    print(a)
5  }
```

■コード解説

1 行目 オブジェクト a に 0 を代入 (初期化)

2 行目 for 文開始。中括弧で括られている 5 行目までの中身を反復計算する。

3 行目 オブジェクト a に，もとの a に 1 を加えた値を代入 (上書き) している。

4 行目 オブジェクト a を出力させている。

5 行目 for 文ここまで。

ここでは 3 行目の「1 を加える」，4 行目の「表示する」という操作を繰り返していることになります。ポイン

トは 2 行目の書き方ですね。for の後ろの小カッコ () の中身が，繰り返しに使う要素の指定です。後ろの 1 : 4 は R 言語特有の書き方で，「1 から 4 まで」すなわち 1,2,3,4 を意味しています。変数 i がこの順番で変わるよ，ということを意味しているので，i は 1 回目，2 回目，3 回目，4 回目，と進んでいくことになります。code:2.4 のような書き方をすると，その振る舞いがよくわかるかと思います。

code : 2.4　反復操作 2

```
1  a <- 0
2  for (i in c(1, 3, 5, 15, 12)) {
3    print(paste(a, "に", i, "を加えます"))
4    a <- a + i
5    print(a)
6  }
```

4 行目には a <- a + i という代入があります。「a に $a+i$ を代入せよ」という意味であり，数学のイコール記号だと意味がわかりませんが ($2 = 2 + 1$ なんて式はおかしいですもんね)，ここでは $a + i$ の計算をしたものを，新たにオブジェクト a}**に上書きせよという意味になります。またここでのポイントは，反復用インデックスiが$1,3,5,15,12$の順に変わっていくということと，変わっていくインデックスiの値そのものも計算に使えるということです。2 点目については注意が必要で，反復用インデックスが$1,2,3,4$と変わるような例の場合，途中で**\verbi <- 1|などとしてしまうと，いつまでもインデックスが前に進まず永遠に計算が終わらないことになります。そのようなミスが生じないように，注意してくださいね。

また，この for 文は入れ子にして使うことができます。たとえば i が 1 から 5 まで変わり，j が 1 から 3 まで変わりながら計算をする，ということを考えて，次のような表現が可能です。

code : 2.5　反復操作 3

```
1  A <- matrix(1:15, ncol = 3, nrow = 5)
2  for (i in 1:5) {
3    for (j in 1:3) {
4      print(paste("Aの",i,"行",j,"列目の要素は",A[i,j]))
5      A[i, j] <- A[i, j] + (i * j)
6    }
7  }
8  print(A)
```

■コード解説

1 行目　オブジェクト A は 3 行 5 列の行列で，1 から 15 までの数字が順に入っている。
2 行目　for 文開始。i が 1 から 5 まで変わる。
3 行目　for 文その 2 開始。j が 1 から 3 まで変わる。
4 行目　オブジェクトの中身を表示させる。
5 行目　行列 A の i 行 j 列目の要素に対し，$i \times j$ の計算結果を加えたものを代入 (上書き) させる。
6 行目　for 文その 2 がここまで。
7 行目　for 文その 1 がここまで。
8 行目　最終計算結果の表示。

行列の行，列それぞれについて順番に，各要素を指定しながら代入していくという計算です。挙動を確認しておきましょう。

2.1.3 条件分岐

「卵が 200 円より安かったら買ってくる」というようなお使い指示，ありますよね。これは「200 円以上の時は買わない」という意味でもあります。この 200 円かどうか，という条件に対して，その後の動きが「買う」「買わない」に分岐するので，条件分岐と呼ばれます。これをプログラムで書くと次のようになります。

code : 2.6 条件分岐

```
1  egg <- 250
2  if (egg < 200) {
3    print("卵を買います")
4  } else {
5    print("卵を買いません")
6  }
```

■コード解説

1 行目 オブジェクト *egg* に 250 を代入。
2 行目 if 文開始。() 内が条件で，条件が該当すれば次の中括弧で括られている領域までを実行する。
3 行目 「卵を買います」と画面表示させる。
4 行目 条件が該当した時の実行内容を閉じつつ，該当しない場合の実行内容を書く領域を展開する。
5 行目 「卵を買いません」と画面表示させる。
6 行目 条件が該当しない場合の実行内容を閉じる。

一行目の egg にさまざまな数字を入れて検証してみてください。思った通りの振る舞いができているでしょうか。この if 文は小括弧の中身が**条件節**ということになります。条件が成立していることを**真**または **TRUE** と表現し，成立しないことを**偽**または **FALSE** と表現します[*1]。今回は egg 変数が 200 未満であることを条件としています。もし 200 円も含めたいのであれば (200 円以下にしたいのであれば)，egg <=200 と書く必要があります。

ところで，ちょうど 200 円のときに，という一致を表す条件はどう書けば良いでしょうか？ if(egg=200){...}としたいところですが，このままでは括弧の中身が「egg オブジェクトに 200 を代入せよ」という命令と同じになってしまいます。条件節で使う「同じかどうか判定する」の記号はとくに，==で表します。「卵がちょうど 200 円であれば」という条件節は，if(egg==200){...}と書くのが正しい表記です。逆に「同じでないとき」を表現したい場合は，!=と書きます。他にも条件節は，「A かつ B のとき」，「A または B のとき」のような表現をしたくなることがよくあります。このような条件節の表記や計算のことを**論理演算**といい，「かつ」は&&，「または」は||で表現します。コード 2.7 に論理演算の例を示しました。これを実行して，R が TRUE か FALSE のどちらで返答してくるか，確認してみてください。

code : 2.7 論理演算

```
1  X <- 2
2  X > 3
3  X < 3
4  X == 3
5  X != 3
6  X > 1 && X < 3
```

[*1] TRUE,FALSE が大文字なのは重要で，真偽を表す R の特別な用語です。こうした用語のことを予約語といいます。

```
7  X < 1 || X > 3
```

■コード解説

1 行目　オブジェクト X に 2 を代入しておきます。
2 行目　X は 3 より大きい?
3 行目　X は 3 より小さい?
4 行目　X は 3 と等しい?
5 行目　X は 3 と異なる?
6 行目　X は 1 より大きく，かつ，X は 3 より小さい?
6 行目　X は 1 より小さい，または，X は 3 より大きい?

条件分岐をする場合は，条件が思った通りに設定できているか，分岐した後のルートが間違いなく書かれているかなどに注意が必要です。というのも，我々が日常言語で使っている「もし～なら XXX する」といった表現は曖昧なことがあり，書いた通りにしか解釈しないコンピュータは，思った通りに動いてくれないということがあるからです。Twitter 上で有名になったジョークに，次のようなものがあります[*2]。

> ある妻がプログラマの夫に「買い物にいって牛乳を 1 つ買ってきてちょうだい。卵があったら 6 つお願い」と言った。夫はしばらくして，牛乳を 6 パック買ってきた。妻は聞いた「なんで牛乳を 6 パックも買ってきたのよ！」夫いわく「だって，卵があったから……」

これはプログラミング的思考の例として示されており，気の利かない夫とされていますが，言外の意味を含めすぎた妻の条件分岐指示がまずかったといえます。より適切な指示の例は，次のようなものです[*3]。

> プログラマを夫にもち長年苦労してきた妻「1 パック 8 個以上入った 190 円以上 210 円以下の鶏卵のパックがあったら買ってきて。それがなかったら 1 パック 8 個以上入った 189 円を超えない最も高い鶏卵のパックを買ってきて。売り場について 15 分以内に見つけられなかったときは私に電話してから指示にしたがって」

これでもまだ指示が厳密でない！条件漏れの可能性がある！というコメントがありますが，「書いた通りにしか動かない」ことの重要さがお分かりいただけるかと思います。

2.2　関数を作る

複雑なプログラムも，ここまでの代入，反復，条件分岐の組み合わせからなっています。回帰分析や因子分析のような統計モデルを実行するとき，統計パッケージのユーザとしては，統計モデルを実現してくれる関数にデータを与えて答えを受け取るだけですが，そのアルゴリズムはこれらプログラミングのピースを紡いでつくられているのです。

ここでは関数を自分で作ることを考えます。といっても身構える必要はありません。表計算ソフトウェアで同じような操作を繰り返すときにマクロに記録するように，R 上で同じようなコードを何度も書く機会があるならば，それを関数という名のパッケージにしておこう，ということです。関数化しておくことで手続きをまとめることができ，小単位に分割できるため並列して開発したり，バグを見つけやすくなるという利点があります。

[*2] https://twitter.com/beamtetrode350b/status/1406773935069229057
[*3] https://twitter.com/takl/status/1127065591380971521

2.2.1 基本的な関数の作り方

関数が受け取る値のことを**引数** (ひきすう，argument) といい，また関数が返す値のことを**戻り値** (もどりち，value) といいます。$y = f(x)$ という式は，引数が x で戻り値が y な関数 f，と言い換えることができるでしょう。

R の関数を書く基本的な構文は次のようになります。

code : 2.8 関数を作る基本的な構文

```
1  function_name <- function(argument) {
2     # function body
3     return(value)
4  }
```

ここで function body とあるのは計算本体です。例えば与えられた数字に 3 を足して返す関数，add3 を作ってみましょう。プログラムは次のようになります。

code : 2.9 引数を 1 つとる関数

```
1  add3 <- function(x) {
2    x <- x + 3
3    return(x)
4  }
5  # 実行例
6  add3(5)
```

また，2 つの値を足し合わせる関数は次のようになります。

code : 2.10 引数を 2 つとる関数

```
1  add_numbers <- function(a, b) {
2    sum <- a + b
3    return(sum)
4  }
5  # 実行例
6  add_numbers(2, 5)
```

ここで示したように，引数は複数取ることも可能です。また，既定値 (default value) を設定することも可能です。次の例を見てみましょう。

code : 2.11 引数に既定値を与える

```
1  add_numbers2 <- function(a, b = 1) {
2    sum <- a + b
3    return(sum)
4  }
5  # 実行例
6  add_numbers2(2, 5)
7  add_numbers2(4)
```

R の出力 2.2: 自分で作った関数を実行した結果 (コード 2.9 から 2.11)

```
> add3(5)
[1] 8
> add_numbers(2, 5)
[1] 7
> add_numbers2(2, 5)
[1] 7
> add_numbers2(4)
[1] 5
```

関数を作るときに，(a, b = 1) としているのは，b に既定値として 1 を与えていて，特に指定がなければこの値を使うよう指示しているということです。実行例において，引数が 2 つ与えられている場合はそれらを使った計算をし (2+5)，1 つしか与えられていない場合は第一引数 a に与えられた値を，第二引数 b は既定値を使った計算をする (4+1)，という挙動になります。

ここから推察できるように，われわれユーザが使う統計パッケージの関数にも実は多くの引数があり，既定値が与えられているということです。これらの引数は選択的に指定することができるものですが，通常は一般的に使われる値や計算の細かな設定に関するものであり，開発者がユーザの手間を省くために提供しているものです。関数のヘルプを見ると指定可能な引数の一覧が表示されますので，ぜひ興味を持って見てもらいたいと思います。

2.2.2 複数の戻り値

R での戻り値は 1 つのオブジェクトでなければなりません。しかし，複数の値を返したいということがあるでしょう。そのような場合は，返すオブジェクトを list などでひとまとめにして作成すると良いです。以下に簡単な例を示します。

code : 2.12 複数の値を戻り値にする

```
1  calculate_values <- function(a, b) {
2    sum <- a + b
3    diff <- a - b
4    # 戻り値として名前付きリストを作成
5    result <- list("sum" = sum, "diff" = diff)
6    return(result)
7  }
8  # 実行例
9  result <- calculate_values(10, 5)
10 # 結果を表示
11 print(result)
```

R の出力 2.3: 名前付きリストを戻り値にした結果 (コード 2.12)

```
> result <- calculate_values(10, 5)
> print(result)
$sum
[1] 15

$diff
[1] 5
```

519

520 こうしてみると，今まで R で「分析する」と言って使ってきた関数，たとえば `lm` や `fa` なども関数であり，戻
521 り値も複数の値がリストで帰ってきて，オブジェクトに受け取っていたんだな，ということがよく分かりますね。
522 分析結果は名前付きリストで返ってくることが多いですから，関数の返してきた戻り値の中から必要な数字，
523 値を$マークをつけて抜き出せることがわかります。次のコードを見て見ましょう。

code : 2.13 戻り値の中から必要な値を抜き出す

524
525
526
527
528
529
530
531
532

```
result <- lm(mpg ~ wt + hp, data = mtcars)
# 一般的な結果の表示
summary(result)
# 構造の確認
str(result)
#  必要な値を抜き出す
result$coefficients
```

R の出力 2.4: 戻り値の中から必要な値を抜き出す (コード 2.13)

```
> result <- lm(mpg ~ wt + hp, data = mtcars)
> summary(result)

Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268, Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12

> str(result)
List of 12
 $ coefficients : Named num [1:3] 37.2273 -3.8778 -0.0318
  ..- attr(*, "names")= chr [1:3] "(Intercept)" "wt" "hp"
 $ residuals    : Named num [1:32] -2.572 -1.583 -2.476 0.135 0.373 ...
  ..- attr(*, "names")= chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...
 $ effects      : Named num [1:32] -113.65 -29.116 -9.125 0.734 0.574 ...

 ＜中略＞

 > result$coefficients
(Intercept)          wt          hp
37.22727012 -3.87783074 -0.03177295
```

summary() 関数は, 結果の要約です。回帰分析の結果を表示する関数で, 回帰係数や適合度が表示されます。しかし, 戻ってきたオブジェクトの中身は実はこれだけではありません。str() 関数を使うと, オブジェクトの構造 (structure) が確認できますが, 実はかなりの情報が帰ってきています。これを全部表示するのは大変で, かえってわかりにくくなってしまいますから, summary() 関数はその一部しか表示しないのです。

でも何かの折に, このオブジェクトの一部を取り出したいと思うかもしれません。たとえば回帰係数の数字だけが欲しい, ということもあるかもしれません。そんな時は, オブジェクトの名前を参照しながら必要な値を取り出すことができます。

いかがでしょうか。自分で関数を作ると, 他の関数の仕組みがどうなっているのかな, ということへの理解も深まると思います。また, 統計モデルはなんか複雑なことをやってるんだろうな, とそれ以上の想像がつかないこともあるかと思いますが, 関数も中身はここで扱った代入, 反復, 条件分岐の組み合わせで出来上がっ

ているのです。時間はかかるかもしれませんが，ゆっくり考えていけば，必ず読めるし理解できます。関数の中身を理解することは，統計モデルの理解にもつながります。

2.3 まとめ

ここで説明したのは，レゴのピースのようなものです。ピース 1 つ 1 つは小さくて，それだけではなんの形も作り上げることができませんが，細かなピースでも組み合わせ次第で大きなオブジェを作ることができます。プログラミングにはたった 1 つの正解というのはなく，どういう形であれいと通りに動くものができればそれで構いません。書き方は人それぞれでよく，結果が伴うかどうかがポイントです。どのピースをどのように組み上げてもいいので，思い通りのものが作れるようにトレーニングをしておきましょう。

2.4 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) を 1 つ提出してください。

ファイル名 `chNN_学籍番号_氏名.R` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；R でプログラムを書く R の基本的な書き方を，3 つの小問で確認します。すべて 1 つの R ファイルにまとめて提出してください。

1. 次の計算をするコードを書いてください。答えは `ans1` という名前つきベクトルにまとめること。要素名は上から順に `a`，`b`，`c`，`d`，`e`，`f`，`g` とします。
   (a) $\frac{5}{6} + \frac{1}{3}$
   (b) $9.6 \div 4$
   (c) $2.3 + \frac{1}{2}$
   (d) $3 \times (2.2 + \frac{4}{5})$
   (e) $(-2)^4$
   (f) $2\sqrt{2} \times \sqrt{3}$
   (g) $2 \log_e 25$
2. 1 から 15 までの数列に対して，次の条件にあった出力をするプログラムを書きなさい。なお，割り算の余りを計算する R の関数は `%%`，文字を出力する関数は `print` です。あわせて，出力される 15 個

を順に並べた文字列ベクトルを ans2 に入れること (たとえば 1 番目は"1"，3 番目は"Fizz"となります)。

- その数が 3 で割り切れる場合には，その数の代わりに「Fizz」を出力する
- その数が 5 で割り切れる場合には，その数の代わりに「Buzz」を出力する
- その数が 3 でも 5 でも割り切れる場合には，その数の代わりに「FizzBuzz」を出力する

3. 行列 $\boldsymbol{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 2 & 0 \end{pmatrix}$ と $\boldsymbol{B} = \begin{pmatrix} 1 & 5 \\ 2 & 3 \\ 3 & 8 \end{pmatrix}$ を用意し，$\boldsymbol{AB}$ の掛け算をしたいとします。R の行列の積を求める記号 (%*%) ではなく，for 文をつかって，この計算をするプログラムを書きなさい。計算結果が %*% による結果と一致することも確かめてください。求めた積の行列を ans3 に入れること。

第 3 章

データ生成モデリング

3.1　データ生成モデリング

「心理学統計法」と名のつく講義のほとんどは，記述統計や推測統計，とくに帰無仮説検定を扱うことが中心であるのが現状です[*1]。心理学において帰無仮説検定を行う理由は，心理学実験の効果を検証すること，それも手元の標本についての記述ではなく母集団においてもその効果があると言えるかどうかを検証するためです。手元のデータに基づいて，目に見えない母集団全体のことに推論するというのは，基本的に不良設定問題です。つまり，確実な正解を導き出すにはヒントが少なすぎる，圧倒的に不利な状況で知恵を絞る必要があるのです。こうした不利な状況下ですから，いくつかの仮定をおいて，確率的に推論するのでした。その推論の方法として，**モーメント法 (moment method)**，**最尤法 (maximum likelihood estimation)**，**ベイズ法 (Bayesian estimation)** があるのでした[*2]。

これらの推定方法が**実験計画 (experimental design)**(要因計画ともいう) と組み合わせで用いられ，帰無仮説検定となるのでした。心理学における実験的なアプローチは，実験群と統制群に無作為に割り当てた集団に対し，介入・処置のあとでの群平均を見ることでその効果を検証します。人間を相手にする研究ですから，当然誤差や個人差がデータには含まれますが，無作為割り当てと平均化によってそれらはキャンセルアウトされ，平均の比較をすることで効果を見ることができると考えるのでした。また標本の平均ではなく，それを用いて母平均を推測することで，結果の一般化を考えます。母平均の推定には点推定と区間推定とがあり，確率的表現を用いた区間推定をつかって慎重に結論を導き出すのです。ただし区間推定は判断基準が明確ではありませんから，帰無仮説と対立仮説という 2 つのモデルを戦わせて決着を見る，というやり方をして一応の決着を見るのでした。こうした「実験計画」+「推測統計学」=「帰無仮説検定」は，長らく心理学のスタンダードとして君臨しています。

こうしたアプローチについて，昨今批判があることについては，今は触れないでおきましょう[*3]。それよりもこうした問いの立て方や解決策に注目してみたいと思います。心理学は物理学を範として，科学 science の仲間入りをしたい，というモチベーションが強く根付いている世界であり，また人間というのは嘘をついたり間違えたりするものだ，ということが骨身に染みてわかっていますから[*4]，データを分析する際にも**客観性**を大事にすることが殊更重視されます。客観性の反対は主観性，つまり本人の思い込みや考え方の癖がもっとも

[*1] とくに公認心理士に必要な授業として，科目名「心理学統計法」を名乗る必要があり，そこで教える基本的な内容としてこれらが含まれています。なお測定論やそれに関する多変量解析法はそれほど中心的話題ではありません。みんな使うのになあ。

[*2] これら 3 つの方法について，十分に思い出せない人は一年時の授業資料を振り返ってみてください。

[*3] Amrhein et al. (2019) などが指摘するように，誤った使い方をされることが多く心理学研究の意義そのものが疑われるほどになっている現状があります。また豊田 (2020)ではさらに辛辣に批判がなされています。

[*4] というか心理学の研究というのは，いかに人間がダメで偏った考え方をする生き物であるかを，滔々と明らかにしていくという側面もあります。

邪魔になるのです。そこでデータに対しては真っ白な気持ちで向き合うものだ，という姿勢をとることになります。

言い方を変えると，分析をする前はデータについて何も考えてないよ，という態度をとるわけです。もちろんデータは要因計画の結果として得られるものですから，計画を立てる際は主観的な誤謬が微塵も入り込むことないように緻密に練り上げるのですが，出てきた結果は結果，数字の羅列に過ぎないと考えるのです。その結果を統計的に分析するときは，それが記憶実験の数字であろうが臨床実験の結果であろうが気にすることはなく，淡々と帰無仮説検定の俎上に乗せていくことになります。こうしたアプローチができるからこそ，そしてそれを許す統計ソフトウェアがあるからこそ，心理学の研究を積み重ねていくことができたのだという一面があります。こうした研究アプローチは，データ駆動型分析と言えるでしょう。つまりデータができてから，分析が始まるという考え方です。

これに対して，データがどのように生まれてきたのか，そのメカニズムを考え，その仕組みを明らかにしていこうというのが**データ生成モデリング**です。松浦 (2016)は統計モデリングを「確率モデルをデータに当てはめて現象の理解と予測を促す営みのこと」と定義しています。すなわちこのアプローチは，まずデータがどのようなメカニズムで生まれてきたのかを，簡潔な数式を使って表現します。そのモデルをデータに当てはめ，このモデルからデータが出てきたと言えるかどうか，他のモデルの方が今のデータをうまく説明するのではないか，といった比較検証をしていくことになります。データ駆動型分析では，こうしたメカニズムが実験計画の中に暗黙理に埋め込まれていたと言えるかもしれません。データ生成モデル駆動型の分析は，メカニズムを明示して検証する，というところが違います。

データが作られるメカニズムを考えるアプローチの利点は，予測にも向いています。メカニズムが正しい，あるいはうまく現状のデータを再現できるのであれば，おそらく今後も同じようにデータが作られていくでしょう。であれば将来の予測ができる，という考え方です。たとえば市場の動向の予測，消費者の傾向の予測ができればそのメリットは想像に難くありません。昨今はデータサイエンスという言葉が流行していますが，そこにはこうした研究アプローチの隆盛があるのです。もちろん，心理学においてもモデリングのアプローチは有用ですし，従来通りの平均値の比較をする上でも，さまざまなメリットがあります。

この授業では，データ生成モデリングのアプローチをとって，心理学的にもどういう意義があるのかを考えていきましょう。

これを考える上で重要なのが**確率モデル (stochastic model)** です。言葉の通り，データ生成過程を確率を使って表現します。なぜ確率を？と思うかもしれませんが，手元のデータは誤差や個人差を含んで微小に変わる，偶然の成分が必ず入っているからです。こうした偶然をハンドリングし，偶然の中にも理論的な予測をするという点で確率が必要になるのです。また確率モデルで考えるときに，最尤法よりもベイズ法の方がよく使われます。確率モデルが複雑になっていくにつれ，最尤法では計算コストが非常に高く，実質的に解が得られないということも少なくありません。ベイズ法によるアプローチはこの問題をクリアしてくれるからです。それでは改めて，ベイズ法について復習しておきましょう。

3.2 ベイズ推定の基礎

データ生成モデルが確率の言葉で記述される，というのはすでに述べたところです。推測統計というのがそもそも不良設定問題，すなわち少なすぎるヒントから未知なるものを当てるという状況に置かれた学問であり，この「わからないこと」を確率で表現するからです。この確率についての考え方として，ベイズの公式というのがあるのでした。ベイズの定理は次のように書き表されます。

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

ここで $P(X)$ は X についての確率，という表現です。$P(A|B)$ は条件付き確率というもので，B が与えられた時の A の確率，という意味です。

ベイズの定理の用語は次の通りです。まず右辺の分子に注目しましょう。$P(D|\theta)$ とありますが，ここで D はデータを表しています。θ はデータを生む確率のパラメータです。$P(D|\theta)$ はパラメータ θ のもとでのデータ D が得られる程度を表すもの，という意味になります。これは**尤度 (likelihood)** と呼ばれるもので，パラメータの関数になっているのでした。たとえば正規分布からデータが生成されると考えるのであれば，手元のデータがパラメータ θ から出てくる可能性がどれぐらいあるのか，を表現していると言えるわけです。この尤度関数は，データを生み出すメカニズムの表現そのものです。

右辺の分子の第二項，$P(\theta)$ はパラメータの確率です。正規分布からデータが生成される例で言えば，尤度はあるパラメータの値からデータが得られる程度を表現しているのですが，そのパラメータが出てくる確率はそもそもどの程度であるか，を考えているのです。これは別名**事前分布 (prior distribution)** とも呼ばれます。実際にデータが出てくる前の段階で，そもそもそのパラメータがどれほど出やすいかという確率を表現しており，これは今回のデータを取る前までの事前のデータや経験に基づいている，ということができます。

右辺の分母，$P(D)$ はデータ全体が得られる確率であり，**周辺尤度 (marginal likelihood)** とか**エビデンス (evidence)** と呼ばれます。**正規化定数 (normalizing constant)** ということもあります。これについては理解しにくいところもあるかもしれませんが，後に述べる理由によって，ひとまず深く考える必要はありません。

左辺はこの計算の結果得られる**事後分布 (posterior distribution)** を表しています。$P(\theta|D)$ はデータ D で条件づけられたパラメータ θ の確率です。我々は確率モデルを作るわけですが，そのパラメータがどうなっているかは事前にはわかりません。が，データを取ることで「データが与えられたのであれば，パラメータはこうだよ」というのがわかるわけです。あるいは事前にパラメータはこのあたりにあるのではないか，と経験上考えていたとしても，それがデータを取ることによって更新される (確信が強くなったり，違うかもしれないと思い直したり)，ということを意味しています。

この式を言葉で表現し直すと，次のようになります。

$$\text{事後分布} = \frac{\text{尤度} \times \text{事前分布}}{\text{周辺尤度}} = \text{尤度} \times \frac{\text{事前分布}}{\text{周辺尤度}}$$

尤度がメカニズムそのものだ，ということはすでに書きました。たとえば**回帰分析**においても，尤度を計算できます。普通の回帰分析は，誤差が確率的に生じると考え，誤差以外のところは線形モデルで表現します。すなわち，従属変数 Y_i に対して，独立変数 X_i があったとすると，

$$\begin{aligned} Y_i &= \hat{Y}_i + e_i = \beta_0 + \beta_1 X_i + e_i \\ e_i &\sim N(0, \sigma) \end{aligned}$$

より，

$$Y_i \sim N(\beta_0 + \beta_1 X_i, \sigma)$$

となるのでした。ここで $N(\mu, \sigma)$ は平均 μ，標準偏差 σ の正規分布を表し，データ Y_i が正規分布から生成されているというモデルになっています。回帰分析の場合は**最尤法**で未知数 β_0, β_1, σ を求めたりしましたが[*5]，その名の通りこのモデルが示す尤度を最大にする未知数の求め方を最尤法というのでした。

ですが，尤度は確率を表すものではありません。確率とは非負の実数で，すべての確率を足し合わせる (あるいは積分する) と 1.0 にならなければなりませんが，尤度は足し合わせても 1.0 にならない数字なので

[*5] あるいは**最小二乗法 (least square method)** で求めるのですが，これは幸い最尤法の結果と一致します。正規分布を使った線形回帰モデルの場合，データの記述統計的値と確率モデルによる推定値が一致するので，気づかないまま推測統計学に足を踏み入れてしまえるのでした

す。ですから,「手元のデータがパラメータからでてくる**可能性**」という変な言い回しをしていました。「出てくる**確率**」とは言えないのですね。そこで, 尤度を確率に置き換えたい, と考えたときに便利なのがベイズの定理なのでした。ベイズの定理は尤度に事前分布をかけ, 周辺尤度で割るという操作によって, 事後分布が得られる式, と読むこともできます。事後分布は**確率分布**です。尤度にある数字をかけて (事後の) 確率分布にしていると考えるといいでしょう。ちなみに事前分布も確率分布ですが, **周辺尤度**は確率ではありません。たとえば度数を総度数で割った相対頻度は確率と考えることができますが, それと同じように, 分子をとある数字で割って, 全体を 1.0 に整えるための定数なのです。**正規化定数**というのはそういう意味ですね。定数倍 (正確には定数の逆数) をかけて大きさの調整をしているだけですので, ベイズの式はさらに次のように書き直すことができます。

$$\text{事後分布} \propto \text{尤度} \times \text{事前分布}$$

ここで $\propto$ は比例する, という関係を表しています。最終的には事後分布の形が知りたいのですが, その形を決めているのは分子の尤度と事前分布だけだ, ということになります。

ここで事前分布が**一様分布 (uniform distribution)** であれば, 事後分布の形状には影響せず, 事後分布は尤度そのものの形を反映することになります。従来のデータ駆動型分析では, データに対して事前の仮定を一切置かないということでしたが, ベイズ流の分析でもそのことは同様に表現できるわけです。

以上がベイズの定理についての簡単な復習でした。しかしベイズの定理自体は, 1740 年代には明らかになっていたことであり, それがなぜ 21 世紀の今になって見直されてきたのでしょうか。それには色々な理由がありますが, その 1 つは最近まで「ベイズ流の分析は絵に描いた餅」だったからです。すなわち理屈ではこのような形になることは明らかだったのですが, 実際に計算するのは難しいことが多々ありました。その問題を解決したのが**マルコフ連鎖モンテカルロ法 (Markov chain Monte-Carlo method; MCMC)** と呼ばれる方法です。

3.3 マルコフ連鎖モンテカルロ法

ベイズの定理から, 事前分布と尤度が分かれば, 計算式を解いて事後分布の形を算出できます。しかし**確率分布**の式が複雑になればなるほど, その計算はとても難しくなります。確率関数を複数組み合わせたり, 求めるべきパラメータの数が増えて行ったりすると, とてもじゃないけど計算して答えを出すことができない, ということになります。そのせいもあって, ベイズ統計学は長らく実践には向かない手法だったのですが, **マルコフ連鎖モンテカルロ法 (Markov chain Monte-Carlo method)**, 略して **MCMC** が出てきてからその状況が一変しました。

MCMC 法は近似的な答えを探す 1 つの方法です。MCMC という名前は「マルコフ連鎖」と「モンテカルロ法」の 2 つのパートからできあがっています。マルコフ連鎖は, 目標となる確率分布状態になるような推移規則を作る方法であり, モンテカルロ法は乱数を発生させるアルゴリズムです。この 2 つが合体することで, 確率分布の形がわからなくてもサンプルが得られるような, 乱数発生器を作ることができるようになったのです。

ベイズ統計のゴールは事後分布を作ることです。事前分布を一様分布にして, 尤度はベルヌーイ分布にする, といった簡単な場合ですととくに問題ないのですが, ある確率分布のパラメータがあって, さらにそれを生成する確率分布があるといった, 階層的に入れ子になっているようなモデルが出てくると,「最終的な事後分布の形がわからない」という状況になります。正規分布とかポアソン分布といった, 名前がついている確率分布であればその特徴がはっきりわかるのですが, それらを組み合わせて作られるものは名もなき合成関数になり, どんな形状をしているのか, まったく想像つかないことがあります。マルコフ連鎖を使うと, そのなもなき

合成関数の形状をとにかく作り上げることができる，そんな数学的技術です。

モンテカルロ法は乱数発生技術です。乱数というのは規則性のない数ですが，これを作るのはなかなか難しいものです。人間が 0-9 の数字を適当に書いていくと，知らず知らずのうちに均等でないパターンができてしまいます[*6]。コンピュータに (たとえば R に) 乱数があるじゃないか，と思われるかもしれませんが，コンピュータはあくまでも計算機でどこまでも合理的です。乱数によって動いているように見えますが，乱数に見えるような数字を生成するアルゴリズムがその中には埋め込まれていますから，正確には擬似乱数でしかありません。コンピュータの作る乱数は，もちろん人間が適当に思いつく乱数よりもより規則性が少ないですが，それでも基本的に

- 何らかの関数 g によって，内部状態 S_t をアップデートする；$S_{t+1} = g(S_t)$
- 内部状態 S_t から何らかの関数 h によって，実現値が生成される；$x = h(S_t)$

というステップを反復 ($t = 1, 2, 3 \ldots$) することで生成するのです。こうした乱数列ができれば，それを正規分布の形だとか二項分布の形に当てはめて出力することは簡単なのです。このステップ・バイ・ステップの計算法としてある確率分布に従う乱数発生技術があり，これがマルコフ過程とひっついてできたのが MCMC 法ということになります[*7]。

MCMC 法は，ですから，どんな形の確率分布であっても乱数のサンプリングは生成できる，という技術なわけです。事後分布の関数の形，形状がわからなくてもそこからの乱数は作れるという技術であり，コンピュータ技術の性能が発展した今では大量の乱数を生成することも瞬時に行われます。

乱数を用いるアプローチはいくつかの利点があります。たとえば**確率分布**の平均である期待値など，分布の特徴を記述するための計算は積分を含むので解析的に解くのは知識も技術も必要です。しかしその確率分布から乱数を発生させると，その平均値を求めることでその近似値を得ることができます。確率分布の計算が記述統計の計算に置き換えられるのであり，また統計ソフトウェアにとって大量のデータの記述統計量を描くのは造作もないことなのです。乱数による近似値は，あくまでも近似値，近くて似ている値に過ぎませんが，精度を上げるためにはその乱数の数を増やしてやるだけで良いことになります。この点もいいですね。

また求めたいパラメータが複数ある場合，たとえば正規分布だと平均と標準偏差が未知数ですし，回帰分析では平均の中に切片と傾きといった未知数が入っているわけですが，このような場合の事後分布は同時確率空間ということになります。すなわち平均と標準偏差という 2 つの変数の場合でも，可視化するなら 3 次元空間が必要です (x 軸に平均，y 軸に標準偏差，z 軸に確率密度)。こうした多次元空間において，あるパラメータだけについて期待値を計算したい，という場合はそれ以外のパラメータについては**周辺化**といって積分して全部の可能性をつぶしてしまう必要があるのですが，この計算も解析的にやるには実に大変なものです。しかし多次元の事後分布空間から生成された乱数があるなら，注目したい変数だけについての記述統計をすれば，他の変数を周辺化したこと同じになります。なんと便利なのでしょう。

このように，乱数を使ったアプローチは計算上非常に有利な特徴を揃えています。乱数を大量に発生させられるような計算機のパワーは，最近の PC でしたら十分持っていますから，最近になってベイズ統計学やモデリングアプローチが生きてきたわけですね。さらにありがたいことに，事後分布から乱数を発生させるためのプログラミング言語ツールが登場したのも大きいでしょう。古典的には BUGS というソフトウェアが，

[*6] 嘘の 538(ゴサンパチ) という標語があって，人間がふと思いつきで数字を作ろうとすると 5，3，8 が多くできてしまうと言われています。エビデンス (出典) がわからないので本当かどうかわかりませんが・・・。

[*7] 余談ですが，スマホアプリなどで「ガチャを引く」というのも内部では乱数が生成されていて擬似的に「偶然あたりが出た」のを装っているに過ぎません。私はゲームなどをする時，擬似乱数に思いを馳せ「どこかの誰かに遊ばされている」と思うとやる気がなくなるので，ガチャを引くようなシーンは興醒めしてしまいます。やはりマルチプレイヤーゲームのように，人間が背後にいたほうがよっぽど意外な行動が見られますし，ゲームよりもリアルの世界の方が擬似的でない本物の偶然を楽しむことができておもしろいと思います。皆さんはどうお考えですか。

最近では JAGS や Stan といったのがそれで, **確率的プログラミング言語 (probabilistic programming language)** と呼ばれたりします。これらの言語では, 尤度と事前分布をモデルとして表記し, それにデータを与えてやることで, 事後分布からの乱数を生成します。いわば万能乱数発生器なのです。こうした環境が整ったことで, 簡単に分析できるようになりました。

3.4 乱数によるアプローチの例

それでは MCMC を使った実践に入る前に, 乱数を使って何ができるのか, R で確かめてみましょう。

3.4.1 乱数による近似の例

まずは馴染み深い**正規分布 (normal distribution)** の例からいきましょう。

正規分布に感する R の関数は `dnorm,pnorm, qnorm ,rnorm` などがあります。この `d,p,q,r` は他の確率分布を表す関数の前につける接頭語で, `d` は確率密度, `p` は累積確率,`q` はある累積確率になるときの確率点, そして `r` が乱数発生を意味しているのでした。

code : 3.1 乱数のコードの例

```
1  rm(list = ls())
2  set.seed(12345)
3  dnorm(0, mean = 0, sd = 1)
4  pnorm(0, mean = 0, sd = 1)
5  qnorm(0.6, mean = 0, sd = 1)
6  rnorm(10, mean = 0, sd = 1)
7  set.seed(12345)
8  rnorm(5, mean = 0, sd = 1)
```

■コード解説

1 行目 環境の初期化
2 行目 乱数発生開始点の設定
3 行目 標準正規分布の, 確率点 $x = 0$ における確率密度の計算
4 行目 標準正規分布の, 確率点 $x = 0$ までの累積確率
5 行目 標準正規分布の, 累積確率 60% になるときの確率点
6 行目 標準正規分布から乱数を 10 個出力する
7 行目 乱数発生開始点の再設定
8 行目 標準正規分布から乱数を 5 個出力する

このコードを使って確認しておいて欲しいところは, 3 つあります。まず `d,p,q,r` をつけたときの意味です。それぞれ正規分布のどういう数字を返しているのか, しっかり確認しておいてください。次に 7 行目にある乱数の発生です。これを実行すると, 標準正規分布にし違う乱数が 10 個出力されます。乱数ですので, 数字の並びに規則性はありません。バラバラの数字が出ていることを確認してください。最後に 8 行目, 9 行目の内容です。8 行目は 2 行目と同じく, 乱数の開始点を定めています。R で出力される乱数は, 規則性がない数字とは言え, 計算によって算出している数字ですから規則性は拭いきれず, 再現してしまいます。どこから計算を始めるか, という開始点は**シード値 (seed value)** と呼ばれ, ここに入力した数字が開始点になります。シード値の設定にとくに意味はなく, 好きな数字にしていただいて結構です。ポイントは, 任意の数字で

あっても同じ数字に設定すると，乱数はそこから計算して算出されますので，9 行目で実行した 5 つの乱数は 7 行目の最初の 5 つと同じ数字が再現されているというところです。規則性のない数字なので再現されては困ると思うかもしれませんが，科学計算のアプローチという意味では再現性が担保できることも重要なのです。

では乱数を使う例をもう少し見てみましょう。次は**ベルヌーイ分布 (Bernoulli distribution)** について考えてみたいと思います。これはコイントスをして表が出るか，裏が出るかという 0/1 の離散的結果を生み出す分布です。残念ながら R にはベルヌーイ分布の関数はなく[*8]，二項分布の特殊例として使うことにします。**二項分布 (binomial distribution)** とは，N 回コイントスをして K 回表が出る確率を表す分布ですが，これのコイントス回数が 1 回であればベルヌーイ分布と同じことになります。

code : 3.2　乱数のコードの例 2

```
1  rbinom(10, size = 1, prob = 0.5)
2  rbinom(10, size = 1, prob = 0.3)
3  rbinom(10, size = 1, prob = 0.7)
4  # パッケージの利用
5  library(extraDistr)
6  rbern(10, prob = 0.5)
```

■コード解説

1 行目　二項分布の乱数を 10 個発生させる。確率は 0.5 で。
2 行目　二項分布の乱数を 10 個発生させる。確率は 0.3 で。
3 行目　二項分布の乱数を 10 個発生させる。確率は 0.7 で。
4-6 行目　`extraDistr` パッケージによるベルヌーイ乱数例

ここで確認して欲しいのは，コイントスを 10 回するとして，確率 0.5 の場合はほぼ半々，0.3 の場合は半分以下，0.7 の場合は半分以上表 (1) が出ている，ということです[*9]。ここで確率を幾つにするかは，われわれが自由に設定できます。また乱数を幾つ発生させるかも自由です。これらの数字を色々変えて遊んでみましょう。

たとえば乱数の数を 100 回，1000 回，10000 回と増やしたとします。その結果をすべて表示するのは大変ですが，その平均値を計算してみるとどうなるでしょうか。

code : 3.3　乱数のコードの例 3

```
1  N100 <- rbinom(n = 100, size = 1, prob = 0.5)
2  N1000 <- rbinom(n = 1000, size = 1, prob = 0.5)
3  N10000 <- rbinom(n = 10000, size = 1, prob = 0.5)
4  mean(N100)
5  mean(N1000)
6  mean(N10000)
```

■コード解説

1-3 行目　二項分布の乱数を 100 個，1000 個，10000 個発生させる。確率は 0.5 で。

[*8] `extraDistr` パッケージを用いると，`bern` という関数名でベルヌーイ分布が使えます。
[*9] 正確には二項分布なので，1 が出た回数が出力されているのです。試行数 (`size`) を 1 にしているので，1 回中 1 回表が出た＝表 (1) が出た，と解釈しても，結果的に同じことであるというだけです。

4-6 行目 それぞれの平均値を計算する

これの実行結果は，私の環境では出力 3.1 のようになりました。

R の出力 3.1: 乱数出力の結果

```
> mean(N100)
[1] 0.62
> mean(N1000)
[1] 0.52
> mean(N10000)
[1] 0.5003
```

もともと確率を `prob = 0.5` と設定しましたから，理論的には 0.5，つまり半々の確率で表 (1) が出たり裏 (0) が出たりするはずですが，最初の 100 回では 62 回なのでやや表が多めに出たようです。乱数ですので，そういうことはあります。ただ，この傾向も，1000 回，100000 回と繰り返していくと，ほぼ偶然による偏りはなく，半々の比率になっていくのが確認できると思います。このように，乱数が十分多ければ，確率分布の近似値として使うことができるというわけです。

ちなみに今回は平均をとりましたが，これは確率分布の**期待値 (expectation)** を計算したこと同じになります。

3.4.2 乱数を用いた確率分布のプロット

乱数を使って近似できる，という例を可視化で見てみましょう。また正規分布を例にしてみたいと思います。

code : 3.4 確率分布の可視化 1

```
1  x <- seq(-4, 4, 0.05)
2  plot(x, dnorm(x))
3  library(tidyverse)
4  ggplot(data.frame(x = c(-4, 4)), aes(x = x)) +
5    stat_function(fun = dnorm)
```

■コード解説

1 行目 -4 から 4 までの範囲で，0.05 刻みの数列を作る

2 行目 低水準プロット。各確率点の確率密度をプロットする

3 行目 パッケージの読み込み

4-5 行目 `ggplot` による美しい描画。`stat_function` は関数の結果を図にするもの

これは理論的な正規分布の形を図示するものです。美しいですね。でも同様のことが，乱数を使ってもできます。

code : 3.5 確率分布の可視化 2

```
1  N <- 1000
2  X <- rnorm(N, mean = 0, sd = 1) %>% as.data.frame()
3  ggplot(data = X, aes(x = .)) +
4    geom_histogram()
```

■コード解説

1 行目 発生させる乱数の数

2 行目 乱数を発生させ，data.frame 型に組み上げる

3-4 行目 ggplot による美しい描画。geom_histogram になっているところに注意

このコードの結果描かれるカーブは，それほど美しいものではないかもしれません。しかし発生させる乱数の数が増えればどうでしょうか。どんどん形が似ていくことがわかると思います。ここでのポイントは，ヒストグラムを描いたらその稜線が**確率分布**の形になること，乱数の数が増えれば十分な近似になることです。よく確認しておいてください。

3.4.3 乱数を用いた確率分布の要約

最後に，確率分布の特徴を記述するコードの書き方について練習しましょう。乱数発生によるアプローチは，記述統計量で確率分布の特徴を記述できます。確率分布の期待値は，その算術計算の平均で良いのですが，中央値やパーセンタイル，分散や標準偏差も分布を記述する重要な指標です。さらに確率分布における，確率密度が最も高くなる点は，「最も生じる確率が高い点」という意味で重要ですが，それを求めるためには特別な関数を書く必要があります。

まずは data.frame 型にした正規乱数を使って，記述統計量を算出する例を見てみましょう。

code : 3.6 確率分布の要約

```
 1  N <- 10000
 2  X <- rnorm(N, mean = 0, sd = 1) %>%
 3    as.data.frame() %>%
 4    dplyr::rename(val = 1)
 5  X %>%
 6    summarise(
 7      Exp = mean(val),
 8      SD = sd(val),
 9      Median = median(val),
10      U50 = quantile(val, prob = 0.50),
11      U90 = quantile(val, prob = 0.90),
12      L90 = quantile(val, prob = 0.10)
13    )
```

■コード解説

1 行目 乱数を 10 万個作ることにします。

2 行目 標準正規分布の乱数発生

3 行目 データフレーム型に整形

4 行目 変数名がつけられていないので，val という名前をつけることに。ここでの 1 は 1 列目を意味している。

6 行目から データフレーム X を使って計算

7 行目 summarise 関数は要約計算。変数名 = 計算式で表す。

8 行目 期待値を mean 関数で算出

9 行目　確率分布の標準偏差を sd 関数で算出[*10]

10 行目　中央値を median 関数で計算

11-13 行目　パーセンタイル点を算出。50 パーセンタイル点は中央値に同じ。

また，確率密度が最も高くなる点を算出するための関数を自分で作ってみることにします。次のようにコードを入力してください。

code : 3.7　確率分布の要約

```
1  map_estimation <- function(z) {
2    density(z)$x[which.max(density(z)$y)]
3  }
4  # 試してみる
5  X %>%
6    summarise(
7      MAP = map_estimation(val)
8    )
```

■コード解説

1 行目　関数の宣言。map_estimation という関数名を作る。この関数は引数 z を取る

2 行目　density 関数は与えられた数列の関数を考え，そのカーネル密度を計算します。which.max 関数はその中でも最も密度の高い値を返すものです。

3 行目　関数の終わり

5-8 行目　さきほどの例で試してみる。

このようにして，確率密度関数の近似値が簡単な関数で求められます。

次回以降は，モデルに基づいて計算された事後分布から得られる乱数について，これらの関数を使ってその特徴を記述していくことになります。関数や指標の意味をしっかりと理解しておいてください。

3.5　課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの　分析を実行する R コード (.R) を 1 つ提出してください。

ファイル名　chNN_学籍番号_氏名.R としてください。NN は回の番号です。

動くこと　提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方　小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは ans1，小問 2 は ans2，小問 3 は ans3 です。複数の数値を答えるときは，ans2 <- c(mu_eap = 9.7, mu_map = 9.9) のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。round や signif で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは

[*10] R の sd 関数は不偏分散の平方根を計算しており，厳密には標本標準偏差の計算で良いが，サンプルサイズが十分に大きいのでほとんど影響しません。

940 構いませんが, 提出するオブジェクトの中身は丸めないこと。

941 わからないところがあれば, 曜日別 TA か小杉までメールで連絡し, 指導を受けてください。

942 **■大問；正規分布からデータを生成して確かめる** 平均 50, 標準偏差 10 の正規分布を考えます。この
943 分布から乱数を生成し, 理論値と見比べてみましょう。

944 1. この分布に従う乱数を 10 万点生成し, その平均値, 中央値, 標準偏差, および 2.5，15，75，97.5
945 パーセンタイルを算出してください。答えは `ans1` という名前つきベクトルにまとめ, 要素名を `mean`,
946 `median`，`sd`，`p2.5`，`p15`，`p75`，`p97.5` とすること。
947 2. 小問 1 で求めた記述統計量が, 理論値の近似値になっていることを確認します。`dnorm`,`pnorm`,`qnorm`
948 などを使って対応する理論値を算出し, 乱数から得た値とどれぐらい違うかを示してください。理論値
949 は `ans2` という名前つきベクトルにまとめ, 要素名は `ans1` と同じにすること。
950 3. 生成した乱数のうち，30 以上 60 未満の値が含まれる割合を計算してください。あわせて, その割
951 合の理論値を `pnorm` を使って算出し, 両者を比較してください。答えは `ans3` にまとめ, 要素名を
952 `empirical`(乱数から求めた割合)，`theory`(理論値) とすること。

第 4 章

いんたーみっしょん；Stan の概略と環境の準備について

このセクションでは，確率的プログラミング言語 Stan を導入するにあたっての，周辺知識を解説します。授業中に解説するものではありませんし，ここに書かれていることのすべてを理解していないと Stan を使えないというわけではありませんが，導入や利用にはトラブルやエラーも多く，その際に前提知識，周辺知識があるとないとでは理解度が大きく異なります。「わからなくても，なんとなく動いた」という状態で受講し続けるのは自身のためにならないだけでなく，**面白くない**です。知識があってはじめて価値がわかるということもありますので，教養としてご一読いただければと思います。

4.1 はじめに

この授業では統計言語としての R，それを使う統合開発環境としての RStudio，そして**確率的プログラミング言語 (probabilistic programming language)** としての Stan を利用します。大学での PC ルームの利用にはこれらの環境がすでにある程度準備されていますが，最新バージョンではありませんので，自分の PC に環境を準備することを強く推奨します。今後の研究室配属やその後の卒業研究などでも活用することになると思いますので，まずは自身の PC 環境に準備することを考えてみてください。

この付録資料は，これらの環境を準備する方法について解説するものですが，提供されるソフトウェアのバージョンや対応する環境などは日々発展するものですので，必ずしも最適な情報提供になり得ません。基本的な情報は提供いたしますが，執筆時[*1]での情報であることも多く，より新しい情報についてはインターネットなどでキーワード検索を行なって，より良いものを探してみてください。

PC の操作が苦手に思っている人のために付言しますが，うまくいかなかったからといって癇癪を起こしたり，諦めたりしてはいけません。相手は機械ですから，命じられたことを愚直に実行しているだけです。また文房具のようなものですから，恐れて嫌うのではなく，愛して可愛がってあげることが肝要です。もしみなさんの中に，お持ちの PC に名前をつけていない人がいるようでしたら，今すぐ命名することをオススメします。パソコンが，と思うと腹が立ちますが，わたしの XXX ちゃんが，と思うとエラーも「ちょっとご機嫌斜めなのかしら」と受け入れやすくなります[*2]。

また，困った時は教員，TA，先輩などに相談することが重要ですが，その際には症状や経緯を正確に伝えることが必要です。ペットを病院に連れていくときに「何もしてないのに勝手に病気になった。治してほしい」

[*1] この記事は 2026 年 9 月 16 日に加筆修正されました。

[*2] ちなみに私が初めて手に入れたノート PC には「秀吉」という名前をつけました。Mac に変わってからは代々数学者の名前をつけることにしています。私はいま，Gauss ちゃんや Hermite ちゃんを持ち歩いています。

といわれても獣医さんは困ると思います。**普段の様子や症状についての丁寧な報告**を心がけてください。とくに PC は機械ですので，何もしていないのに壊れるということはありません。自分が自覚していないことであっても，「なにかをした」から様子がおかしくなるのです。自分はそんな大それたことはしていない[*3]，と思っても必ず何かトリガーがあったはずなのです。どこをクリックしたか，どういうコマンドを入れたかという**履歴をしっかりと把握し，丁寧に報告する**ことを心がけてください。

4.2 Stan の位置付け

確率的プログラミング言語 (probabilistic programming language) とは，確率モデルを記述し，データと合わせることで統計的推論をしてくれるコンピュータ言語のことです。Stan ができる前は，**BUGS** や **JAGS(と)** いうものがありましたが，BUGs は開発が終わってしまいました[*4]。今，こうした言語は Stan が最先端だといって間違い無いでしょう。これらの言語は，具体的には確率モデルとデータの関係を記述することで，事後分布からの乱数を生成するというものです。以下ではこの言語の利用形式について解説します。

4.2.1 コンパイラとインタプリタ

プログラミングはコンピュータを動かすための仕様書を書くことです。ここで「コンピュータ」というのは計算機という意味だと思ってください。もちろんコンピュータは数字の計算だけでなく，音楽を奏でたりゲームを楽しませてくれたり，と色々なことができるのですが，その背後にあるのはとにかく 0/1 の数値演算です。0 か 1 かという数字がどうして映像や音声，通信になるのか，と疑ってしまうほど異なっているようですが，それでもすべて数字の計算から構成されています。

コンピュータができ始めたごく初期の頃は，これを動かすのに 0 と 1 からなる数字の羅列で仕様書を用意していました。流石にそれではわかりにくく表現力にも乏しいので，次にできたのがマシン語とよばれる 16 進数による表現でした。この段階でも，知らない人にとっては暗号か，意味のある連なりに思えない文字列にすぎません。画面に線を引いて欲しい時に `line` という命令で伝えられるようになって，やっと普通の人間にも意味がわかるレベルになってきます。このように，人間がわかるレベルでコンピュータに命令が伝えられるような言語のことを**高級言語**と言います。高級言語は人間寄りで，直接コンピュータが理解できませんので，この高級言語を機械の言葉に翻訳するためのアプリケーションを介在させます。それがいわゆる**プログラミング言語 (programming language)** と呼ばれるものです。

プログラミング言語は，古くは BASIC，PASCAL，FORTAN などと呼ばれる書式のものがあり，その後出てきた C 言語，その改良版である C++ 言語[*5]などが有名です。他にも JAVA や Object C，Python などが有名ですね。R も言語の一種で，統計解析に特化したものです。また，この授業で扱う Stan という**確率的プログラミング言語 (probabilistic programming language)** は，確率モデルの分析に特化したものです。みなさんがデータサイエンス業界に進むのであれば，Python や R を使えるようになっておくといいでしょう。これらの言語の特徴の 1 つは，パッケージを追加することで機能が拡張できることにあります。とくに**機械学習**系 (AI など) のパッケージが豊富なのが Python です。またこれらの言語は基本的にフリーソフトウェアであ

[*3] たとえば計算途中だけど時間が来たので電源をオフにした，といったことでも，内部ファイルにアクセスしているときの中断であれば，十分に PC を破壊することができます。

[*4] JAGS はまだ開発が続いているようで，R から使うこともできます。

[*5] この ++ という書き方は，C 言語特有の「変数に 1 加える」という表記法であり，C 言語の改良版という意味で C++ と命名されています。読み方はシープラスプラスですが，シィプラプラの愛称でも知られています。

り，導入にお金がかからないところもポイントですね。タダで始められるおもちゃみたいなものです！[*6][*7][*8][*9]

プログラミング言語の分類には，その設計思想や書き方などを基準にすることもできますが，実行方法を基準にするものとして**インタプリタ**型と**コンパイラ**型とに分けることができます。インタプリタ型は，interpret，つまり翻訳型です。これは毎回の命令を機械語に翻訳して実行していきます。R はインタプリタ型言語で，毎回の命令を逐一翻訳して実行していきます。R を実行するとき，コンソールに>というマークが出ていますよね。これは R が聞き耳を立てている状態，入力待ちの状態なのです。ここに命令を書くと (たとえば 2+3 と書く)，R はすぐさま答えを返してきます (たとえば [1] 5 と返す)。基本はこのように一問一答，ひとつひとつの命令を毎回機械語に翻訳して実行し，その答えを出力するという手続きをとっています。もちろん私たちの分析プログラムは，一行で書ける簡単なものではありませんから，複数行に渡る長々としたファイルを書くことが多いでしょう。こうしたファイルはプログラムともいわれますし，一行一行の命令のことを**スクリプト (script)** ということもあります。プログラムファイルあるいはスクリプトファイルを開いて，一行ずつ実行するのが R の基本的なスタイルであり，RStudio はスクリプトファイルを編集する**エディタ (editor)** がセットになっているのが便利な点なのでした。

これに対して，インタプリタ型は，スクリプトファイルをまとめて機械語に翻訳 (コンパイル) し，実行ファイルというのを別途作ります。その上で，実行ファイルを実行すると計算が進められるのです。どうしてこんな手間がかかることをするのでしょうか。ひとつには，ひとつひとつの命令分を逐一翻訳しているのでは，実行スピードが遅くなるということが挙げられます。命令を逐一聞き耳を立てて待ち，全ての命令を聞き終わるまで途中の指示を記憶しておいて，命令文が終わってはじめて行動に移す，というのでは記憶容量も時間もかかるわけですね。これに対して，一連の計算命令文書を一括で渡すことができれば，機械は内部的に効率よく計算手順を配置して実行できます。コンパイラ型は計算スピードが速いのです。ちなみに実行ファイルは機械がわかる言葉に書き換わっているので，人間が見ても意味がわかりません。また，OS や CPU に理解できる形に書き換えていますので，MacOS の実行ファイルを Windows で実行する，ということはできません。翻訳後の言語が違うからです。スクリプトファイルは環境を超えて共有できますが，それを翻訳したり最適化したりする仕組みは，個別の環境に依存します[*10]。

Stan はコンパイラ型です。Stan のファイルを読み込んで，機械にわかるように「事後乱数生成命令文」に翻訳して実行します。確率モデルから乱数を生成するのは非常に高度な計算機能ですから，コンパイルしてまとめておくことでスピードアップの効率が断然良くなるわけです。コンパイラ型の利点はこのスピードにありますが，欠点は「命令に変更やミスがあれば翻訳し直さなければならない」というところです。あり得ない命令を出しているようであれば，コンパイルの段階で「おかしな文法だよ」とエラーを出して翻訳を止めてくれます。翻訳が通れば良いかというとそうではなく，翻訳はあくまでも文法上の手続きであって，数値や内容に間違いがあっても翻訳自体は完了させることができるのです。翻訳 (コンパイル) には少し時間がかかりますから，文法

[*6] 余談ですが，Scratch や任天堂 Switch の「ナビつき! つくってわかる はじめてゲームプログラミング」などにみられる，GUI ベースのプログラミング言語もあります。これはプログラミングの要素がアイコンやキャラクターになっていて，それらを組み合わせることで計算させるという方法をとっています。その操作のしやすさ (ミススペルがない！) から，小学生などにも導入できる素晴らしい取り組みです。同様の発想は，LOGO 言語などにもみられました。

[*7] Perl や Java Script など，Web ブラウザ上で動くプログラミング言語もありますが，これはブラウザ上での操作に限定されており，数値計算にはあまり向かないのでここには取り上げていません。さきほどあげた JAVA と JAVA Script は別物なので注意してください。

[*8] GUI アプリケーションを作ることに特化した言語もあります。数値計算と違って，マウスがどこでクリックされたか，と言った「動き」を契機としてプログラムが走る，イベントドリブン型の言語で，Microsoft 社の Vsial Basic などが有名です。これは Microsoft Office 製品の中に埋め込むこともでき (VBA)，これを使うと Excel でも高度な統計解析ができたりします。Excel を使った統計ソフトウェアとして代表的なものに HAD があります清水 (2016)。他にも Delphi などの言語がありました。

[*9] シ (2016)には，数え方にも癖がありますが，117 ものプログラミング言語が紹介されています。

[*10] ちなみに R や Rstudio はもちろん，Word や Excel などのオフィスアプリ，果ては OS そのものも，コンパイルされた実行ファイルを PC が計算して実行しているに過ぎません。

上も内容上も間違いのない命令分をしっかり準備しておくことが重要になってきます。

4.2.2 ファイルのやり取りについて

ここで，R/RStudio をつかって Stan を使う際の，ファイルのやり取りを見ておきましょう (図 4.2)。

Stan のコードは R のコードの中に書くこともできますが，別のファイルに保存しておくのがベターです。Stan ファイルは拡張子を `.stan` とすることが一般的です。このファイルの中身はプレーンテキストでコードを書いていますから，一般的なエディタ[*11]で編集できます。RStudio のエディタ画面も必要十分なエディタ機能を持っていますので，RStudio をエディタとして利用するのがいいでしょう。

RStudio で File > New File として新しいファイルを開くときに，Stan ファイルとして開くことが可能です。Stan ファイルとして開くと，初心者向けの配慮からか，最初からちょっとしたコードがすでに書かれています。コードの書き方を示すサンプルコードなのですが，実際にこのコードを使って何かすることはありませんので，中身は全部削除してしまいましょう。それでも RStudio はその画面が Stan ファイルのものだということは認識していますから，そこで書いたファイルは Stan ファイルとして扱ってくれます。ここでまちがって新しく R Script で開いてしまった，というときに，保存するときだけ `.stan` をつけておけばいいか，という対応をすると失敗します。RStudio では拡張子をあえて表示していませんので，ファイル名として `hoge.stan` と書くと実際には `hoge.stan.R` というファイル名になっています。拡張子は最後のピリオドで判断されますので，これは R ファイルなのです。このミスを防ぐために，ファイル名にピリオドは使わないこと，ファイルの種類を変える時はエディタ画面の右下でファイル種別を選択してください (図 4.1)。

図 4.1 ファイル種別の変更

RStudio を使って Stan を書くと便利なことがいくつかあります。ひとつは強調表示機能で，ブロックや関数名など Stan で使うことが決まっている専門用語は色やフォントが変わって表示されます。`transformed parameters` のように長い専門用語を書くときはミススペルが心配ですが，書いた後で強調されなければスペルミスがある，ということがわかります。もうひとつの利点は文法チェックの機能です。コンパイル型なので，Stan に書かれた命令文は実行前に一括で翻訳されますが，スペルミスや型の違いなど，文法的に間違っているところがあればこれも自動的にチェックして警告記号がでます。また Stan ファイルが開かれているペインの左上に `Check` というボタンがあり，これを押すことでファイル全体の文法チェックをしてくれます。もし文法上の問題がなければ，「hogehoge.stan is syntactically correct.(hogehoge.stan というファイルは文法的には正しいですよ)」というメッセージがコンソールに表示されます。逆にいうと，これが表示されないというときは何か間違っているので，コンパイルに進む前に修正しましょう。

さてこのようにして Stan ファイルを準備します。また分析用のデータセットが外部ファイルにある場合などは，これまで同様，当該プロジェクトフォルダ内に置いておくと良いでしょう。これらはいずれも，R のコードで

[*11] エディタとは ASCII ファイル，いわゆるプレーンテキストを編集するアプリケーションの総称で，OS にデフォルトで「メモ帳」などの名称で含まれています。デフォルトのアプリは文字を読み書きできるだけですので，もう少し発展的，あるいは便利な拡張機能を持っている専用のアプリを使うことが一般的です。筆者は最近もっぱら VS Code というエディタを利用しています。他にも MacOS でしたら，「mi」というアプリを好んで使っていました。Windows でしたら「秀丸」というアプリが有名です。Ubuntu には gedit というアプリがデフォルトで入っていますし，歴史やユーザの多さでは vim や emacs などがあります。Linux 界隈で，vim 派か emacs 派かの話題は，きのこたけのこ戦争ばりに激しい戦いになります。

呼び出して使うことになります (図 4.2 の 1. と 2. のステップ)。R では，Stan ファイルやデータファイルを読み込んで，これを PC 内部にある Stan に渡します (図 4.2 の 3. のステップ)。Stan は R と違う言語であり，Stan 独自の計算機能で計算してくれるわけですから，R と Stan の橋渡しが必要になります。R ではこれを `cmdstanr` や `rstan` というパッケージを経由して行います。これらのパッケージが Stan を呼び出してくれるわけです[*12]。

データと確率モデルの関係を記した Stan ファイルと，データのそのものを受け取った Stan は，コンパイルして PC 内に乱数発生装置を作ります (図 4.2 の 4. のステップ)。事後分布の関数を書き出すのではなく，その形状とそれを代表する乱数を作る状態をもつわけです。R の指示をうけて，そこから必要な数だけ乱数を作り出します (図 4.2 の 5. のステップ) 。R はこれを受け取って，統計解析を始めるわけです。R は統計に特化した言語ですから，集計や可視化などの作業はお手のもの。事後分布から得られた大量の乱数は，事後分布の特徴を反映した大量のデータセット，事後分布の具体例たちになっているのでした。

分析の進め方は以上のようになります。R でデータを整形し，Stan のファイルも別途用意して，R から Stan を呼び出す，Stan か返してきたデータを R で解析する，ということです。R の拡張子は `.R` で，Stan の拡張子は `.stan`，データファイルの拡張子は `.csv` などでしょうか。いずれにせよ，複数の種類のファイルをやり取りしながら進めるので，相互の関係についてしっかり理解しておく必要があります。

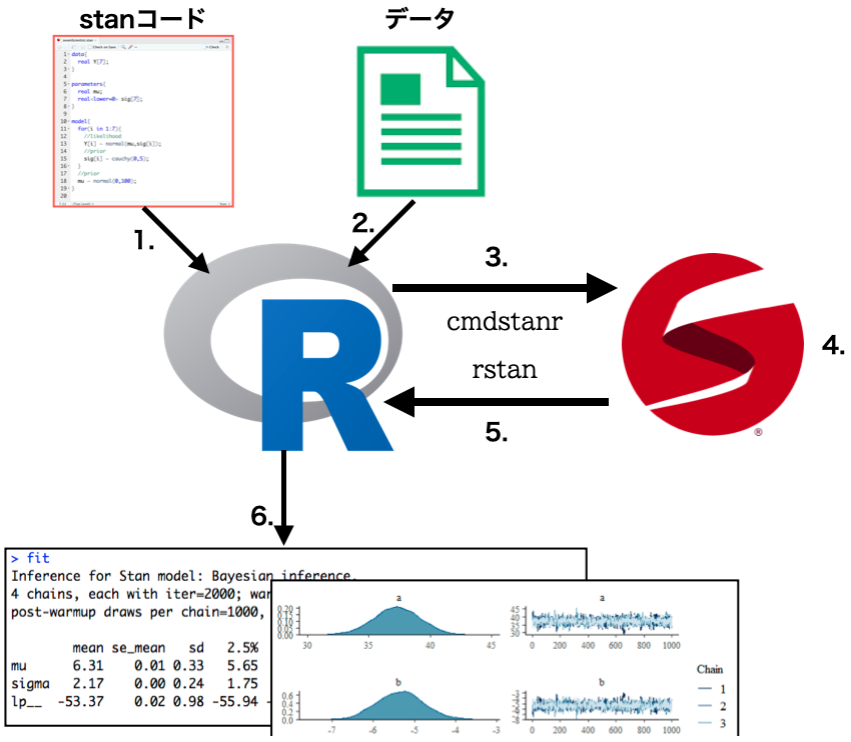


図 4.2　R/RStudio と Stan ファイルのやり取り

[*12] このように，Stan は独立した計算環境であり，R 以外のアプリケーションから呼び出して使うことが可能です。Python から使いたい場合は，PyStan というパッケージ経由で，Julia から使いたい場合は StanJl，コマンドラインから使いたい場合は cmdstan といったように，いろいろなルートがあります。

4.2.3 2 つのルート

ここでは R から Stan を呼び出す 2 つのルート，具体的には rstan パッケージと cmdstanr パッケージについて説明します。この 2 つのパッケージは，いずれも Stan を呼び出してつかう，R と Stan の間を取り持つインターフェイスの役割を持ったパッケージです。間を取り持つだけなので，Stan ファイル自体は同じであっても構いません。同じ Stan ファイルを rstan から呼んでも，cmdstanr から呼んでも，結果は同じです。ただしインターフェイスが違うので，結果の扱い方が変わってきます。他にも違うところがいくつかありますので順に説明していきますが，結論からいうとこれから[*13]は cmdstanr のほうを使うほうがお勧めです。

歴史的には rstan のほうが古くから存在します。このパッケージも最初の頃は導入に一苦労するものでしたが，2016 年ごろから CRAN を通じて配布されるようになりました。要するに，他のパッケージと同じように，install.packages("rstan") と R で書くだけで導入できるようになったのです。このようにして導入するとわかりますが，rstan は多くの依存パッケージがあります。Stan はコンパイルするのに OS に応じた C++ コンパイラを借用しますが，R から C++ を呼び出すパッケージや R と C++ の間に入る StanHeader と呼ばれる緩衝材など，関連する多くの環境を整えてやっと R から Stan へのルートが通じるのです。ユーザにとってはそのような苦労は知ったことではない，という感じですが，「間に多くの調整が入る」ということがエラーの温床になっていました。つまり，いくつかの内部的ステップのどこかでバグがある，互換性がない，というようなことがあると，Stan ファイルが同じでも，バージョンが変わるとコンパイルできないということがよくあったのです。Stan 本体の方も開発が盛んですから，Stan の新しいバージョン，新しい機能を使いたいと思っても，パッケージが対応するのを待たなければなりません。パッケージが対応しても，途中の媒介パッケージが追いついていなければバグになったりします。これらを使う R や OS もアップデートしていきますから，その度に「動かなくなる」ということはよくありました。研究の再現性が問題になる昨今ですが，自分の研究とデータであっても，自分の環境で再現できないということが少なくなかったわけです[*14]。そうなると，今動いているからそれでいい，と環境のアップデートを控えるようになりますが，ご存知の通り OS やアプリのアップデートはバグやセキュリティホールの修正など，安全につかうための基本的な機能に関わってきますから放置もできない，という問題があるわけです[*15]。

このような不安定な環境になる原因は，R と Stan の間にあるさまざまな障壁と，それをなんとかして調整しようという媒介パッケージの存在でした。そこでなるべくシンプルに R と Stan をやりとりする方法はないものか，ということで考えられたのが cmdstanr です。これはコマンドライン[*16]から Stan を呼び出す cmdstan を R から呼び出して使おう，というものです。Stan を R になんとか取り込んで動かす，というのではなく，Stan の部分は OS のプリミティブな環境にお任せ，R は結果をもらうだけ，というシンプルな設計にしようというわけです。たとえば rstan は Stan の計算結果を R オブジェクトとして取り込みますが，cmdstanr は実は MCMC の結果は csv ファイルのような形で吐き出しており，それを読み込んで使います。御用聞きや仲介業者を介して情報をもらうのではなく，現地で直接買い付けしてくるようなイメージでしょうか。このように間に

[*13] 執筆時点，2026 年 9 月現在

[*14] 実際 R や OS のアップデートに関わるエラーは大変で，筆者は一度 OS をアップデートした後半年ほど Stan が使える環境が作れず，研究が頓挫したことがあります。そのために別途，PC を購入し直したぐらいです。

[*15] OS のアップデートなど，通知が来ても「今支えているからいいや」と無視する人もいますが，できれば OS やアプリは最新版であるほうが良いのです。セキュリティホールの問題などを放置しておくと，インターネットを介して自分の PC の中身が全部見られて公開される危険性があります。Stan しか使わないネットに繋がない PC というのがあればいいのかもしれませんが，文房具としての PC はもっと一般的な用途がありますから，そうもいってられませんよね。また，一昔前の OS アップデートは，アップデートの失敗で PC が動かなくなるというようなこともありましたから，アップデートが公開されてすぐに対応するのは「人柱」などと揶揄されていましたが，最近はそういったこともほとんどありませんので，安心して常に最新の状態にしておいてください。

[*16] MacOS や Linux では端末，ターミナル，Windows ではコマンドプロンプトなどと呼ばれる，PC に直接コマンドで命令を出すプリミティブな実行環境です。

挟むものを減らすことによって，不安定な要素をなくしたわけです。

cmdstanr は Stan と R の複雑な絡まりを排していますので，Stan の開発が進めばすぐにそれを R に届けることができます。たとえば 2026 年 9 月現在で，CRAN にある rstan が抱えている Stan のバージョンは 2.32 ですが，cmdstanr が呼び出す Stan は 2.39 です。cmdstanr のほうが新しいものに対応できていますね。このように，cmdstanr がでてきてからはこちらの対応，反応が早く，また動作も安定的ですから，よりお勧めしやすくなっています。

これまでに出ている R で Stan を使う方法を解説したテキストの多くは，rstan をつかっていますので，今それを使うためには多少の読み替えが必要です。とはいえ，Stan のコード自体はそのまま使えるものがほとんどで，R とのインターフェイス，やりとりの方法が違っているだけです。このテキストでは第 2 版から cmdstanr を中心にするように切り替えました。

4.3 導入の概略

環境の導入は，OS の種類によって変わります。Widows なのか Mac なのか，あるいは Linux なのかといった違いに合わせて導入を考えてください。OS の種類で言えば，Linux は全体的に CUI 操作が主であり，堅牢かつ合理的な使い方ができるものです。如何せんマイノリティですので，ググってもあまり情報が出てこないところが玉に瑕です。Mac は Linux ベースの OS になっているので，比較的堅牢かつ合理的な使い方ができます。ハードウェアも OS も 1 つの会社 (Apple) が作っているので，サポートもしやすいという利点がありますが，日本では少数派なのが残念なところです。Windows は多数派の OS ですが，OS メーカはソフトウェア会社でハードウェアが別会社なことが多く，利用される範囲は広いのですが，その分問題が生じたときにケースバイケースになりがちです。多くのハードウェアの違いを吸収するために最大公約数的な設計をするせいか，ソフトウェアデザインの統一性がないところが欠点です。ユーザ数は最も多いので，一生懸命調べたら同じような問題で困っている同じような機体の人に出会える可能性が高いのがせめてもの救いです。また光明として WindowsOS も Linux ベースの仕組みを取り入れ始め (WLS)，比較的合理的な振る舞いができるようになったことが挙げられますが，OS のグレードによって導入のしやすさが異なります。

ちなみに ChromeOS や iOS, Android などタブレット・スマートフォンでの統計環境の利用は限界があります。これらは計算結果を利用するフロントエンドとしては非常に高機能なのですが，インタラクティブに使うことには不向きです。いわば書籍のように多くの情報を一方的に与えてはくれるのですが，ユーザが計算するといった働きかけの補助になるような (ノートとペンのような) 使い方ができませんので注意してください。

さて，環境の導入として 2 つのルートを紹介しましょう。第一のものが最も基本的なアプローチで，自分の手元の環境を作り上げるというものです。これを**ローカル環境**での構築と呼びます。ローカル環境は自分だけの環境ということなので，自分の責任でもってしっかりと環境構築をできます。第二のアプローチとして，ある程度できあがった環境を丸ごと取り込む，**仮想環境**での構築という方法があります。PC の OS の中に別のマシン・別の OS をさらに憑依させて使うようなやりかたで，これができると取り込む環境は完成しているので細かい設定をする必要がありません。問題は「憑依させる」準備がいろいろ大変であるということです。また憑依させた PC はブラウザ経由でアクセスすることになることにも注意してください。

4.3.1 導入方法 1；ローカル環境での構築

ローカル環境での構築は，R，RStudio の導入から入ります。ここは既にできている人もいるかもしれませんが，念のためバージョンが最新のものかどうかを確かめておいてください。ローカル環境構築の方法がわからない人は，「RStudio」「インストール」などのキーワードでウェブ検索し，自分の環境にあっ

1157 た例を見つけてそれをみながら進めると良いでしょう。参考までにいくつかあげておきますと，新しい
1158 ものでは，高知工科大学柳井先生のこちら http://yukiyanai.github.io/jp/resources/ とか，Quiita の記事
1159 https://qiita.com/hujuu/items/ddd66ae8e6f3f989f2c0 が良いでしょう。書籍では，手前味噌で恐縮ですが小
1160 杉 (2019)などが良いでしょう[*17]。

1161 次に確率的プログラミング言語，Stan の導入を行います。Stan を R から使うためのパッケージとして
1162 `cmdstanr` と `rstan` という 2 種類があるという話はしました。かつては `rstan` のほうがよく使われていたの
1163 ですが，いまは `cmdstanr` のほうが安定的に動作し，開発も盛んですので，この授業では `cmdstanr` を使い
1164 ます。このパッケージの導入には，公式サイト https://mc-stan.org/cmdstanr/articles/cmdstanr.html を参考に
1165 しましょう。「cmdstanr インストール」で検索すると色々な紹介サイトが出てきますので，自分と同じ環境でなる
1166 べく日付の新しいものから参考にしていくといいでしょう。

1167 インストールにあたっては，Stan がコンパイルされるときに利用する C++ 言語環境が必要です。これは
1168 MacOS であれば Xcode の導入が，Windows であれば Rtools での導入が必要になります。

1169 **■MacOS ユーザの場合** AppStore で Xcode と検索し，Xcode をインストールすれば OK です。

1170 **■Windows ユーザの場合** Rtools という R 周辺のツールをインストールする必要があります。https:
1171 //cran.r-project.org/bin/windows/Rtools/ にいって，自分の R のバージョンにあった RTools をインストール
1172 してください。この他にも Windows ユーザは導入にあたって色々障壁にあたることがありますので，こちらの
1173 サイト https://norimune.net/3609 なども参考にしてみてください。

1174 4.3.2 導入方法 2；仮想環境での構築

1175 第二のルート，仮想環境を構築する場合ですが，これについては我らが国里先生が大変詳しいサイトを
1176 作ってくださっています。日本心理学会のチュートリアルワークショップ，「再現可能な日本語論文執筆入門:
1177 jpaRmd で実現する再現可能で低コストな日本語論文執筆のはじめの一歩」で使われた時の資料集がそれ
1178 で，このサイト https://ykunisato.github.io/jpa2021-tws-jpaRmd/ の事前準備編をよく読むと導入できます。
1179 通信環境にもよりますが，慣れているとものの 10 数秒でシステム導入できるという優れものです。面倒な設
1180 定が怖い人は，是非試してみてください。

1181 4.4 導入方法 3；外部サーバの利用

1182 最近，Google 社がオンラインで利用できる分析環境，Google Colaboratory を提供してくれています。こ
1183 れは Google 社が提供する，ブラウザで実行できるプログラミング環境です。Google が教育，研究用に提供
1184 しているもので，90 分間は無料で利用できます。90 分を超える場合でも，書いたプログラムを保存しておけ
1185 ば，新しいセッションを開始することで続けて利用できます。これを用いる利点は，どのユーザにも同じ環境を
1186 提供できることです。

1187 基本言語環境は Python で提供されており，Jupyter Notebook を使いますが，次のアドレス (に含まれる
1188 コード) を使えば言語環境を R に変えて利用することができます。

1189 https://colab.research.google.com/notebook#create=true&language=r

1190 ここでウィンドウに R のコードを入力し，実行していきます。`cmdstanr` もインストールできます。

[*17] インストールに際しては，RStudio Desktop の Free edition を選んでください。RStudio Cloud や RStudio Server は別のものです。

まずパッケージ本体を `install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev", getOption("repos")))` で入れ，続けて `cmdstanr::install_cmdstan()` で Stan 本体を導入します。Stan 本体はビルドに数分かかりますので，気長に待ってください。

4.4.1 導入後の確認

インストールが終わったらサンプルコードを実行して「動くかどうか」を確かめてみてください。サンプルコードは Getting started with CmdStanR のサイトに掲載されています。実行して文字列がズラズラっと出力されればとりあえず動いたと思っていただいて結構です。赤い文字やエラーなどが表示されうる，あるいは全く表示されない場合は，インストールがうまくいっていないことを疑いましょう。上に戻って，丁寧に説明に目を通して一歩ずつ進めてください。機械に命令することですので，自分勝手にショートカットしたり変更したりしないことが重要です。

4.5 Stan を使ってみよう

具体的な Stan コードの書き方や統計モデルへの応用については，本書の次の章から順に説明していきます。それに先立って，全体的な注意をしておきたいと思います。具体的には，配列の書き方，実行と出力のスタイル，そして本書の以下の章で使う関数の紹介です。

4.5.1 配列の書き方について

Stan の文法は，バージョン 2.32 で配列の宣言の仕方が変わりました。古い資料やコードを読むときに戸惑うところですので，先に触れておきます。

データ X が N 人分あったとします。数学的フォームでは，$X_1, X_2, X_3, \cdots, X_N$ というようなものですね。これはコンピュータ上では X と名付けられた配列，サイズ N というように考えます。これは，かつて Stan では次のように書くものでした。

code : 4.1 Stan2.32 以前の書き方

```
int n[5];
real a[3, 4];
real<lower=0> z[5, 4, 2];
vector[7] mu[3];
```

これらは上から順に，サイズ 5 の整数変数 n，サイズ 3×4 の実数変数 a，下限 0 のサイズ $5 \times 4 \times 2$ の配列 z，長さ 7 のベクトル mu が 3 つ，ということを意味しています[*18]。

この書き方が，Stan のバージョン 2.32 以降は次のような書き方になりました。

code : 4.2 Stan2.32 以降の書き方

```
array[5] int n;
array[3, 4] real a;
array[5, 4, 2] real<lower=0> z;
array[3] vector[7] mu;
```

[*18] ベクトルは数字をセット，人まとめとして演算するものです。最後の変数は，7 つの数字のセットが 3 つという意味で，7 つセットで 1 つのまとまりですよ，ということを明示的に宣言していることになります。

このコード 4.2 はさきほどのコード 4.1 と同じ意味で，書き方が変わっただけです。一瞥してわかるように，サイズを先に array という言葉で宣言しましょう，というだけですね。

気をつけてほしいのは，古い書き方はもう使えないということです。過渡期には両方が通ったのですが，現在の環境で古い書き方を試すと次のようなエラーが出て，コンパイルする前の段階ではねられます。

R の出力 4.1: 古い書き方はエラーになる

```
Ill-formed declaration. ";" expected after variable declaration.
  It looks like you are trying to use the old array syntax.
  Please use the new syntax:
  array[5] real y;
```

親切にも「新しい書き方を使いなさい」と正解まで教えてくれますので，落ち着いて直せば大丈夫です。とはいえ，世の中にある Stan の教科書やウェブの記事は古い書き方で書かれているものがまだたくさんあります。そのまま書き写しても動きませんので，上の対応関係を頭に入れて，読み替えながら使ってください。

本書のコードは，すべて新しい書き方に統一しています。

4.5.2 実行のオプションと出力の読み方

さて今度は，実際に走らせるときの指定の仕方と，返ってくる出力の読み方を見ていきましょう。

指示の仕方

まずは実行方法です。Stan では事後分布からの乱数を生成しますが，それにあたって「乱数の数」「ウォームアップ期間の長さ」「チェインの数」などオプショナルに指定できるものがあります。これは見てもらったほうが早いでしょう。

code : 4.3 cmdstanr のスタイル

```
fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  iter_sampling = 5000,
  iter_warmup = 1000
)
```

ここで指定しているのは，データを dataSet として設定し，同時に 4 本のチェインを発生させ，それを並列で走らせて，1 本あたり 5000 個の乱数を作る，というものです。それとは別に 1000 個は，まだ機械が温まっていないので候補から除外します。複数のチェインを作ること，温まっていない部分を捨てる理由などは後ほど説明しますが，結果的に合計 20000 個の乱数が作られることを知っておいてください。この数字は $5000 \times 4 = 20000$ という計算からでてきます。

■他の本を読むときのために 世の中に出ている Stan の教科書の多くは，rstan パッケージを使って書かれています。こちらは rstan::sampling 関数を使い，並列計算をさせるために options(mc.cores = parallel::detectCores()) という一行を事前に入れておく，というスタイルです。紛らわしいのは乱数の数の数え方で，rstan の iter は捨てる分も含めた数ですから，上と同じことをするには iter = 6000, warmup = 1000 と書くことになります。Stan ファイル自体はどちらでも同じものが使えますので，R とのやりとりの部分だけ読み替えてください。

出力の読み方

続いて出力結果です。cmdstanr パッケージで推定した結果は，次のような形で表示されます。

cmdstanr の出力 4.1: cmdstanr パッケージの出力

```
variable   mean median   sd  mad     q5    q95 rhat ess_bulk ess_tail
lp__ -51.56 -51.20 1.34 1.07 -54.18 -50.12 1.00     7799    10071
mu1   59.99  60.00 5.63 5.47  50.82  69.16 1.00    16325    12764
mu2   40.05  40.05 5.48 5.18  31.09  49.05 1.00    16511    12466
sig   15.48  15.03 3.07 2.75  11.41  21.07 1.00    14489    12137
```

前から順に，事後分布の平均値 (mean)，中央値 (median)，標準偏差 (sd)，平均絶対偏差[*19](mad)，5%,95% パーセンタイルなど分布の特徴[*20]と，Rhat や有効サンプルサイズに関する情報 (ess_bulk,ess_tail) が示されています。まえがき・あとがきのようなものはなく，結果だけがシンプルに出力されていますね。それぞれの内容については，あとの章で順に説明していきます。

cmdstanr は，MCMC の結果を一時フォルダに csv ファイルとして吐き出しており，上の表示はそれを要約したものです。乱数そのものを取り出して自分で加工したいときは，fit$draws() で取り出し，posterior パッケージの関数でデータフレームの形に整えます。次の節で，そのための関数を用意しておきます。

4.5.3 本書で利用する準備関数

欲しいものは事後分布からの乱数をデータセットにしたものであり，そのデータセットさえ持っていれば，既存の関数を使わなくとも自分で工夫して加工すれば良いでしょう。

そこで本書では，MCMC サンプルを抜きだしてデータセットに変換する関数を作り，これを利用して話を進めることにします。ここではその関数の中身を解説しておきます。

MCMC サンプルをデータフレームにする関数

まずは MCMC サンプルをデータセットにして渡してくれる関数です。この関数は伴走サイトのコードの冒頭に必ず含まれていて，以後はこれを使って結果の要約を示すといった使い方をしていますので，中身を知っておいてもらった方が良いでしょう。

cmdstanr パッケージで MCMC サンプルを得た場合，そのオブジェクトから乱数部分だけを抜き出す関数は draws 関数であり，これをオブジェクトに直接作用させます。これで取り出したものをデータフレーム型にして返す関数として，次のようなものを作りました。

code : 4.4 MCMCtoDF 関数 (cmdstanr 版)

```
1  MCMCtoDF <- function(fit) {
2    fit$draws() %>%
3      posterior::as_draws_df() %>%
4      tibble::as_tibble() %>%
```

[*19] **中央値絶対偏差 (median absolute deviation)** とは，中央値のばらつきを表す指標で，各データ点の中央値からの偏差の絶対値をとり，その中央値を計算したものです。標準偏差とは違う分布の幅を表現する方法で，中央値を使って計算しますから外れ値に強いという特徴があります。

[*20] ここで示されるのは 5% と 95% のパーセンタイル，すなわち囲まれる区間の面積が 90% です。rstan パッケージの出力は 2.5% から 97.5% のパーセンタイル，すなわち 95% になっていますので，他の本と見比べるときは注意してください。

```
5      dplyr::select(-lp__, -.iteration) %>%
6      dplyr::rename(iter = .draw, chain = .chain) %>%
7      tidyr::pivot_longer(-c(chain, iter)) -> MCMCsample
8    return(MCMCsample)
9  }
```

■コード解説

1 行目 関数を作る宣言。引数として cmdstanr の出力オブジェクトをとります。

2 行目 引数で引き受けたオブジェクトに draws 関数を作用させ, サンプルだけ取り出します。

3 行目 取り出したサンプルをデータフレーム型にする posterior パッケージの as_draws_df 関数を適用します。このとき, チェーン番号 (.chain), チェーンの中でのステップ番号 (.iteration), 全チェーンを通した通し番号 (.draw) という 3 つの隠し変数が付いてきます。

4 行目 tibble 型にします。別にしなくてもいいんですが, 著者の好みです。

5 行目 変数の中から lp__(対数事後確率) と.iteration を除外します。

6 行目 隠し変数のうち 2 つを残して, わかりやすい名前に付け替えます。通し番号.draw を iter に, チェーン番号.chain を chain にします。チェーン番号を残しておくのは, 後で収束の具合 ($\hat{R}$) をチェーンごとに比べて計算するためです。

7 行目 chain と iter の 2 つは残して, それ以外の変数を縦に並べた tidy なデータにし, MCMCsample というオブジェクトに保存します。

8 行目 戻り値として MCMCsample を返します。

rstan を使う場合は, 2 行目を fit %>% as.array() %>% の 2 行に置き換えれば, あとは同じです。伴走サイトには cmdstanr 版と rstan 版の両方を置いてあります。

■MCMCtoDF 関数の結果 出力結果は以下のような形式になります。

R の出力 4.2: MCMC サンプルをデータセットにしたもの

```
# A tibble: 60,000 × 4
   chain  iter name  value
   <int> <int> <chr> <dbl>
 1     1     1 mu1    61.7
 2     1     1 mu2    41.4
 3     1     1 sig    11.5
 4     1     2 mu1    63.1
 5     1     2 mu2    39.7
 6     1     2 sig    17.8
 7     1     3 mu1    62.8
 8     1     3 mu2    39.4
 9     1     3 sig    17.4
10     1     4 mu1    57.1
# i 59,990 more rows
```

ここで chain とあるのはチェーン番号, iter とあるのは MCMC のステップの通し番号, name とあるのが変数名, value とあるのがその値です。tidy な形になっていますから, このまま分析や描画に用いることができます。

MCMC サンプルの結果を要約する関数

MCMC の結果を分析するにあたっては，bayestestR パッケージなど既存のパッケージを使うと便利でしょう。ですが，こうしたパッケージはどんどん開発が進んでアップデートされたり，rstan か cmdstanr かで書き方が変わったりするなど，テキストで紹介するには難しいところがあります。そもそも MCMC サンプルを加工して記述統計を示したり，描画したりするわけですから，上で紹介したように MCMC サンプルを抜き出すことができれば自力でもできるはず。ということで，自作の関数を用意しました。本書では以下，この関数をつかって解説しますので中身を知っておくと良いでしょう。

■MAP 推定関数　確率分布の特徴を報告するときに，その確率分布に従う乱数を使って，期待値や中央値を計算するのは簡単です。記述統計としての平均値やパーセンタイルでよいからです。しかし確率分布の確率密度が最も高くなるところ，すなわち **MAP 推定値 (MAP estimation)** の計算はちょっと難しいですね。というのも，関数であれば微分して極値を求めれば良いのですが，MCMC サンプルは関数ではなくて値なので，ヒストグラムを書くしかなく，最大の値を求める計算がむずかしいからです。

でも大丈夫，R の描画関数を駆使して対応することができます。MAP 推定値を計算する関数は，次のように書きます[*21]

code : 4.5　MAP 推定する関数のコード

```
map_estimation <- function(z) {
  density(z)$x[which.max(density(z)$y)]
}
```

これは R の density 関数でヒストグラムに密度関数を当てがい (カーネル密度推定)，その関数の特徴から値を返すようになっています。

■MCMC サンプルの要約関数　先ほどの MAP 推定する関数を含めつつ，また MCMC サンプルをデータフレームにする関数を使いつつ，MCMC サンプルの要約を表示する関数を次のように作りました。

code : 4.6　MCMC の要約を報告するコード

```
MCMCsummary <- function(MCMCsample, prob = 0.95) {
  lower <- (1 - prob) / 2
  upper <- 1 - lower
  width <- round(prob * 100)
  MCMCsample %>%
    dplyr::arrange(name, chain, iter) %>%
    dplyr::group_by(name) %>%
    dplyr::summarise(
      EAP = mean(value),
      MED = median(value),
      MAP = map_estimation(value),
      SD = sd(value),
      L = quantile(value, prob = lower),
      U = quantile(value, prob = upper),
      Rhat = posterior::rhat(matrix(value, ncol = max(chain)))
    ) %>%
    dplyr::rename(!!paste0("L", width) := L,
```

[*21] このコードは関西学院大学社会学部の清水裕士先生に教えてもらったものです。記して謝意を表します。

```
18                      !!paste0("U", width) := U) %>%
19      dplyr::mutate(dplyr::across(dplyr::where(is.numeric),
20                                  ~ pillar::num(., digits = 3)))
21  }
```

■コード解説

1 行目 関数を作る宣言。1 つ目の引数は MCMCtoDF 関数の戻り値です。2 つ目の引数 prob は確信区間の幅で, 何も指定しなければ 0.95(95% 区間) になります。90% 区間がほしければ prob = 0.90 と指定します。

2 行目 区間の下端にあたるパーセンタイルを計算します。95% 区間なら, 残りの 5% を上下に半分ずつ分けるので, 下端は 0.025 です。

3 行目 区間の上端にあたるパーセンタイルです。95% 区間なら $1 - 0.025 = 0.975$ です。

4 行目 列の名前に使うために, 区間の幅を百分率の整数にします。prob = 0.95 なら 95, prob = 0.90 なら 90 です。

5 行目 引数で引き受けたオブジェクトを加工していきます。

6 行目 変数名, チェーン番号, ステップ番号の順に並べ替えます。15 行目でチェーンごとに区切って計算するための下ごしらえです。

7 行目 変数名でデータをグループ化します。

8–16 行目 変数ごとに記述統計を計算します。9–12 行目は EAP, MED, MAP, SD です。13 行目と 14 行目は 2–3 行目で求めたパーセンタイルの値で, 区間の下端 L と上端 U になります。15 行目の Rhat は収束の目安 $\hat{R}$ で, 6 行目で並べた値をチェーンごとに 1 列ずつの行列 (列の数はチェーン番号の最大値＝チェーンの数) にし, posterior パッケージの rhat 関数に渡しています。1.1 を超えていたら, チェーンどうしの結果が食い違っている (収束していない) と考えます。

17–18 行目 13–14 行目で仮に L と U としておいた列の名前を, 区間の幅に合わせて L95 と U95(90% なら L90 と U90) に付け替えます。rename 関数はふつう rename(**新しい名前 = 古い名前**) と書きますが, ここでは新しい名前が paste0("L", width) という計算の結果なので, そのままでは書けません。そこで 2 つの記号を使います。!!(「バンバン」と読みます) は, うしろの式をその場で計算して, 結果 (たとえば文字列"L95") を差し込むという合図です。また, 左側に計算の結果を置くときは= の代わりに:=を使う決まりになっています。つまりこの 2 行は, rename(L95 = L, U95 = U) と書いたのと同じことを, 幅に応じて自動でやっているのです。

19–20 行目 出力する数値データは, 小数点以下 3 桁まで表示させるようにします。

この関数は, 先ほどの MCMCtoDF とセットで使い, 次のような結果を得ます。

R の出力 4.3: MCMCsummary の結果

```
> fit %>% MCMCtoDF() %>% MCMCsummary()
# A tibble: 3 × 8
  name        EAP       MED       MAP        SD       L95       U95      Rhat
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu1      60.010    60.035    60.575     5.574    48.909    71.017     1.000
2 mu2      39.987    40.036    39.957     5.586    28.828    50.990     1.000
3 sig      15.521    15.055    14.065     3.080    10.843    22.810     1.000
```

ここにあるように，各変数の EAP 推定値，MED 推定値，MAP 推定値，95% 確信区間が表示されます[*22]。

課題などで 95% 以外の区間を求められたときは，prob を指定します。90% 区間なら次のとおりで，区間の列名も L90 と U90 に変わります。

R の出力 4.4: 90% 区間を求める

```
> fit %>% MCMCtoDF() %>% MCMCsummary(prob = 0.90)
# A tibble: 3 × 8
  name        EAP       MED       MAP        SD       L90       U90      Rhat
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu1      60.010    60.035    60.575     5.574    50.969    69.043     1.000
2 mu2      39.987    40.036    39.957     5.586    30.801    49.001     1.000
3 sig      15.521    15.055    14.065     3.080    11.372    21.187     1.000
```

EAP や MAP は変わらず，区間だけが狭くなっていることを確かめてください。

これらの推定値や，そもそも事後分布が何をどのようなことを表しているのか，といった点については今後の授業で説明していきます。以下は結果をしれっとこの関数で表現していきますので，何をやっているのかわからなくなったらここに立ち戻ってきてください。

■警告が出ても慌てない MCMC を実行すると，結果と一緒に英語の警告が出ることがあります。よく見るのは次のようなものです。

```
Warning: 1908 of 20000 (10.0%) transitions ended with a divergence.
```

「発散した遷移があった」という意味です。はじめて見ると，自分のコードが間違っているのではないかと不安になりますが，**これはエラーではなく警告です**。エラーであれば計算は止まりますが，警告は「気になることがあったので念のため伝えておきます」という知らせであって，結果はちゃんと返ってきています。

発散は，事後分布の形が MCMC にとって探索しにくいときに起こります。たとえばこのあと扱う 7 人の科学者のモデルでは，1 人ぶんの測定値 1 点から，その人の誤差の大きさ σ_i を 1 つ推定します。データが 1 点しかないので σ_i の事後分布は裾が長く伸び，そこを歩き回るときに足を踏み外すことがあるのです。モデルの書き方が間違っているわけではありません。

同じように E-BFMI が低いという警告が出ることもあります。これも「事後分布の形が探索しにくい」という同じ種類の知らせです。

本書の範囲では，警告が出ても**そのまま先に進んで構いません**。推定値そのものは安定していて，乱数の種を変えて何度走らせてもほぼ同じ値が返ってきます。警告への対処法，たとえば事前分布を変える，モデルの書き方を変える (再パラメータ化) といった話は，この科目より一段上の内容になります。いまは「そういうものが出ることがある」とだけ知っておいてください。

ただし，同じ警告でも R-hat が 1.1 を超えているという知らせが出たときは別です。これは「複数のチェインが同じところに落ち着いていない」ということなので，**サンプル数を増やすか，モデルを見直す必要があり**

[*22] この区間はコードから明らかなように，quantile 関数で計算しています。下から 2.5%，上から 2.5% を取り除くと 95% の区間が残ることになりますが，このように分布の両端を均等に切った区間を厳密には**等裾区間 (equal-tailed interval; ETI)** というのでした。これに対して，分布の最も信頼できる部分，かつ分布の大部分をカバーし，区間内部のすべての点が，区間外の任意の点よりも高い密度を持っているように推定するものを**最高密度区間 (highest-density interval; HDI)** と呼びます。これらは左右対称の単峰分布であれば同じになるのですが，事後分布の形がぐちゃっとしている時は必ずしもそうなりません。両者の違いについてはクルシュケ (2017)を参考にしてください。また，HDI を出力する関数は bayestestR::hdi() です。

ます。こちらは無視しないでください。

第 5 章

ベイジアンアプローチと確率的プログラミング 1

それではいよいよ，**データ生成モデリング**とベイズ推定による実践例を見ていくことにしましょう。

5.1　7 人の科学者

ここでは リー・ワゲンメーカーズ (2017)より「7 人の科学者」の例を紹介します。そこでのカバーストーリーは次のようなものです。

> 実験スキルが大きく異なる 7 人の科学者が，全員同じ量について測定を行う。彼らが得た数値結果は次の通り。
>
> $$Y = \{-27.020, 3.570, 8.191, 9.898, 9.603, 9.945, 10.056\}$$
>
> 直感的には，最初の二人の科学者はひどく適性を欠いた測定者であり，この量の真の値はおそらく 10 をわずかに下回るくらいであるように思えるのだが･･･？？

さあ，これをみて「あれ，どこが統計的な問題なんだ？」と思った人もいるかもしれません。なんらかの測定をして，データのばらつきがあるんだけど，それが測定者によって違うらしい，というのはわかったかと思いますが，これをどうやって統計的な問いにするのでしょうか。まず，ストーリーの背後に「正確な測定値 (真の値) はわからないけど，定数のはず」という前提があることを確認しましょう。測定の基本として測定誤差があるので真の値を特定出来はしませんが，測定を繰り返すと真の値に近づいていくはずではあります。また，今回の測定結果は 7 人それぞれ別の人による測定であることから，測定誤差の出方が測定毎すなわち測定者毎に異なるだろう，という仮説もあります。

これらを踏まえて謎解きをしていきましょう。わからない量を確率の言葉で表現し，データを使って少しでも正解に近づこうとするのが統計のやり方です。まずここでわからない量は，「正確な測定値」と「個人毎の誤差の大きさ」です。誤差は正規分布に従うと思われますから，データ Y は $Y \sim N(\mu, \sigma)$ と表すことができます。データ生成メカニズムという意味では，「正規分布に従ってデータが作られる」と言ってもいいかもしれません。また，正確な測定値はここでは μ ということになります[*1]。また，データは 7 点あって，それぞれ $Y_1, Y_2, \cdots, Y_7$，あるいは一般に Y_i と書くことにしましょう。ここで添字の i は第 i 回目の測定でもありますし，測定者 i のことでもあります。この測定者 i 毎に誤差の出方の大きさが変わります。誤差の大小，言い換

[*1] 正確な測定値 μ に誤差 $N(0, \sigma)$ がついてデータになる，すなわち $Y = \mu + N(0, \sigma)$ ということですが，誤差の平均は 0 で μ の分だけ正規分布がズレる，そして「分布に従う」を表現する $\sim$ を使いたいので，$Y \sim N(\mu, \sigma)$ となるのです。

1442 えれば測定の精度は誤差の SD で表現されていますから，σ が個人毎に変わる，すなわち σ_i と書くことができます。これが今回のデータ生成モデルです！

1444 ここまでのことをイメージ図であらわすと図 5.1 のようになります。この図の書き方のポイントは，まずデータを一番下に置くことです。次にこのデータがどこから来たのかな，ということを矢印で表現して書きます。データは誤差を伴うなどで，毎回定数にならないでしょうから，**確率分布**をその上に書くことになります。確率分布にはその形状を定めるパラメータがあるはずですから，それを書き加えれば OK です。

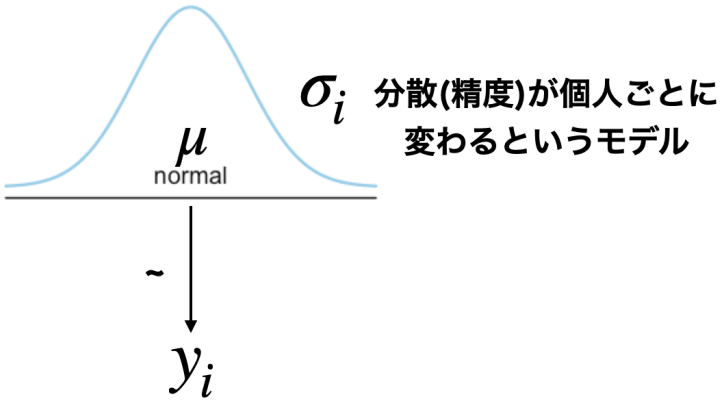


図 5.1 データが生成されるプロセス

1448 しかしここでも，μ と σ_i は結局なんなんだ，どういう数字なんだということがわからないままです。わからない量は確率の言葉で表現するのがベイジアンの生き様。ここは未知のパラメータがどうやって出てくるかなんてわからないところなので，「わからない」ことを確率で表現する必要があります。パラメータに対して「この辺りにあるだろう」と事前に想定する確率分布のことを**事前分布 (prior distribution)** というのでした。

1452 ここでは μ は平均 0，SD100 の正規分布からきていることにしましょう。正規分布というのは左右対称で平均値・中央値・最頻値が一致する単峰の分布です。これは正確な測定値を表しているのでしたから，なんらかの値を取るとしても 1 つの値だろう，というのは無理のない仮定でしょう。複数の値を取る可能性があるのなら多峰性の分布を仮定したらいいと思いますが，今回はそうではないだろう，と仮定するのです。平均 0 で SD100 ということは，-300 から +300 の範囲にある可能性が 99.7% だということでもあります[*2]。実際のデータが -27 から 10 ぐらいの範囲に入っているので，-300 から $+300$ の間というのも十分過ぎるぐらい広めの範囲にしてある数字と言えるでしょう。このように，うっすら情報を持っているけどほとんど意味がない程度に縛りをかけるのを，**弱情報事前分布 (weakly informative prior distribution)** と言います。たとえばこれが身長のデータであれば，$\pm$3m の範囲は十分すぎ，なんなら負の数は取るはずがないので過剰な安全策を置いているとも言えます。それでも「なんらかの事前分布を持つのは，主観的な思い込みだ」というのであれば，$\pm\infty$ の一様分布を考えるか，ベイズ法をやめるかということになります。

1463 次に σ_i です。これは標準偏差パラメータなので負の数を取ることはあり得ません。ここでは**半コーシー分布 (half-Cauchy distribution)** を事前分布におくことにします。コーシー分布とは正規分布によくにた形で

[*2] 正規分布は，平均 $\pm$1SD の範囲に全体の 68% が，$\pm$2SD の範囲に 95% が，$\pm$3SD の範囲に 99.7 が入ることが理論的にわかっているのでした。わからない人は R で計算して確認すること！

すが, 正規分布より裾が重く[*3], 標準偏差 (分散) の事前分布に適していると言われている分布です Gelman (2006)[*4]。このコーシー分布は正規分布のように左右対称ですが，0 を中心に半分に折りたたんで正の値しか取らないように考えたのが半コーシー分布です。今回はスケールパラメータに 5 を置いていますが, とくにこの数字に深い意味はありません。気になるようでしたら 10 でも 100 でも変えていただいて結構です[*5]。

これらをおくことで, モデルの設計図ができあがりました (図 5.2)。あとはこれをもとに, **確率的プログラミング言語 (probabilistic programming language)** に書き起こしていけば良いでしょう。

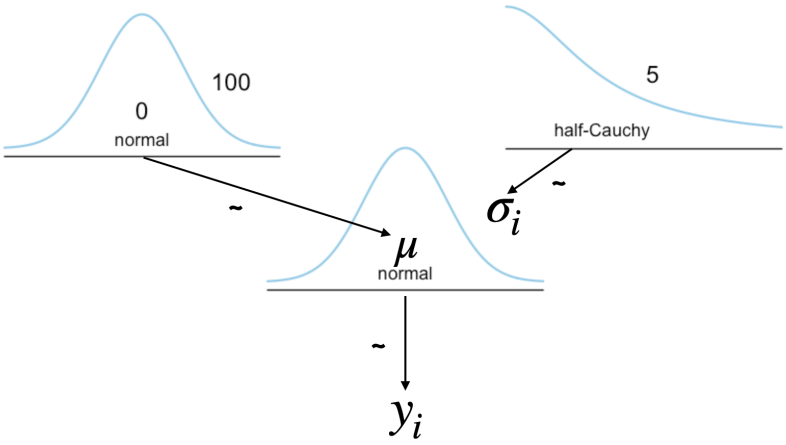


図 5.2　モデルの全体像

5.2　Stan コードの書き方

では図 5.2 という設計図に基づいて，Stan のコードを書いていきましょう。

5.2.1　Stan コードの文法

Stan は事後分布からの乱数を生成するための言語です。その言語仕様は C や C++ ににているところがあります。R で書くのと違うところとして「ブロック単位の記述」「セミコロンによる行の区切り」「変数の宣言」の 3 つを押さえておきましょう。

■ブロック単位の記述　Stan に書くべきことは**ブロック**の中に記述します。ブロックは中括弧 ({ ... }) で括ったもので, 次の 6 つの種類があります。

1. `data` ブロック
2. `transformed data` ブロック
3. `parameters` ブロック
4. `transformed parameters` ブロック

[*3] より極端な値が出る可能性が多いという意味です
[*4] この論文では他にも，student の t 分布などを提案していますが, パラメータ数がより少ないコーシー分布を選びました。
[*5] 半コーシー分布がどんな数字になるのか気になる, という人は R で `rcauchy` 関数を使って乱数を発生させ, その挙動を見てみればいいでしょう。乱数発生によって大体の分布感 (分布の感覚?) を得ることができる, ということを思い出してください。

5. `model` ブロック
6. `generated quantities` ブロック

Stan のプログラムに含まれるブロックは，この順番でなければならないと決まっているので注意してください。この中で，1,3,5 番目，すなわち `data`,`parameters`,`model` ブロックが基本的に必要なもので，2.4.6 番目のブロックはその派生や応用です。まずは基本ブロックから説明します。

`data` ブロックは Stan と外部とのやりとりをするブロックです。外部からデータを取り込んで，それに基づいて乱数を発生させます。Stan で計算せずに外部から与えるものはすべてこのブロックの中で書く必要があります。`parameters` ブロックは，パラメータ，すなわち Stan で推定したいものを書くところです。Stan は事後分布からの乱数発生機であり，事後分布は**確率分布**ですからその形状はパラメータによって特徴づけられます。事後分布の形が分からないというのは，事後分布のパラメータがどのような値になっているのか分からない，ということと同じ意味です。その「分からない」「知りたい」パラメータをリストアップしておくブロックです。`model` ブロックは，確率モデルすなわち尤度と事前分布を書くところです。ここで書かれた尤度と事前分布が，`data` ブロックのデータと組み合わさって，事後分布が Stan 内部で計算され，そこからの乱数が次々取り出されてくることになるのです。

■セミコロンによる行の区切り このブロックの中に，Stan 言語の文を書いていくことになるのですが，ここで 1 つ覚えておいて欲しいのが，**Stan の句点はセミコロン**ということです。句点がない文章は読みにくいどころか，機械にとっては意味不明な文字列になりかねませんので，必ず 1 つの命令・一文がおわったらセミコロンで閉じるようにしてください。これを忘れてエラーになるのが初心者にもっとも多いミスのひとつです。

■変数の宣言 たとえば R では数字でも分析結果でも，オブジェクトに代入でき，オブジェクト名は任意につけることができました。またオブジェクトはいつどのタイミングで発生させても OK で，思いついたらすぐオブジェクトに代入，ということができました。Stan をはじめ，いくつかの高級言語ではこれに厳格な制限をかけるものがあります。すなわち，どんなオブジェクトを作るかを最初に**宣言 (declaration)** する必要があるのです。Stan では使われるオブジェクト (変数) が，どういう数字になるものなのか，どの範囲の数字になるのかをブロックの最初に宣言します。この数字の種類のことを**型 (type)** といい，その宣言された数字の型に適合しない数字はエラーとなって弾かれます。なんでそんな窮屈な，と思うかもしれませんが，逆に「ありえない数字は許さない」という意味で，エラーを未然に防いでくれることにもなるのです。

変数の形として，Stan は整数 (**int 型**)，実数 (**real 型**)，ベクトル (**vector 型**)，行列 (**matrix 型**) などがあり，また範囲を `lower` や `upper` で指定します。たとえば標準偏差は負の数を取らないので，標準偏差の変数を宣言したいときは，`real<lower=0> sig;` のように書きます。

5.2.2 Stan コードを書いてみよう

されこれらを踏まえてコードを書いてみるわけですが，設計図を書いたときにデータを下に，それに紐づく確率分布を上に上に，と下から上に書いていったように，`model` ブロック →`parameters` ブロック →`data` ブロック，と書き進んでいったほうがわかりやすいでしょう。

Stan のコードはメモ帳など，エディタで開くことのできるテキストファイルでいいのですが，拡張子を `.stan` にしておきましょう。RStudio で書く場合は，`.stan` にしておくと Stan のファイルだと認識してくれるので，ブロックや型宣言の用語などを強調表示してくれるようになります。RStudio でコードを書き始めるときは，ファイル (File) ＞新しいファイル (New File) と進むと Stan というファイルがあります。これを選ぶとすでに Stan のサンプルコードが書かれているファイルが出てきますが，そのファイルの中身は全部消して，次のコード 5.1

のように書き，ファイル名をつけて保存しましょう。ここでは sevenScientist.stan とファイル名をつけたものとします。

code : 5.1 7 人の科学者コード

```
1  data{
2    array[7] real Y;
3  }
4
5  parameters{
6    real mu;
7    array[7] real<lower=0> sig;
8  }
9
10 model{
11   for(i in 1:7){
12     //likelihood
13     Y[i] ~ normal(mu,sig[i]);
14     //prior
15     sig[i] ~ cauchy(0,5);
16   }
17   //prior
18   mu ~ normal(0,100);
19 }
```

■コード解説

model ブロック 尤度と事前分布を書くところ。確率分布からのサンプリングは変数を左に，～で**確率分布**につなぎます。normal とか cauchy が確率分布の名前なので RStudio ではハイライト表示されているはず。一行の終わりはセミコロンで。複数のデータや変数，Y_i や σ_i は，i ごとに変わるということをあらわしていますが，プログラム的には Y[i] のように書いて，大括弧で変数を括って添字であることを表現します。データは 7 件あって，それぞれについて i さんの測定値 Y_i と測定誤差 σ_i があるので，for 文で i を繰り返しています。変数 sig[i] についての事前分布も同様に繰り返しています。ちなみに変数名 mu，sig，Y などは任意で，予約語でなければ好きな名前にしてもらって構いません。

parameters ブロック model ブロックで，変数 mu や sig という任意に命名して使っていたわけですが，それは何なのかを宣言してやらねばなりません。これらは求めたいパラメータなのですから，このブロックに型と共に宣言するわけです。変数 mu は実数型，変数 sig も実数型ですが 7 つの要素を持つ配列，しかも負の数を取らないので下限はゼロですよ，という宣言をしているところになります。範囲の設定に<>の記号を使っていること注意してください。配列については次の data ブロックで説明します。

data ブロック 最後にデータブロックです。モデルの変数でもパラメータでもない，外部から与えられるものなので，変数名 Y で宣言します。宣言のときに 1 つの変数に複数の要素がある (**配列**といいます) ことを明示するため，array[size] と書きます。この大括弧 ([ ]) の中身は整数で，今回は 7 件のデータなのでサイズは 7 としてあります。そしてその配列の数字が整数 (Integer) なのか，実数 (Real) なのか，といった型宣言をしたうえで，変数名を書きます[*6]。

[*6] Stan の以前のバージョンでは Y[i] や sig[i] のように，大括弧 ([ ]) を後ろにつけて書いていました。しかしバージョン 2.32.0

いかがでしょうか。書き方 (文法) がわかると比較的単純，あるいは率直な書き方をしていることがお分かりいただけるかと思います。尤度と事前分布とデータを書くだけで，Stan は事後分布からの乱数を生成してくれます。確率密度関数を書いたり解いたりしなくても，結果の近似計算ができるというのはとても便利なことではないでしょうか。

5.3 Stan を使った MCMC の実践

さて Stan ファイルが準備できましたから，これはいったん保存しておいて，今度はこれを R の方から利用してやることになります。R からは cmdstanr パッケージを使って，Stan を呼び出して結果を返してもらう，という流れです。Stan の言語は一度機械語に翻訳 (コンパイル) されるので，その間はしばらく待つ必要がありますが，それが終わると Stan から事後分布の乱数が次々生成され，R に返ってきます。R はそれら乱数からのサンプルを事後分布の代表値からなるデータセットとして扱って，記述統計や可視化を通じて確率分布の解釈に使うのですね。

それではどのように R から呼び出すのか，実際のコードを見てみましょう。共通する準備の部分のあとに，呼び出しのコードを示します。

■共通部分 環境をクリアしたり，必要なパッケージを読み込んだりする部分は毎回共通です。今回は次の 5 行を共通部分として読み込んでおいてください (code5.2)。

code : 5.2 環境の準備と共通部分

```
1  rm(list = ls())
2  library(tidyverse)
3  library(bayesplot)
4  # データ
5  x <- c(-27.020, 3.570, 8.191, 9.898, 9.603, 9.945, 10.056)
```

■cmdstanr パッケージによる呼び出し その上でさっそく cmdstanr パッケージを使って，Stan を呼び出し事後乱数生成機を作ってみましょう。これは次のようにコードを書きます (code5.3)。

code : 5.3 cmdstanr パッケージによる呼び出し

```
1  library(cmdstanr)
2  model <- cmdstan_model("sevenScientist.stan")
3  fit1cmdstan <- model$sample(
4    data = list(Y = x),
5    chains = 4,
6    parallel_chains = 4
7  )
8  sample <- fit1cmdstan$draws() %>%
9    posterior::as_draws_df() %>%
10   as_tibble()
```

■コード解説

1 行目 パッケージの呼び出し

はその表記は誤りとされ，ここで解説しているような array 表記がスタンダードになります。古い記法のままだと警告が出たり，RStudio も古いバージョンだと新しい記法に変更要請 (赤い波線がコードについたり) が出たりしますが，今すぐ問題になるというものではありません。とはいえ，なるべく新しいバージョンを使ったほうがよいので，表記法も新しいものに心がけましょう。

2 行目 モデルのコンパイルを実行して `model` オブジェクトに入れます。コンパイルには少し時間がかかります。

4-8 行目 乱数発生。ここで R のデータを Stan に渡したり，並列化の指定をしたりします。実行すると画面に色々表示されるかと思います。

9-11 行目 得られた乱数をデータセットに整形。まず `posterior::as_draws_df` で MCMC サンプルだけを取り出し，`as_tibble` で tibble 型に変換します。

コンパイル → サンプリング，という流れになります。

またサンプリングの際に，R からデータを渡すことになりますが，データは R 環境では `x` というオブジェクト名だったものを，Stan の `data` ブロックで宣言した `Y` という変数にして渡すのだ，ということが書かれています。データは複数のものを与えることがあるので，`list` 関数で並べて渡すことにします。そのほかの設定については，順次説明します。

最終的に `tibble` 型でえられた MCMC サンプルのデータセットができあがります。これはパッケージによって少し変数名が異なりますが，内容は同じで，今回の設定だと 4000 行のデータになっているはずです。つまり事後分布からの乱数を 4000 個発生させたことになります。

そしてこの各行が 1 回のサンプリングで得られた**事後分布からの代表値**です。一部を見てみましょう (出力 5.1)。1 回目のサンプリングで，$9.98, 23.4, 4.63, 1.13, 0.123, 5.17, 0.0264, 1.24, -9.10$ という数字のセットがあります。変数名にあるように，前から $\mu, \sigma_1, \sigma_2, \cdots, \sigma_7$ の代表値と，`lp__`の値が得られています。最後の `lp__`は**対数尤度 (log posterior)** の略で，ここでは正確には事後対数尤度と呼ばれるものです。今回の代表値から計算された尤度の対数ということで，パラメータの推定値とは違いますのでここでは気にしないことにしましょう。ということで改めて，1 回目のサンプリングで $9.98, 23.4, 4.63, 1.13, 0.123, 5.17, 0.0264, 1.24$ という数字のセットが得られました。このように，今回求めたかったパラメータは 8 つあったわけですが，その 8 次元からなる空間の 1 つの座標点を取り出した 1 回目，ということです。2 回目は $9.90, 27.2, 13.1, 2.86, 1.23, 0.779, 0.0449, 0.446$ というセットでした。以下 3,4,.. と続き，4000 サンプルを得たわけです。このように結果は 8 次元からなるパラメータの**同時確率空間 (joint probability space)** から得られており，一行一行が 1 回 1 回のサンプリングステップなのです。

R の出力 5.1: MCMC サンプルの結果 (ごく一部)

```
      # A tibble: 4,000 × 9
      mu sig.1 sig.2 sig.3 sig.4 sig.5   sig.6 sig.7   lp__
    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl> <dbl>  <dbl>
  1  9.98  23.4  4.63  1.13 0.123 5.17   0.0264 1.24   -9.10
  2  9.90  27.2 13.1   2.86 1.23  0.779  0.0449 0.446  -7.72
  3  9.99  20.9  5.12  1.11 0.132 4.48   0.0397 0.937  -8.97
```

5.4 MCMC 結果の診断

さて，MCMC が実行できましたが，その中身を少しみてみましょう。サンプリング結果を代入したオブジェクト名をそのまま入力すると，要約された結果が示されます。

cmdstanr の出力 5.1: MCMC の結果出力 (cmdstanr パッケージの場合)

```
variable  mean median     sd   mad     q5    q95 rhat ess_bulk ess_tail
  lp__   -9.70  -9.35   1.91  1.69 -13.28  -7.30 1.00      914      1694
  mu      9.64   9.81   0.67  0.36   8.28  10.36 1.01      879      1101
  sig[1] 54.19  31.91 105.83 16.95  15.41 142.60 1.01      663       215
  sig[2] 10.48   6.71  20.30  3.96   2.88  25.82 1.00     1272       935
  sig[3]  4.30   2.76   6.19  2.01   0.67  12.16 1.01     1281      1104
  sig[4]  2.52   1.31   3.88  1.47   0.11   8.58 1.01      603       252
  sig[5]  2.68   1.37   5.65  1.39   0.19   9.02 1.00     1291      1535
  sig[6]  2.66   1.26   5.11  1.47   0.09   9.46 1.01      492       312
  sig[7]  2.87   1.42   5.64  1.56   0.15   9.49 1.01      904      1049
```

ここで確認して欲しいのは，**Rhat** です。これは「複数チェインの収束の程度」を表しています。まず MCMC によってサンプルを得る方法は乱数だったことを思い出してください。乱数ですから毎回同じ数字からスタートするわけではありません。そしてある乱数は次の数字を生むステップになっており，一歩一歩，事後分布の形に近づいていくものです。しかし違う数字からスタートしても，目標とする事後分布は同じはずですから，最終的にはこのステップの鎖はよく絡まったもの，つまりどこからスタートしても同じあたりをウロウロするものになるはずです。ですから，MCMC がきちんとできているかどうか，事後分布を正しく代表した数字になっているかどうかを確認する必要があります。この **Rhat** という数字は，1.0 であればピッタリ一致という数字で，1.1 よりも小さければ十分「絡まっている」と判断できます。今回はいずれも十分な収束が得られたと言えるでしょう。

このことを視覚的に確認することもできます。**トレースプロット (trace plot)** というのがそれで，MCMC のステップを可視化し，十分に絡まっていることを見ておくのです (図 5.3)。

ちなみにトレースプロットの描画のコードは,code5.4 の通りです。`cmdstanr` で推定したものを使う場合は，`bayesplot` パッケージの力を借りて描画します。

code : 5.4 トレースプロットの描画

```
1  fit1cmdstan$draws("mu") %>% bayesplot::mcmc_trace()
```

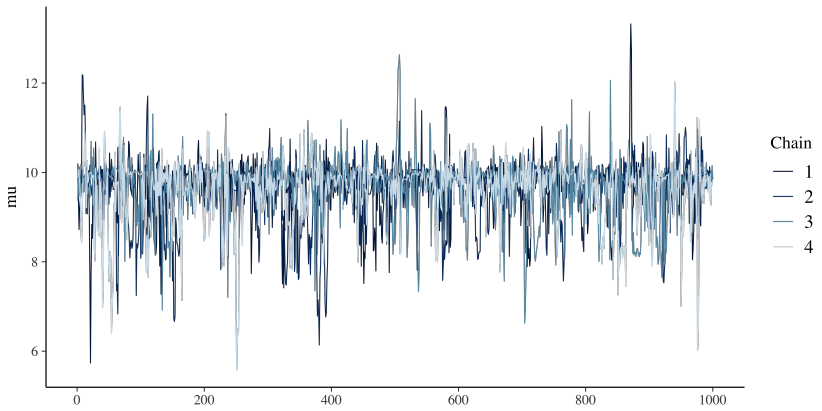


図 5.3 トレースプロットの例

これについては悪い例を見た方が早いかもしれません。図 5.4 に，チェインが絡まない場合のトレースプ

ロットを示しました。このように，複数のチェインがそれぞれバラバラ，迷子になっている場合は，手元に生成された乱数が事後分布の適切な代表になっていないと判断することになります。

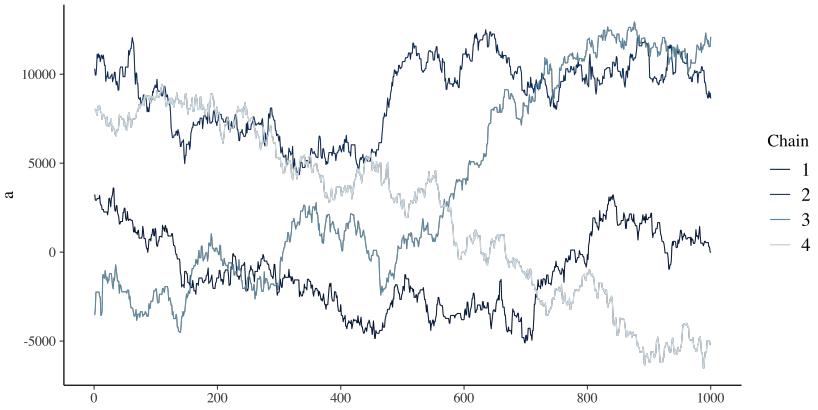


図 5.4　チェインが絡まない場合

次に，**有効サンプルサイズ (effective sample size)** を見ておきましょう。出力の `ess_bulk` や `ess_tail` がそれです。これは MCMC サンプルがうまく取れたかどうかを診断した結果で，ここが少なくとも 3 桁ほどなければ不十分なのだ，と考えます。いくらあったらいいかについては，どれだけ MCMC サンプルを発生させたかに依存するので一概には言えませんが，少なくとも効果的に得られているのが 100 サンプルぐらいないとダメなんだ，ということです。

ちなみに，MCMC サンプルのチェイン (発生させた系列数) や，MCMC サンプルの数は，関数のオプションで指定します。今回は指定なしだったので，デフォルトの値で 4 チェイン，各々 1000 サンプルずつ作るようになっていました。これらを明示的に設定するには次のようにします (code5.5)。

code : 5.5　cmdstanr パッケージによるオプション指定

```
fit1cmdstan <- model$sample(
  data = list(Y = x),
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 4000
)
```

ここにある `chains` というのが生成する MCMC サンプルの連鎖系列の数です。「どこからスタートしても同じ事後分布にたどり着く」ことを検証するために，少なくとも複数のサンプルが必要で，デフォルトでは 4 本のチェインを作っていることになります。`cmdstanr` パッケージでは，並列化する本数も別途 `parallel_chains` で指定します。

また生成する乱数の数ですが，ここに**ウォームアップ (warmup)** という単語が出てきています。これは MCMC サンプルの際，最初の数ステップは調整のための準備運動期間中なので，この期間に得られた MCMC サンプルは代表値として使わない，という意味です。`cmdstanr` パッケージの場合，捨てるステップ数とサンプルするステップ数を明示的に指定していますからわかりやすいですね。最終的に得られる MCMC サンプルの数は $4000 \times 4 = 16000$ で計算できます。`rstan` パッケージを使った本では，全体の反復 (iter) 回数からウォームアップの分を引き算する数え方をしていますので，読み替えるときに気をつけてください。

5.5 MCMC の結果の解釈

さあ長々と話をしてきましたが，どうやら Rhat や有効サンプルサイズにも問題がなかったので，MCMC は事後分布からの適切な代表値を産んでいたと考えられますから，その結果を考えてみましょうか。

ここで，前の章 (セクション 4.5.3,Pp.51) で準備した関数を使ってみましょう[*7]。これらをコード 5.6 のように使うと結果が得られます。私の結果では出力 5.1 のようになりました。

code : 5.6 stan の結果を準備し関数に与える

```
1  fit1cmdstan %>% MCMCtoDF() %>% MCMCsummary()
```

MCMC の結果 5.1

```
# A tibble: 8 × 8
  name          EAP       MED       MAP        SD       L95       U95      Rhat
  <chr>   <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu          9.646     9.827     9.927     0.675     7.925    10.540     1.005
2 sig[1]     44.656    30.128    27.319    64.348    13.698   158.406     1.004
3 sig[2]      9.596     6.496     4.766    12.541     2.588    33.265     1.004
4 sig[3]      4.535     2.687     1.902     7.448     0.513    18.787     1.006
5 sig[4]      2.657     1.169     0.520     6.279     0.070    13.439     1.018
6 sig[5]      2.693     1.459     0.396     5.314     0.098    12.649     1.003
7 sig[6]      2.607     1.148     0.331     7.322     0.021    12.972     1.047
8 sig[7]      2.802     1.301     0.668     6.456     0.098    13.997     1.006
```

ここでは**事後確率最大値 (maximum a posteriori)** を見ることにしましょう。これによると，μ すなわち真の測定値は 9.927 ぐらいじゃないか，という推測がなされています。10 より少し小さいくらいではないか，と思っていたようですが，こうして計算してみると確かにそれぐらいのようですね。そして 7 人それぞれの測定精度が σ_i として算出されています。これをみると，最初の測定者は 27.319，2 番目の人は 4.766 になっています。これが大きいということは，誤差の幅広さを表しているのですから，測定精度が悪いということを意味します。確かに最初の二人は非常に精度が悪く，4 番目から 7 番目のベテラン測定者が 0.3 から 0.7 ぐらいの誤差であることに比べれば，圧倒的に悪いということが言えますね。

もちろんこれは点推定値で，真の測定値が 9.927 に違いないとか，最初の人の測定精度が 27.319 だと断言すると，おそらくほぼ確実に外れた予測ということになるでしょう。そこで幅を持った予測，すなわち**区間推定 (interval estimation)** を行えば良いのです。区間を考えれば，真の測定値は 7.925 から 10.540 の範囲にあるだろうとか，最初の測定者の精度も最善なら 13.698，最悪 158.406 ぐらいである可能性があるわけです。

ベイズ推定のポイントとして，これらは事後分布，すなわち**確率分布による予測**であるということが挙げられます。真の測定値 μ のありそうな範囲が 7.925 から 10.540 にある確率が 95% だ，という言い方ができるところです。**モーメント法**による**信頼区間 (confidence interval)** とは違い，ベイズ法の予測は**確信区間 (credible intervals)** という言い方になります[*8]。

[*7] 伴走サイト https://stats.kosugitti.net/ からは，これらの R コードをダウンロードできます。Stan ファイルは自分で準備し，読み込み先やファイル名などは適宜変更してください。

[*8] 信頼区間の場合は，ここにあると予測したとして，その予測が当たる確率を表しています。すなわち当たるか外れるか，0/1 の話をしているのであって，この「範囲に存在する」という大きさ・幅の話をしているわけではないことに注意です。

いかがでしたでしょうか。今回はモデリングとベイズ推測の実際，Stan コードの書き方を実践してみました。この「7 人の科学者」がおもしろい点は，少ないサンプル (たった 7 件のデータ！) で推測ができること，簡単なコードで検証できること，というのもありますが，なにより標準偏差 (分散) を検証対象とすることにあると私は思います。心理学の実験のほとんどが，平均の比較，操作の効果の話になっているのですが，データの散らばりというのも測定精度のように心理学的に意味のあるものとして考えられ，検証の対象にしたっていいのです。もちろん正規分布でなくてもいいですし，さまざまな分布のさまざまなパラメータに意味があるなら，それをわからないものとして検証しようと言える，というのはとても可能性が広がる話ではないでしょうか。

5.6 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；8 人目の科学者 本講で扱った 7 人の科学者の問題を拡張します。8 人目の科学者が現れて，測定値 18.25 だったと報告してきました。

1. この人のデータも含めた分析ができるように，Stan コードと R コードを書き換えてください。Stan コードには，母数として `mu` と `sig` を宣言すること。
2. 書き換えたコードで MCMC 推定を実行し，`mu` について，EAP 推定値，MAP 推定値，90% 確信区間の下限と上限を報告してください。`ans2` の要素名は `mu_eap`，`mu_map`，`mu_lower`，`mu_upper` とすること。MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. 8 人の測定値の単純平均を計算し，小問 2 で求めた `mu` の EAP 推定値と比べてください。あわせて 7 人だったときについても同じ 2 つを求め，8 人目が加わったことで単純平均とモデルの推定値がそれぞれどれだけ動いたかを数値で示してください。`ans3` の要素名は `simple7`，`simple8`，`mu_eap7`，`mu_eap8` とすること。

第 6 章

モデリングの目から見た検定 1；二群の平均値の差

前回はデータ生成メカニズムという観点で考えることで，標準偏差 (分散) を「測定精度」と考えた検討ができることを示しました。今回は，心理学でもよく使われている平均値の差を検討対象とするモデルを考えることにします。

皆さんは帰無仮説検定のことを覚えているでしょうか。正規分布を仮定した母集団からの標本統計量は，正規分布に従うことを利用して，母平均の区間推定を行い，群間に差があるかないかといった判断を確率的に行うのが帰無仮説検定でした。心理学では要因計画と帰無仮説検定が合体し，群間の差の形でデータが得られるようにすることで結論を導き，考察を重ねてきました。また帰無仮説検定という手法は，誤用や誤解が多く，ひどい時には悪用もされるということについてもみてきたと思います。ところで，帰無仮説検定の考え方はデータが得られたところから考え始めるデータ駆動型の分析モデルでした。ではデータがどのように出てきているのかを考える，**データ生成モデリング**の考え方をつかうと，この平均値の差についての検討はどのように姿を変えるのでしょうか。

ここでは **t 検定**を例に話を進めていきたいと思います。

6.1 t 検定の過程と実際

t 検定は，二群の平均値の差を比較し，結論を下すという最も典型的な帰無仮説検定の例の 1 つです。とくに独立した二群の検定は，最も単純な要因計画 (一要因二水準 Between デザイン) ということができるでしょう。これについて，よく思い出せない人はデータ解析基礎の資料や，山田・村井 (2004)，清水 (2021) などを読んで，分析の流れや仮定をもう一度確認しておいて欲しいと思います。

非常に駆け足ながら概略を説明してみますと，次のようになります。

1. 標本の母集団が正規分布していると考える。母集団から無作為に選ばれた標本が二群あるとする。
2. 二群の一方に何らかの処置を加え，他方には何もしない (あるいは検討したい処置と同等の効果のない処置を施す[*1])。前者を実験群，後者を統制群という。データ (標本) は正規分布するはずだから，その平均値を見ることで誤差や個人差は相殺される。つまり群間の平均値の差が効果の大きさであるということができる。

[*1] たとえば「単語リストは声に出して覚えたほうが記憶の定着度が高い」ということを検証したい場合，声に出す群と出さない群で比較することになりますが，声に出さないことが (頭の中で反芻するなど) 従属変数に与える別の効果を持っていることがあるので，音のない映像を見せるなどして効果のない同等の処置をした群を比較対象にする，ということを考えたりするわけです。

3. ところで標本平均値などの**標本統計量**も正規分布に従うことがわかっている。母集団が $N(\mu, \sigma)$ であれば, 標本平均は $N(\mu, \frac{\sigma}{\sqrt{n}})$ に従う (ここで n はサンプルサイズ)。
4. 標本平均が従う分布 (散らばりの位置と幅) が分かれば, 母平均がどのあたりにあるのかは区間推定することが可能。実験群・統制群ともに母平均の値を推定する。これが異なっていれば標本を超えて母集団で効果があった, と結果を一般化できる。もちろん点推定値は母平均の値とピッタリ同じとは思えないし, 標本平均もピッタリ同じになるはずがないから, 点推定値ではなく区間推定で考えたい。
5. ところで標本平均の従う分布の中には, σ つまり母 SD がパラメータとして入っている。母数がわからないという前提のもとでは, どの程度の幅で散らばるのかがわからないことと同じ。そこで σ の代わりに $\hat{\sigma}$ を用いて推定することを考える。この場合, 標本平均は正規分布ではなく t 分布に従うことがわかっている。
6. t 分布を使った区間推定をしても, 差があるのかないのか判断するために何らかの基準が必要である。そこで「差がない」という主張と「差がないとは言えない」という主張を戦わせて, 判定すると言う形式を取る。前者の主張を**帰無仮説**, 後者の主張を**対立仮説**という。また判断も確率的になるので, 勝敗を決める基準を 5% とする。この確率は**有意水準**とか**危険率**と呼ばれる。
7. データから t 分布に従う統計量を計算し, その値が出てくる確率を算出する。この確率は p **値**と呼ばれる。これが有意水準よりも小さいようであれば, 帰無仮説の仮定の下で算出した数字が滅多に生じ得ないことを意味するから, 帰無仮説が間違っていたのだと判断してこれを**棄却**し, 対立仮説を**採択**する。

さて, この二群の平均値を検定することの流れですが, とくに「帰無仮説のもとでの t 値を算出して判定する」というあたりがややこしかったかもしれません。数式で書くと嫌われそうですが, 帰無仮説というのは二群の平均値に差がない, という仮定だったので, 実験群から考えられる母平均 μ_A と統制群から考えられる母平均 μ_B に差がない, つまり $\mu_A = \mu_B$, からの $\mu_A - \mu_B = 0$ という位置母数が 0 の理論分布のを考えている, というところがポイントです。検定統計量である t 値は次のように計算できるのでした。

$$t = \frac{\bar{X}_A - \bar{X}_B}{\sqrt{\frac{(n_1-1)\hat{\sigma_A}^2+(n_2-1)\hat{\sigma_B}^2}{n_1+n_2-2}(\frac{1}{n_1}+\frac{1}{n_2})}}$$

とっても複雑な式に見えますが, 複雑そうなのは分母で, 分子は標本平均の差を表しているに過ぎません。参照する t 分布は, 標準正規分布が $N(0,1)$ だったように, 位置と幅のパラメータを持ち, $t(0, df)$ で考えられる分布にこの統計量を照らし合わせると, 差 0 で自由度に応じて変わる幅 df の分布から「どの程度極端な値が出てきたのか」の確率を計算できるわけです。ちなみに分母は, 母分散ではなく標本から計算される分散 s^2 からバイアスを除いた不偏推定量 $\hat{\sigma}^2$ を使っています。この計算はサンプルサイズ n ではなく $n-1$ で割ることによって計算されるのですが, 2 つの群それぞれについて $n_j - 1$ で割ったものを足し合わせるために, いったん $n_j - 1$ 倍して二群のサンプルサイズ -2 で割り直す, という作業をしているため, 分母がとくに複雑に見えているだけです。

ともあれ, こうして検定するんだったという流れを思い出したところで, データ生成モデリングの観点からこれを考え直してみましょう。仮定としておいてあるのは, 両群ともに同一の正規分布から得られた標本であり, 操作によってその平均値が異なっているはず, ということだけです。つまり実験群のデータは $X_{i,A} \sim N(\mu_A, \sigma)$, 統制群のデータは $X_{i,B} \sim N(\mu_B, \sigma)$ という分布が前提とされているのです。

これをそのまま設計図にし, コードにしてみましょう。設計図として図 6.1 が, これをもとにコードが `code:6.1` のようにかけていればいいでしょう。コードが設計図をほぼそのまま文字起こししていることを確認してください。

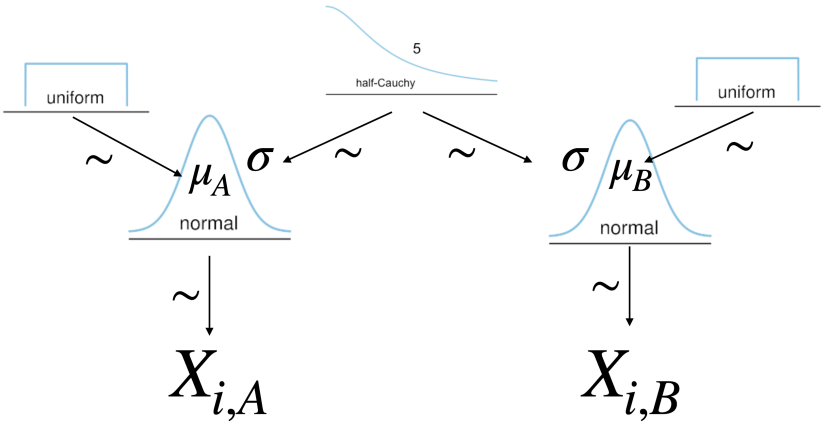


図 6.1　平均値の差を比べるときの設計図

code : 6.1　二群の平均値のコード

```
1  data{
2      int<lower=0> N1; // Number of Subjects in Group 1
3      int<lower=0> N2; // Number of Subjects in Group 2
4      array[N1] real X1; // Data in Group 1
5      array[N2] real X2; // Data in Group 2
6  }
7
8  parameters{
9      real mu1;
10     real mu2;
11     real<lower=0> sig;
12 }
13
14 model{
15     // likelihood
16     X1 ~ normal(mu1,sig);
17     X2 ~ normal(mu2,sig);
18     // prior
19     mu1 ~ uniform(0,100);
20     mu2 ~ uniform(0,100);
21     sig ~ cauchy(0,5);
22 }
```

ここで Stan コードにちょっとした工夫を 2 点入れています。1 つ目は data ブロックにある，変数 $N1, N2$ の存在です。二群のデータについて，サンプルサイズがとくに決まっていませんから，ここで外部から入力することにしています。2 つの群それぞれのサンプルサイズをデータとして取り込み，その数と同じだけデータ数を配列として宣言しているところがポイントです。もう 1 つのポイントは，尤度のところです。丁寧に書くならば，次のようにするべきでしょう (コード 6.2)。

code : 6.2　丁寧な尤度の記載

```
1      ...
```

```
model{
    for( i in 1:N1){
        X1[i] ~ normal(mu1 , sig);
    }
    for( i in 1:N2){
        X2[i] ~ normal(mu2 , sig);
    }
    ...
}
```

このように，それぞれの群にいて for 文を回し，各データ点が正規分布から出てきているよ，ということを明示するのです。しかしそうしなかったのは，Stan が「分かりきったことは書かなくていいよ」という優しい設計になっているので，変数 X1 の要素すべてがある分布に従うのであれば，まとめて X1 ~ normal(mu1, sig); と書いても良い，つまり X1 の配列要素を逐一指定しなくても同じ尤度関数をあてがってくれるというのを利用しています。

ともかくこれでできるのでした。では少し具体例をつかって，これがどういう結果をもたらしているのかを確認しましょう。

6.1.1 t 検定の具体例

> 統計学の新しい指導法の教育効果を見るため，全国の大学の心理学科から学生を無作為に 16 名選び，従来の指導法グループ (統制群) と，新指導法のグループ (実験群) にランダムに 8 名ずつわりつけました。プログラム終了後，心理統計のテストを行いました。統制群のスコアは $20, 40, 60, 40, 40, 50, 40, 30$，実験群のスコアは $30, 50, 70, 90, 60, 50, 70, 60$ となりました。新たしい指導法は，心理学科の学生に効果があると言えるでしょうか？ 5% 水準で検定してください。

これを t 検定するのは簡単ですね。計算式は複雑でも，機械がやってくれるから楽なのが NHST[*2]のいいところです。

code : 6.3 帰無仮説検定のコード

```
groupA <- c(30, 50, 70, 90, 60, 50, 70, 60)
groupB <- c(20, 40, 60, 40, 40, 50, 40, 30)
## t検定
t.test(groupA, groupB, var.equal = TRUE)
```

結果は出力 6.1 のようになります。検定統計量である t 値，検証のための自由度 df，帰無仮説のもとでの出現確率 p 値，が表示されています。5% より小さいので有意差あり，という判断ができます。もっとも，t 値とか自由度とかは，データに関係のない数字なのでピンと来ないというところもあるかと思います。

[*2] **帰無仮説検定 (null hypothesis significance test)** の略です

R の出力 6.1: 検定の結果

```
> t.test(groupA, groupB, var.equal = TRUE)

 Two Sample t-test

data:  groupA and groupB
t = 2.6458, df = 14, p-value = 0.01919
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
  3.786937 36.213063
sample estimates:
mean of x mean of y
       60        40
```

6.1.2 モデルで推定してみる

つづいて，先ほどのコードを使ってモデリングによる推定をしてみたいと思います。私の環境下では，次のような数字になりました。

MCMC の結果 6.1

```
# A tibble: 3 × 7
  name        EAP       MED       MAP        SD       L95       U95
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu1      60.006    59.976    59.700     5.608    48.869    71.420
2 mu2      39.990    39.997    39.650     5.679    28.654    51.128
3 sig      15.534    15.048    14.070     3.111    10.893    22.877
```

これを見ると，未知のパラメータ μ_1, μ_2, σ の値がそれぞれ推定されていますが，「判定」のような結果は出てきていません。というのも，勝負するような形で考えているのではなく，パラメータがどこにありそうか，ということを示す**確率分布**を求めているからです。それでも，**EAP** をみると 60006 と 39.990，95%CI をみると実験群は [48.869, 71.420]，統制群は [28.654, 51.128] とあり，実験群の平均値が 51 より小さい値になる確率はほとんどなく (2.5% 以下)，統制群の平均値が 49 以上より大きくなることもまたほとんどないわけですから，確実に差があると言ってもほぼ過言ではない，と言い切れるでしょう。

モデリングの利点は，t 値や p 値のような直接のデータと関係ない値を出すのではなく，データに直接関係する数字で考えさせてくれるので，イメージしやすいところもあるかもしれませんね。

6.1.3 分散の等質性

ところで，t 検定の場合は「分散が同じであるかどうか」という条件によって，算出方法を補正するという考え方があるのを覚えていますでしょうか。**Welch の補正**というやつで，こちらの方が一般に条件が緩いものですから，t 検定では普通こちらの方が使われます。

R の出力 6.2: Welch の補正の例

```
> t.test(groupA, groupB, var.equal = FALSE)

 Welch Two Sample t-test

data:  groupA and groupB
t = 2.6458, df = 12.274, p-value = 0.021
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
  3.570406 36.429594
sample estimates:
mean of x mean of y
       60        40
```

出力 6.2 にあるように，オプションとして指定するだけで瞬時に答えが出てきます。t 値，自由度，p 値が先ほどと異なって出ていますが，結果はほぼ変わりがありません。

モデリング上ではこれをどう表現すれば良いでしょうか。これは実は簡単で，分散が実験群と統制群で異なるというのですから，別々に推定してやれば良いのです。

code : 6.4　分散が異なる場合の推定モデル

```
1          ...
2  parameters{
3      real mu1;
4      real mu2;
5      real<lower=0> sig1;
6      real<lower=0> sig2;
7  }
8
9  model{
10     // likelihood
11     X1 ~ normal(mu1,sig1);
12     X2 ~ normal(mu2,sig2);
13     // prior
14     mu1 ~ uniform(0,100);
15     mu2 ~ uniform(0,100);
16     sig1 ~ cauchy(0,5);
17     sig2 ~ cauchy(0,5);
18 }
```

6.2　差の分布

さて，二群のデータから考えられるそれぞれの母平均が推定されました。今回は実験群と統制群の平均値差が大きく離れており，一方の 95% 上限が他方の 95% 下限を上回っているという状況でしたので，「差がある」と判断できましたが，そうでないこともありそうです。つまり，微妙な差のとき，ですね。次のようなデータで試しに推定してみましょう。まずは 5% 水準で検定をしてみます。

code : 6.5 帰無仮説検定のコード

```
groupA <- c(30, 50, 70, 90, 60, 50, 70, 60)
groupB <- c(30, 45, 60, 40, 60, 50, 40, 30)
## t検定
t.test(groupA, groupB, var.equal = FALSE)
```

今度は $t(12.175) = 2.0761, p = 0.0597$ で有意とは言えなくなりました[*3]。このデータを，先ほどのモデルで推定してみましょう。目にも分かりやすくするために，推定された平均値の事後分布をプロットしてみたいと思います (図 6.2)。

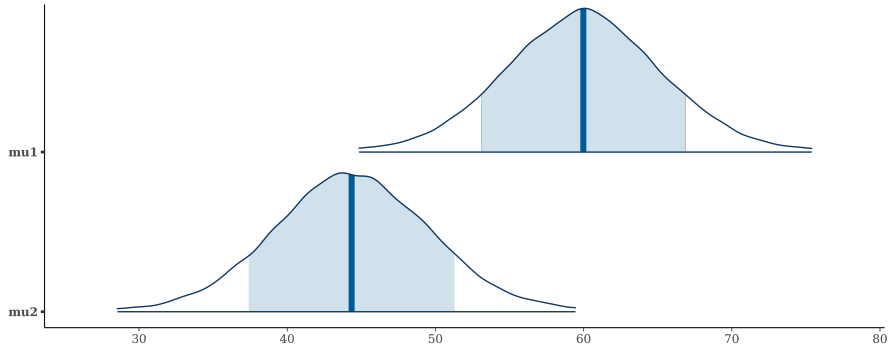


図 6.2 二つの平均値パラメータの事後分布

今回は 80% 確信区間と，確率分布の両端を 99% のところまで描画してみました。2 つの平均値パラメータは，分布の代表値で見ると異なっているのですが，重複している領域が多く一概に「一方が他方より大きい」と言いにくい結果になっています。帰無仮説検定のようにズバっと「差がある」「ない」と言い切れればいいのですが，このような場合はどうしたらいいでしょうか。

正解は「このままにしておく」です。急いで差があるとかないとか言い切るのではなく，これが区間推定の結果そのものですから，この程度の重複があり得るもの・それほど明確な答えは出ないもの，として考え続けるしかありません。

しかしまあ，このままでは考えるヒントが少ないので，ここで少し工夫をしてみましょう。帰無仮説検定では，$\mu_A = \mu_B$ という帰無仮説が反駁できるかどうかが問題なのでした。今回，μ_A や μ_B を代表する数字の候補が色々ありますので，これを使って「実際どの程度差があったのか」を計算してみれば良いのです。MCMC サンプルは，毎回推定したいパラメータの同時分布からの代表値を持ってきているという話をしました。たとえば 1 回目のサンプルは $\mu_A = 62.4, \mu_B = 25.7$，2 回目のサンプルは $\mu_A = 60.4, \mu_B = 36.8$ と言った具合に，です。それらは事後分布からのサンプル，事後分布を代表した実現値の 1 つですから，これを使って $\mu_A - \mu_B$ の計算をできます。すると**差の分布**の代表値が MCMC サンプルで計算できることになります。

これを使って，$\mu_A - \mu_B = 0$ になる確率はどれぐらいか，というのを考えれば良いのかもしれません。もっとも，連続変数におけるある一点の確率はゼロですし[*4]，代表値といってもピッタリゼロになることはほぼあり得ないでしょうから，「どの程度ならゼロと見做すか」ということを考える必要があります。この範囲は**実質的に等価な範囲 (region of practical equivalence; ROPE)** と呼ばれ，たとえばこのテストの例では ± 5 点ぐらいは誤差みたいなものだけど，5 点以上変わったら意味のある違いだ，ということであれば「5 点以上点差が出た確率」を計算できます！

[*3] 5.9% だから惜しい！とか「有意傾向にある」なんて変なこと言い出したらダメですよ。勝負は 5% だと決めて始めたのですから。

[*4] 確率は相対的な面積で表される数字ですから，面積を持たない点については確率が定義できないのです。

この計算は，MCMC サンプルを取り出した R のオブジェクトを使って計算することもできますが，実は Stan の中で計算させてしまうこともできます。`generated quantities` ブロックが，この MCMC サンプルを取り出したあとの処理をするブロックに該当します。ここで計算される量のことを**生成量 (generated quantities)** と呼びます。実際にどのように使うのかをみてみましょう。

code : 6.6　生成量ブロックの利用

```
1       ...
2   generated quantities{
3       real diff;
4       int<lower=0, upper=1> FLG;
5       diff = mu1 - mu2;
6       if(diff > 5){
7           FLG = 1;
8       }else{
9           FLG = 0;
10      }
11  }
```

コード 6.6 には生成量ブロックを使う例を示しました。ここもブロックですので，変数の宣言が必要です。今回はまず `diff` という変数を実数型で宣言してみました。これはその下で $\mu_1 - \mu_2$ として定義されていることからわかるように，差の分布 (の代表値) を生成す流のです。これを見ると，差の生じる確率を考えることができます。次に整数型 (int 型) で `FLG` という変数を宣言しました。上限が 1 で下限が 0，というか実質的には 0 か 1 の数字しか入らない，ある/なし，成立する/しないといった情報しか持たない変数です。いわゆる「フラグが立ったかどうか」の指標であり，その実態は下の `if` 文で表現されています。ここでは先ほど計算した `diff` が 5 よりも大きいのであれば 1，そうでなければ 0 を代入する，というコードになっています。

これを実行すると，パラメータのサンプルに加えて，そのパラメータから計算されるさまざまな量も同時に算出されます。出力例を見てみましょう。

MCMC の結果 6.2

```
# A tibble: 6 × 7
  name        EAP       MED       MAP        SD       L95       U95
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 diff     15.650    15.605    14.940     8.156    -0.516    31.706
2 FLG       0.911     1.000     1.000     0.285     0.000     1.000
3 mu1      60.035    60.035    60.112     6.737    46.602    73.575
4 mu2      44.385    44.374    44.291     4.594    35.249    53.696
5 sig1     18.509    17.472    15.960     5.369    11.414    31.792
6 sig2     12.496    11.752    10.761     3.683     7.629    21.594
```

ここにあるように，差の 95% 確信区間は `[-0.516, 31.706]` であることがわかります。差が大きければ 31.70，小さければ逆転して-0.52 になる可能性すらあるのです。この確信区間は 0 を含んでいますから，差がない可能性は否定できません。そういう意味で，帰無仮説検定的表現を借りるなら，5% 水準で差がないとは言い切れない，ということができるでしょう。

とはいえ，せっかく分布として情報が得られているのに，点推定にして勝負をするのはもったいないともいえるわけです。結論を急がずに，どの程度の差があるのかをじっくり考えてみるのがいいでしょう。また変数 `FLG` を見ると，その平均が 0.908 となっています。これは変数 `diff` が 5 よりも大きい時は 1，そうでなけれ

ば 0 という条件の数字で，その平均は 1 になった数を総 MCMC サンプル数で割った相対度数になっているのですから，確率を表す数字だと考えても良いことになります。つまり，差が 5 よりも大きくなる確率は 90.8% である，と考えることもできるのです。これを応用すると，さまざまな仮説を考えることができそうですね。

6.3 帰無仮説検定を省みる

さてこのように考えてくると，帰無仮説検定について異なる視点で見ることができるようになったのではないでしょうか。ここでは，帰無仮説検定の独特さを 3 つの点から考えてみたいと思います。

6.3.1 不平等な対決

帰無仮説検定は帰無仮説 vs 対立仮説という勝負に持ち込んで判断する方法である，ということを冒頭に述べました。平均値の差の検定の文脈で言えば，帰無仮説は $H0 : \mu_1 = \mu_2$ であり，対立仮説は $H1 : \mu_1 \neq \mu_2$ です。このように，帰無仮説は「差がない」の一点張りであり，対立仮説は「それ以外ならなんでもあり」という状態を指しているのでした。そもそも，帰無仮説というのは非常に限定的な仮説だったのです。無に帰してほしい仮説とは言え，あまりにも不利な勝負なのでした。その上で，勝負は「差がない」「ある」のどちらかにしかなりません。ごく微妙な差であっても，判定によれば差が「あった」といえますし，ギリギリでもなければ「なかった」というしかないのです。このような過度に結果を際立たせる報告は，科学的な研究実践の文脈では拙速なことになりかねませんので，注意が必要なのでした。

データ生成モデリングのやり方で同じデータを分析してみると，差も分布の形で描かれますから，「あったか，なかったか」という単純な問題にせずに色々考えてみたくなるのではないでしょうか。

6.3.2 量的な判断を

もっとも，出力 6.2 の t 検定の結果をよくみてみると，t 値や p 値の下に 95% 信用区間が示されており，ここでは [3.570406, 36.429594] という数字が出ています。そうです，帰無仮説検定をやっているようでも，しっかり区間推定した結果も表示されていたのです。これは差の信用区間で，この区間が 0 を跨いでいないということは，差がないとは言えない ($\mu_A - \mu_B = 0$)，ということを意味しています。この区間の幅をみると，3 から 36 と随分ひろくとられているように思えます。つまり，今回の検定結果は差があるとなったけれども，それほど確かなものではないかもしれないぞ，と慎重に判断することもできたはずなのですね。

この差についての量的な評価については，**効果量 (effect size)** という指標で表現されるのでした。効果量とは，標準化された差の大きさです。標準化するというのは，標準偏差で割って幅を整える，あるいは標準偏差何個分という単位で表現することでもあります。比べられるスコアはさまざまですから，その標準偏差で表現することで一般的に議論できるのです。平均値の差の検定においては，**Cohen の d(Cohen's d)** などが指標としても用いられますが，それは $\frac{\bar{X}_1 - \bar{X}_2}{\sigma}$ で計算されるのでした。

これは生成量を使って簡単に計算できます。今回の場合，計算する場合は，生成量に次のようにすれば良いのです。

code : 6.7 生成量ブロックで効果量の算出

```
1      ...
2  generated quantities{
3      real diff;
4      real cohen_d1;
5      real cohen_d2;
```

```
6       diff = mu1 - mu2;
7       cohen_d1 = diff / sig1;
8       cohen_d2 = diff / sig2;
9   }
```

どちらの群の標準偏差を基準にするか，ということを考える必要がありますが[*5]，相対的な大きさの比較も簡単にでき，しかもその結果も分布の形で得られる，すなわち効果量の確信区間 (効果が少なくともどの程度ありそうかとか，大きければどの程度ありそうかとか) を考えることもできるのです。

もちろんこれらの計算は，帰無仮説検定の文脈，言い換えれば伝統的な統計手法 (**モーメント法**による推論) であってもできたことなのですが，データ生成モデルという観点からみるといっそう理解がしやすいのではないかと思います。

6.3.3 方向性を持った仮説も

今回は最後に，「実験群が統制群よりも 5 点以上大きくなる確率」というのを考えました。この「一方が他方よりも大きい」という仮説は，帰無仮説検定の文脈では**片側検定 (one-tailed test)** と呼ばれます。一般的に「差がある」という対立仮説は，プラスであれマイナスであれ違いが生じている，という意味であり，統計量は分布の右側でも左側でも，とにかく極端な値になりさえすれば良いという判断でした。そうではなく「$A > B$ のはずだ」という仮定であれば，小さくなる可能性を考えなくていいのですから，統計量のどちらか一方の極について考えれば良いことになります。

とはいえ検定ですから，ある有意水準で一方が他方より大きいと判断すると間違える確率が云々，という判定に落とし込むこという点では同じでした。

今回は生成量を使って，5 点以上大きくなる**確率**を求めることができました。これは検定統計量や p 値のように架空の値ではなく，もとのデータに直結した数字や確率になっていますから，解釈が比較的自然にできるというのが利点です。プログラム上の表現も，たとえば今回のように `if` 文を使って表現するのは非常に直感的で，「もしあれがこうなってそれがこうなったらどうなる？」といろいろな仮説を考えていくこともできますね。帰無仮説検定のロジックは，結果判定のために仮説を限定的な型にはめ込んでしまいます。しかしその型を飛び越えていろいろなことをやってもいいのだ，できるんだというのは，データがどうやって生まれてきているかを考えている「創造主」としての特権かもしれません。

ただし注意してもらいたいのは，これらの検証の仕方は，今回のデータと仮定されたモデルという前提の上で成立する割合を確率とみなしているに過ぎない，ということです。データが変わればその数字も変わるでしょうし，そもそもデータが正規分布から出てきていないのかもしれません。本当は XX 分布から出てきているのに，YY 分布を仮定したモデルで計算したら，仮説が正しい可能性が 100% !といっても虚しい (明らかに間違っている) ことになりますね。心理学のデータの場合はえてして，どういう分布やメカニズムになるのかがはっきりわからないものです。ただ，あまりにもわからなさ過ぎて，さまざまな影響が混ぜ合わさっているので，色んな要素が相殺しあって結局ほとんど正規分布とみなしていいよ，ということだったりします。心という目に見えないものに，不良設定問題を解くためのツールで立ち向かうという，無理に無理を重ねるような推論の世界であるということに，自覚は持っていたいですね[*6]。

[*5] 二つの標準偏差の平均をとった，プールされた標準偏差で計算することもあります。

[*6] いいんです，それでも。だってそんなよくわからない人間というのが好きなんですから。

6.4 今回のまとめ

我々がデータを分析する時は，そのデータの数字がどうであったか，ということを真っ先に考えたいはずです。身長の差は何センチあったのか，最終学歴によって生涯年収は何円差がつくのか，といった具体的な単位に基づく**実質的な差**が一番大事なはずです。心理学をやっていくと，尺度をはじめとした「絶対的なスコア」が出てきませんから，尺度で何ポイント差があったのか，ということがあまり意味のある実態と対応しないことも少なくありません。そう言った時のために，**標準化された差**を考えるようになったのです。その計算は非常に面倒ですが，機械がすぐに計算してくれることによって結論を急ぎすぎ，単位も実際のデータとも関係のない統計量を参照して**有意差**を求めるようになってしまったのでは意味がありません。

データがどういうメカニズムで生まれてきたかを考え，母数をベイズ推定するというやり方は，いちいちコードを書かなければならないこともあって，非常に手間がかかるようにも思えます。しかし自分が何を仮定していたのか，どういうメカニズムの元で考えていたのかを自覚し，またその推定値を使って自由に仮説を考えることで，決まりきった分析手続きに落とし込むという単調な作業から解放され，クリエイティブにデータに向き合えることでもあります。

6.5 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；フライドポテトの研究 あるコンビニで，ホットスナックのフレンチフライを 2 つ買ったところ，どうも一方より他方の方が長くて大きい気がした。そこでそれぞれの袋から取り出して長さを測定してみた (単位 cm)。測定結果が表 6.1 である[*7]。ポテトの長さは正規分布に従い，また両群の分布の幅は同じであると仮定します。なおこの数値は伴走サイトのコードでも取得できます。

1. 群 A と群 B の平均値を推定するモデルを Stan で書いてください。母数として，群ごとの平均 `mu1`，

[*7] このデータは実際に筆者がコンビニの二つの店舗でポテトを購入し，長さを測定した結果に基づいています。

表 6.1　二つの店舗のポテトの長さ

A	8.4	11.3	8.1	11.2	5.8	6.3	7.1	10.9	7.1	6.5	5.0	3.0	7.2	6.5	6.4	6.4	9.3	8.3
B	6.7	7.2	4.2	11.0	7.5	8.9	7.0	8.0	7.2	4.2	6.0	9.0	8.6	9.0	5.0			

mu2 と, 両群に共通の標準偏差 sig を宣言すること。

2. このモデルでベイズ推定を実行し，mu1 と mu2 それぞれについて，EAP 推定値，MAP 推定値，90% 確信区間の下限と上限を報告してください。ans2 の要素名は mu1_eap，mu1_map，mu1_lower，mu1_upper，mu2_eap，mu2_map，mu2_lower，mu2_upper とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル, ウォームアップを 1000 として得ること。
3. 両群のポテトの長さの平均が，3cm 以上違うようであればクレームをつけに行こうと思います。生成量を使って, 群 A の平均が群 B の平均より大きい確率と, 平均の差の絶対値が 3cm 以上である確率の 2 つを求め, クレームをつけに行くべきかどうかを判断してください。ans3 の要素名は p_A_longer，p_diff_over3 とすること。

第 7 章

モデリングの目から見た検定 2；パタメータの世界とデータの世界

前回は，二群の平均値差の検定を行うデータ生成モデルを考えました。平均値差の検定には正規分布が仮定されますから，データ生成モデルも正規分布からデータが作られていると考えれば良かったわけです。仮定に忠実に設計図を書き，それに対応した Stan コードを書くと母平均の推定値が出てきます。検定の時も母平均の推定値を使って考えるのですが，検定統計量にしてしまうので実感が湧きにくいところがあったかもしれません。データ生成モデルの場合は，直接パラメータを考えるのでイメージがしやすいという側面がありました。

さらに，MCMC による推定ですから事後分布からの代表値が，実現値として手元のオブジェクトに代入されています。これをつかった計算をすることで，たとえばパラメータの差を計算でき，その分布を考えることができるのでした。こうした計算は Stan の `generated quantities` ブロックを使うことで，MCMC の結果と同時に考えることもできるのでした。

さて今回も，この `generated quantities` ブロックをつかって，生成量からいろいろ考えてみましょう。

7.1 事後予測分布

前回，二群のデータ $X_{i,A}$ および $X_{i,B}$ に対して，次のようなデータ生成モデルを考えました。

$$X_{i,A} \sim N(\mu_A, \sigma), X_{i,B} \sim N(\mu_B, \sigma)$$

このパラメータ μ_A, μ_B, σ の推定値，$\hat{\mu_A}, \hat{\mu_B}, \hat{\sigma}$ の分布からの代表値が MCMC によって得られるのでしたね。MCMC サンプルは iteration ごとに 1,2,3...M 個得られたとすると，その期待値すなわち **EAP** は，次の式で求めていることと同じです[*1]。

$$\hat{\mu_{A}}_{eap} = \frac{1}{M}\sum \mu_A^i = \frac{1}{M}(\mu_A^1 + \mu_A^2 + \cdots + \mu_A^M)$$

$$\hat{\mu_{B}}_{eap} = \frac{1}{M}\sum \mu_B^i = \frac{1}{M}(\mu_B^1 + \mu_B^2 + \cdots + \mu_B^M)$$

$$\hat{\sigma}_{eap} = \frac{1}{M}\sum \sigma^i = \frac{1}{M}(\sigma^1 + \sigma^2 + \cdots + \sigma^M)$$

[*1] ここで上つきの数字は MCMC のステップ番号を表しているもので，冪乗の数字ではありません。また M は Stan のデフォルトでは 4000 です。

さて, 今回は得られたデータ $X_{i,.}$ から μ, σ を推定したわけですが, これはいわばデータ生成メカニズムの設定値を見つけたことと同じですね。例え話で考えるなら, こんな感じです。ある企業 A が製品 a を量産しているとします。ライバル会社 B が, この製品 a の類似品を作ろうと考えたとします。そこで B 社は A 社の製品 a を作っている製造機械と同じ型番の機械を購入します。この機械には製品を生成するに当たっていくつかの設定をしなければならないとします。アナログな例えで恐縮ですが, ツマミが 2 つ 3 つ付いていて, それをひねれば何らかの製品ができるといった感じです。でも A 社が製品 a をつくるのに, どういう設定にしているのかはわからない。そこで製品 a をいくつかサンプルとして入手します。入手したサンプル a1,a2... と見比べながら, この辺りかな？この辺りかな？とツマミを調節し, サンプル a1,a2... と同じような製品 b1,b2... ができるのはこの辺の設定だな, というのを見つけたという感じ。

この例え話を MCMC の用語を使っていうと, 入手したサンプル a1,a2... がデータです。このデータをつくる製造機械が**確率分布**です。機械には設定のツマミがついているんですが, そのツマミは**パラメータ**のメタファーになっています。サンプルを参考に, この辺りかな, この辺りかな, とツマミの値を探るステップが MCMC の各段階で, まずは $\mu_A^1, \mu_B^1, \sigma^1$, 次にそれからちょっと動かして $\mu_A^2, \mu_B^2, \sigma^2$, またちょっと動かして･･･と 4000 回試すことで「だいたいこのデータを作るための設定はこの辺り」というのが決まってくるわけですね。「この辺り」というのを点推定する場合は **EAP** や **MAP** などを使いますし, 幅を持って推定したい場合は**確信区間**で表現するのでした。

ということで, いくつかの手元のデータから, 製造メカニズムの設定を手に入れました。このとき B 社はきっと, 「じゃあこの推定値をつかって (模造品である) 製品をジャンジャン作っていこう」となるでしょう。この推定値から作られる新しい製品 (模造品, 新しいデータとも言えます) のことを MCMC ではとくに**事後予測分布 (posterior predictive distribution)** と言います。分布となっているのは, 作られる製品も散らばりますので, その散らばり方を分布で考えるからです。

事後予測分布は, 点推定値を使って計算することもできます。つまり, データが $X_i \sim N(\mu, \sigma)$ というメカニズムで作られていて, それから推定値 $\hat{\mu}, \hat{\sigma}$ を得たわけですから, これを使って $X_{new} \sim N(\hat{\mu}, \hat{\sigma})$ とすれば良いのです。これはただの乱数生成でもありますから, たとえば R で `rnorm(mu,sigma)` とするときに `mu` と `sigma` に推定値を入れてやればいいでしょう。でも点推定値では決め打ちが過ぎますので, 区間推定した上限と下限も入れますか？いや, それならいっそ, MCMC で得られた事後分布からの代表値を全部入れてしまえばいいのではないでしょうか。

それを実現するために,`generated quantities` ブロックを使うといいでしょう。コード 7.1 にその例を書いてみました。データやモデルは前回の二群の平均値差の検定と同じものです。

code : 7.1　事後予測分布を作るコード

```
1  data{
2      int<lower=0> N1; // Number of Subjects in Group 1
3      int<lower=0> N2; // Number of Subjects in Group 2
4      array[N1] real X1; // Data in Group 1
5      array[N2] real X2; // Data in Group 2
6  }
7
8  parameters{
9      real mu1;
10     real mu2;
11     real<lower=0> sig1;
12     real<lower=0> sig2;
13 }
14
```

```
model{
    // likelihood
    X1 ~ normal(mu1,sig1);
    X2 ~ normal(mu2,sig2);
    // prior
    mu1 ~ uniform(0,100);
    mu2 ~ uniform(0,100);
    sig1 ~ cauchy(0,5);
    sig2 ~ cauchy(0,5);
}

generated quantities{
    array[N1] real Xpred1;
    array[N2] real Xpred2;
    for(i in 1:N1){
        Xpred1[i] = normal_rng(mu1,sig1);
    }
    for(i in 1:N2){
        Xpred2[i] = normal_rng(mu2,sig2);
    }
}
```

ここで，generated quantities には，normal_rng という関数が入っています。この_rng は確率分布の後ろにつけて，乱数を発生させろという意味です。R でいう r****みたいなもんですね。このコードでは各群と同じサイズだけの事後予測データセットを作っています。まあ同じ μ と σ からの乱数ですから，データの数だけ作ったりしなくてもいいんですがイメージしやすくするためにそうしてみました。この μ と Xpred を図にしてみたのが図 7.1 です。この図の左側は μ_1 の**事後分布**が，右側には X_1 の**事後予測分布**が描かれ

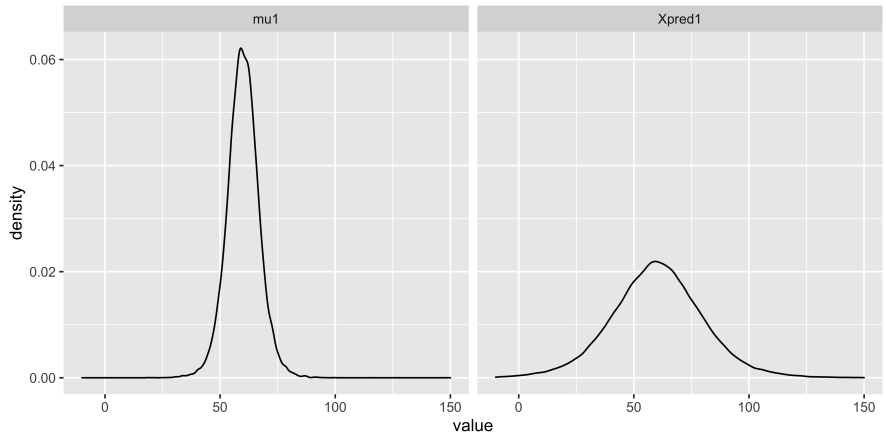


図 7.1　事後分布と事後予測分布

ています。左側は**パラメータの世界**の話であり，右側は**データの世界**の話になっていることに注意してください。

パラメータ μ_1 は，EAP で 59.96,95%CI[48.81,71.22] となっています。そこからでてくるであろうデータは，25 から 100 弱の幅に分布していますね。平均が 60 であっても，SD が 20 弱ありますから，データのレベルではその散らばる幅が大きくなるのも当然ですよね。でもこれはとても大事なことで，我々は「群間に差があった」となるとその効果について色々考えますが，それはパラメータの話，あるいは**平均因果効果**の話で

あって，個々のデータではまた事情が違うのです。たとえば，男女差があると言っても平均的な男性，そうでない女性などさまざまなケースがあるように。たとえば，平均的に 80% 成功する手術があると言われても，自分の身に置き換えると生きるか死ぬかは割合の問題ではないように。

また，この事後予測分布の形は，もとのデータの分布の形と似ているかどうかを視覚的に確認するために使われます。図 7.2 に，事後予測分布ともとのデータのヒストグラムを合わせて表示してみました。事後予測分布は MCMC サンプルの数だけありますが，ここでは一部だけ取り出しています。これを見ることで，もと

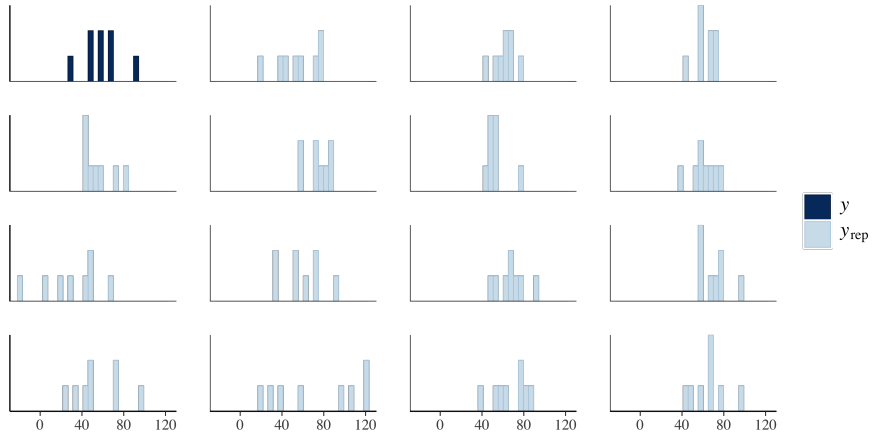


図 7.2 データの分布と事後予測分布の視覚的な確認

のデータと事後予測分布の形が似ているかどうかがわかります。もし模倣した機械の設定値が正しいのであれば，手元にある商品サンプルと同じような分布をするデータ分布が得られるはずですね。視覚的な確認ですから，見て「うーん似てるな，似てないな」というのを見て考えるだけではあります。今回はデータの数が 8 件と少ないので何ともイメージしにくいところですが，データが多くなるとデータの分布の形状で比較することもしやすくなるでしょう。もし分布の形が似ていなければ，購入した製造機械が違うものだったのかもしれない，ということになります。というのも実際のデータ分析の場合は，製造機械 (確率分布，モデル) が正しいかどうかはわからないので，それすらも仮定，検証すべき対象だからです。

7.2 データレベルの仮説

事後予測分布の利用方法は他にもあります。事後予測分布は，確率モデルと推定されたパラメータを利用した「新しいデータの生成」だったわけですが，この新しく生まれるであろうデータについての，データの世界での仮説を考えることができるのです。

帰無仮説検定で扱っていたのは，パラメータの世界の仮説でした。たとえば二群の平均値差の検定について言えば，帰無仮説は $\mu_1 = \mu_2$ であり，対立仮説は $\mu_1 \neq \mu_2$ です。これはギリシア文字 μ を使って書いてあることから明らかな通り，母数の仮説であり，$\bar{X}_1 = \bar{X}_2$ のような，データについての仮説ではありませんね。ではデータについての仮説というのはどういうものがあり得るでしょうか？たとえば次のようなものが考えられます。

- 無作為に選んだ統制群のデータと実験群のデータを比べたとき，実験群が統制群を上回っている確率はどれぐらいあるだろうか？
- 無作為に選んだ統制群のデータと実験群のデータの差が，基準点 c より大きくなる可能性はどれぐらいあるだろうか？

第 1 の点は**優越率 (probability of dominance)** という指標です。これはパラメータの世界の比較ではなく，実データの比較ですからイメージしやすいのではないでしょうか。たとえば男女差が平均的にはあるとされていたとしても，それはパラメータの違いであって，実際これから新しく出会う男性 (女性) が自分より優れている (劣っている) 可能性は，と考えた方が実質的な意味がありそうです。性差の話の多くは，平均値差だけでしか議論されませんが，効果量[*2]で考えると小さいことが少なくありません。それでも有意差が検出されてしまうのは，非常に大きなサンプルサイズをとっているから，ということもあったりします。サンプルサイズを大きくとるのは研究者の工夫や努力なのですが，小さな効果を取り上げて針小棒大に騒ぎ立てるのは科学的に良い態度とは言えません。そんな時に優越率で考えてみると，差があると言っても無作為に二群から新しい情報を得た場合の優越率が 50% ちょっとしかない，つまり超えるか超えないかが半々ぐらいですから「何の情報ももたらさない」のと同じこと，ということになったりします。くどいようですが，実データのレベルでの比較になるので，群間の差の表現としてよりわかりやすいということが言えるでしょう。第 2 の点は，優越率に一定の違いの差 c を含めて考えるもので，**閾上率 (probability beyond threshold)** といいます。これもデータレベルでの仮説の 1 つで，たとえば新しいトレーニング方法を使えばタイムが 3 秒小さくなる確率は X%，などということが分かれば具体的にイメージしやすい伝達方法になると思いませんか。

ちなみにこの優越率，閾上率などの指標は古くから指摘されてきていたものであり[*3]，ベイズ推定を使わない，**モーメント法**による推定でも計算可能な量でした。まぁしかし，このような統計量が実際に使われているケースってほぼ見かけないんですけどね。しかし，これらの指標はベイズ推定を使うことによって，というより乱数による近似計算をすることによって，遥かにイメージしやすくなっています。ここで示された数値はいずれも，`generated quantities` ブロックの中に `if` 文をつかって書いてやれば表現できることだからです！

実際に考えてみましょう。これら「条件が成立する確率」については，成立すれば 1，成立しなければ 0 というフラグをたてて MCMC サンプルを生成し，その平均値すなわち相対頻度で考えてやれば良いのでした。

code : 7.2 優越率や閾上率を計算するコード

```
data{
    int<lower=0> N1; // Number of Subjects in Group 1
    int<lower=0> N2; // Number of Subjects in Group 2
    array[N1] real X1; // Data in Group 1
    array[N2] real X2; // Data in Group 2
    int<lower=0> C; //constant
}

...

generated quantities{
    real Xpred1;
    real Xpred2;
    int<lower=0,upper=1> FLG1;
    int<lower=0,upper=1> FLG2;
    Xpred1 = normal_rng(mu1,sig1);
    Xpred2 = normal_rng(mu2,sig2);
    //probability of dominance
    if(Xpred1 > Xpred2){
        FLG1 = 1;
    }else{
```

[*2] 標準化された平均値差のことでしたね。

[*3] たとえば南風原・芝 (1987)による優越率の解説は 1987 年の論文です。

```
22          FLG1 = 0;
23      }
24      //probability beyond threshold
25      if(Xpred1 - Xpred2 > C ){
26          FLG2 = 1;
27      }else{
28          FLG2 = 0;
29      }
30  }
```

コード 7.2 は，generated quantities ブロックで 2 つの FLG 変数を用意しています。FLG1 が優越率，FLG2 が閾上率のために用意したものです。このブロックで，推定されたパラメータ mu1,mu2,sig1,sig2 からそれぞれ生成される二群の「新しいデータ」を作り (Xpred1,Xpred2)，成立するかどうかを検証します。FLG1 は if 文の中にあるように，Xpred1 > Xpred2 が成立すれば 1，しなければ 0 になる数字です。FLG2 は，同じくその差分が一定の値 C を超えていれば成立，超えていなければ不成立というフラグです。ちなみにこの値 C は data ブロックで外部から読み込むことにしてあります。

これと前回のデータを使って推定させてみましょう。

code : 7.3 優越率，閾上率を推定するコード

```
1  groupA <- c(30, 50, 70, 90, 60, 50, 70, 60)
2  groupB <- c(20, 40, 60, 40, 40, 50, 40, 30)
3  dataSet <- list(X1 = groupA, X2 = groupB, N1 = 8, N2 = 8, C = 3)
4  sampling2 <- modelC$sample(
5    data = dataSet,
6    chains = 4,
7    iter_sampling = 5000,
8    iter_warmup = 1000,
9    parallel_chains = 4
10 )
```

MCMC の結果 7.1

```
# A tibble: 8 × 7
  name         EAP       MED       MAP        SD       L95       U95
  <chr>  <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 FLG1       0.800     1.000     0.999     0.400     0.000     1.000
2 FLG2       0.764     1.000     1.001     0.425     0.000     1.000
3 mu1       59.933    59.936    59.833     6.914    46.147    73.895
4 mu2       40.006    40.002    40.141     4.656    30.869    49.362
5 sig1      18.598    17.495    15.846     5.480    11.310    32.054
6 sig2      12.640    11.941    11.071     3.725     7.623    21.903
7 Xpred1    59.880    59.828    58.098    20.590    18.660   100.984
8 Xpred2    39.887    39.978    40.457    13.984    11.642    67.691
```

ここにあるように，優越率は 80.0%，閾上率は 76.4% ということになりました。実験群 (GroupA) のほうが統制群 (GroupB) よりもスコアが高くなる確率が 80%，スコアが 3 点以上大きくなる確率は 76% もあるのですから，実験群の効果は結構おおきいぞ，といったことがわかるようになります。

こうしたデータに基づく仮説は，単位がもとのデータに対応しているから想像しやすく，より実践的な意味を想像しやすいですね。

まとめておきます。パラメータの世界に関する仮説は，推定されたパラメータはどれぐらいの値なのか，どれぐらいの「差」「違い」があるのかを考えることでした。あるいは，推定されたパラメータはどれぐらいの幅に分布するのか，という差の分布を考えることで，効果の有無の確からしいさを量的に評価できるのでした。データの世界に関する仮説は，データ生成機がどういうデータを作るのか，ひいてはそれが今のデータと同じ形をしているかを見ることで，モデルが正しいかどうかを検証できます。また未来のデータの分布はどんな形になるのか，という意味では，データという具体的な数字でもって未来予測をできます。

ただしいずれの世界においても注意しておいて欲しいのは，データ生成機すなわち確率モデルが正しい場合であり，これが間違っていると `generated quantities` などで作られる数字もまったく意味のないものになってしまう，ということです。もちろん **t 検定**など帰無仮説検定を行う時も，データが正規分布していなければ意味のない検定結果になるので，「そもそも統計的な仮定が間違ってた」というのでは意味がないのはいずれも同じです。推測統計学は手元の少しのデータから，未知の世界を予測検証するという不良設定問題に対応するためのものですから，仮定や前提が正しいかどうかについては常に敏感でなければなりません。

7.3 パラメータ・リカバリ

そもそもの仮定が間違っていた，というのを考えすぎて何もできなくなっても良くないですから，そこはいったん OK だとしましょう。そう考えると，仮定・前提をつけた上での話になりますが，未知の世界や未来を予測できるというのは，とんでもなく強力なツールであるような気がします。株価や競馬の予想ができれば，大儲けできる薔薇色の人生が待っているかも。ということで，ベイズ推定の勉強にも身が入るというものです。推定を可能にしてくれた MCMC に感謝！

半ば冗談，半ば本気なのですが，Stan や JAGS などの**確率的プログラミング言語 (probabilistic programming language)** が登場したおかげで，事後分布の計算が可能に，簡単便利にできるようになったというのは大変ありがたいことです。これのおかげでどんなモデルでも，設計図が書ければ答えが出せちゃう。複雑な確率の式とか微分積分を考えなくても，なんとかなるような気がします。しかし簡単なツールが出てきたときの常で，ちょっと注意が必要なのですが，MCMC は優秀すぎて正しくない計算式からでも答えが出てしまう，という可能性があるのです。

MCMC がうまく行ったかどうかは，**Rhat** や**有効サンプルサイズ**，**トレースプロット**などをチェックすればよい，というのはすでにお話しした通りです。しかしここで指摘したいのはそうではなく，これらの MCMC がうまく行っていたとしても，推定結果が正しいとは限らない可能性です。MCMC は「あり得そうな答えの可能性」を大量に出力し，その分布で我々は考えるのですが，その答えの可能性群が全部的外れなところに行ってるかもしれないのです。実際にデータを分析していると感じるのですが，MCMC はかなり複雑なモデルを考えても答えを出してくれます。とにかく答えてくれる機械が手に入ったとしても，その信憑性をチェックすることを忘れてはいけません。

ではこれをチェックするためにはどうすればよいでしょうか。ひとつは，答えが先にわかっている問題を作って，機械が正解を当てられるかどうかを検証するということが考えられます。この「わかっている正しい値」を当てることができるのであれば，実際のデータを用いた「わからない値」を推測する方法として有用だということがわかります。正解を当てることができないようであれば，この計算機械は当てにならないので，実際のデータを使っても正しく推測できているとは言えないでしょう。

この検証方法のことを**パラメータリカバリ (parameter recovery)** と言います。実際のデータ分析をする場合は，いきなり未知の答えに推測機をあてがうのではなく，推測機が事前に設定したパラメータをリカバリ (回復) できることを確認してから利用するようにしましょう。実際の研究実践の場合，その分析手順は，以下のようになります。

1. データ生成メカニズムを考える (紙とペンによる設計図の作成)
2. モデルを式で表現する
3. モデルの式からデータ生成をシミュレーション
4. Stan でコードを書く
5. シミュレーションデータを Stan が再現していることを確認する
6. 実際のデータでやってみる
7. 結果の解釈 (図による確認も)

ここで第 3 のステップ, 仮想データの生成から第 5 のステップ再現の確認が, パラメータリカバリという作業になります。推定したい平均値などのパラメータを事前に仮に入れてみて, その答えが復元できるかどうかを見るのですね。

なんだかまだるっこしい！と思う人もいるかもしれません。自分で設定したパラメタ通りになるのって当たり前じゃないか, 何がおもしろいのか？というわけですね。ですが意外と当たり前でないことがあるんです。間違えた答えであれば, 実践では役に立ちません。あるいは, いきなりデータでいいじゃない, 固いこと言うなよ, と思う人もいるかもしれません。これが設定ミスの場合は答えを出さない･答えられないという機械であればそれでもいいのかもしれませんが, MCMC は大概なんでも答えてしまうので, いきなりの運用は怖いと思いませんか。またあるいは, 基礎実験とかそのほかの授業でも, 今までそんなのやったことないよ, という意見もあるかもしれません。大学の授業で行うような演習･実習は, そもそも効果がはっきりあること, 再現する現象であることがある程度確からしいものを選んでやっていますからいいのですが, これから皆さんは研究者の卵として, 新しい現象, 十分に確認されていない現象を研究対象にすることになります。そのとき, いきなり出てきた結果がどれほど信用できるでしょうか。実際, 最近では心理学実験が再現できないケースも数多く指摘されており, その理由のひとつとして統計モデルの乱用 (悪用･誤用) にあったことは何度もお伝えしてきた通りです。さらに, ごく微小な効果を偶然検出してしまった, という可能性もあります。事前に推定機の精度がわかっていればこうした問題を回避できるのです。この話はベイズ推定をする場合だけでなく, 帰無仮説検定など従来のやり方にも通じ, **効果量**や**例数設計**の話に直結します。

7.3.1 二群の平均値差の例

ここまで扱ってきた, 二群の平均値差を検証する例で考えてみましょう。もともとの設計図 (図 6.1) がここでも役に立ちます。データがどういうメカニズムで出てきたかを考えていたわけですから, これを使えばデータ生成を考えることができますね。乱数生成のアプローチはデータを取り始める前にも有用なのです！

今回はパラメータを仮に設定してやることができます。二群の平均とその差を次のように決めてしまいましょう。

code : 7.4 平均値差の設定

```
1  mu1 <- 50
2  diff <- 10
3  mu2 <- mu1 + diff
4  sig1 <- 5
5  sig2 <- 8
```

第一の群の平均は `mu1 <- 50` としています。第二群の平均は `mu2 <- 60` というようにしてもいいのですが, 差を考えるという意味を強調するためにいったん変数 `diff` を作って差の量を定め, それを `mu1` に加えるという方法をとっています。2 つの群は正規分布に従いますから, それぞれの標準偏差を `sig1`,`sig2` として設定しました。ここから仮想データを作ります。データの生成は乱数に従うので, `rnorm` 関数の出番

です。

code : 7.5 仮想データの生成

```
1 set.seed(12345)
2 N <- 10
3 X1 <- rnorm(N, mu1, sig1)
4 X2 <- rnorm(N, mu2, sig2)
```

これで実際に生成された仮想データは次のようになっています。

R の出力 7.1: 検定の結果

```
> X1
[1] 52.92764 53.54733 49.45348 47.73251 53.02944 40.91022 53.15049
[8] 48.61908 48.57920 45.40339
> X2
[1] 59.07002 74.53850 62.96502 64.16173 53.99574 66.53520 52.90914
[8] 57.34738 68.96570 62.38979
```

ちなみにこの 2 つのデータセット, 平均は $\bar{X}_1 = 49.33528, \bar{X}_2 = 62.28782$ となっています。理論的にはそれぞれ $50, 60$ であるはずなのですが, 乱数による実現値なので少し誤差が出ていますね。これは実際の研究状況でもある話です。理論的な値と, 目の前の実現値というのは必ず少しずれがあるもの。その中で, きちんと理論値を推定できるかどうかが問題なのですから。さて, このデータを使って分析していきましょう。

code : 7.6 パラメータリカバリ・検証

```
1  ## t検定(モーメント法による推定と判定)
2  t.test(X1,X2)
3  ## ベイズ推定(MCMCによるベイズ推定と差の分布)
4  dataSet <- list(X1 = X1, X2 = X2, N1 = N, N2 = N, C = 3)
5  modelC <- cmdstanr::cmdstan_model("ttest03.stan")
6  sampling2 <- modelC$sample(
7    data = dataSet,
8    chains = 4,
9    iter_sampling = 5000,
10   iter_warmup = 1000,
11   parallel_chains = 4
12 )
```

まずは t 検定の結果ですが, ここでは $t(14.797) = 5.2059, p = 0.0001113$ で有意差ありと判定されました。実際 `diff <- 10` で差がある設定なのですから, 正しく判定できて良かったな, というところです。ベイズ推定の方はどうでしょうか。結果は次のようになりました (ここでは前回のコード 6.6 を再利用しました)。

MCMC の結果 7.2

```
# A tibble: 6 × 7
  name        EAP       MED       MAP        SD       L95       U95
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 diff    -12.936   -12.957   -13.250     2.762   -18.458    -7.445
2 FLG       0.000     0.000    -0.001     0.000     0.000     0.000
3 mu1      49.349    49.350    49.499     1.477    46.381    52.302
4 mu2      62.285    62.285    62.256     2.355    57.600    66.977
5 sig1      4.482     4.257     3.868     1.194     2.842     7.406
6 sig2      7.189     6.870     6.363     1.821     4.615    11.579
```

これを見ると，mu1 <- 50 と設定したときの EAP 推定値が 49.349，95%CI で [46.381, 52.302] ですから，ほぼ正しい点推定値，また確信区間の中に真値を含んでいます。推定はうまく行っている，ということです。mu2 についても同様で，真値が 60 であり，EAP 推定値が 62.285，95%CI で [57.600, 66.977] です。点推定値は真値から 2 点ほどずれていますが，95% の区間推定にすれば真値を含んでおり，推定はうまく行っているようです。標準偏差の推定値についても，それぞれうまく行っており，差についても EAP 推定値が-12.936，95%CI で [-18.458, -7.445] です[*4]。確かに確信区間に真値は含んでいるのですが，17.45 は真値 10 から少し外れすぎかな... というようなことがわかりますね。

このように，今回の結果をみると概ね正しく推定できていると言えるでしょう。その精度についても「どの程度誤差が生じうるものなのか」が具体的にわかったことと思います。しかし次のような設定の場合はどうでしょう。

code : 7.7 仮想データの生成その 2

```
1  mu1 <- 50
2  diff <- 18
3  mu2 <- mu1 + diff
4  sig1 <- 10
5  sig2 <- 15
6  N <- 3
```

この設定でデータを作って，推定した結果は次のようなものです (出力 7.3)。

MCMC の結果 7.3

```
# A tibble: 6 × 7
  name        EAP       MED       MAP        SD       L95       U95
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 diff     -5.434    -5.630    -5.104    12.855   -31.789    21.966
2 FLG       0.171     0.000     0.000     0.376     0.000     1.000
3 mu1      53.962    53.972    54.117     4.147    45.756    62.294
4 mu2      59.396    59.671    59.608    12.124    32.811    84.278
5 sig1      6.194     5.034     3.778     4.332     2.356    17.443
6 sig2     20.651    17.148    13.108    12.966     8.682    53.683
```

MCMC の収束は問題なく，推定値は出力されていますが，本来 $\mu_1 = 50$ であるところが 53.962，95% 確信区間は [45.756 ,62.294] であり，$\mu_2 = 68$ であるべきところに至っては EAP 推定値が 59.396，95%

[*4] 生成量は diff = mu1 - mu2 で作っているので，正負が逆転しているのはご容赦ください。

確信区間は [32.811 ,84.278] です。確信区間はなんとか平均値を挟み込みましたが，38 から 84 までと 46 点も幅がある推定はあまり有効な予測とは言えませんね。差は 18 あるという設定なのですが，EAP 推定値はわずか 5.434 であり，95% 確信区間は [-31.789, 21.966] です。確信区間にゼロをふくんでいますから，「差がないかも」ということになります。実際，t 検定の結果は帰無仮説を棄却できませんでした ($t(2.2337) = 0.52828, p = 0.6451$)。**タイプ 2 エラー (type II error)** を犯してしまっています。

どうしてこんなにうまくいかなかったのでしょうか。今回は正規分布という正しい確率モデルを使っていたのに，です。その答えは明らかに，`N <- 3` という点にあります。つまり，データが 3 件しかないので予測の精度が悪くなったのです。このように，モデルが正しくても推定精度が十分ではない，あるいは判定をするときに間違った判断をする可能性がある，ということがわかります。推定精度や効果の判定については，サンプルサイズ，データの散らばり (群内分散)，データ間の差の大きさ (群間分散) などの要因が影響してきます。ここで設定値を色々変えてみて，どういう関係にあるのかをみておくのも良いでしょう。

このように仮想データを使ったパラメタリカバリを通じて，推定や検定の精度を設計したりチェックしたりできることがわかりました。これを踏まえて考えると，前回の例では $N = 8$ のデータで分析をしましたが，$N = 8$ だと平均値差を 8〜12 点ほど外した予測をしてしまうかもしれない，ということがわかります。逆に「5 点差までの推定誤差に収めたい」というような希望があった場合，データは何件ぐらい取れば良いでしょうか？このような問いについても，N の設定値をさまざまに変えてみることで，実験や調査を実際に始める前に予想を立てておくことができます。これが**例数設計**です。例数設計の重要さを確認するとともに，こうした乱数を作るアプローチで簡単に実践できますので，実際の研究の前にはぜひ一度試してみてください。

7.4 今回のまとめ

生成量をつかうことで，次のような検証をできるのでした。

- 事後予測分布を作ることで，想定した確率モデルが正しかったかどうか検証する。
- 事後予測分布を利用して，優越率や閾上率などデータに基づく仮説を検証する。

さらに，データ生成モデルを応用して，そもそも擬似データをつくることで推定モデルの精度や，どれぐらいのサンプルサイズが必要かといったことを，実践の前に検証できます。

データ生成アプローチと乱数発生技術の組み合わせで，より慎重かつ確実な研究アプローチができることを理解してください。

7.5 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，

2463 `ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の
2464 名前はそれぞれの課題文に書いてあります。

2465 **数値は丸めない** オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif`
2466 で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは
2467 構いませんが，提出するオブジェクトの中身は丸めないこと。

2468 わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

2469 **■大問；フライドポテトの研究その 2** 第 6 講と同じ，二つの店舗のポテトのデータ (表 7.1) を使います。
ただし今回は，両群の分布の幅が同じであるという仮定を外します。

表 7.1 二つの店舗のポテトの長さ

A	8.4	11.3	8.1	11.2	5.8	6.3	7.1	10.9	7.1	6.5	5.0	3.0	7.2	6.5	6.4	6.4	9.3	8.3
B	6.7	7.2	4.2	11.0	7.5	8.9	7.0	8.0	7.2	4.2	6.0	9.0	8.6	9.0	5.0			

2470

2471 1. 群ごとに異なる標準偏差を推定するモデルを Stan で書いてください。母数として `mu1`，`mu2`，`sig1`，
2472 `sig2` を宣言し，あわせて `generated quantities` ブロックで，それぞれの群から新しいポテトを 1
2473 本取り出したときの予測値 `Xpred1`，`Xpred2` を生成すること。
2474 2. このモデルでベイズ推定を実行し，`sig1` と `sig2` それぞれについて，EAP 推定値と 90% 確信区間
2475 の下限と上限を報告してください。2 つの区間が重なっているかどうかを見て，両群の分布の幅は同
2476 じと考えてよさそうかを判断すること。`ans2` の要素名は `sig1_eap`，`sig1_lower`，`sig1_upper`，
2477 `sig2_eap`，`sig2_lower`，`sig2_upper` とすること。なお MCMC サンプルは，4 チェインで 1 チェ
2478 インあたり 5000 サンプル，ウォームアップを 1000 として得ること。
2479 3. 今後，店舗 A で購入するポテト 1 本が，店舗 B で購入するポテト 1 本よりも長い確率を求めてく
2480 ださい。母数 `mu1`，`mu2` ではなく，生成量 `Xpred1`，`Xpred2` を使うこと。あわせて 2cm 以上長い
2481 確率も求め，それが 30% を超えるかどうかを判定してください。`ans3` の要素名は `p_A_longer`，
2482 `p_A_2cm_longer` とすること。

第 8 章

モデリングの目から見た検定 3；多群の平均値差モデル

8.1　要因計画モデル

さてここまでは二水準の平均値差を考えるモデルでしたが，ここからそれを三水準に増やしてみましょう。水準数が多くなると**要因計画 (factorial design)** や**実験計画 (experimental design)** といった話になってきます。**分散分析 (analysis of variance; ANOVA)** は要因計画と帰無仮説検定の合わせ技で，効果がある/ないの判定を分散の比をつかって行うことでした[*1]。

今回は要因計画をデータ生成モデルから考え，**ベイズ推定 (Bayesian inference)** をすることになります。**帰無仮説検定**のような Yes/No 判断はせず，平均値や差の大きさを直接見積もることになるのは，二群の時と同じです。

まずは簡単な**群間計画 (between design)** の話から進めましょう。対応のない二群の時のように，それが三群になるモデルですので，モデルの設計図は非常に単純です (図 8.1)。設計図を見れば，コードはそのまま

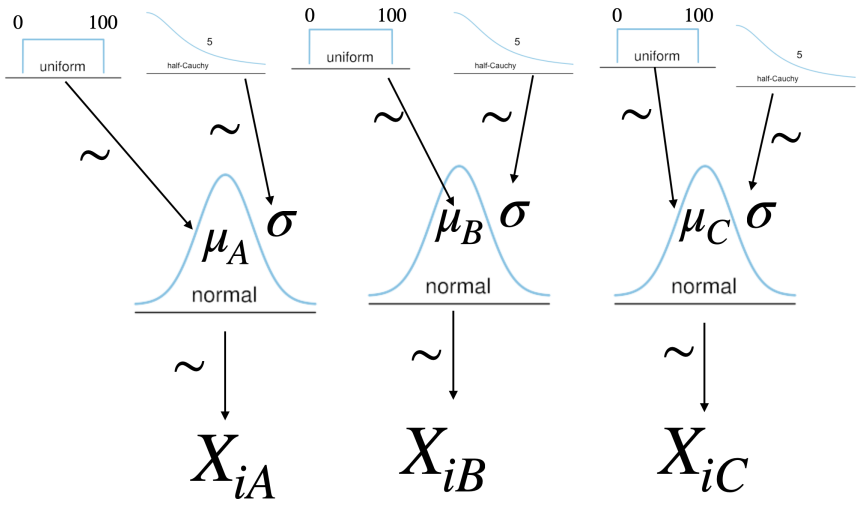


図 8.1　多群の平均値の差を比べるときの設計図

[*1] 忘れた人は，データ解析基礎のテキストに戻って確認してください。

置き換える形でかけますね。設計図の通りに書いてみたコードが code:8.1 にあります。

code : 8.1　三群の平均値のコード

```
1  data{
2    int<lower=0> N1;
3    int<lower=0> N2;
4    int<lower=0> N3;
5    array[N1] real X1;
6    array[N2] real X2;
7    array[N3] real X3;
8  }
9
10 parameters{
11   real mu1;
12   real mu2;
13   real mu3;
14   real<lower=0> sig;
15 }
16
17 model{
18   // likelihood
19   X1 ~ normal(mu1,sig);
20   X2 ~ normal(mu2,sig);
21   X3 ~ normal(mu3,sig);
22   // prior
23   mu1 ~ uniform(0,100);
24   mu2 ~ uniform(0,100);
25   mu3 ~ uniform(0,100);
26   sig ~ cauchy(0,5);
27 }
28
29 generated quantities{
30   real diff12;
31   real diff13;
32   real diff23;
33
34   diff12 = mu1 - mu2;
35   diff13 = mu1 - mu3;
36   diff23 = mu2 - mu3;
37 }
```

最後の `generated quantities` ブロックにあるように，各群の差の分布も計算して出せるようにしてあります。これをみながら，群 A と B，A と C，B と C などどこに差があるのか，その大きさはどれぐらいかということを考えることができます。左の大きさを標準偏差 `sig` で割ることで効果量を出すこともできます。

二群の平均値差の時もそうでしたが，検定と違って差の大きさを直接検証できているところがポイントです。また，検定の場合は分析によって有意差が確認できた場合，そのあと**多重比較 (multiple comparison)** へと進むのでした。これは帰無仮説が $H_0 : \mu_1 = \mu_2 = \mu_3$ となっていて，これを否定することから「どこに差があったか」をさらに詳しくみる必要があったからです。この時，統計的検定の繰り返しによって**タイプ 1 エラー (type I error)** が増えること，すなわち $\alpha = 0.05$ と 5% 水準で検定していても，検定を繰り返すと 5%

を大きく上回る問題が生じるため，有意水準を調節するなど工夫が必要だという話でした[*2]。

ところが，ベイズ推定の場合はこうした問題が生じません。なぜなら，確率を伴う判断をしていないからです。帰無仮説検定の場合は，「帰無仮説が正しいという条件のもとで」計算された検定統計量 (F 値や t 値) が，帰無仮説の条件下ではどれほど生じやすいかということを考え，その確率 p 値を計算していましたが，この p 値はパラメータがどこにあるかといった確率ではありません。判断のための指針としての確率に過ぎないのです。一方ベイズ推定する場合，推定されたパラメータの**事後分布**は，パラメータがこの辺りにあるだろう，という確率であり，生成量である差の分布もさの大きさがこれぐらいだろうという範囲，大きさに直接関わる確率です。ですから何 % 区間で考えてもいいですし，その判断が間違うかどうかの確率というのは問題に含まれないのです[*3]。検定の多重性の問題が生じない，というのは複雑な要因計画を考える場合には非常に助かりますね。

8.2 パラメータの変形と制約

8.2.1 パラメータの変形

ところで，先ほどは差の分布を `generated quantities` ブロックで表現しましたが，違う表現も可能です。差分も未知なるパラメータと考えて推定するのです。

順を追って説明しましょう。まず，最初の群の平均値を μ_1 とします。第二の群平均 μ_2 は，μ_1 と δ_1 だけ違う，つまり $\mu_2 = \mu_1 + \delta_1$ と考えるのです[*4]。同様に，第三の群平均 μ_3 は $\mu_3 = \mu_1 + \delta_2$ と考えます。こうすると，推定するべきパラメータは $\mu_1, \delta_1, \delta_2$ であり，`generated quantities` ブロックを使わなくても，直接差分パラメータを推定できるようになります。これを書いてみたのが，code:8.2 になります。

code : 8.2 差分を直接推定するコード

```
1  ...
2  parameters{
3    real mu1;
4    real delta1;
5    real delta2;
6    real<lower=0> sig;
7  }
8
9  model{
10   // likelihood
11   X1 ~ normal(mu1,sig);
12   X2 ~ normal(mu1+delta1,sig);
13   X3 ~ normal(mu1+delta2,sig);
14   // prior
15   mu1 ~ uniform(0,100);
16   delta1 ~ normal(0,100);
```

[*2] お前は何をいってるんだ，と思った人は，データ解析基礎のテキストに戻って確認しておいてください。大雑把に解説しますと，5% 水準というのは一回の判断で間違いが含まれる可能性を 5% にしましょう，という判定基準のことでしたが，二回判定を繰り返すと $1-(0.95^2) = 0.0975$ と 9.75% 水準になってしまうということでした。3 群の平均値を比べるときは，3 箇所比較する必要があり，**t 検定**を 3 回も繰り返すと 5% 水準ではなくなる，という問題です。

[*3] もっとも，すべて研究者の仮定したモデルのもとで，という話ではありますから，その仮定が間違っていたら全部意味のない数字である，ということにはなるかと思います。とはいえ，帰無仮説検定もデータが正規分布に従うという仮定で行うのですから，これについてはどっこいどっこい，というところでしょうか。

[*4] ここで δ はデルタと読む，ギリシア文字の d のことです。先ほどは生成量として `diff` と書いていましたが，ここでは未知のパラメータなのでギリシア文字に書き換えました。

```
17    delta2 ~ normal(0,100);
18    sig ~ cauchy(0,5);
19  }
```

data ブロックに違いはありませんので省略してあります。変更点として，まず parameters ブロックで推定するべきパラメータが変わっていることを確認してください。次に model ブロックで，3 つのデータがそれぞれ normal(mu1,sig),normal(mu1+delta1,sig),normal(mu1+delta2,sig) から出てきているように書き変わっています。正規分布の位置パラメータをずらすことで書き換えているのです。最後にその下，事前分布の設定ですが，差分 δ_1, δ_2 は正負どちらにも出てくる可能性があり，その大きさがわかりませんので，平均を 0 にした正規分布としてあります。これで推定した結果は，先ほどの生成量を使った結果と変わりはありません。数式の展開で書き方が変わっただけで，モデルの形は同じだからです。

ここで transformed parameters ブロックの紹介をしておきましょう。code:8.2 は悪くはないのですが，モデルのところ，normal(mu1+delta1,sigma) としているのがちょっと美しくないですね。元のコードにあった，normal(mu2,sigma) のほうがわかりやすいのに，と思った人もいると思います。まあ今の所，そこまで複雑ではないのでそれほど問題にはならないでしょうが，これからもしパラメータが複雑な構造をするようになったとき，尤度のところはもう少しスッキリ書きたいということも生じてくるでしょう。そういうときに transformed parameters ブロックを使います。このブロックは，parameters ブロックの次に書くもので，言葉の通りパラメータを変形 (トランスフォーム) するためのものです。

これを使って書いたのが code:8.3 です。

code : 8.3　パラメータ変換を入れたコード

```
 1  ...
 2  parameters{
 3    real mu1;
 4    real delta1;
 5    real delta2;
 6    real<lower=0> sig;
 7  }
 8
 9  transformed parameters{
10      real mu2;
11      real mu3;
12      mu2 = mu1 + delta1;
13      mu3 = mu1 + delta2;
14  }
15
16  model{
17    // likelihood
18    X1 ~ normal(mu1,sig);
19    X2 ~ normal(mu2,sig);
20    X3 ~ normal(mu3,sig);
21    // prior
22    mu1 ~ uniform(0,100);
23    delta1 ~ normal(0,100);
24    delta2 ~ normal(0,100);
25    sig ~ cauchy(0,5);
26  }
```

ここで注目すべきは，まず `transformed parameters` ブロックで `mu2,mu3` という変数名が宣言され，それが式によって表現されていることです。ここでパラメータ `mu1,delta1` が `mu2` に変形されていることがわかります。このようにして作られたパラメータであれば，`model` ブロックの中で使ってやることができます。現に，尤度のところが `X2 ~ normal(mu2,sig)` のようにスッキリした形になっていますね。ただし注意すべきは，事前分布のところが変わっていない，ということです。事前分布はパラメータに対しておかれますので，ここで `mu2 ~ uniform(0,100);` などと置くべきではありません。

このように変換することで，コードが多少なりともスッキリしたと思います。面倒だな，と思うかもしれませんが，コードがスッキリすることは自分にとっても誰にとっても「読みやすい」ことにつながります。そして読みやすいコードは，間違い (バグ) があったときに修正しやすいのです。あまりごちゃごちゃしたコードを書くと，問題が生じたときに対応が難しくなりますから，なるべくこうした作法を活用して，コードは単純明快なものにしておくと良いでしょう。

8.2.2 パラメータの制約

さてここでこのモデルをもう一段階，発展させてみましょう。今のコードは第一の群，μ_1 を基準に，μ_2, μ_3 を構成していました。別に第二，第三の群を基準にしても良かったですよね。つまり基準点は恣意的になっています。それで悪いということではないのですが，μ_1 だけ特別扱いです。そうではない作法として，全体平均 μ を考えて，次のように定式化してみましょう。

$$\mu_1 = \mu + \delta_1 \tag{8.1}$$
$$\mu_2 = \mu + \delta_2 \tag{8.2}$$
$$\mu_3 = \mu + \delta_3 \tag{8.3}$$

こうすることで，全体平均からの差分としてモデルを表現できます。ここで全体平均 μ はどういう意味があるでしょうか。群の違いを独立変数，結果のスコアを従属変数とした線形モデルの一環として考えると，全体平均は群を通じて変化のない水平線 (図 8.2) という関係を表していることになります。横一線で群ごとに違いがないのですから，いわばこれは**帰無仮説**のモデル，すなわち**帰無モデル (null model)** とでもいうべきものです。

あるいは，群ごとの平均値の違いはこの全体平均からの差分で表現されます。この群ごとの違いは，その群に入ったからこそ生じた**効果**であるとも言えます。つまり，δ は効果の大きさなのです。翻ってかんがえると，全体平均は「なんの処置もしなければ理論的にこの値になるはず」という操作なしの状態，天然の状態とでもいえるでしょう。**ランダム化比較実験 (randomized control trial)** は，被験者をランダムに割り付けることによって，平均的に見れば個別の効果が相殺し合うことを用いています。平均化することで誤差が相殺するので群ごとの比較ができますし，群の平均値が変化したということは，その群に属したことの効果が現れたと考えるわけです。帰無モデルの状態は，この個別の効果を相殺させた状態であることを表しています。

さてこのように考えて，これを数式の形，モデルの形に書き起こしていくことを考えましょう。ところがこのままだと，未知のパラメータは $\mu, \delta_1, \delta_2, \delta_3$ と 4 つあることになります。今までは μ_1, μ_2, μ_3，あるいは $\mu_1, \delta_1, \delta_2$ の 3 つだったのに，です。同じ数式を書き直しただけなのに，パラメータの数が変わるのはおかしいですね。実はこれには 1 つ，制約が欠けているのです。今回は全体平均 μ からの相対比較になっていますから，$\delta_1 + \delta_2 + \delta_3 = 0$ という関係が隠れていることになります[*5]。そこで，この制約をモデルの中に書き込まな

[*5] もしこれが $= 0$ にならなければ，全体平均の計算に矛盾が生じますよね。全体平均 μ は，$\mu = \frac{1}{3}(\mu_1 + \mu_2 + \mu_3)$ ですから，

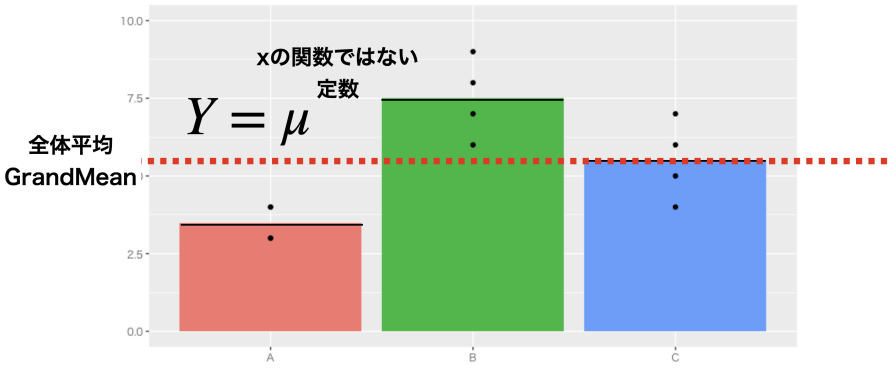


図 8.2 全体平均は水平線

ければなりません。$\delta_1 + \delta_2 + \delta_3 = 0$ という式から，変数を移項することでこれを表現します。すなわち $\delta_3 = 0 - (\delta_1 + \delta_2)$ のようにします。

これを使って書いたのが code:8.4 です。

code : 8.4 制約を入れたコード

```
1  ...
2  parameters{
3    real gm;
4    real delta1;
5    real delta2;
6    real<lower=0> sig;
7  }
8
9  transformed parameters{
10     real delta3;
11     real mu1;
12     real mu2;
13     real mu3;
14     delta3 = 0 - (delta1 + delta2);
15     mu1 = gm + delta1;
16     mu2 = gm + delta2;
17     mu3 = gm + delta3;
18 }
19 ...
```

このようにすることで，先ほどと等価なモデルを別の形で書き表すことができるようになりました。

ここまでをまとめておきます。三群の平均値差を求めるモデルを，3 つの方法で書いてみました。

1. 3 つの群平均 μ_1, μ_2, μ_3 を個別に推定するモデル
2. ある群とそこからの差分，$\mu_1, \delta_1, \delta_2$ を推定するモデル
3. 全体平均とそこからの差分，$\mu, \delta_1, \delta_2, \delta_3$ を推定するモデル。ただし $\sum \delta = 0$ という制約をつける

$\mu = \frac{1}{3}(\mu + \delta_1 + \mu + \delta_2 + \mu + \delta_3) = \frac{1}{3}(3\mu + \sum \delta_j) = \frac{1}{3}3\mu + \frac{1}{3}\sum \delta_j$ となり，最初の項が $\frac{1}{3}3\mu = \mu$ ですから，第二の項は $\frac{1}{3}\sum \delta_j = 0$ でないと計算があいません！

この方法はいずれも同じ推定結果を返します。第一のモデルの生成量から差分を計算すれば，δ_1, δ_2 を計算したことと同じです。

同じことを異なる方法で実装するのは，生産的でないように思うかもしれません。しかし第 3 の形式にしておくと，モデルの一般化が容易になります。すなわち，多群の平均値差の比較をするときに，群の数が変わるたびにコードを書き換える手間がなくなるのです！

8.3 モデルの洗練

ここまで，二群，三群の平均値差を求めるモデルを書いてきました。書き方はわかっているので，比較する群が四群，五群と増えても書き足していくだけでなんとかなりそうです。

しかし，たとえば四群に増えたときに $\delta_1, \delta_2, \delta_3$，五群に増えたときに $...\delta_3, \delta_4$ と手作業で増やしていると，N 群までふえたときは $...\delta_{N-2}, \delta_{N_1}$ と何行も書き足していかなければなりません。しかもその度にコンパイルしているようでは，とても面倒なことはすぐに想像がつきます。

そこで，群の数が変わっても同じコードで対応できるように，一般化を目指してコードを改良しましょう。そのためには，何群の比較なのかを示す群の数を，data ブロックで外部から取り込むようにすれば良いでしょう。その変数を使って差分の数，効果の数を計算してやれば良いのです。たとえば比較する群の数を G とすると，全体平均 μ と，$G-1$ 個の差分をパラメータとして推定すれば良いことになります。

その考え方を使って書いたコードは，たとえば code:8.5 のようになります。

code : 8.5 群の数を一般化したコード

```
1  data{
2    int<lower=0> Lv;              // 水準数
3    int<lower=0> N;               // 各群のサンプルサイズ
4    array[Lv,N] real X;           // 変数の値
5  }
6
7  parameters{
8    real gm;                      // 全体平均
9    array[Lv-1] real raw_delta;   // 全体からの差。水準数マイナス1個
10   real<lower=0> sig;            // 誤差の分散
11 }
12
13 transformed parameters{
14   array[Lv] real delta;           // 差の大きさを作り直す
15   array[Lv] real mu;              // 再構成される群ごとの平均
16
17   for(i in 1:(Lv-1)){
18     delta[i] = raw_delta[i];    // ほとんどコピー
19   }
20
21   delta[Lv] = 0 - sum(raw_delta); // 総和が0になるように最後だけ書き換える
22
23   for(i in 1:Lv){
24     mu[i] = gm + delta[i];        // 群ごとに再構成
25   }
26
27 }
28
```

```
29  model{
30    // Likelihood
31    for(l in 1:Lv){
32        for(i in 1:N){
33            X[l,i] ~ normal(mu[l],sig);
34
35        }
36    }
37
38    // Prior
39    gm ~ uniform(0,100);
40    raw_delta ~ normal(0,100);
41    sig ~ cauchy(0,5);
42  }
```

■コード解説

data **ブロック**　群の数 Lv, 各群のサンプルサイズ N, 各データ点 X を入力します。ここで X が**二元配列**になっていることに注目してください。X[1,1] は第一群の第 1 番目のデータ，X[1,2] は第一群の第 2 番目のデータ，X[2,1] は第二群の第 1 番目のデータ, 同様に X[i,j] は第 i 群の第 j 番目のデータを表すことになります。この形式は二元配列 (2 次元の変数セット) になっていると言います[*6]。宣言の時に, 上で取り込んだ変数 Lv や N を使うことができます。最大 Lv 個の群, 各群 N 人のデータがあるという前提です。

parameters **ブロック**　全体平均 gm と, 差分, 誤差の SD をパラメータとしています。差分はここでは raw_delta とし, 上で宣言した Lv をつかって Lv-1 個の要素を持つ配列にしています。raw_delta って変な名前, と思うかもしれませんが, この後これを変換して δ を作りますので, 作る前の生 (raw) の δ という名前にしてみました[*7]。

transformed parameters **ブロック**　以後の話をわかりやすくするために, 各群の平均 mu を構成することにします。mu の数は水準数と同じ Lv です。そして第 i 群の平均 mu[i] を作るために, mu[i]=gm+delta[i] という書き方をしたいので, 水準数と同じ数だけ delta[i] を作りたいと思います。もとの raw_delta が 1,2,...,Lv-1 個ありますので，delta も Lv-1 番目まではそれと同じものをコピーします (for 文で繰り返し代入していってます。)。そして最後に Lv 番目の要素を，0 からこれまでの要素をすべて足した (sum(raw_delta)) ものを引くことで作っています。ちなみに sum は stan の持っている総和の関数で, 配列要素のすべてを足し合わせるというものです。こうして delta が Lv 個できあがりましたので, 各水準の平均値を mu[i]=gm+delta[i] という数式で一般的に書くことができるようになりました。

model **ブロック**　尤度のところに, 群それぞれについて巡回する for 文, その中で 1,2,3...N 人まで巡回する for 文を組み込んで二重にぐるぐると回しています。事前分布は設定の通りです。

このコードを使って, 実際に推定してみましょう。データは表 8.1 のものを使います。

推定してみましょう (R コードは 8.6, 結果は 8.1)。各群の平均値, 全体平均からの偏差などが, 事後分布

[*6] 行列でもいいじゃないか, と思われるかもしれません。それでも結構です。Stan は行列の時，real ではなく matrix で宣言することができます。しかしこのように real 型にしておいて, 後ろのカッコで配列の次元を表現すると, 何次元でも拡張することができるので今回はそのようにしました。

[*7] 変数名は任意ですので, 著者の命名法がわかりにくいなあと思ったら, 自分で好きな変数名にしてもらって構いませんよ。その場合は, 以後すべての変数を読み替えてくださいね。

表 8.1 独立した三群から得られたスコア

groupA	6	6	5	5
groupB	7	4	7	6
groupC	4	3	4	6

として出力されているのがわかると思います。これをみて，たとえば差の事後分布の 50% 確信区間でみると δ_2, δ_3 はその区間に 0 を挟んでないので，一定の効果はあるだろうなどと判断できるわけです。もちろん生成量をつかって，ある程度の差がある確率がどのくらいとか，優越率などデータレベルの仮説を立てることも可能です。

code : 8.6 三群の平均値の比較

```
1  Example <- matrix(c(6, 6, 5, 5, 7, 4, 7, 6, 4, 3, 4, 6),
2                    ncol = 4, byrow = T)
3  dataSet <- list(Lv = 3, N = 4, X = Example)
```

MCMC の結果 8.1

```
# A tibble: 10 × 7
   name              EAP       MED       MAP        SD       L95       U95
   <chr>         <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
 1 delta[1]          0.257     0.256     0.227     0.569    -0.880     1.394
 2 delta[2]          0.749     0.754     0.804     0.566    -0.387     1.878
 3 delta[3]         -1.005    -1.009    -1.114     0.576    -2.158     0.162
 4 gm                5.251     5.251     5.242     0.400     4.452     6.039
 5 mu[1]             5.507     5.507     5.562     0.699     4.114     6.902
 6 mu[2]             5.999     6.001     5.986     0.697     4.603     7.386
 7 mu[3]             4.246     4.238     4.182     0.695     2.860     5.647
 8 raw_delta[1]      0.257     0.256     0.227     0.569    -0.880     1.394
 9 raw_delta[2]      0.749     0.754     0.804     0.566    -0.387     1.878
10 sig               1.339     1.261     1.134     0.391     0.823     2.301
```

8.3.1 データサイズの一般化

さてこのモデルは，比較する群の数が増えても外部からデータとして群の数を与えることができますので，毎回コンパイルする必要がありません。やったね！これで完璧，と言いたいところですが，1 つ気になるのはサンプルサイズです。各群のサンプルサイズ N をデータとして与えるようになっていますが，たとえば A 群は 10 人，B 群は 12 人，C 群は 14 人と言ったように，群ごとにサンプルサイズが異なるとこのコードでは対応できません。実際に実験をやる場合，各群のサイズを整えて人を集めたいところですが，調査研究などでは群ごとに同じ人数を与えるような調整ができないこともあり，そういう意味ではこのコードではうまく対応できそうにありません。

そこで，群の人数が異なる場合でも対応できるように，さらにこのコードを拡張していきたいと思います。そのためにまず，データを**整然データ (tidy data)** の形に整形しましょう。

先ほどのサンプルデータ (表 8.1) は，3×4 の長方形をしていました。これはデータのサイズが整っているからできることで，サイズが変わってしまうと表の中に欠損ができてしまうことになります。また，たとえば B 群

の 3 番目の人の値を見る時，われわれは 2 行 3 列目のデータにさっと目が行きますが，機械的には群の情報が行名に，人の群内整理番号が列名にあるので，1 つのデータを見る時に行・列それぞれを参照することになります (図 8.3 左)。ここで「どの群か」「群の中の何番目の人か」という情報はいずれも変数で，人によって変わるものですから，データの持つ情報の 1 つなのです。データの持つ情報，変数なのに参照先が変わっていることになります。

このデータの情報をまったく損なうことなく，縦長に並べ直したのが図 8.3 の右側です。これは 1 つの行がそれぞれの観測に対応しています。このような形式にすると，何行目かということを指定するだけで，どの群の何番目の人なのかという情報が手に入ります。またデータの中に欠損があっても，整然データの形にする時にこれは観測に含まれないので，データの一部が欠けることがない という利点があります。

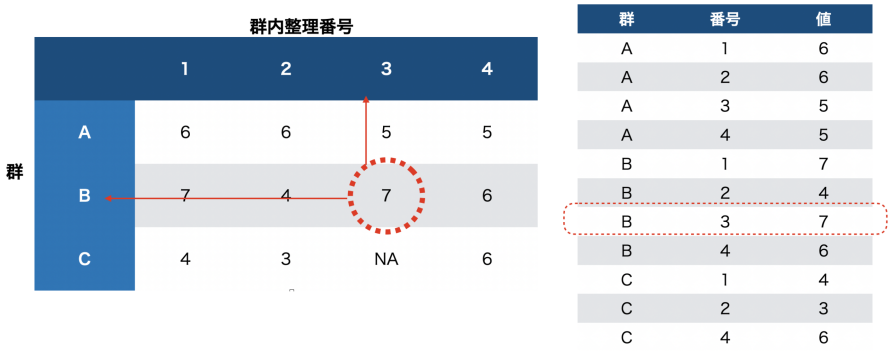


図 8.3 一般的なデータと整然データ

このようなデータ形式にして，これを与える形でデータ分析をするとデータサイズに依存しないコードが書けます。具体的には code:8.7 のようにします。

code : 8.7 整然データに対応したコード

```
data{
  int<lower=0> Lv;                              // 水準数
  int<lower=0> L;                               // データ数
  array[L] int<lower=0,upper=Lv> idx;           // データのID
  array[L] real X;                              // 変数の値
}
...(中略)...
model{
  // Likelihood
  for(l in 1:L){
    X[l] ~ normal(mu[idx[l]],sig);
  }

  // Prior
  gm ~ uniform(0,100);
  raw_delta ~ normal(0,100);
  sig ~ cauchy(0,5);
}
```

parameters ブロックと transformed parameters ブロックは code:8.5 と同じなので省略しました。まず data ブロックをみてください。群の数を Lv で入力するのはそのままですが，次にデータの総数 L を撮るようにしています。図 8.3 の例で言えば，C 群 3 列目のデータがありませんから，データ長は全部で $L = 11$ になります。次に L 行のデータがどの群に属するのかを指示するインデックス変数 idx を L 個用意しています。L 行目のデータが第何群に属しているのかの数字が入ります。たとえば 1 行目は A 群なので 1，5 行目 (L=5) は B 群なので 2, と言った数字が入るようになっています。さいごにデータの値そのものである X も，データ長 L と同じだけの 1 次元配列を用意してやります。

技術的に注目すべきは model ブロックの尤度のところで，for 文がデータ長 L の反復をしているだけです。つまり 1,2,3...,L 行目のデータを順に参照していくのです。そして l 行目のデータがどの群に属するかは，変数 idx[l] が指し示してくれますから，mu[idx[l]] としてやることで指定できていることになります。idx[l] は，l 行目のデータが属している群の数字が代入されていますから，たとえば 1 行目 (l=1) の場合は mu[idx[1]] = mu[1] としていることと同じ，5 行目の場合は mu[idx[5]]=mu[2] としていることと同じ，ということになります。ここでは入れ子になった参照が行われています。ちょっとテクニカルですが，この技術を使うと表現力も広がりますので，仕組みをしっかり理解しておいてください。

8.4 パラメータリカバリ

さあ，最後にパラメータリカバリをして，このコードが正しく推定できるか，あるいはどの程度のサンプルサイズがあればどの程度の精度で推定できるのかを検証しておきましょう。

私たちはデータがどのように造られるか，というそのメカニズムの方からアプローチしているわけです。これはリバースエンジニアリングと呼ばれる考え方ですね。そして今から仮想データを作ろうという時も，そのデータ生成モデルをそのまま利用すればいいのです。仮想データの生成は，データ生成メカニズムをリバースエンジニアリングで明らかにしていくアプローチのちょうど正反対，リバース・リバース・エンジニアリングです。

具体的には，たとえば次のようなコード (code:8.8) でデータを作ることができるでしょう。

code : 8.8 リバースエンジニアリング

```
N <- 100
Lv <- 5
gm <- 50
sig <- 3

raw_effect <- runif(Lv - 1, -10, 10)
effect <- c(raw_effect, 0 - sum(raw_effect))
mu <- gm + effect

X <- rnorm(N * Lv, mu, sig)
dat <- data.frame(
  Idx = rep(1:Lv, times = N),
  value = X
)

modelC <- cmdstanr::cmdstan_model("BetweenAnova2.stan")
dataSet <- list(Lv = Lv, L = NROW(dat), idx = dat$Idx, X = dat$value)
```

■コード解説

1 行目　各群のサンプルサイズ N を設定します。各群共通のサイズになります。

2 行目　水準数です。ここでは 5 群の平均値の比較をすることにしました。

3 行目　全体平均の設定です。事前分布に [0,100] の一様分布を置いてますから，真ん中ぐらいにしてあげました。

4 行目　誤差の散らばりを設定します。

6 行目　水準数 -1 の効果を設定します。手入力でもいいのですが面倒なので，-10 から 10 までの範囲で一様乱数によって生成しました[*8]。

7 行目　水準数 Lv の効果にするため，先ほど作った水準数 -1 の `raw_effect` に `0-sum(raw_effect)` の計算結果をつけくわえています。関数 c は結合させる combine という意味です。

8 行目　各群の平均値です。全体平均に効果を足しています。全体平均は 1 つの数字，効果は水準数の要素を持つベクトルですから，計算ができないように思えますが，このような場合 R は自動的にサイズ合わせのため値の再利用を行います。つまり，$50 + (\delta_1, \delta_2, \delta_3, \delta_4, \delta_5)$ を $(50, 50, 50, 50, 50) + (\delta_1, \delta_2, \delta_3, \delta_4, \delta_5)$ と解釈してくれるのです。

10 行目　水準数 $\times$ 各群のサイズの乱数を発生させています。乱数は正規分布によるもので，平均や SD は上で指定した通りです。ここでも値の再利用が行われていて，ベクトル `mu` は $\mu_1, \mu_2, \mu_3, \mu_4, \mu_5, \mu_1, \mu_2, ...$ とくり返して当てはめられていきます。

11-14 行目　数値を `data.frame` 型にくみ上げていってます。`rep` 関数は，指定した回数だけ繰り返すもので，ここでは `1:Lv` すなわち 1,2,3,4,5 という数字をデータサイズ分繰り返していることになります。

16 行目　ここでは `cmdstanr` パッケージを使ってコンパイルしています。

17 行目　推定に使うデータセットです。データ長は `data.frame` の長さを返す関数 `NROW` を使って与えています。インデックス変数は `dat` オブジェクトの `Idx` 変数，データは同じオブジェクトの `value` 変数を指定しています。

いかがでしょうか。作ったデータ，オブジェクト，ベクトルなどの意味がわからない場合は，その都度 R でオブジェクト名を入力し，何が格納されているかを確認しながら進めると良いでしょう。ここで作った仮想データを，先ほどの Stan オブジェクトに与えて乱数を生成し，パラメータリカバリがどの程度の精度でできるかを考えてみてください。サンプルサイズや誤差の大きさなどを変えながら確認してみると良いでしょう。

8.5 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの　分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名　`chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと　提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方　小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の

[*8] 一様乱数の関数は，R では `runif` です。

名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；整然データに対応した多群比較　本講の最後に紹介した，整然データに対応した多群比較のモデルを自分で書き，正しく動くことを確かめます。

1. 整然データ (1 行が 1 観測で，群を表すインデックス変数を持つ形) を受け取るモデルを Stan で書いてください。本講の書き方にならい，母数は全体平均 `gm`，全体からの差 `raw_delta`，誤差の大きさ `sig` とし，`transformed parameters` ブロックで群ごとの平均 `mu` を作ること。群の数は `data` ブロックで外から与えられるようにします。
2. 本講のパラメータリカバリのコードを使って人工データを生成し，小問 1 のモデルで推定してください。ただし，データを作る前に `set.seed(2026)` を実行し，全員が同じ人工データになるようにすること。5 つの群それぞれについて，平均の EAP 推定値を報告してください。`ans2` の要素名は `mu1_eap` から `mu5_eap` の 5 つとすること。あわせて，データを生成したときの真の値も並べて示してください。
3. 各群の平均について，真の値が 90% 確信区間に入っているかどうかを判定してください。区間に入った群の数を `ans3` に入れ，要素名を `n_covered` とすること。入っていない群があれば，どの群かもあわせて示してください。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。

第 9 章

モデリングの目から見た検定 4；対応のある群の比較

9.1 対応のある群

9.1.1 多次元正規分布

ここまでは**群間計画 (between design)** のモデリングをしてきましたが，今回は**群内計画 (within design)** のモデリング的アプローチになります。

Within モデルは別名として，**反復測定 (repeated measure)** と呼ばれることがあり，とくに二群の場合は**対応のある t 検定 (paired t-test)** と言ったりします。同じ人から 2 回データを取って，その変化をみる**事前・事後デザイン (pre-post design)** がその典型例です。2 回以上とる場合もあるので，その場合は反復測定とよばれるわけです。

この場合のデータ生成モデルは，何が違うのでしょうか。**群間計画**の t 検定は別名**独立した二群の** t 検定と呼ばれるように，2 つの群のデータがバラバラに出てきているのですが，今回は**対応のある二群**ですから，独立してないということになります。独立していない，つまり関係がある。データ分析上は，2 つのデータに相関がある場合を考える必要があります。同じ人の事前・事後のデータであれば，当然変化のもとになる「その人」という共通要因があるわけで，同じ人からデータを得ていますから当然相関していることを前提に考えなければなりません。

統計モデル上，独立した二群であれば別々の正規分布からデータが出てきていると考えますが，今回は 1 つの分布から 2 つのデータが出てきていることになります。ここで使うのは**二次元正規分布 (two-dimensional normal distribution)**，さらに群の数が多い場合に一般化した**多次元正規分布 (multivariate normal distribution)** あるいは**多変量正規分布 (multivariate normal distribution)** と呼ばれる分布を使います。この正規分布の特徴は，それぞれの変数 (次元) に注目すれば正規分布なのですが，各次元に相関関係があることを組み込まなければなりません。

これまでの 1 次元の正規分布は，平均 (位置)μ と標準偏差 (幅)σ で特徴付けられました。多次元の正規分布も，各次元について平均と標準偏差があります。平均も複数の次元のセットになっているので，平均ベクトル $\boldsymbol{\mu}$ として表現することになります。標準偏差もベクトルで考える必要がありますし，さらにすべての組み合わせにおける相関関係がありますので，分散共分散行列 $\boldsymbol{\Sigma}$ を考えることになります。丁寧に数式で表現すると，1 次元正規分布に従う確率変数は次のように書きます。

$$X \sim Normal(\mu, \sigma)$$

これに対して多次元正規分布に従う確率変数ベクトル $\boldsymbol{X}$ は次のように書きます。

$$\boldsymbol{X} \sim MultiNormal(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

この分散共分散行列の中身を見てみますと, 次のようになっています。

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1m} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{m1} & \sigma_{m2} & \cdots & \sigma_m^2 \end{pmatrix}$$

この行列の**対角 (diagonal)** 要素には分散 σ_j^2 があり, 非対角要素には共分散 σ_{jk} が入っている**正方対称行列**になっていますね[*1]。またこうしてみると, 変数 j の分散 σ_j^2 は j と j の共分散, すなわち σ_{jj} であることもわかりやすいですね。ところでこれを考えることでどこに相関の要素が入っているのでしょう？改めて, 分散や共分散, 相関係数の定義式から考えてみたいと思います。

$$\text{分散} : \sigma_j^2 = \frac{1}{n}\sum(x_{ij} - \bar{x_j})^2 \tag{9.1}$$

$$\text{共分散} : \sigma_{jk} = \frac{1}{n}\sum(x_{ij} - \bar{x_j})(x_{ik} - \bar{x_k}) \tag{9.2}$$

$$\text{相関} : \rho_{jk} = \frac{\sigma_{jk}}{\sigma_j \sigma_k} \tag{9.3}$$

この関係から逆に, 共分散 σ_{jk} は次のように表すことができます。

$$\sigma_{jk} = \sigma_j \sigma_k \rho_{kj}$$

ということで, 共分散の式は分布の幅を表す標準偏差と, 相関係数からできあがっていることがわかりましたね。

9.1.2 対応のあるデータの生成モデル

ではこれを使ってモデルの設計図を書いてみましょう。設計図は図 9.1 のようになるでしょう。

あとはこれを使ってコードを書くだけです。この時のポイントは, データの渡し方にあります。

ベクトル型とマトリックス型

2 次元正規分布に従うデータは, 必ず 2 つセットになっています。1 回の観測で 2 つの数字を持つものです。これを Stan で表現するときには, 変数を `vector` 型で表現しておく必要があります。今回の場合は `vector[2] mu;` などとします。これで `mu` という変数が 2 つセットで準備されることになります。これまで同じ変数名で複数のデータを持つ場合, たとえば `real mu[2]` のようにしていましたが, なぜ今回もこの形式, つまり**配列**にしないのか, と思われるかもしれません。配列でも 2 つの数字を扱ったりすることは可能なのですが, ベクトル型にしておく利点が別にあります。ベクトルにはベクトルを使った演算がありますが, Stan にはベクトルや行列専用の関数がありますので, 変数をベクトルで宣言しておいてやるとこれらの関数を使って高速化できるのです。

また Stan には, `matrix` 型という行列の型もあります。`matrix[N,M] X;` のように宣言してやると, サイズ $N \times M$ の行列変数 X が 1 つ用意されます。`matrix[N,M] X[L];` のように宣言すると, サイズ $N \times M$

[*1] これら行列の要素や名称については, データ解析応用 1 で扱った行列の要素や名称を思い出してください。

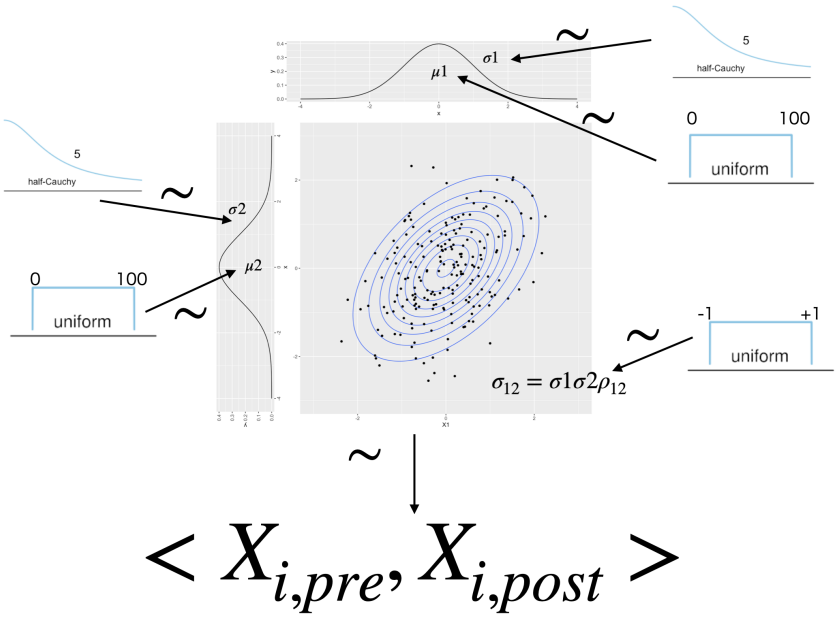


図 9.1 対応ある二群のデータ生成モデル設計図

の行列変数 X を L 個用意することになります。他にも特殊な型として，cov_matrix 型とか corr_matrix 型などがあります。これらは正方行列ですから，サイズの指定は 1 つでよく，たとえば cov_matrix[2] とすれば 2 × 2 の正方行列を用意したことになります。Stan における変数や配列の型について，松浦 (2016)を参考にまとめたものを表 9.1 に，イメージ図を図 9.2,9.3,9.4 に示します。

表 9.1 Stan の配列と型

型の例	宣言例	解説
整数	int X	整数ひとつ
実数	real X	実数ひとつ
範囲つき実数	real<lower=0> X	0 より大きい数字の入る X
整数の配列	int X[N]	N 個の整数
実数の配列	real X[N,M]	$N \times M$ 個の実数からなる 2 次元配列
実数の配列	real X[N,M,L]	$N \times M \times L$ 個の実数からなる 3 次元配列
ベクトル	vector[K] X	長さ K のベクトルひとつ
ベクトル	vector[K] X[N]	長さ K のベクトルが N 個
ベクトル	vector[K] X[N,M]	長さ K のベクトルが $N \times M$ 個
行列	matrix[J,K] X	サイズ $J \times K$ の行列がひとつ
行列	matrix[J,K] X[N]	サイズ $J \times K$ の行列が N 個
特別な行列	cov_matrix[J] X	サイズ $J \times J$ の正方対称行列がひとつ
特別な行列	corr_matrix[J] X	対角が 1 のサイズ $J \times J$ の正方対称行列がひとつ

これを踏まえて，今回の対応のある二群のコード例を見てみましょう (Code::9.1)。

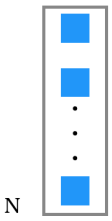
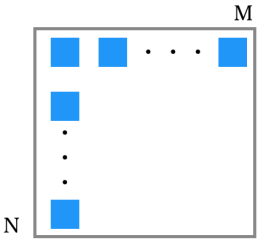
型	宣言例	サイズ
整数	int X	
実数	real X	
範囲つき実数	real<lower=0> X	
整数の配列	int X[N]	
実数の配列	real X[N,M]	

図 9.2　変数の型とサイズ (1)

code : 9.1　対応ある二群のデータ生成モデル

```
1  data{
2    int N;
3    array[N] vector[2] X;
4  }
5
6  parameters{
7    vector[2] mu;
8    real<lower=0> sd1;
9    real<lower=0> sd2;
10   real<lower=-1,upper=1> rho;
11 }
12
13 transformed parameters{
14   cov_matrix[2] SIG;
15   SIG[1,1] = sd1 * sd1;
16   SIG[1,2] = sd1 * sd2 * rho;
17   SIG[2,1] = sd2 * sd1 * rho;
18   SIG[2,2] = sd2 * sd2;
19 }
20
21 model{
22   X ~ multi_normal(mu,SIG);
23   //prior
24   mu[1] ~ uniform(0,100);
```

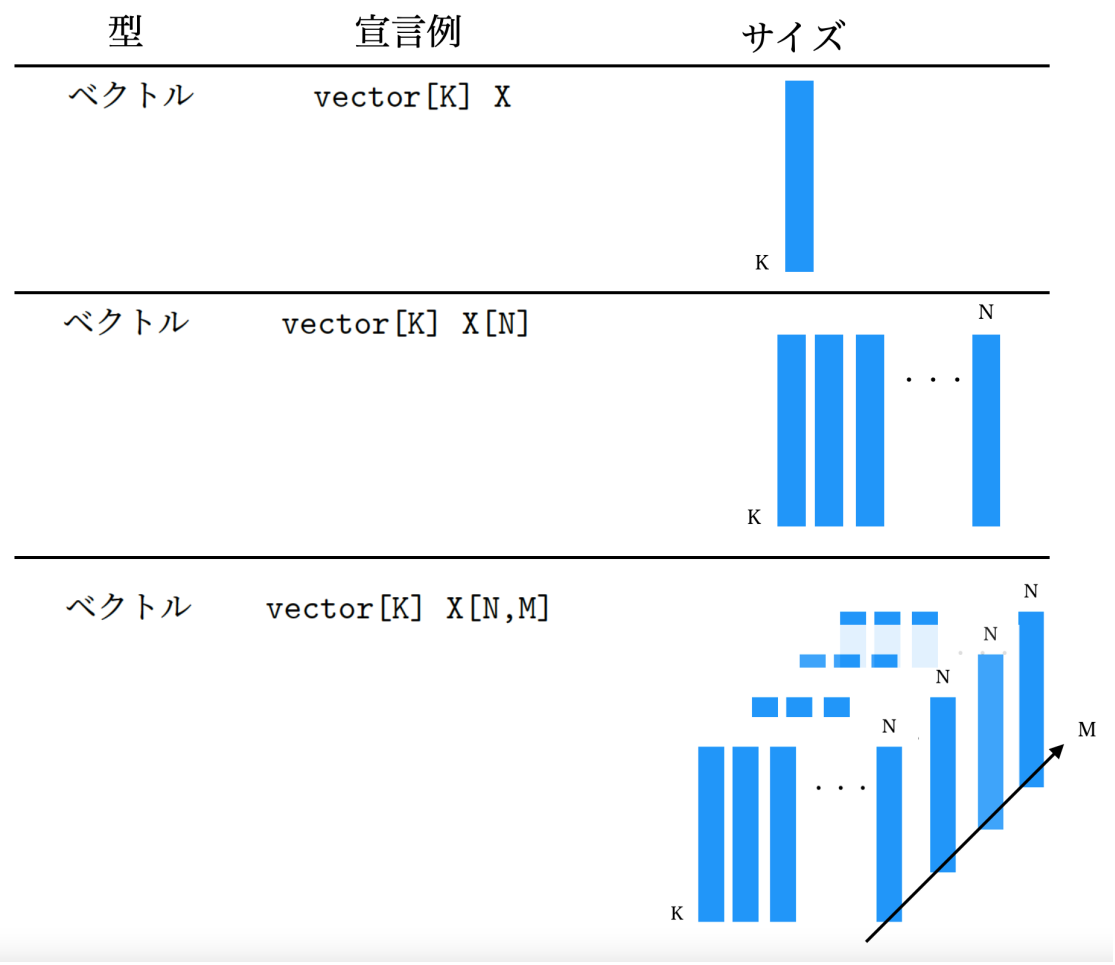


図 9.3 変数の型とサイズ (2)

```
25     mu[2] ~ uniform(0,100);
26     rho ~ uniform(-1,1);
27     sd1 ~ cauchy(0,5);
28     sd2 ~ cauchy(0,5);
29   }
```

■コード解説

data ブロック サンプルサイズ N のデータですが，ペアになっている数字なので vector[2] で宣言した変数 X を N 個用意しています。

parameters ブロック パラメータとして，平均をベクトル，標準偏差や相関係数はスカラで宣言しています。標準偏差や相関係数の範囲指定にも注意してください。

transformed parameters ブロック 各標準偏差，相関係数を使って分散共分散行列 SIG を構成しています。わかりやすくするために，各要素について書き下して書いています。

model ブロック 尤度はデータベクトルが多次元正規分布関数 multi_normal から出てきていることを示しています。多次元正規分布関数の引数は，ベクトルと行列です。

この書き方ですと，三群に拡張した時に大変になるのは目に見えていますね。こんな時のために，ベクトル

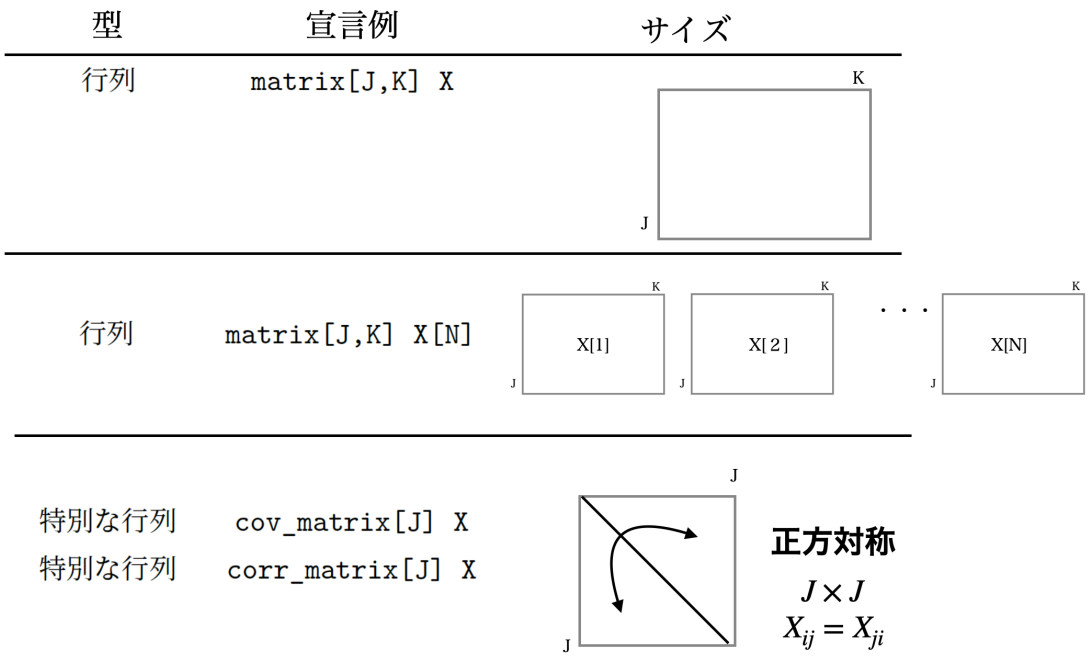


図 9.4 変数の型とサイズ (3)

と行列用関数をつかって書いておく方法があります。Code::9.2 には, 多次元にまで拡張したコードを示しています。

code : 9.2 コードの一般化

```
1  data{
2    int N;
3    int K;
4    array[N] vector[K] X;
5  }
6
7  parameters{
8    vector[K] mu;
9    vector<lower=0>[K] sds;
10   corr_matrix[K] rho;
11 }
12
13 transformed parameters{
14   cov_matrix[K] SIG;
15   SIG = quad_form_diag(rho,sds);
16 }
17
18 model{
19   X ~ multi_normal(mu,SIG);
20   //prior
21   mu[1] ~ uniform(0,100);
22   mu[2] ~ uniform(0,100);
23   rho ~ lkj_corr(1);
24   sds ~ cauchy(0,5);
```

```
25  }
```

■コード解説

data **ブロック**　データのサイズ K も外侮から指定できる変数とし，それを使ってサイズ K のベクトル X を N 個用意しています。

parameters **ブロック**　パラメータとして，平均と標準偏差をベクトルで，相関係数も行列として宣言しました。標準偏差ベクトルの範囲指定の方法に注意してください。

transformed parameters **ブロック**　Stan の関数 quad_form_diag を使って分散共分散行列を作成しました。この関数は，正方行列 $\boldsymbol{X}$ とベクトル $\boldsymbol{v}$ から，$diag(\boldsymbol{v})\boldsymbol{X}diag(\boldsymbol{v})$ という計算をするものです。ここで diag は k 個の要素を対角項に持つ $k \times k$ の対角行列を作ることを意味します。今回の例で具体的に説明しましょう。ここでの正方行列は相関行列で，ベクトルは標準偏差を要素に持つ sds ですから，

$$\begin{pmatrix}\sigma_1 & 0\\ 0 & \sigma_2\end{pmatrix}\begin{pmatrix}1 & \rho_{12}\\ \rho_{12} & 1\end{pmatrix}\begin{pmatrix}\sigma_1 & 0\\ 0 & \sigma_2\end{pmatrix} = \begin{pmatrix}\sigma_1 & \sigma_1\rho_{12}\\ \sigma_2\rho_{12} & \sigma_2\end{pmatrix}\begin{pmatrix}\sigma_1 & 0\\ 0 & \sigma_2\end{pmatrix} = \begin{pmatrix}\sigma_1^2 & \sigma_1\sigma_2\rho_{12}\\ \sigma_1\sigma_2\rho_{12} & \sigma_2^2\end{pmatrix}$$

となり，分散共分散行列が構成されていることがわかりますね。

model **ブロック**　注目すべきポイントは相関行列の事前分布です。相関行列の事前分布には，Stan では lkj_corr という関数を使うのが一般的で，この引数が 1 になっているのは無情報分布であることを意味しています。

このコードで推定される平均値，mu ベクトルの 2 つの要素が，事前・事後の平均値に該当します。ですからこの差分を計算すれば「平均的にどれぐらい変化があったか」という**平均因果効果**を見積もったことになります。この差分については，**生成量**のブロックで算出しても構いません。もちろんその差分から標準化した効果の大きさを算出することもできますし，パラメータについての仮説，事後予測などデータについての仮説など，さまざまに検証できることはこれまでの通りです。

9.2 ID をもったデータ構造

さてでは今回の乱数生成機の精度を検証するために，パラメータリカバリをしてみましょう。

仮想データの作り方は，データ生成モデルの裏返しです。2 群の平均値差を検証するためのデータでしたら，次のようにして作ることができます。

code : 9.3　対応ある二群の仮想データ

```
1  mu <- c(50,50)
2  sd1 <- 10
3  sd2 <- 5
4  rho <- 0.7
5  SIG <- matrix(ncol=2,nrow=2)
6  SIG[1,1] <- sd1 * sd1
7  SIG[2,2] <- sd2 * sd2
8  SIG[1,2] <- sd1 * sd2 * rho
9  SIG[2,1] <- sd1 * sd2 * rho
10 N <- 100
11 library(MASS)
12 X <- mvrnorm(N,mu,SIG)
```

```
13  # Stanに与えるデータセット
14  dataSet <- list(N=N, X=X)
```

ここでは二群の平均をいずれも 50, 標準偏差を 10 と 5 にし, 相関係数を 0.7 としたサンプルを 100 個作ることにしました。発生させる乱数は MASS パッケージの mvrnorm 関数を使います。この関数は, 発生させるサンプルサイズと, 平均ベクトル, 分散共分散行列を引数に取ります。これを使って推定させた結果の例が出力 9.1 になります。

MCMC の結果 9.1

```
  # A tibble: 9 × 7
  name           EAP       MED       MAP        SD       L95       U95
  <chr>    <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu[1]       47.590    47.594    47.608     0.913    45.808    49.356
2 mu[2]       49.506    49.508    49.570     0.500    48.523    50.493
3 rho          0.693     0.697     0.703     0.052     0.582     0.784
4 sd1          9.087     9.059     9.071     0.640     7.919    10.434
5 sd2          4.980     4.960     4.930     0.352     4.345     5.724
6 SIG[1,1]    82.975    82.063    81.989    11.761    62.715   108.872
7 SIG[1,2]    31.597    31.189    30.790     5.515    22.086    43.574
8 SIG[2,1]    31.597    31.189    30.790     5.515    22.086    43.574
9 SIG[2,2]    24.923    24.599    24.207     3.550    18.883    32.761
```

中央値による点推定値 (MED 推定値) を見ると, ほぼ真の値の通りになっていますね。

ところで, 今回与えたデータ X は行列の形をしています。一行が 2 つの数字を持つペアになっているのですが, これは**整然データ**の形にはなっていませんね。整然データであれば, 次のようなデータセットになっているはずです (R の出力 9.1)。

R の出力 9.1: 対応ある二群の仮想データ・整然版

```
# A tibble: 200 × 3
      ID name  value
   <int> <chr> <dbl>
 1     1 pre    45.1
 2     1 post   45.8
 3     2 pre    45.1
 4     2 post   51.2
 5     3 pre    52.4
 6     3 post   53.7
 7     4 pre    46.8
 8     4 post   48.6
 9     5 pre    45.5
...
```

このようなデータ形式に対応できるように, Stan のコードを書き直してみましょう。ポイントは, 個体識別インデックスと, 事前・事後を表す変数のインデックスの両方を準備する, というところにあります。

まず R から与えるデータの方から考えましょう。整然データの形に並び替えるのに加えて, 事前・事後といった水準を表す変数も数字で与えなければなりません。先の例 Code::9.4 では pre,post となっていたと

ころを，pre=1,post=2 と置き換えた別の変数を用意しました。

code : 9.4 対応ある二群の仮想データ･整然版

```
1  ...
2  X <- mvrnorm(N, mu, SIG)
3  tidy_data <- X %>%
4    as.data.frame() %>%  as_tibble() %>%
5    rename(pre = V1, post = V2) %>%
6    rowid_to_column("ID") %>%
7    pivot_longer(-ID) %>%
8    mutate(cond = if_else(name == "pre", 1, 2))
```

■コード解説

2 行目 真値の設定をして，行列 X を作っています。

3 行目 X を加工して，tidy_data という変数に作り替えます。加工のプロセスは下記の通りです。

4 行目 data.frame 型を経て tibble 型に変形します[*2]。

5 行目 この段階で，変数名が V1,V2 となっていますが，わかりやすくするために pre,post に書き換えました。

6 行目 行番号を ID という変数名を持つものに変えています。

7 行目 ID をキーに，横長だったデータを縦長に (tidy に) 変換する関数です。

8 行目 縦長になった変数 (Code::9.4 はこの段階の出力です) から，name 変数の値が pre であるものを 1 に，そうでないものを 2 にした新しい変数 cond を作りました。

このコードでどういうデータセットができたのか，R のコンソールで tidy_data として確認すると良いでしょう。1 つ 1 つのステップを確認したい場合は，途中で止めて一歩ずつ進むのも手です。

さて Stan にデータセットを与えるときは，このデータセットから 1. データ全体の長さ，2. 被験者の数，3. 被験者 ID，4. そのデータがどの水準なのかを示す識別子，5. 実際の値を取り出して渡すことになります。続いて Stan コードの例を見てみましょう。

code : 9.5 整然データ対応番

```
1  data{
2    int L;
3    int N;
4    array[L] int<lower=0,upper=N> IDindex;
5    array[L] int<lower=1,upper=2> Condition;
6    array[L] real val;
7  }
8
9  transformed data{
10   array[N] vector[2] pairX;
11   for(l in 1:L){
12     pairX[IDindex[l],Condition[l]] = val[l];
13   }
14 }
```

[*2] tibble 型にする必要は別にありません。著者の趣味です。tibble 型は data.frame 型の拡張版で表示した時に変数の型などがわかりやすいという利点があります。matrix 型からいきなり tibble 型に変更すると警告が出るので，いったん data.frame 型を経由しました。

```
15  ...(中 略)...
16  model{
17    pairX ~ multi_normal(mu,SIG);
18    //prior
19    mu[1] ~ uniform(0,100);
20    mu[2] ~ uniform(0,100);
21    rho ~ uniform(-1,1);
22    sd1 ~ cauchy(0,5);
23    sd2 ~ cauchy(0,5);
24  }
```

■コード解説

data **ブロック** 1. データ全体の長さ L，2. 被験者の数 N，3. 被験者 IDIDindex，4. そのデータがどの水準なのかを示す識別子 Condition，5. 実際の値 val を与えています。最後の 3 つはデータ長と同じサイズです。

transformed data **ブロック** データを変形する transformed data ブロックの登場です。ここではペアとしての情報がある vector 型にした変数を作っています。その変数 pairX は，N 人分の情報があり，それぞれ要素番号 1,2 がベクトルの要素に対応しています。ここでも入れ子になった識別子を使っていて，pairX[IDindex[l],Condition[l]] は l 行目の個人 IDindex[l]，l 行目の条件 Condition[l] から，たとえば IDindex[l] ＝ 15，Condition[l]=2 であれば 15 番目の人の事後の値 pairX[15,2] を値として代入する，ということをしています。

model **ブロック** 作った pairedX というベクトルに対して多次元正規分布からのデータを与えています。

少しややこしくなったように思えますが，データの構造が複雑化すると整然データの方がスッキリすることの方が一般的です。コードをよく読んで，どのような動きをするのかイメージをしっかり掴むようにしましょう。

9.3 個人差と変化量のモデルへ

さて，コードは随分と複雑になってきましたが，やっていることは「事前・事後で変化はあるか」ということだけです。それなのに多次元正規分布が出てくるなんて，面倒ですねえ！もう少し簡単な表現はできないものでしょうか。たとえば事前 + 効果=事後のように。

もちろん可能です。むしろその方が自然なモデリングと言えるかもしれないですね。ただその時，事前・事後が同一人物であることを忘れてはいけません。小杉の事前の状態 + 効果=山田の事後の状態，では意味がわからないからです。記号を使って書くと，$X_i^{pre} + effect = X_i^{post}$ のように，添字 i で同じ個人であることを明記しておく必要があります。

これを踏まえて，少し話を拡張した 3 水準モデルを考えてみましょう。たとえば次のようなカバーストーリーはいかがでしょう。

> 臨床心理学者がある介入法をつかってメンタルケアに取り組んでいます。4 人のクライアントにこの手法を適用し，抑うつ度が改善されていくかどうかチェックし，この介入法に効果があるのかどうかを検証したいと思っています。

チェックの結果が表 9.2 のようになっていて，ここの数字が抑うつ度スコア (低ければ低いほど健康) だと思ってください。記号で書く時のノーテーション (表記法) は，値 X_{ij} が i さんの第 j 期のスコア，ということにした

表 9.2 介入時期とスコアの推移

ID	1 期	2 期	3 期
1	10	5	9
2	9	4	5
3	4	2	3
4	7	3	5

いと思います。

これを使って毎回の変化の大きさを見積もりたいと思います。前回やった，多水準モデルのことを参考に見ていきましょう。ポイントは，前回は全体平均からの差分で効果を考えていたところが，今回は個人差があるので個人ごとの平均 μ_i を基準におくところです。

最初の人のスコア，$X_{11} = 10, X_{12} = 5, X_{13} = 9$ はそれぞれ，その人の平均 μ_i から考えて，$\mu_1 + \delta_1, \mu_1 + \delta_2, \mu_1 + \delta_3$ となるはずだと考えましょう。理論通りにならない誤差を含めて考えれば，先ほどの表 9.2 は表 9.3 のようになっているはずなのです。

表 9.3 介入時期とスコアのモデル式

ID	1 期	2 期	3 期
1	$X_{11} \sim N(\mu_1 + \delta_1, \sigma)$	$X_{12} \sim N(\mu_1 + \delta_2, \sigma)$	$X_{13} \sim N(\mu_1 + \delta_3, \sigma)$
2	$X_{21} \sim N(\mu_2 + \delta_1, \sigma)$	$X_{22} \sim N(\mu_2 + \delta_2, \sigma)$	$X_{23} \sim N(\mu_2 + \delta_3, \sigma)$
3	$X_{31} \sim N(\mu_3 + \delta_1, \sigma)$	$X_{32} \sim N(\mu_3 + \delta_2, \sigma)$	$X_{33} \sim N(\mu_3 + \delta_3, \sigma)$
4	$X_{41} \sim N(\mu_4 + \delta_1, \sigma)$	$X_{42} \sim N(\mu_4 + \delta_2, \sigma)$	$X_{43} \sim N(\mu_4 + \delta_3, \sigma)$

つまり，$X_{ij} \sim N(\mu_i + \delta_j, \sigma)$ ですね。個人のベースライン μ_i に，各時期の効果 δ_j が加わったものを中心に，正規分布に従う誤差を纏ってデータになる，という考え方です。ただしここでも注意が必要で，パラメータがいろいろありますが，効果は相対的なもの，すなわち $\delta_1 + \delta_2 + \delta_3 = 0$ という総和ゼロの制約を考える必要があります。実質的には自由度 2，すなわち $\delta_3 = 0 - (\delta_1 + \delta_2)$ となることを忘れないでください。

そしてもう 1 つ大事なことが。μ_i は人によって違う個人差成分ですが，この散らばりも正規分布に従うものとして考えることができますね。すなわち平均と散らばりをもった集合体が従う分布としての正規分布です。この平均を ψ，標準偏差を τ として[*3]，

$$\mu_i \sim N(\psi, \tau)$$

と考えることにしましょう。これを設計図に書き込んだのが，図 9.5 です。

このように，相関があることを個人 i に紐づけられた変数として表現し，個人差からの差分として対応のあるモデルを考えることができます。ここで，個人差に関係なく影響を与えている効果のことを**固定効果 (fixed effect)** と呼び，個人差のように交換可能な要素のことを**変量効果 (random effect)** と呼びます。また図 9.5 に示されているように，データ生成分布の中に個人差の分布が入れ子になって含まれていますね。このようなデータは階層モデルといい，とくに今回のように線形な階層モデルは**階層線形モデル (hierarchical linear model)** とよばれます。この展開については，次回以降にお話しすることになるでしょう。

ではこのモデルを実装していきましょう。設計図に沿えば Stan コードは書けるはずです。ここには Stan に

[*3] ψ はギリシア文字でプサイと読みます。τ も同じくギリシア文字でタウと読みます。

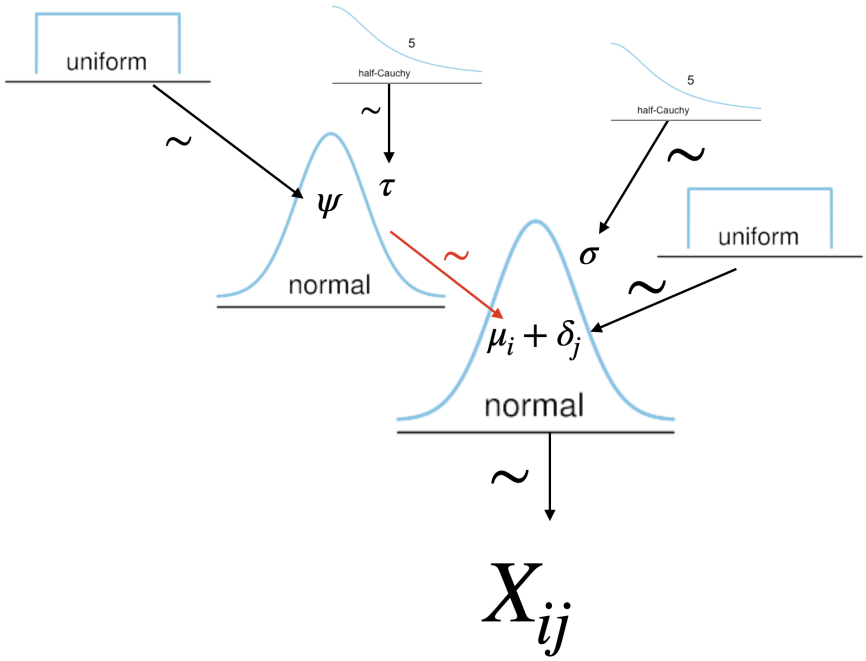


図 9.5 階層的データ生成モデル

与える R のコード例を用意しましたので, 自分なりのコードを書いて検証してみてください[*4]。

code : 9.6 整然データ対応番

```
1  dat_raw <- data.frame(ID = 1:4,
2                        period1 = c(10,9,4,7),
3                        period2 = c(5,4,2,3),
4                        period3 = c(9,5,3,5))
5  # 整然データにする
6  tidy_dat <- dat_raw %>%
7    pivot_longer(-ID) %>%
8    mutate(name = as.factor(name) %>% fct_relevel("period1","period2")) %>%
9    mutate(cond = as.numeric(name))
```

9.4 課題

今回の課題は大問 1 つ, 小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (.R) と, モデルを記述した Stan コード (.stan) の 2 つです。

ファイル名 chNN_学籍番号_氏名.R, chNN_学籍番号_氏名.stan としてください。NN は回の番号です。

動くこと 提出されたコードは, こちらの環境でそのまま実行して確認します。提出する前に, 新しいセッションを開いて最初から最後まで通して実行し, エラーが出ないことを確かめてください。動かないものは

[*4] これに限らず, コードに正しい書き方はありません。自分なりの書き方で結構です。判断すべきは正しく機能するかどうかであって, 書き方はあくまでも一例にすぎません。

未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；対応のある群の比較 Code::9.6 を参考にしながら，表 9.2 の事例を分析します。同じ人から 2 回測定したデータですので，個人差を考慮したモデルを考えましょう。

1. 2 時点を同時に扱う多変量正規モデルを Stan で書いてください。表 9.2 のうち **1 期と 2 期**を使います。本講の書き方にならい，母数は 2 時点の平均 `mu`，時点ごとの標準偏差 `sd1`，`sd2`，時点間の相関 `rho` とすること。
2. このモデルでベイズ推定を実行し，2 時点の平均の差 (`mu[2]-mu[1]`) について，EAP 推定値，MAP 推定値，90% 確信区間の下限と上限を報告してください。あわせて `rho` の EAP 推定値と 90% 確信区間の下限と上限も示すこと。`ans2` の要素名は `diff_eap`，`diff_map`，`diff_lower`，`diff_upper`，`rho_eap`，`rho_lower`，`rho_upper` とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. 2 時点の平均の差が 0 より大きい確率を求め，条件によって値が変化したといえるかどうかを判断してください。`ans3` の要素名は `p_diff_over0` とすること。

第 10 章

モデリングの目から見た検定 5；カテゴリカル分布をつかって

ここまでさまざまなモデルを使って，パラメータ間あるいはデータの間の差を検討するということを考えてきました。ここで差分を計算できているということは，数値が**連続的**であったということもできます。いいかるならば，**間隔尺度水準**以上の尺度水準で得られたデータに対する検証だったわけです。

ところがデータの種類によっては**順序尺度水準**や**名義尺度水準**でしか得られないこともあります。試験に合格したのか，失敗したのか。男性か，女性か。47 都道府県のどこ出身なのか。あるいはたとえば，正解率や成功率，相対的な比率などでデータが現されたり考えたりすることもあります。比率は連続的な数字ではありますが，$\frac{X}{Y}$ のように数え上げによる度数の比較ですから，元は**離散的**な，質や種類を区別するための数字による (度数に該当するかどうか) 情報だと言えるでしょう。たとえば比率のデータに対して t 検定のような分析をするのは適切ではなく[*1]，カテゴリカルな分布を考えて検証する必要があります。

また，複数のカテゴリカル変数の組み合わせによって**クロス集計表 (cross-tabulation table)** あるいは**分割表 (contingency table)** を利用することもあります。男性の X 割が A 党に，女性の Y 割が B 党に投票した，といった表は，性別 $\times$ 支持政党のクロス集計表ですが，これを見ることで性別による支持政党の偏りがあるのかないのか，といったことがわかります。クロス集計表は社会科学的なデータにも多く見られ，カテゴリカルな性質の組み合わせについての情報を提供してくれます。この組み合わせに偏りがあるのかないのか，と言ったことを考える際にも，確率モデルとしてはカテゴリカルな分布を必要とします。

10.1 離散的な分布

それでは代表的な離散変数についての**確率分布**を見ていきましょう。離散分布の場合，確率関数は密度 density ではなく，質量 mass，つまり**確率質量関数 (probability mass function)** と呼ばれることに注意してください。カテゴリーの中に含まれるかどうか，どれぐらいの量がそこに含まれているかということを直接表しているからです。

■ベルヌーイ分布　離散的な分布の基本は**ベルヌーイ分布 (Bernoulli distribution)** です。この分布から出てくる数字は 0 か 1 であり，確率 θ で 1 が，$1-\theta$ で 0 が出る分布です。たった 2 つの状態しか区分しませんが，正答と誤答，賛成と反対，男性と女性，生と死などさまざまなもののメタファーとして利用できるため，応用範囲はむしろ広いと言えるでしょう。パラメータは θ ひとつですので，わかりやすいですね。Stan での関

[*1] t 検定は正規分布するデータを前提にしている検定方法・確率分布であり，比率のデータはとうてい正規分布とは考えられないからです。正規分布の定義域は $-\infty$ から ∞ なのに対し，比率は 0 から 1 でしかないことからも明らかです。

数は bernoulli と書き[*2]，y ~ bernoulli(theta); のように使います。

今後出てくることになりますが，ロジスティック関数と組み合わせて使うことが多い分布です。ロジスティック関数は，連続的な数字を 0 から 1 の範囲に変換する関数で，変換したものをベルヌーイ分布のパラメータにする合わせ技が非常に便利だからです。たとえば学力などは正規分布すると考えられ，$-\infty$ から ∞ の値を取りうることになりますが，その能力を反映してテストに正答するか誤答するか，ということを考えると学力 θ をロジスティック関数にいれた ($logistic(\theta)$) ものをベルヌーイ分布にいれる (X ~ bernoulli(logistic(theta))) からです。この合わせ技はよく使われるので，Stan では bernoulli_logit という関数があるほどです。

■二項分布 パラメータ θ をもつベルヌーイ分布に従うコイントスを N 回やったときに，何回表が出るか，というときに使うのが**二項分布 (binomial distribution)** です。10 問中何問正解するかとか，N 個提示した刺激のうち覚えていたのはいくつかとか，N 回シュートして入ったのは何回かといった，割合や比率に関係する分布です。パラメータは N と θ の 2 つあり，Stan での関数は binomial です。y ~ binomial(N, theta); のように書きます。

■カテゴリカル分布 ベルヌーイ分布や二項分布はコイントスの「表」のことしか考えていませんでした (裏は「表ではない」という考え方)。これに対してカテゴリカル分布はサイコロの出目のように，1,2,3,4,5,6 のどれが出るか，という多段階の乱数発生を考える分布です。こん関数のパラメータはサイズ K のベクトルで，たとえばサイコロの場合は $\boldsymbol{\theta} = (1/6, 1/6, 1/6, 1/6, 1/6, 1/6)$ のようになります。Stan での関数は categorical と書き，使う時は y ~ categorical(theta); のようにします。このとき theta はベクトルで宣言されていなければなりません。

■多項分布 カテゴリカル分布と似ているようですが，この分布は N 回サイコロを降った時の各出目の回数であり，1 が k_1 回，2 が k_2 回，$\cdots$，6 が k_6 回出た，というようにカウントしたものを出力します。ABO 型の血液分類で，サンプルの中に A 型が何人，B 型が何人‥というときに，この母比率を推定する時は多項分布のパラメータを推定する，ということになります。二項分布の多変量版だと考えてもいいかもしれません松浦 (2016)。この分布に与えるパラメータはベクトル $\boldsymbol{\theta}$ であり，Stan では y ~ multinomial(theta); のように使います。総数 N はデータベクトル $\boldsymbol{y}$ の要素の総和 ($N = \sum_{i=1}^{K} y_i$) ですから指定しなくても構いません。ここで注意すべきは，theta はサイズ K のベクトルであり，すべての要素を足し合わせると 1 になる必要があります。Stan では合計 1 のベクトル専用の型があり，それは simplex と呼ばれます。

10.2 χ^2 検定

カテゴリカルなデータ，度数分布などをどうやって研究の時に使うのか，具体的な例とともに見ていきましょう。ベイズ推定の話に入る前に，帰無仮説検定の話から先に考えてみたいと思います。

表 10.1 には，3 つの携帯電話キャリアのユーザ数をとある大学で調査した例です。合計 123 名から回答を得て，それぞれ 51 名，45 名，27 名ということが判明しました。これを見て「A 社は多いね，学生人気だね」などと解釈してもいいのですが，これでは推測統計学の域を出ていませんね。違うサンプルを対象にすれば違う度数になる可能性があるからです。そこで推測統計学的な発想をします。

ここで検証したいのは「どこかのカテゴリが多く，どこかのカテゴリは少ない」，つまり「散らばりに偏りがある」ということです。逆に考えると帰無仮説が出てきます。つまり「散らばりに偏りはない」が帰無仮説であり，

[*2] 対数尤度で書く場合は bernoulli_lpmf となります。lpmf は log probability mass function の略です。

表 10.1　携帯キャリア調査

キャリア	A 社	D 社	S 社	合計
観測度数	51	45	27	123
期待度数	41	41	41	

対立仮説は「偏りがある」ということになります。散らばりに偏りがない，という帰無仮説の世界の元では，期待度数は総数を均等に割った値になるはずです。表 10.1 には**期待度数 (expected frequency)** という行を設けました。ちなみに得られたデータそのものは**観測度数 (observed frequency)** と言います。

今回のデータが帰無仮説の元ではどれぐらいあり得ない数字だったのでしょうか。これを考えるために，χ^2 値を計算します。χ はギリシア文字のカイであり，x(エックス) とは違うので注意してください。χ^2 値はカイ二乗値（にじょうち）と読みます。この値は次のように計算します。

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

ここで O_i は観測度数，E_i は期待度数です。今回の場合は次のような数字になります。

$$\chi^2 = \frac{(51-41)^2}{41} + \frac{(45-41)^2}{41} + \frac{(27-41)^2}{41} = 2.439 + 0.390 + 4.780 = 7.609$$

計算式から明らかなように，期待度数からのずれの大きさを表現していますから，期待通り＝帰無仮説通りであればこの値は 0 になります。今回は $\chi^2 = 7.609$ ですが，この統計量がどれぐらい出てきやすいかということは，自由度 2 の χ^2 分布をつかって検証できます。ここでの自由度は選択カテゴリ数 -1 です[*3]。自由度 2 の χ^2 分布から 7.609 という実現値が出てくる確率は $p = 0.022$ で 5% より小さいですから[*4]，統計的に有意な偏りがある，と判断できます。

χ^2 は偏りを検証するための数字です。今回は均等である，という帰無仮説から算出しましたが，母集団比率が事前にわかっているのであればそれを用いて，手元のサンプルがどれぐらい偏っているかを検証するということも可能です。たとえば日本人の血液型はおよそ A 型が 40%，O 型 30%，B 型 20%，AB 型 10 %といわれていますから，期待度数をこの比率で割り振るとサンプルが偏っていたかどうかの検証になります[*5]。他にも (比率によらず) 理論的な度数がわかっていることがあれば，どれぐらい合致しているかを検証できます[*6]。

また同様のロジックで，クロス集計表の検定を行うこともできます。クロス集計表の場合は**独立性の検定**と呼ばれることもあります。偏りがあれば関係がある，独立ではないということですね。

例として，表 10.2 には，先ほどの携帯キャリア調査の結果に男女の区別もつけてみました。これを見て「男性は D 社が好きで，女性は A 社が好き」といった偏りがあると判断できるかどうかを検証できます。

各セルの期待度数は行の周辺度数 × 列の周辺度数 ÷ 総度数で計算でき，各観測度数から期待度数を引いて二乗したものを，さらに期待度数で割ったものを足し合わせることで χ^2 にするのは同じです。これが (行の数 -1)×(列の数 -1) の自由度をもつ χ^2 分布に従いますので，今回は $\chi^2 = 6.4326, df = 2, p = 0.0401$

[*3] 合計数が 123 と決まっていますから，最初の二つのカテゴリに入る数字が決まれば残りの一つは自動的に決まります。自由に値が変えられるのは二つまでなのです。

[*4] この値は R で `1-pchisq(7.609,df=2` とすることで得られます。もっと直接的にするなら，`chisq.test(c(51,45,27))` とすると χ^2 検定が実行されます。

[*5] 母比率の検定と言います。

[*6] **適合度**の検定といいます。モデルフィットの指標としても使われ，たとえばデータ解析応用 1 で説明した**構造方程式モデリング**の指標などでも用いられています。

表 10.2 携帯キャリア調査と性別の関係

キャリア	A 社	D 社	S 社	合計
男性	21	30	13	49
女性	30	15	14	74
合計	51	45	27	123

であることから有意 (に偏っている), と判断できます[*7]。これは**分散分析**の時と同様に, どこかに偏りがあるという全体的な傾向を示しただけであることに注意が必要です。

このようにして度数の検定を行うことができます。これに対して, データ生成メカニズムを考える場合は, ある比率に沿って度数が出てきていると考えますから, もっと直接的に母比率の検定を考えることができますし, MCMC によるアプローチを使うと「どこが, どの程度大きいと言えるか」, といったことについても検証できます。次にその方法を見ていきましょう。

10.3 カテゴリカル分布のモデリング

ではまず, 表 10.1 にあげた携帯キャリア調査の結果を, データ生成モデルから考えてみましょう。設計図にするまででもない感じで, 51:45:27 というデータの比率から考えられる, 母比率ベクトル $\boldsymbol{\pi} = \{\pi_1, \pi_2, \pi_3\}$ を考え, そこからデータベクトル $\boldsymbol{y}$ が多項分布に従って出てくる, と考えるわけです。

Stan のコードは Code::10.1 のようになります。ポイントは特殊なベクトル `simplex` で $\boldsymbol{\pi}$ を宣言しているところでしょうか。またデータベクトルは `vector[K] X;` としたいところですが, モデルからいってここは `int` 型である必要があり, 型指定をしたベクトルというのがないので, 配列にしました。

code : 10.1 多項分布からのデータ生成モデル

```
data{
    int K;
    array[K] int X;
}

parameters{
    simplex[K] pi;
}

model{
    X ~ multinomial(pi);
}
```

これを次のような R コードで実行してみましょう (Code:10.2)

code : 10.2 多項分布のパラメータ推定

```
model <- cmdstanr::cmdstan_model("categorical1.stan")
fit <- model$sample(
  data = list(K = 3, X = c(51, 45, 27)),
  chains = 4,
  parallel_chains = 4,
```

[*7] この計算は `chisq.test(matrix(c(21,30,13,30,15,14),ncol=3,byrow=T))` で算出しました。

```
6     refresh = 500
7   )
```

構造も何もないモデルですから，推定はすぐに終わると思います。$51 : 45 : 27 = 0.4146 : 0.3658 : 0.2195$ ですから，ほぼ標本分布と同じような形で事後分布が推定されました (図 10.1)。これで終わり，といった

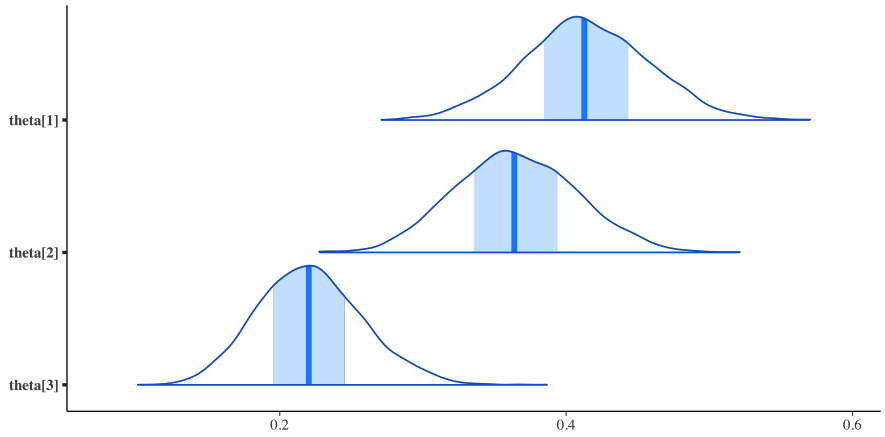


図 10.1　比率ベクトルの推定結果

らそれまでなのですが，仮説検定のように考えてみましょう。三社それぞれが均等に選ばれている，つまり $\pi_1 = \pi_2 = \pi_3$ というのはあり得そうにないですが (ピッタリ一致するはずはない)，たとえば

- A 社よりも D 社の方が好まれている
- S 社よりも D 社の方が好まれている
- D 社が最も好まれている

といった仮説は生成量を使って検証できそうです。1 つ目は $\pi_1 > \pi_2$，2 つ目は $\pi_2 > \pi_3$ が成立する割合から考えれば良いからです。3 つ目の仮説はこの 2 つと同じじゃないか，と思うかもしれませんが，正確には $\pi_2 > \pi_1$ かつ $\pi_2 > \pi_3$ という両方の可能性が成立していること，と考える必要があります。こうした「A かつ B」のような命題は**連言命題**といいます。

これを検証する生成量の作り方は，次のようになります。他にもいろいろ考えられそうですね。

code : 10.3　多項分布からのデータ生成モデル

```
1  ...前略...
2  generated quantities{
3      int<lower=0,upper=1> FLG1;
4      int<lower=0,upper=1> FLG2;
5      int<lower=0,upper=1> FLG3;
6
7      if(pi[1] > pi[2]){ FLG1 = 1; } else { FLG1 = 0; }
8      if(pi[2] > pi[3]){ FLG2 = 1; } else { FLG2 = 0; }
9      if(pi[2] > pi[1] && pi[2] > pi[3]){ FLG3 = 1; } else { FLG3 = 0; }
10 }
```

10.3.1 対応のない二変数の場合

続いてクロス表の検定に行きましょう。表 10.2 にあるように，携帯キャリアの選び方と性別の関係をみたいと思います。男性と女性は独立したカテゴリですから，先ほどの多項分布モデルが 2 つ同時にあることと同じです。

男性は 21:30:13 という観測度数が得られていますが，これは $\boldsymbol{\pi_M} = c(\pi_1^M, \pi_2^M, \pi_3^M)$ という母比率を持っており，また女性は 30:15:14 という観測度数から，$\boldsymbol{\pi_F} = c(\pi_1^F, \pi_2^F, \pi_3^F)$ という母比率だったのではないか，と推測することになります。

「男性と女性とで携帯キャリアの選択率に違いがあるか」ということが検証したい命題になるかと思いますが，何を持って違いがあるとするか，ということを厳密に考えて生成量を作らなければなりません。たとえば，

- 男性は女性よりも A 社を好む $\rightarrow \pi_1^M > \pi_1^F$
- S 社は女性の方が多く選んでいる $\rightarrow \pi_3^M < \pi_3^F$
- 男性は D 社が一番好きで女性は A 社が一番好き $\rightarrow \pi_2^M > \pi_1^M$ かつ $\pi_2^M > \pi_3^M$ かつ $\pi_1^F > \pi_2^F$ かつ $\pi_1^F > \pi_3^F$

というように考えられます。とくに 3 番目の命題は，「一番好き」というのを「他のどの群と比べても大きいという条件が同時に成り立っている」と考える必要がありますし，「男性は･･･で，女性は･･･」というときの「で」を論理的には「かつ」，確率的には「同時に」という意味で捉えないといけないところがポイントです。

このように，生成変数を使うとさまざまな仮説を検証することが可能です。集計表に関するデータは，官公庁が公開しているものなどを含め，身の回りのさまざまなところで目にできます。これらの資料を見た時に，標本統計量だけでなく母比率にまで思いを馳せて，検証可能な仮説を色々考えてみると良いでしょう[*8]。

ところでこの生成量を使った連言命題の検証については，今回のデータから考えられる事後分布に依存していることを忘れないようにしてください。今回のデータから言える「命題が成立する確率」は，事前分布とデータに基づいている結果ですから，「仮説が正しい確率」とまで言い切るのは危険です。あくまでもモデルに基づく推定であることを忘れないようにしましょう。

10.4 κ 係数の算出

先ほどのクロス集表の例では，男性と女性という独立した群から作られていましたので，2 つの母集団を別々に考えることができました。しかしたとえば，「法案 A,B それぞれに対して賛成か反対か」といった調査をした場合には，「A にも B にも賛成」「A には賛成，B には反対」「A には反対，B には賛成」「A にも B にも反対」という 2×2 のクロス集計表が得られますが，これは一人の人間が 2 つの質問に答えていますので**対応のある**データになります。対応のあるデータの場合は，先ほどと同じように独立した確率分布を考えるわけには行きません。

また「ある病院にいったら健康だと診断されたが，別の病院に行ったら異常が発見された」とか，「ある専門家の見立てでは問題のある行動といわれたが，別の専門家の見立てでは問題ないといわれた」，「ある映像

[*8] たとえばテレビのバラエティ番組や情報番組においても，該当アンケート結果についてコメンテータが色々物申すことが少なくありません。「エビフライのしっぽは食べるのか，残すのか」とか「焼き鮭の皮は食べるのか残すのか」といった些細なことについて，無作為抽出とは思えないような該当インタビューをし，僅差でも多数派を勝者と見立てて騒ぎ立てる様をみる度に，筆者は「せめてちゃんと検定，推定してから結論づけるべきだ」と思い，常々腹立たしく感じていました。しかし妻に「そういう些細なことを，僅差でも大袈裟にいうことで，お茶の間に盛り上がる話題を提供しているのであって，学術報告ではない」と諭されたことがあって，なるほどそういうものかと納得した次第です。

を見て判断する課題で，審査者 A と B の判断が一致したりしなかったりする」といったシーンを考えてみてください。これらのシーンでは，2×2 のクロス集計表が表 10.3 のようになっていると考えられます。ここでは Yes/No としてありますが，Hit/Miss でも健康/異常でもいいのですが，ともかく 2 つの判断が合致したかどうか，ということが問題になるシーンです。

表 10.3　対応のあるカテゴリ判断

		A		
		Yes	No	
B	Yes	a	c	
	No	b	d	
				n

このような時も，行と列の判断が独立しているとは言えません。むしろどの程度重複しているかのほうが問題になるわけです。こういったときは，判断が一致した程度を**カッパ係数 (kappa coefficient)** で表現できます[*9]。この係数は，観測された一致度，すなわち

$$p_o = \frac{(a+d)}{n}$$

と，偶然の一致度

$$p_e = \frac{(a+b)}{n}\frac{(a+c)}{n} + \frac{(b+d)}{n}\frac{(c+d)}{n} = \frac{(a+b)(a+c)+(b+d)(c+d)}{n^2}$$

から，次のように計算されます[*10]。

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

この式の分子は「合致した割合から偶然の一致をひいたもの」であり，分母は「偶然ではない割合」になっていることから，一致の程度を表す指標として考えられるのです。この κ 係数は -1 から $+1$ の範囲の値を取ります。

このように，2 つの群わけが独立ではない場合の確率はどう考えればいいでしょうか。$a:b:c:d$ を $\pi_1:\pi_2:\pi_3:\pi_4$ と考えるのでは 4 つのセルが独立だと考えていることになってしまいます。そうではなく，順番にまず A が確率 α で Yes と判断する，ということを考えましょう。No と判断する確率は $1-\alpha$ です。次に B の判断ですが，A が Yes と判断した時に B が Yes と判断する確率を β としましょう。また A が No と判断した時に B も No と判断する確率を γ と考えます。

そうすると，両者が Yes と答える確率 $\pi_a = \alpha\beta$，A が No で B が Yes と答える確率 $\pi_b = (1-\alpha)\beta$，A が Yes で B が No と答える確率 $\pi_c = \alpha(1-\gamma)$，両者が No と答える確率 $\pi_d = (1-\alpha)\gamma$ のようにあらわすことがで切るようになります (図 10.2)。κ 係数はこれら π_a〜π_d の組み合わせで計算できますから，生成量を使えば算出できますね。

それではこれをコードにしていきましょう。確率ですから，変数の範囲は 0 から 1 に制限し，事前分布もこの範囲の一様分布にしています。

[*9] カッパ係数は河童ではなく，ギリシア文字の k にあたる κ です。

[*10] A が Yes と判断する割合 $\times$B が Yes と判断する割合を，A が No と判断する割合 $\times$B が No と判断する割合に足している数字が p_e です。割合 (=確率) の積なので二つの条件が偶然に一致する割合を，Yes-Yes のケースと No-No のケースで算出し，合わせたものと理解すればいいでしょう。

		Aの判断: Yes	Aの判断: No
Bの判断	Yes	$\pi_a = \alpha\beta$	$\pi_c = (1-\alpha)(1-\gamma)$
	No	$\pi_b = \alpha(1-\beta)$	$\pi_d = (1-\alpha)\gamma$

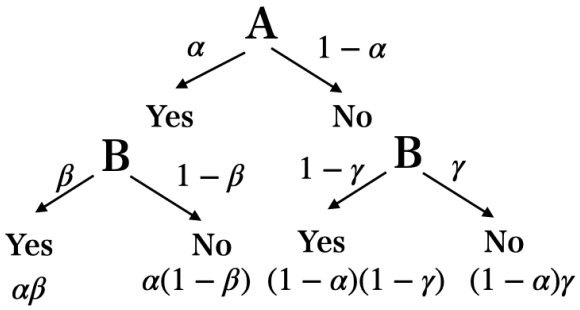


図 10.2　対応のあるカテゴリ判断の確率

code : 10.4　κ 係数を計算するモデル

```
1  data{
2      array[4] int Y;
3  }
4
5  parameters{
6    real<lower=0,upper=1> alpha;
7    real<lower=0,upper=1> beta;
8    real<lower=0,upper=1> gamma;
9  }
10
11 transformed parameters{
12   simplex[4] Pi;
13   Pi[1] = alpha * beta;
14   Pi[2] = (1-alpha) * (1-gamma);
15   Pi[3] = alpha * (1-beta);
16   Pi[4] = (1-alpha) * gamma;
17 }
18
19 model{
20   Y ~ multinomial(Pi);
21   alpha ~ uniform(0,1);
22   beta ~ uniform(0,1);
23   gamma ~ uniform(0,1);
24 }
25
26 generated quantities{
27   real po;
```

```
28    real pe;
29    real kappa;
30    po = Pi[1] + Pi[4];
31    pe = (Pi[1]+Pi[2])*(Pi[1]+Pi[3]) + (Pi[2]+Pi[4])*(Pi[3]+Pi[4]);
32    kappa = (po-pe)/(1-pe);
33  }
```

カテゴリカルな分布はこのように度数に関するモデルを考えることができますし，この分布を応用して潜在的な変数がカテゴリカル分布に従うと考えると，分類わけを表現することもできます。さまざまな応用例が思いつくのではないでしょうか。

10.5 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；お菓子の味の好み とある棒状のお菓子にはさまざまな味のバリエーションがありますが，コーンポタージュ味，チーズ味，めんたい味，野菜サラダ味，たこ焼き味あたりが人気上位に入るそうです。そこで実験を行って，最も美味しい味に投票してもらったところ，コーンポタージュ味が 105 票，チーズ味が 80 票，めんたい味が 75 票，野菜サラダ味が 60 票，たこ焼き味が 45 票でした。

1. この投票結果を多項分布でモデル化し，5 つの味それぞれが選ばれる確率を推定するモデルを Stan で書いてください。母数として，長さ 5 の単体 (`simplex`) である `pi` を宣言すること。
2. このモデルでベイズ推定を実行し，5 つの確率それぞれについて，EAP 推定値と 90% 確信区間の下限と上限を報告してください。味の順序は問題文のとおり，コーンポタージュ，チーズ，めんたい，野菜サラダ，たこ焼きとします。`ans2` の要素名は `pi1_eap` から `pi5_eap`，`pi1_lower` から `pi5_lower`，`pi1_upper` から `pi5_upper` の 15 個とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. MCMC サンプルを使って，次の 2 つの確率を求め，それぞれ言えるかどうかを判断してください。`ans3` の要素名は `p_corn_over_cheese`，`p_corn_best` とすること。

- コーンポタージュ味がチーズ味よりも好まれている確率
- コーンポタージュ味が他のどの味よりも好まれている確率

第 11 章

一般化線形モデル

11.1 一般線形モデル

ここまで検定と線形モデルが同じ形をしていることについては，何度か指摘してきました。検定モデルは，

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$$

という形をしており，ここでとくに X_i が統制群に属するか，実験群に属するかを 0/1 で表現したものであったことを再確認しておきましょう。たとえば二水準モデルの場合，統制群の場合 $X_i = 0$ ですから，データは $Y_i = \beta_0 + 0 \times \beta_1 + \varepsilon_i = \beta_0 + \varepsilon_i$ であり全体平均に誤差がついただけの値になります。実験群の場合は $Y_i = \beta_0 + 1 \times \beta_1 + \varepsilon_i = \beta_0 + \beta_1 + \varepsilon_i$ になりますから，この β_1 の部分が効果の大きさを表しているのでした。

水準数が増えても同じことです。3 水準モデルの場合はベクトルを使って表記した方がわかりやすいので，データセットを $\boldsymbol{Y}$ と表すと，

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon} \tag{11.1}$$

であり，詳しく書くと

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ \vdots & \vdots & \vdots \\ 1 & -1 & -1 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

のようになります[*1]。ここで行列 $\boldsymbol{X}$ はその群に属しているかどうか (その群の効果が発動するかどうか) を 0/1 で表しているデザイン行列 (design matrix) であり，有無という状態に一対一対応した名義尺度水準の数字です。説明変数が名義尺度水準になっているだけで，これが間隔尺度水準以上の量的なデータであれば，回帰分析 (regression analysis) になるわけです。回帰分析は一般に，

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \varepsilon$$

あるいはベクトルで書くと

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon} \tag{11.2}$$

となります。この通り式で書くと 11.1 と 11.2 にはなんら違いがないことがわかりますね。

このように要因計画と線形モデルは同じである，ということを含意して，これをまとめて一般線形モデル (general linear model) と言います[*2]。

[*1] $\beta_3 = 0 - (\beta_1 + \beta_2)$ となっていることに注意。

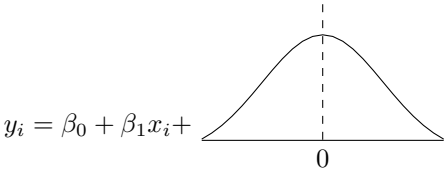
[*2] 一般，というのは同じだという意味があって，辞書には「私は彼女と同じい罪を犯したも一般だ」といった用法が載っています。

11.1.1 回帰分析の確率モデル

ところで，この一般線形モデルでは誤差が正規分布に従うという仮定を置いていました。すなわち，$\varepsilon \sim N(0, \sigma)$ というわけです[*3]。線形モデルの部分，すなわち $\hat{Y}_i = \beta_0 + \beta_1 X_i$ のところは確率的ではありませんから，データ全体では次のようになっていることになります。

$$y_i = \underbrace{\beta_0 + \beta_1 x_i}_{\text{確率的でないところ}} + \underbrace{e_i}_{\text{確率的}}$$

誤差 e_i は平均 0 を中心に確率的に散らばりますが，その前のところは確率的ではありませんので，これを組み合わせた式全体としては，最後に確率的な散らばりがひっついているような形になります。数式ではありませんが，イメージでいうと次のような形でしょうか。



このことから，モデル全体の確率的挙動は，

$$Y_i \sim N(\beta_0 + \beta_1 X_i, \sigma) \tag{11.3}$$

ということになります。これが正規分布の確率モデル的表現です。一般線形モデルですから，説明変数 X が連続的でも離散的でも同じモデルです。また，この正規分布は平均と SD でその形状が定められるのでした。平均は分布の位置を表すので，別名**位置母数 (location parameter)** といいます。ちなみに SD のことは別名**尺度母数 (scale parameter)** と呼ぶことがあります。一般線形モデルは平均的な位置を線形で予測して，データはそれに付随した誤差が幅 σ でぶれる，と考えます。そういう意味では，図 11.1 のように，回帰線にそって誤差分布があり，結果として実現値が得られているイメージ図のほうがわかりやすいかもしれません。

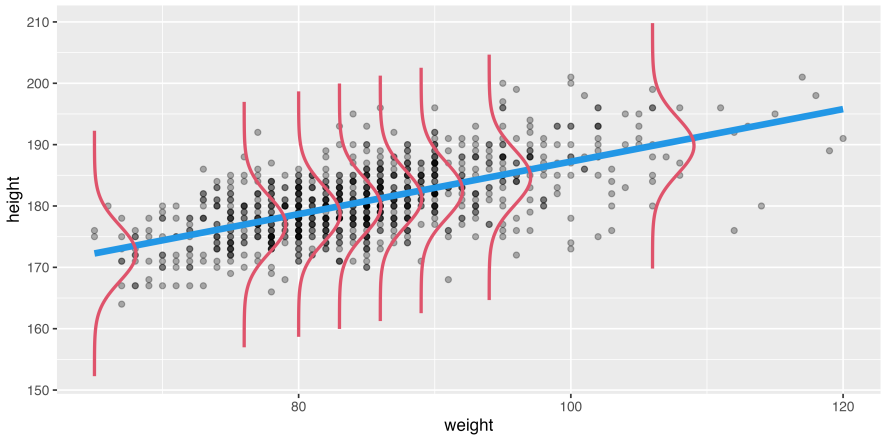


図 11.1 回帰線に伴う誤差分布

[*3] 誤差の平均はゼロであるのは，偶然誤差の仮定から必然的です。

これを確認した上で，では回帰分析をベイズ的に推定するコードを書いてみましょう。設計図も一応準備しておきます。図 11.2 では，μ が $\beta_0 + \beta_1 X_i$ で作られているので，矢印のところは $\sim$ ではなく $=$ になっています。また，未知数 β_0, β_1 については平均 0，SD100 のとても幅の広い正規分布を置いています。一様分布でもいいのですが，原理的に $\pm\infty$ までの幅があり，どこか 1 つの値に収束するだろう＝単峰性があるだろう，ということで正規分布を選んでいます。誤差 SDσ については，半コーシー分布[*4] 設計図をそのままコードに起こせば一丁上がりです。事後予測分布も生成するコードを書いてみました。

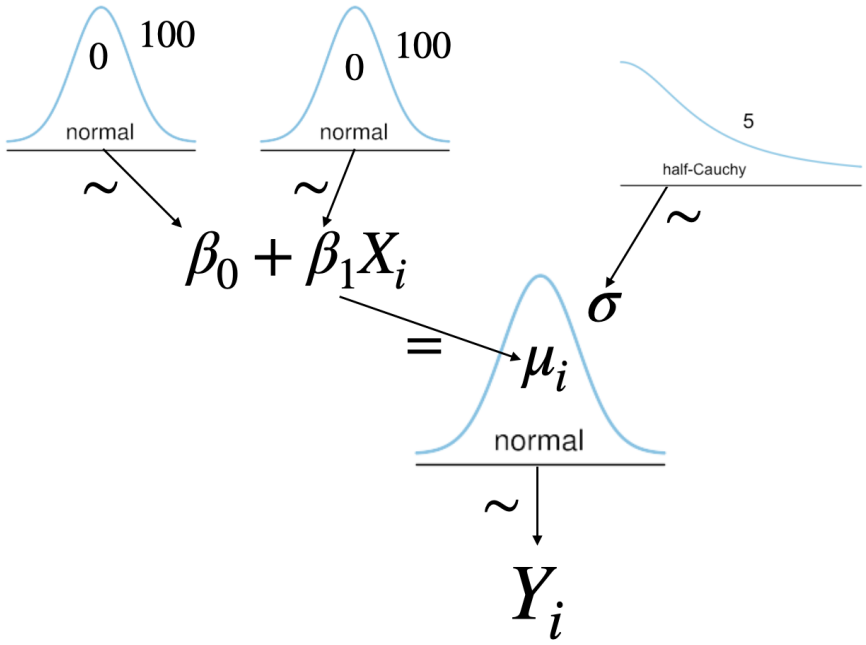


図 11.2　線形モデルの設計図

code : 11.1　線形モデルの Stan コード

```
data{
  int N;
  array[N] real X;
  array[N] int Y;
}

parameters{
  real beta0;
  real beta1;
  real<lower=0> sig;
}

transformed parameters{
   array[N] real mu;
   for(i in 1:N){
      mu[i] = beta0 + beta1 * X[i];
```

[*4] 正の値しか取らないように，コーシー分布を 0 のところで半分に折り畳んだ形にした分布です。Stan 上では `lower=0` の制限をかけることで正の範囲からしかサンプリングしない，ということで表現します。

```
17     }
18   }
19
20   model{
21     // model
22     for(i in 1:N){
23       Y[i] ~ normal(mu[i],sig);
24     }
25     // prior
26     beta0 ~ normal(0,100);
27     beta1 ~ normal(0,100);
28     sig ~ cauchy(0,5);
29   }
30
31   generated quantities{
32     array[N] real predY;
33     for(i in 1:N){
34       predY[i] = normal_rng(mu[i],sig);
35     }
36   }
```

これがどのような挙動をするのか，実際のデータセットを使って検証してみましょう。サンプルデータとして，BaseballDecade.csv を使います。このデータセットはプロ野球データ Freak[*5]さんから取ってきたものです[*6]。

この野球データセットにはさまざまな情報が含まれていますが，身長と体重のデータだけ取り出して回帰分析をしてみましょう。身長を体重から予測することにします。ここで身長は正規分布に従っていると考えられますから，一般線形モデルが適切ですね。データの散布図を書いて，線形関係にありそうかどうか確認しておきます (図 11.3。

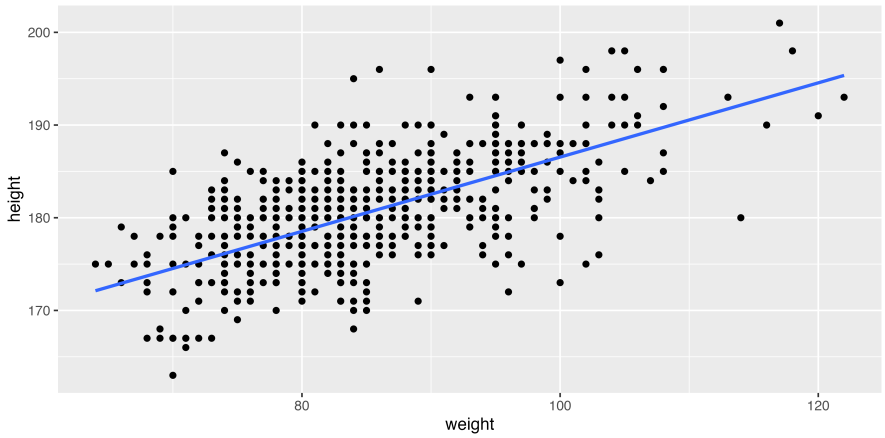


図 11.3 身長を体重で回帰するモデル

そこそこ相関がありそうですね。では回帰モデルを当てはめてみましょう。

[*5] https://baseball-data.com/

[*6] データセットはシラバスのサイトで提供しています。

code : 11.2　一般線形モデルを実行するコード

```
dat <- read_csv("BaseballDecade.csv") %>% filter(Year == "2020年度")
dataSet <- list(N = NROW(dat), Y = dat$height, X = dat$weight)
## cmdstanrで実行する場合
modelC <- cmdstan_model("LM.stan")
fit <- modelC$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 2000
)
## 事後予測を取り出す
predY <- fit$draws("predY", format = "matrix")
# 事後予測分布の描画
bayesplot::ppc_dens_overlay(y = dataSet$Y, yrep = predY[1:10, ])
bayesplot::ppc_intervals(
    y = dataSet$Y,
    yrep = predY,
    x = dataSet$X,
    prob = 0.5,
    prob_outer = 0.95
)
```

■コード解説

1 行目　データを読みこみます。

2 行目　stan に与えるデータセットをリストで作ります。

3-11 行目　cmdstanr でコンパイルしてサンプリングします。

13 行目　事後予測を取り出します。`fit$draws` で `predY` だけを取り出し，`format = "matrix"`で行列の形にしています。

15 行目　事後予測分布の一部です。予測値は MCMC サンプル数 (ここでは 8000 点)× サンプルサイズ (野球選手 664 人) ありますが，このうち最初の 10 人についての実際のデータ `y` と予測値 `yrep` を重ねて表示しています。全員分表示させると非常にサイズが大きくて重たいからです。

16-22 行目　データの散布図に予測値とその 50% および 95% 確信区間をプロットしました。

今回の分析結果を単純に表示するなら，次のようになります (出力 11.1)。

cmdstanr の出力 11.1: ベイズ推定の結果

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
beta0	146.48	146.49	1.59	1.60	143.89	149.14	1.00	2347	2615
beta1	0.40	0.40	0.02	0.02	0.37	0.43	1.00	2337	2655
sig	4.40	4.39	0.12	0.12	4.21	4.60	1.00	2937	3104

これまで習ってきた**最尤法 (maximum likelihood method)** の結果と出力の違いを比較しましょう (出力 11.1)。

R の出力 11.1: ML 推定の結果

```
Call:
lm(formula = height ~ weight, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-13.541  -2.792   0.068   2.866  15.066

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 146.49143    1.57882   92.79   <2e-16 ***
weight        0.40050    0.01844   21.72   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 4.392 on 662 degrees of freedom
Multiple R-squared:  0.4162, Adjusted R-squared:  0.4153
F-statistic: 471.9 on 1 and 662 DF,  p-value: < 2.2e-16
```

最尤法では，切片が 146.49143，傾きが 0.40050 という**点推定**だけが表示される形になっていますが，MCMC を使ったベイズ推定では切片の平均 (**EAP 推定値**) が 146.48，中央値 (**MED 推定値**) が 146.49，90% 確信区間が [143.89,149.14] となっています。同じく傾きの平均が 0.40，中央値も 0.40，90% 区間が [0.37,0.43] となっています。結果が分布として得られますから，初めから幅を持った推定になっていることが確認できます。

事後予測分布 (図 11.4) をみると，いくつか予測幅から外れているデータ点はありますが，概ね回帰線の範囲に捉えられており，線形モデルの当てはまりがそこそこ良いことが確認できます。事後予測分布が元データとよく当てはまるということは，データ生成メカニズムとして正規分布や線形モデルを想定したわれわれのモデルが悪くないんだ，と解釈していいでしょう。

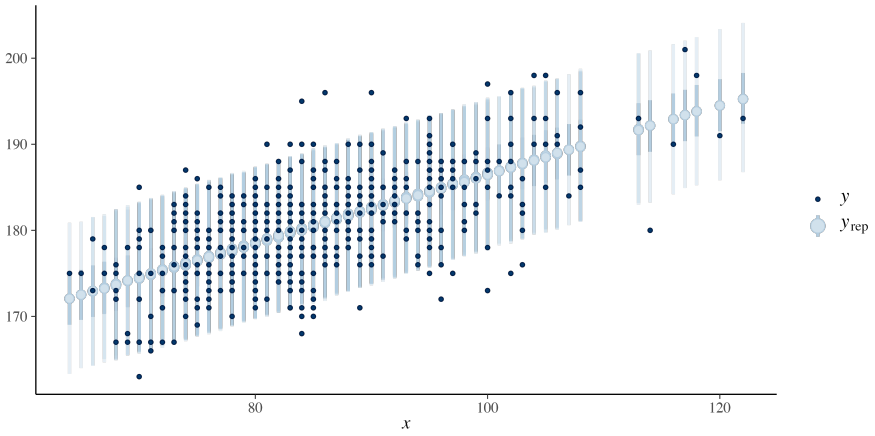


図 11.4 事後予測分布

11.2 データに合わせた確率分布

今回は身長が従属変数，体重が独立変数でした。ここで独立変数を**名義尺度水準**のもの，すなわち離散変数だと考えると要因計画のようになります。Stan のコードは書き換える必要がありますが，線形モデルとして表現できるところは同じです[*7]。

では**従属変数**を離散変数に置き換えたらどうなるでしょうか。野球のデータの例で考えます。野球は野手と投手に大きく分けることができます。投手は野手と違って毎日登板するということはありませんから，当然のことながら出場試合数は野手と比べ物になりません[*8]。図 11.5 には，年収 5000 万円を超える一流の選手たちに限定し，試合数で野手か投手かを予測するような分析ができないかと，回帰直線を引いたものを示してあります。この直線はどうみても，データにうまく当てはまっているとは言えませんね。

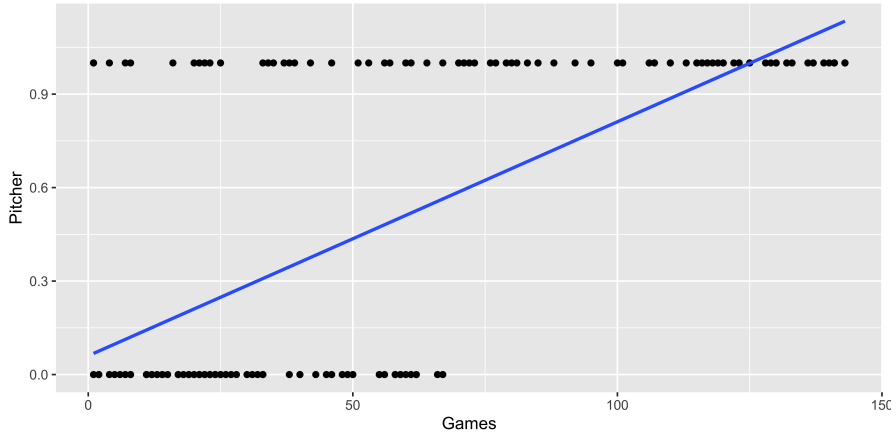


図 11.5　離散変数が目的変数だとおかしなことになる

従属変数が離散的，とくに今回は野手か投手か，という二択の問題になっていますから，データとしては 0/1 しかないわけです。この二種類の数字が正規分布から出てくると考えるのは，流石におかしなことになってしまうでしょう。

そこで前回学んだ**離散確率分布**の登場です。データが二値しか取らないというのは，コインの表と裏のようなものですから**ベルヌーイ分布 (Bernoulli distribution)** に従うと考えるのが自然です。ベルヌーイ分布はパラメータ θ で表 (1) が，$1-\theta$ で裏 (0) がでるというように，パラメータが 1 つで，このパラメータがそもそも「表が出る確率」を表しているわけです。今回は野手であれば 1，投手であれば 0 というように考えることにしましょう。野手か投手かを表す変数で，野手になる確率 θ が試合数によって変わる (予測できる) ということを考えるのです。この予測の仕方については，「試合数が多いのは野手である確率が高い」という線形的な関係が考えられますから，$\theta_i = \beta_0 + \beta_1 X_i$ という式を考えることにします。ここで X_i は試合数を表す変数ですね。

ところがここで，注意すべきことが 1 つあります。θ は野手になる (表が出る) 確率ですから，0 から 1 の範囲の数字しか取れません。プロ野球の試合数は 1 試合から 140 試合ぐらいありますので，この数字をそのま

[*7] 実際，R の最尤推定関数の `lm` は，説明変数が `factor` 型であれば関数を書き換えることなく，分散分析のような結果出力をしてくれるのでした。

[*8] メジャーリーガーの大谷選手は特殊すぎるケースです。投手は一試合で 100 球程度投げますが，当然徐々に握力が弱くなり，身体的な疲労も翌日，翌々日まで残り続けるものなのだそうです。彼は投げた次の日に DH で打席に立ちますから，とんでもないことです。

ま伸ばしたり (β_1 倍) ずらしたり (β_0 を加える) しても，0 から 1 の範囲に収めるのは難しそうです。しかも上手くその範囲に収めたとしても，そもそも直線なのでどうあってもデータと合致しないことになるでしょう。そこで，この数字をロジスティック関数で変換することを考えます。

ロジスティック関数とは次のような式で表されるものです。

$$y = \frac{1}{1 + \exp(-x)}$$

ここで $\exp(x) = e^x$ であり，正規分布の式の中にも出てくる関数です[*9]。関数の形は図にするとよくわかります (図 11.6)[*10]。

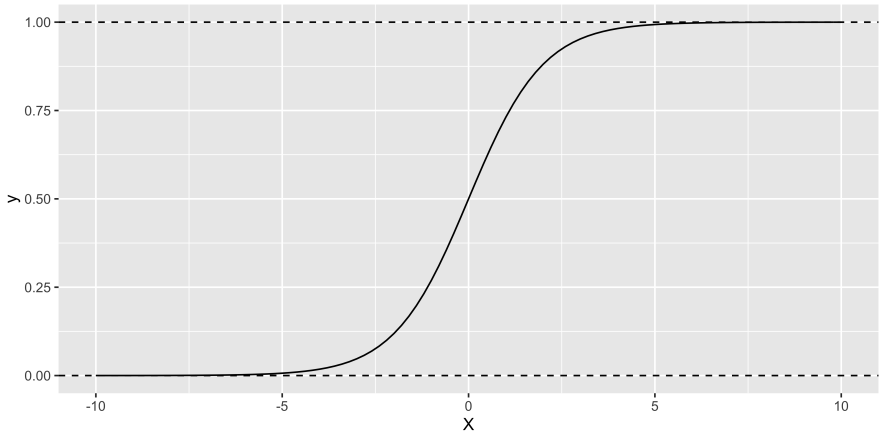


図 11.6 ロジスティック関数

この S 字カーブは，x がどのように変わっても y は必ず 0 から 1 の範囲に入りますので，今回の目的にもってこいです。つまり，今回「野手か投手か」を判断する結果変数 Y_i は，次のような分布に従っていると考えます。

$$Y_i \sim Bernoulli(\theta) = Bernoulli(logistic(\beta_0 + \beta_1 X_i))$$

これを使ってモデルを書いてみましょう。まずは設計図です (図 11.7)

ここまで書ければ，Stan コードに起こすことも簡単ですね。Stan には `inv_logit` という関数がありますのでそれを使うと良いでしょう。

code : 11.3 ロジスティック回帰分析のコード

```
1  data{
2    int N;
3    array[N] int<lower=0,upper=1> Y;
4    array[N] real X;
5  }
6
7  parameters{
```

[*9] e はネイピア数とも呼ばれる自然対数の底，約 2.7182... の実数です。

[*10] すでにデータ解析応用 1 の項目反応理論で出てきた関数ですが，あの時は累積正規分布に近似する関数として紹介していました。今回は $\pm\infty$ の範囲の数字を 0 から 1 に変換するものとして改めて紹介しています。なお，今回は累積正規分布への近似を目的としているわけではないので，係数 1.7 は外してあります。

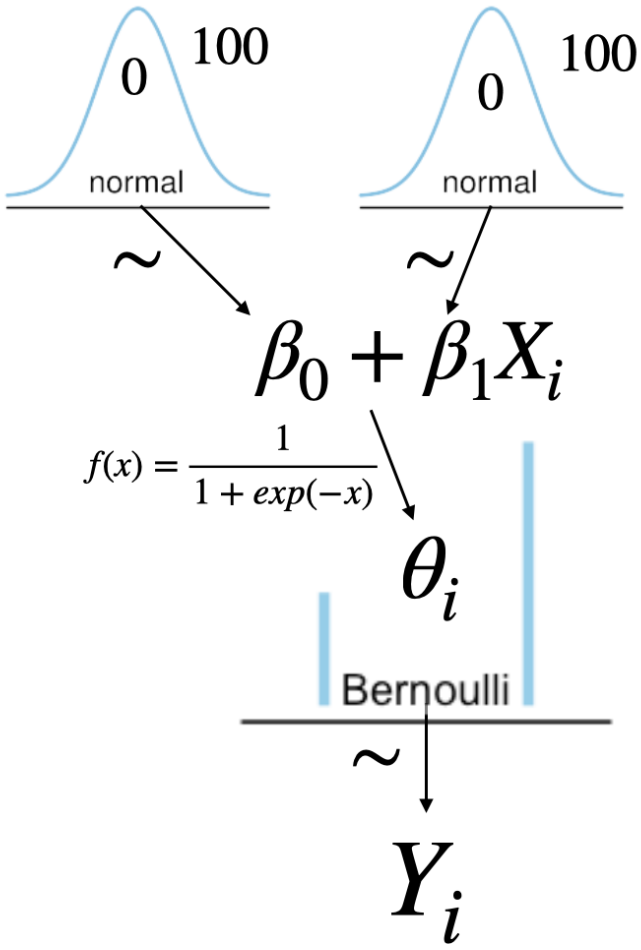


図 11.7　ロジスティック回帰分析の設計図

```
 8    real beta0;
 9    real beta1;
10  }
11
12  transformed parameters{
13    array[N] real theta;
14    for(i in 1:N){
15      theta[i] = inv_logit(beta0 + beta1 * X[i]);
16    }
17  }
18
19  model{
20    // model
21    for(i in 1:N){
22      Y[i] ~ bernoulli(theta[i]);
23    }
24    // prior
```

```
25    beta0 ~ normal(0,100);
26    beta1 ~ normal(0,100);
27  }
```

Code11.3 がこのモデルになります。ベルヌーイ分布を使っているのでベルヌーイ回帰と読んでもいいのですが, 一般には**ロジスティック回帰 (logistic regression)** と呼ばれています。これに適切なデータセットを与えて (R コード 11.4), 推定した結果が出力 11.2 になります。

code : 11.4 ロジスティック回帰分析を実行する R コード

```
dat2 <- dat %>%
  dplyr::mutate(Pitcher = if_else(position == "投手", 0, 1)) %>%
  dplyr::filter(salary > 5000) %>%
  dplyr::select(Games, Pitcher) %>%
  na.omit()

model <- cmdstanr::cmdstan_model("cmdstan/logistic.stan")
dataSet <- list(N = NROW(dat2), Y = dat2$Pitcher, X = dat2$Games)

fit.logistic <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 5000
)

## 簡易表示
fit.logistic$print(c("beta0", "beta1"))
```

■コード解説

1-5 行目 先ほど読み込んだデータに Pitcher 変数を作ります。これは投手であれば 0, 野手であれば 1 になる変数です。その後, サラリーが 5000 万円以上のデータにし, 変数 Game, Pitcher と必要なものだけ抜き出します。最後に na.omit で欠測値のあるデータは除外しました

7 行目 cmdstanr でコンパイルします。

8 行目 stan に与えるデータセットをリストで作ります。

10-13 行目 サンプリングするコードです。

19 行目 結果の出力です。出力 11.2 のように得られます。

cmdstanr の出力 11.2: ロジスティック回帰分析の結果

```
 variable  mean median   sd  mad    q5   q95 rhat ess_bulk ess_tail
    beta0 -4.20  -4.17 0.61 0.61 -5.27 -3.25 1.00     3565     4162
    beta1  0.09   0.09 0.01 0.01  0.07  0.11 1.00     3553     3983
```

出力結果はグラフにした方がわかりやすいでしょう。図 11.8 にあるように, 上下の黒い点がデータ点, 真ん中を通る S 字カーブが回帰線になります。回帰線は **MAP 推定値**で描きましたが, 50%, 95% の幅であり得る可能性の範囲についても薄く書き足しています。これを見ると, ちょうど 50 試合目ぐらいの時にカーブが

Y 軸 0.5, すなわち表裏半々の点を通りますから，50 試合以上出ている人はより野手と判断されがち, と考えれば良いでしょう。

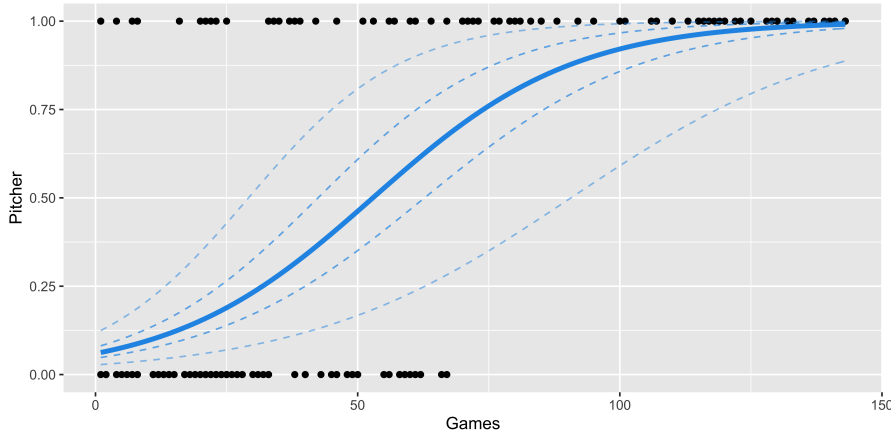


図 11.8　ロジスティック回帰分析の結果

さて, このように従属変数が離散的である場合に, 離散確率分布をつかって線形モデルを当てはめることができるのをみてきました。図 11.8 に示されているように, 出力される関数は S 字のカーブ型をしていますが, 予測モデルとしては**線形**です。今回 $\theta_i = \beta_0 + \beta_1 X_i$ としたように，θ は線形モデルであり, 予測変数 X_i が線型結合されているので, これも広い意味で線形モデルになるわけです。線型結合したものを, そのまま確率分布に入れることができなかったので, 変数を変換してモデルの中に組み込むという工夫が必要でした。この変換関数のことをとくに**リンク関数 (link function)** と言います。線型結合とパラメータの形をつなげる (link する) 関数だからです。

このように, リンク関数をかませてさまざまな確率分布に対応させる線形モデルを総称して, **一般化線形モデル (generalized linear model; GLM)** といいます。この章の初めに出てきた**一般線形モデル**とは違いますので注意してくださいね。日本語では「化」があるかないか, 英語では "'*–ed* がつくかどうかだけの違いですが, 一般線形モデルが正規分布一般だったものに対し, それ以外の確率分布にまで一般化したのがこちらです。

ちなみに今回はベイズ推定で一般化線形モデルを実践してみましたが, 最尤法による推定も可能です。可能ではあるのですが, ベイズ推定の方が拡張性が高いこと, 確率モデルとしてまとめて理解しやすいと筆者は考えています。久保 (2012) は線形モデルの発展段階を推定法とともに, 図 11.9 のように表しています。一般線形モデルは**最小二乗法**による推定でも十分だったのですが, 確率モデルとして考えたときに**最尤法**による推定が必要になってきました。さらに複数の分布を混ぜるような**一般化線形混合モデル (generalized linear mixed model)** にまで発展すると, 最尤法でカバーしきれないところが出てきました (網掛けが部分的になっていることでそれを表しています)。分布の上に分布がくるような, 幾重にも層が厚くなるモデルのことを階層モデルと言いますが, そうなってくるともはや最尤法では推定できません。ここで「形はわからなくても乱数は発生させられる」という **MCMC** の方法が出てきます。MCMC はすでに説明した通り (→Pp.32), 乱数による事後分布の近似法です。この方法はベイズ統計学に基づいた方法ですから, **ベイズ推定**ともいわれるわけです。ベイズ推定は最尤推定の結果と解釈の仕方が少し異なりますから, そこに注意する必要がありますが, ほぼ無敵の推定法と言ってもいいでしょう。

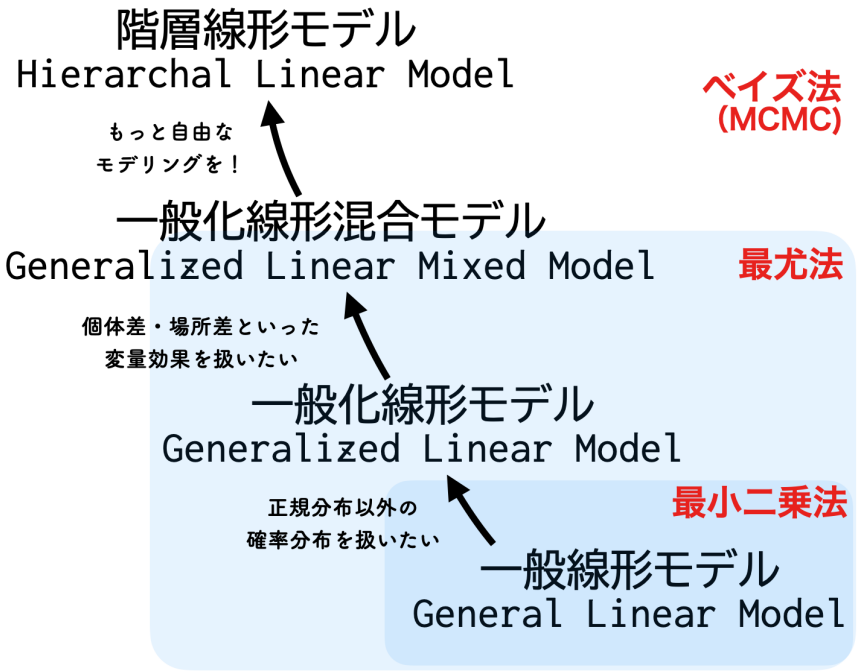


図 11.9 久保 (2012)による線形モデルの発展の図

11.3 リンク関数とパラメータの解釈

さてここでもう一度，先ほどのロジスティック回帰分析の結果をみてみましょう。EAP 推定値でいくと，$\beta_0 = -2.77, \beta_1 = 0.05$ でしたから，$\hat{Y}_i = -2.77 + 0.05X_i$ という関係だったことがわかります。

ところで一般線形モデルのうちは，この傾きは説明変数が一単位増えた時に，従属変数がどれぐらい増えるのかを表す数字でした。これが正規分布に従うモデルであれば，出場試合数が一試合増えれば野手に 0.05 ポイント近づく，という考え方をするのでした。しかし今回は事情が違います。この $\hat{Y}_i$ はこのあと，$\frac{1}{1+exp(-\theta_i)}$ と変換されてしまいますので，X_i が $X_i + 1$ になった値も変換されて確率の数字に変わっていくはずです。ここで少し，数式的な展開を追って何が起こっているか考えてみましょう。

まずは**ロジスティック関数**の表記を，少し改めます。同じことなのですが，$\exp(x) = e^x$ に書き改めて

$$\frac{1}{1+\exp(-x)} = \frac{1}{1+e^{-x}} = \frac{1}{1+\frac{1}{e^x}}$$

とします[*11]。ここで最後の形に $\frac{e^x}{e^x}$ をかけまして，

$$\frac{1}{1+\frac{1}{e^x}} = \frac{e^x}{e^x+1}$$

としましょう。さらに，$\hat{Y} = \beta_0 + \beta_1 X$ で，ベルヌーイ分布のパラメータ θ はこれによって変換されたものを代

[*11] 指数計算のルールで，マイナス乗は逆数になります。$a^{-n} = \frac{1}{a}$ です。

3789 入することになりますから，

$$\theta = \frac{e^{\hat{Y}}}{e^{\hat{Y}} + 1}$$

$$\begin{aligned}(e^{\hat{Y}} + 1)\theta &= e^{\hat{Y}} \\ \theta e^{\hat{Y}} + \theta &= e^{\hat{Y}} \\ \theta &= e^{\hat{Y}} - \theta e^{\hat{Y}} \\ \theta &= e^{\hat{Y}}(1 - \theta) \\ \frac{\theta}{1-\theta} &= e^{\hat{Y}}\end{aligned}$$

$$\log\left(\frac{\theta}{1-\theta}\right) = \hat{Y}$$

3790 という関係が成り立ちます[*12]。つまり，

$$\log\left(\frac{\theta}{1-\theta}\right) = \beta_0 + \beta_1 X$$

3791 ということですね。

3792 この式の右辺は線形モデルです。左辺は**ロジット関数 (logit function)** と呼ばれるものです。線形モデル
3793 はロジット関数を意味し，ベルヌーイ分布のパラメータ θ は確率を表す数字でした。ですから係数の解釈は，
3794 線形モデルの方で一単位増えたことは，ロジット関数が一単位増えたことを意味するわけです。ちょっとややこ
3795 しいですが，線形モデルを確率のパラメータにするには，ロジット関数の逆をすることですから，**ロジスティッ**
3796 **ク関数は逆ロジット関数**とも呼ばれています。そう言えば，Stan の関数も `inv_logit` でしたね[*13]。

3797 - 確率の数字 $\rightarrow$ 線形モデルにつなげる；ロジット関数；$\log\left(\frac{\theta}{1-\theta}\right)$
3798 - 線形モデル $\rightarrow$ 確率の範囲に収める；ロジスティック関数あるいは逆ロジット関数；$\frac{1}{1+exp(-x)}$

3799 また，線形モデルが表していたのは確率の比ですが，それの対数 (log) をとったものでした。対数の中身は
3800 確率 θ と確率 $1-\theta$ の比率ですから，裏が出る確率に比べて何倍表が出やすいか，といったことを表してい
3801 るのです。これをとくに**オッズ (odds)** と言います。賭け事の勝率などを表す言葉ですね。オッズの対数がロ
3802 ジットです。ロジットが正の数であれば分子の方が分母より大きい，つまりより表が出やすいことになり，負の
3803 数であれば分母の方がより大きい，つまり裏の方が出やすいということになります。

3804 さて話は戻って，線形モデルで説明変数が一単位増えた時のことを考えてみましょう。わかりやすくするた
3805 めに，増える前を A，増えた後を B とします。

$$A = \frac{1}{1 + exp(-(\beta_0 + \beta_1 X)}$$

3806

$$B = \frac{1}{1 + exp(-(\beta_0 + \beta_1 (X+1))}$$

3807 これはロジット関数を使って元に戻せますから，

$$\log\left(\frac{A}{1-A}\right) = \beta_0 + \beta_1 X$$

[*12] 最後の一行は，指数の反対が対数で，$\log_e^x = a$ は $e^a = x$ をあらわしていますから，指数を取るために両辺に log をかけています。

[*13] `inv` は inverse,「逆」の意味です。

$$\log\left(\frac{B}{1-B}\right) = \beta_0 + \beta_1(X+1) = \beta_0 + \beta_1 X + \beta_1$$

ここで傾き β_1 のことを考えたいのですが，β_1 はこの式の関係から，

$$\beta_1 = \log\left(\frac{B}{1-B}\right) - \log\left(\frac{A}{1-A}\right)$$

ということになりますね。ここで対数の差分は割り算ですから[*14]，

$$\beta_1 = \log\left(\frac{\frac{B}{1-B}}{\frac{A}{1-A}}\right)$$

となります。右辺が非常にややこしい形をしていますので，log を外してあげましょう。

$$\exp(\beta_1) = \frac{B}{1-B} \Big/ \frac{A}{1-A}$$

右辺は A のオッズに対する B のオッズになりました。オッズの比なのでこれを**オッズ比 (odds ratio)** と言います。左辺はロジスティック回帰分析の係数を exp 関数に入れたものですが，これが説明変数の増加前のオッズと増加後のオッズの比率です。つまり一単位増加した時にどれぐらい生じやすくなるかの比の上昇率が，$\exp(\beta_1)$ だというわけです。

ややこしいですね！具体的な数字で考えてみますと，今回 $\beta_1 = 0.05$ だったわけですから，$\exp(0.05) = 1.051271$ です。つまり，一試合多く出ている選手はそうでない選手に比べて，1.05 倍野手と判断される確率が高い，ということになります。

11.4 まとめ

今日の話を簡単にまとめておきます。

- 一般線形モデルは，正規分布を使った線形モデルで，説明変数が連続変数でも離散変数でも同じ形で表すことができることを示している。
- 一般線形モデルの回帰係数をベイズ推定するモデルは，正規分布の位置母数に線形モデルを入れるだけ。
- 従属変数が離散変数になると，確率モデルは離散確率分布を使う必要がある。そのためには，説明変数の線型結合をリンク関数を使って変形し，適切な形にして推定する必要がある。これを一般化線形モデルといいます。
- 従属変数がベルヌーイ分布に従う例として，ロジット関数をリンク関数としたロジスティック回帰分析の実例をやってみました。
- 一般化線形モデルの場合は説明変数がリンク関数で変換されているから，そのまま解釈するのではなく，変換の意味を理解しながら注意深く解釈しよう。

一般化線形モデルを導入したことで，要するにリンク関数がわかっていれば色々な確率分布が使えるんだな，ということが見えてきたのではないでしょうか。

これは従属変数の形に関わることですので，心理学的実践の上でも注意が必要です。たとえば「記憶のトレーニングをすると思い出せる単語の数が増える」といった研究をする時，従属変数は「思い出した単語の

[*14] $\log_a \frac{M}{N} = \log_a M - \log_a N$ です。たとえば $8 = 2^3, 16 = 2^4$ ですから，$\log_2 168 = \log_2 2 = 1$ は $\log_2 16 - \log_2 8 = 4 - 3 = 1$ と同じことです。

数」になると思いますが, 実験群と統制群とでこれを **t 検定**する, というのが間違っていることはもうお分かりですね。「思い出した数」のようにカウントできる数字はマイナスになることはなく，0 以上の正の整数しか取りません。これは正規分布ではなく, **ポアソン分布 (poisson distribution)** に従うことになります。またたとえば「記憶のトレーニングをすると，50 の単語リストのうち思い出せる単語の割合が増える」と言った研究をする時, 従属変数は今度は「思い出せた率」ということになると思います。これを実験群と統制群で t 検定する, というのが間違っているのも明らかですね。「思い出せた率」という比率のデータは 0 から 1 の範囲の実数で, 正規分布ではないからです。これは前回やりました**二項分布 (binomial distribution)** に従うことになります。

ポアソン分布に従うような変数の場合は, リンク関数として対数関数 log が用いられます (逆リンク関数は指数関数 exp)。また二項分布に従う変数の場合はリンク関数としてロジット関数 (逆リンク関数はロジスティック関数) を用います。何気ない「度数」「比率」のようなデータを分析しようという時に, とりあえず数字が得られているから正規分布でいいや, と考えるのは間違いです。データの性質に応じたモデルを描くように心がけましょう。

11.5 課題

今回の課題は大問 1 つ, 小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と, モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`, `chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは, こちらの環境でそのまま実行して確認します。提出する前に, 新しいセッションを開いて最初から最後まで通して実行し, エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は, 指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`, 小問 2 は `ans2`, 小問 3 は `ans3` です。複数の数値を答えるときは, `ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は, **計算した値をそのまま**にしてください。`round` や `signif` で丸めると, 正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが, 提出するオブジェクトの中身は丸めないこと。

わからないところがあれば, 曜日別 TA か小杉までメールで連絡し, 指導を受けてください。

■大問；打率を年俸で予測する 野球選手のデータのうち, とくに野手に限って考えます。年俸の高い選手は成績が良いから高い契約金が得られるのでしょうから, 逆に言えば年俸で成績がある程度予測できるかもしれません。そこで打率を従属変数に, 年俸を独立変数にした一般化線形モデルを考えます。選手によって試合への出場回数が違いますので, 打席数 N のうち安打数が K 本である, という二項分布に従うモデルを立てましょう。独立変数の年俸は個人差が非常に大きいので, 標準化してから使ってください。元のデータを加工するコードは伴走サイトで提供します。

1. 二項分布とロジットリンクによる一般化線形モデルを Stan で書いてください。母数として切片 `beta0` と傾き `beta1` を宣言すること。
2. このモデルでベイズ推定を実行し，`beta0` と `beta1` それぞれについて，EAP 推定値と 90%

確信区間の下限と上限を報告してください。`ans2` の要素名は `beta0_eap`，`beta0_lower`，`beta0_upper`，`beta1_eap`，`beta1_lower`，`beta1_upper` とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。

3. 年俸が平均の選手 (標準化した年俸が 0) の打率と，年俸が 1 標準偏差高い選手 (標準化した年俸が 1) の打率を，それぞれ MCMC サンプルから推定し，その差を求めてください。あわせて，年俸が打率に対して正の効果を持つ確率 (`beta1` が 0 より大きい確率) も求めること。そのうえで「年俸が高い選手ほど打率が高い」といえるか，またその差は実際に意味のある大きさかを判断してください。`ans3` の要素名は `avg_rate`，`rate_1sd`，`rate_diff`，`p_beta1_positive` とすること。

第 12 章

階層線形モデル

前回は**一般線形モデル**から，**一般化線形モデル**へ，すなわち正規分布以外の確率分布を扱えるモデルへと展開しました。今回はさらにモデルを展開し，**一般化線形混合モデル (generalized linear mixed model; GLMM)** へと展開します。混合 mixed という言葉があるように，複数の確率分布の混ぜ合わせが含まれることが，GLMM の特徴です。

とはいえ，皆さんはすでに第 9 講でこの問題を扱っています。Within 計画・多水準のモデルがそれに当たります。Within 計画のモデルでは，個人差 μ_i に効果 δ_j が加わるという形でモデルを形成しました。ここで個人差 μ_i も正規分布に従うものと考えていました。個人差が従う正規分布，誤差が従う正規分布が混じりあってデータになりますから，これも Mix されたモデルと考えることができるのです。

ただしこの時は，いずれも正規分布のモデルでした。このモデルを正規分布以外のデータに対応させたい，それが今回のモデルになります。

12.1 一般化線形混合モデル

線形モデルを正規分布以外のモデルに対応させるためには，確率分布のパラメータの形にあうように**リンク関数 (link function)** で接続してやる必要があるのでした。たとえばベルヌーイ分布の場合，線形モデルは次のような確率の比 (**オッズ**) を表すものと考えます。

$$\log\left(\frac{\theta}{1-\theta}\right) = \beta_0 + \beta_1 X$$

このリンク関数の逆関数を考えてやると，

$$\theta = \frac{1}{1 + exp(-(\beta_0 + \beta_1 X))}$$

となり，**ロジスティック関数**と呼ばれるこの変換を通じて $0 \leq \theta \leq 1$ の範囲に入るようにしてから，**ベルヌーイ分布**に入れてやるのでしたね。

リンク関数と**逆リンク関数**，この違いがちょっとわかりにくいかもしれません。基本となる線形モデルは $\beta_0 + \beta_1 X$ ですので[*1]，これが確率分布とセットになる方がリンク関数です。あるいは，線形モデルを変えてしまう方が逆リンク関数だ，と考えてください。

[*1] より正確に，より一般的にいうと，説明変数は複数あっても構いませんから，$\beta_0 + \beta_1 X_1 + \beta_2 X_2 \cdots \beta_n X_n$ と書く方がいいのかもしれませんし，もちろんベクトルを使って $\boldsymbol{X\beta}$ とすればいいでしょう。この時，係数ベクトル $\boldsymbol{\beta}$ には切片項 β_0 も含まれていることを思い出してください。また，説明変数が複数になった場合は，**交互作用 (interaction)** 項を考えることもありますが，本書では簡略化のために含めずに考えています。

リンク関数の例をもう 1 つ。**ポアソン分布 (Poisson distribution)** という離散分布があります[*2]。これは 0 以上の整数を取る**カウント変数**に使われる関数です。たとえば野球選手のホームランの数，たとえば青年期における親友の数や交際経験の人数，記憶実験で思い出せた単語の数などは，負の数をとることも小数点を持った実数になることもありません。こうした数を数えるような変数が従うのがポアソン分布ですが，これのパラメータ λ は正の実数です。ということは，線形モデルで算出される値を必ず正の数にしなければなりませんから，逆リンク関数は exp を使うことになります[*3]。

$$\lambda = \exp(\beta_0 + \beta_1 X)$$

$$\log(\lambda) = \beta_0 + \beta_1 X$$

ですから，リンク関数は log になります。

表 12.1 に確率分布，逆リンク関数，リンク関数の関係をまとめました。さらに Stan ではそれぞれの確率分布に加え，リンク関数とセットになった関数がすでに用意されています。`transformed parameters` ブロックで逐一変換しなくても，この関数を使うと `model` ブロックで直接線形モデルを書き込むことができますので，便利です。ここで線形モデルで表されるものを μ, θ, λ などさまざまなギリシア文字を使っていますが，このギリシア文字の違いにとくに意味はありません。一般的に正規分布の**位置母数**には μ が，ベルヌーイ分布のそれには θ が，ポアソン分布のそれには λ が使われることが多く，その慣例に習っただけです。いずれも確率分布の中心的な位置を表す母数であるということだけが重要です。また正規分布のところを見ると，リンク関数も逆リンク関数も同じ形をしていることがわかります。こうした**恒等式**で済んだので複雑なことを考える必要がなかったんですね。

表 12.1 確率分布とリンク関数の関係

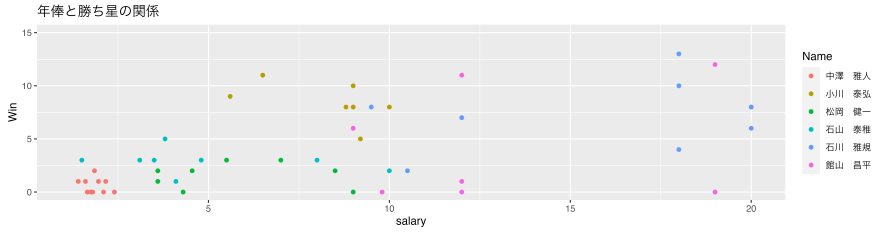
確率分布	逆リンク関数	リンク関数	Stan の専用関数
正規分布	$\mu = \beta_0 + \beta_1 X$	$\beta_0 + \beta_1 X = \mu$	`normal`
ベルヌーイ分布	$\theta = \frac{1}{1 + exp(-(\beta_0 + \beta_1 X))}$	$\beta_0 + \beta_1 X = \log\left(\frac{\theta}{1-\theta}\right)$	`bernoulli_logit`
ポアソン分布	$\lambda = \exp(\beta_0 + \beta_1 X)$	$\beta_0 + \beta_1 X = \log(\lambda)$	`poisson_log`

では一般化線形モデルに「個人差」を混合するモデルを考えてみましょう。ここでは前の章でも扱った野球のデータを例に説明します。図 12.1 には Swallows で 2011 年から 2020 年の間，8 年以上在籍した投手の年俸 (単位は千万円) と勝利数の関係を表しています。年俸が高いと勝ち星が増える，という関係にあると考えてモデルを組みます。

線形モデルですから，勝ち星を Y_i，年俸を X_i として $\hat{Y_i} = \beta_0 + \beta_1 X_i$ という関係を考えます。とはいえ，選手の年俸というのはそれまでの業績によるところも大きく，そもそも個人差があり得るところです。そこで個人差 μ_i を考えて，$\hat{Y_i} = \beta_0 + \beta_1 X_i + \mu_i$ とした線形モデルを考えましょう。また，勝利数は 0 以上の整数しか取りませんから，これはポアソン分布に従うと考えられます。そこでポアソン分布に合わせるために，逆リンク関数を使って $\lambda = \exp(Y_i) = \exp(\beta_0 + \beta_1 X_i + \mu_i)$ とした上で，$Y_i \sim poisson(\lambda_i)$ という確率モデルに入れることにしましょう。このモデルの設計図は図 12.2 のようになります。

[*2] 後学のためにきちんと定義式を書いておくと，変数 X が k の値を取る確率を $P(X = k) = \frac{\lambda^l e^{-\lambda}}{k!}$ とした分布です。ここで e はいつもの**ネイピア数**であり，$k!$ の ! は階乗を表す記号です。

[*3] この関数はよく出てきますから周知のことと思いますが，$exp(x) = e^x$ でこの e もいつものネイピア数です。この関数で x が負の数になっても，$x^{-1} = \frac{1}{x}$ という指数計算の決まりがありますから，結果が負になることはありません。



年俸の推移(単位;千万円)

	Name	2011年度	2012年度	2013年度	2014年度	2015年度	2016年度	2017年度	2018年度	2019年度	2020年度
1	石川　雅規	18.0	20.00	20.00	18.0	18.00	20.00	18.0	12.00	9.5	10.5
2	館山　昌平	12.0	19.00	19.00	NA	9.00	12.00	12.0	9.80	2.4	NA
3	松岡　健一	8.5	9.00	7.00	5.5	4.55	3.60	4.3	3.60	NA	NA
4	中澤　雅人	2.4	1.85	1.65	1.4	1.96	2.16	1.8	1.75	2.1	1.6
5	石山　泰稚	NA	NA	1.50	3.5	3.80	4.10	3.1	4.80	10.0	8.0
6	小川　泰弘	NA	NA	1.20	5.6	6.50	10.00	8.8	9.00	9.2	9.0

勝ち星の推移

	Name	2011年度	2012年度	2013年度	2014年度	2015年度	2016年度	2017年度	2018年度	2019年度	2020年度
1	石川　雅規	10	8	6	10	13	8	4	7	8	2
2	館山　昌平	11	12	0	NA	6	1	0	0	0	NA
3	松岡　健一	2	0	3	3	2	2	0	1	NA	NA
4	中澤　雅人	0	2	0	1	1	1	0	0	0	1
5	石山　泰稚	NA	NA	3	3	5	1	3	3	2	3
6	小川　泰弘	NA	NA	16	9	11	8	8	8	5	10

図 12.1　一流選手の年俸と勝利数

それではこれをもとに，GLMM をやってみましょう。実行するための Stan のコードはコード 12.1 のようになります。

code : 12.1　個人差を含んだポアソン回帰モデル

```
data{
  int L;
  int N;
  array[L] real X;
  array[L] int Y;
  array[L] int index;
}

parameters{
  real beta0;
  real beta1;
  array[N] real mu;
}

model{
  // model
  for(l in 1:L){
    Y[l] ~ poisson_log(beta0 + (beta1 * X[l]) + mu[index[l]]);
  }
  // prior
  beta0 ~ normal(0,10);
  beta1 ~ normal(0,10);
  mu ~ normal(0,10);
}
```

■コード解説

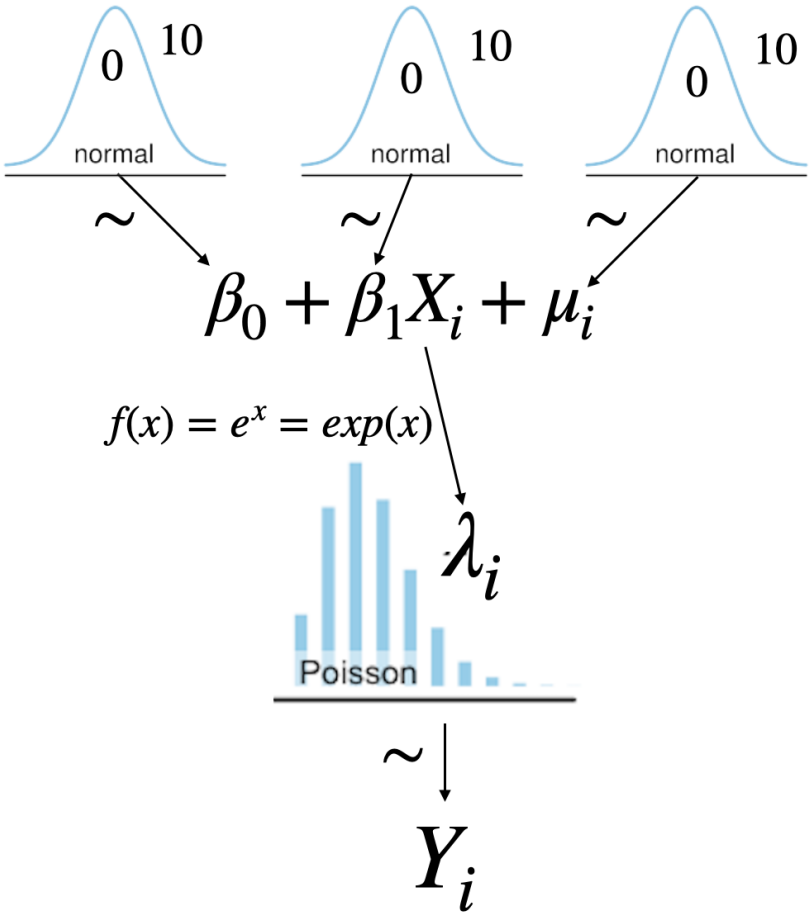


図 12.2　ポアソン分布を使った GLMM モデルの設計図

data **ブロック**　データ長 L と扱う選手数 N, 独立変数 X, 従属変数 Y, 個人差インデックスを宣言しています。ポアソン分布に従う確率変数ですので従属変数は `int` 型にしてあります。

parameters **ブロック**　線形モデルの係数と個人差 μ_i を人数分用意しました。

model **ブロック**　モデル尤度のところは `poisson_log` を使っているので, 線形のコードをそのまま書き込むことができています。事前分布は正規分布にしました。

これを R で呼び出すコードは Code::12.2 の通りです。少しテクニカルなコードになっていますので自分で書けなくても構いませんが[*4], 各ステップで何をしているか読めるようになっておくといいでしょう[*5]。

code : 12.2　ポアソン回帰のコード

```
1  baseball <- read_csv("BaseballDecade.csv")
2
3  dat <- baseball %>%
4    filter(position == "投手") %>%
5    filter(team == "Swallows") %>%
```

[*4] コードの書き方は単一の正解というものがありません。ここに書いてあるのは著者流のやり方だというだけであって, 同じ結果が出るのであれば別のコードの書き方であっても構いません。今回のこのコードはシラバスのサイトで配布しています。

[*5] パイプ演算子 `%>%` を使って変換プロセスをつなげて言ってますが, その途中でデータがどのように加工されているか知りたい場合, `%>% print %>%` と `print` 関数を挟むといいでしょう。ステップごとのデータの変形プロセスがよくわかると思います。

```
 6    group_by(Name) %>%
 7    nest() %>%
 8    mutate(
 9      n = purrr::map_dbl(data, ~ NROW(.)),
10      FLG = purrr::map_lgl(data, ~ anyNA(.$Win))
11    ) %>%
12    filter(n > 7) %>%
13    filter(!FLG) %>%
14    unnest(data) %>%
15    select(Year, Name, salary, Win)
16
17    dat.tmp <- dat %>%
18    mutate(salary = salary / 1000) %>%
19    mutate(ID = as.factor(Name)) %>%
20    mutate(ID = as.numeric(ID))
21
22  dataSet <- list(
23    L = NROW(dat.tmp),
24    X = dat.tmp$salary,
25    Y = dat.tmp$Win,
26    index = dat.tmp$ID
27  )
28
29  model_pois <- cmdstanr::cmdstan_model("glm_poisson.stan")
30  fit <- model_pois$sample(
31    data = dataSet,
32    chains = 4,
33    parallel_chains = 4,
34    iter_warmup = 1000,
35    iter_sampling = 3000
36  )
```

■コード解説

1 行目　ファイルからデータを読み込みます。このデータファイル BaseballDecade.csv には野手，投手の 10 年分のデータが入っています。

3-15 行目　最初のブロックとして，データの加工・変改を行い，それを dat オブジェクトに格納します。

4 行目　フィルターをかけて，投手だけのデータに絞り込みます。

5 行目　フィルターをかけて，Swallows の選手に限ります

6 行目　選手名でグルーピングします。

7 行目　nest 関数でグループ化変数に沿ってデータを畳み込みます。引数をとくに指定しなければ，変数はすべて data という変数名に含まれます。この段階でデータフレームは出力 12.1 のようになっています。nest 関数のイメージを図 12.3 に示しました。

8 行目　畳み込まれたデータセットを使って，そのデータの行数 (何年分のデータがあるか[*6])，勝利

[*6] ここで data とあるのはまとめられたミニ・データフレーム全体を引数にしています。それに対して NROW という関数をあてがうことで行数を数えています。NROW 関数の引数は . ですが，これは map 関数の第一引数全体を再度参照するときの略記号です。purrr::map というのは purrr パッケージの map 関数で，同じ関数操作を繰り返し行うときに使います。

数情報の欠測値の有無を作る変数[*7]を作っています。

9 行目 フィルターをかけて，データの行数 n が 7 より大きいもの，つまり 8 年以上登板している一流の選手だけに絞ったのです。

10 行目 フィルターをかけて，欠測値がないデータだけに絞っています。

11 行目 畳み込みを解除します。

12 行目 年度，選手名，年俸，勝利数だけ変数をセレクトして取り出します。

17-20 行目 このブロックでは Stan で使うためにさらに一時的なデータ加工をし，dat.tmp オブジェクトに格納しています。

18 行目 年俸のデータは数字が大きいので，1000 万円単位にします。分析の際，数字が大きすぎると計算がオーバーフローしてしまうことがありますから，適当な単位にしておくことは実践上の有効なテクニックの 1 つです。

19 行目 名前の変数は文字型ですが，そのまま Stan に渡すことはできないので，Factor 型にした ID という変数に作り替えます。

20 行目 先ほど作った ID は Factor 型ですが，Stan では質的な違いを表す数字だけあれば良いので，数字型に変形します。

22-27 行目 Stan に与えるためのデータセットにするブロックです。

29-36 行目 cmdstanr でサンプリングするためのコードです。

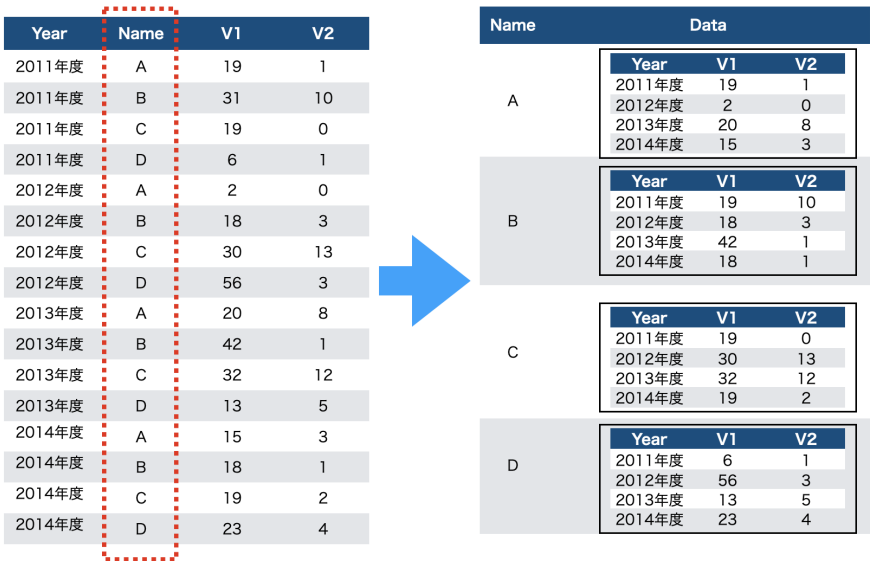
R の出力 12.1: データをグループ化変数でネストする

```
# A tibble: 295 × 2
# Groups:   Name [295]
   Name        data
   <chr>       <list>
 1 永川　勝浩 <tibble [3 × 16]>
 2 前田　健太 <tibble [5 × 16]>
 3 シュルツ   <tibble [1 × 16]>
 4 大竹　寛   <tibble [8 × 16]>
 5 横山　竜士 <tibble [2 × 16]>
 6 ジオ       <tibble [2 × 16]>
 7 バリントン <tibble [5 × 16]>
 8 藤川　球児 <tibble [7 × 16]>
 9 久保　康友 <tibble [7 × 16]>
10 小林宏     <tibble [1 × 16]>
# … with 285 more rows
```

さて，このコードを実行し，結果からモデルを図にしたのが図 12.4 です。図中の赤い転線は，$+\mu_i$ をおかずに全体を混ぜ合わせたときのポアソン回帰です。個人差を表す項目を追加することで，より個々のデータに沿った回帰プロットが実践できていることがわかると思います。

今回は個人差 μ_i だけを考えましたが，たとえば野球の弾の規定が変わってピッチャーが有利になる年とそうでない年があったとか，登板する球場のサイズによって有利不利がある，といったさまざまな個別事情に対し，それを表す変数の項を付加することで，よりデータの詳細を細かくモデルに組み込んでいくことができます。線形モデルは**線形である**という制限から逃げることはできませんが，確率分布や個別の事情を組み込む

[*7] purrr::map の使い方は先ほどと同じで，.$Win は data$Win と同じ意味です。anyNA は NA があるかどうかを TRUE/FALSE で返す関数です。



Year	Name	V1	V2
2011年度	A	19	1
2011年度	B	31	10
2011年度	C	19	0
2011年度	D	6	1
2012年度	A	2	0
2012年度	B	18	3
2012年度	C	30	13
2012年度	D	56	3
2013年度	A	20	8
2013年度	B	42	1
2013年度	C	32	12
2013年度	D	13	5
2014年度	A	15	3
2014年度	B	18	1
2014年度	C	19	2
2014年度	D	23	4

図 12.3　データの畳み込み関数の挙動。左側がもとのデータで，グループ化変数を使って畳み込むと，データフレームの中にグループごとのミニ・データフレームが畳み込まれる。

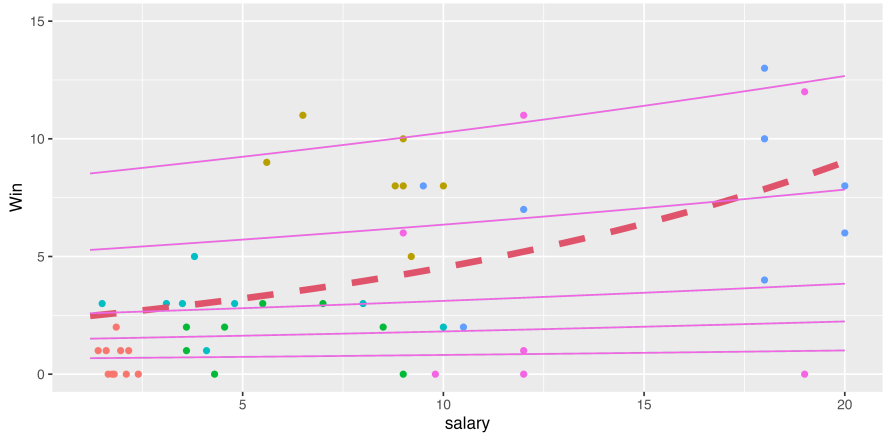


図 12.4　ポアソン分布を使った GLMM の結果

ことで，より上手く個々のデータの特徴を表現できるようになることが，イメージできたのではないでしょうか。

12.2　ネストされたデータ

さて先ほど，nest 関数を使ってデータをネストするという例を見ました。

ネストとは「入れ子にする」という意味で，マトリョーシカのようにデータフレームの中のデータフレーム，データフレームの中のデータフレームの中のデータフレーム，というように，同じ形のものを入れていくイメージです。このネストされたデータは，別名**階層性**を持ったデータだ，ということができます。

たとえば今回の野球データの場合，選手はどこかのチームの一員ですから，チームという上位階層の中に選手がいることになります。野球チームは 12 球団ありますが，それぞれセ・リーグ，パ・リーグに分かれて戦い

4049 ます[*8]。つまりリーグという上位階層があるわけです。

4050 逆に，チーム傘下の選手も，年間 140 試合ぐらいありますから，試合ごとに調子が良かったり悪かったりするでしょう。試合の中でも，野球は 9 回のターン制バトルですから，毎回の活躍というのがあるかもしれません。すなわち，ある選手の中に試合や回がネストされていると考えることもできます。私たちはデータのある側面で区切ってみていることになります。

4054 こうしたデータの階層性は，何も野球選手のデータだけではありません。たとえば調査研究を行う場合，ある小学校でデータをとったというときに，児童は学年でネストされ，学校でネストされ，地区でネストされ，市区町村でネストされ，都道府県でネストされている，ということになります。もちろん 1 つの小学校だけでデータを取るのであれば比較検証はできませんが，ある程度規模が大きくなってくると何らかの階層的な情報を持っていると考えるべきでしょう。

4059 またあるいは，心理学の実験のような少人数からしかデータを得ないという場合であっても，条件 A で n 回試行，条件 B で m 回試行する，といった Within デザインの場合は**反復測定 (repeated measure)** であり，これも被験者ごとに各データがネストされている，と考えることができるわけです。Within データの場合は個人差を考え，GLMM で個人差以外の要因もモデルに組み込むことができるようになったわけですから，データの階層性について考えないわけにはいきません。

4064 何がそんなに問題なのでしょうか。図 12.5 をみてください。左右にあるのは同じデータの分布ですが，左側は階層の区別をしない場合で，右側はその区別をした場合です。線形回帰の直線を引いておきましたが，これを見ると明らかなように階層性を考えなければ，私たちは正の相関があるデータだと判断してしまうでしょう。しかし右側，群ごとに区別してモデルを書いてみると，回帰直線はすべて右下がり，つまり負の相関があるデータだったことがわかります。階層性や群ごとの特徴を無視して分析してしまうと，結果が真逆になってしまう可能性を孕んでいるわけです。

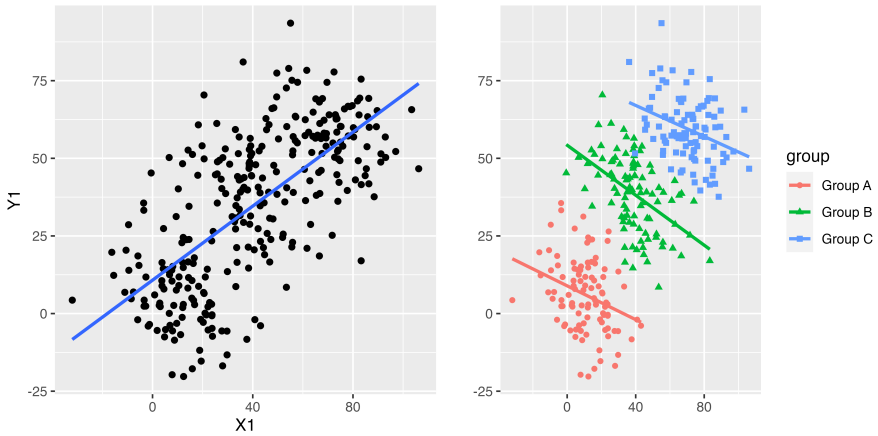


図 12.5 階層データの分析が必要な理由

4069

4070 **データは図にする**ことの重要性は何度指摘してもしすぎることはないと思いますが，ことほど左様にデータの持っている性質はなるべく丁寧に見定めなければなりません。そして階層に区分して分析する必要があるのであれば，モデルもそのようにレベルアップさせなければなりません。そこで出てくるのが**階層線形モデル**

[*8] セントラル・リーグ Central League とパシフィック・リーグ Pacific League の略であり，セ・リーグには巨人 (Giants)，阪神 (Tigers)，広島東洋カープ (Carp)，中日 (Dragons)，ヤクルト (Swallows)，DeNA(Baystars) が含まれます。パ・リーグにはソフトバンク (Hawks)，西武 (Lions)，日本ハム (Fighters)，ロッテ (Marines)，オリックス (Buffaloes)，楽天 (Eagles) からなります。2021 年現在のチーム名であり，かつては大洋ホエールズとか，阪急ブレーブスとか，南海ホークスなどのチームがありました。

(hierarchical linear model) です。

12.3 階層線形モデル

階層線形モデルは，ネストされたデータに対して群ごとの特徴を記述できるよう，線形モデルを拡張したものです。

野球の例で考えてみましょう。階層化せずにデータを眺めて，「グラウンドにはゼニが落ちてるんだ」とばかりに，たくさん年俸をもらっているバッターはホームランを打つし，ピッチャーは勝利数を稼ぐ，という傾向を分析したいとします。年俸が説明変数で，成績が従属変数になるモデルですね[*9]。しかし，ソフトバンクや読売巨人のように，資本力のある親会社のもとでの球団であればそういう傾向もあるかもしれませんが，広島カープのような市民球団[*10]ではそこまで大盤振る舞いできるほどではないよ，といった球団ごとの差異があるかもしれません。

年俸が成績に影響する，というのが全体的な傾向としてあり，影響の程度は群ごとの特徴が多少入り込む，というこの状況をモデルで表現することを考えましょう。変数の影響力の大きさは，β_0, β_1 などの回帰係数で表現されます。全体的な傾向はこれで表現できるとして，個別の効果はこの係数が群 j ごとに違っている，ということになります。この群ごとの違いを，群平均からの差異で表現しましょう。

球団 j に属するプレイヤー i の成績を Y_{ij} と表現したとします。データ全体としては，$\hat{Y_{ij}} = \beta_{0j} + \beta_{1j} X_{ij}$ なのですが，ここでたとえば切片 β_{0j} に群 j ごとの効果があるとすれば，$\beta_{0j} = \gamma_{00} + u_{0j}$ のように，切片の平均 γ_{00} と，切片に加わる群の効果 u_{0j} という，**係数を説明する線形モデル**を想定することと同じです。切片だけでなく傾き β_{1j} の方も，$\beta_1 = \gamma_{10} + u_{1j}$ と線形モデルで考えましょう。個人差 $u_{.j}$ は平均 0 の正規分布に従うと考えます。ここで全体的な傾向，すなわち個人差の平均である γ_{00}, γ_{10} のことを**固定効果 (fixed effect)** といい，個別の効果 u_{0j}, u_{1j} のことを**変量効果 (random effect)** と呼ぶことがあります。

式を整理するとこうなります。

$$
\begin{aligned}
\hat{Y_{ij}} &= \beta_{0j} + \beta_{1j} X_{ij} \\
\beta_{0j} &= \gamma_{00} + u_{0j} \\
\beta_{1j} &= \gamma_{10} + u_{1j} \\
\begin{pmatrix} u_{0j} \\ u_{1j} \end{pmatrix} &\sim MN\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \tau_{00} & \tau_{01} \\ \tau_{10} & \tau_{11} \end{pmatrix} \right)
\end{aligned}
\tag{12.1}
$$

あらまあ。何だか訳がわからなくなりそうですね。いやもちろん，じっくり考えればお分かりいただけると信じていますが，直感的にわかりにくいかもしれません。そこで同じことを違う角度から説明してみたいと思います。

階層性のあるデータとはいえ，たかだか 12 球団ですから，12 回ぐらいの繰り返し分析は頑張ればできますよね。実際それをやったのが図 12.6 にあります。えっへん。この結果を解釈するときは，球団ごとにバラバラの分析をしていることと同じですから，12 球団ごとの切片 β_{0j} と傾き β_{1j} が出てきます。ちょっと面倒ですが，データがそういうものなんだから仕方ありません。

でもこれが，12 球団ではなく，100 球団，1000 球団あったらどうしますか。1000 個の切片と傾きをずらーっと数表に並べられても，目が痛くなりますね。辛いだけです。せっかく大量のデータを手に入れている

[*9] ちょっと待って，逆じゃないか，と思った人もいるかもしれません。つまり成績がいいから年俸が高くなるのでは，ということですよね。その場合は，ある年 X の年俸が翌年 $X+1$ 年の成績を予測する，という形にデータを変えなければなりません。今回は年度ごとの年俸と成績からなるデータセットになっていますから，年俸が成績に影響する，という仮説になっています。

[*10] 広島カープは戦後復興の名目で公的資金が投入されて作られた球団であり，その意味で市民の資本から作られた球団と言えるでしょう。現在も，特定の親会社はなく複数の企業が連携して運営するスタイルをとっています。

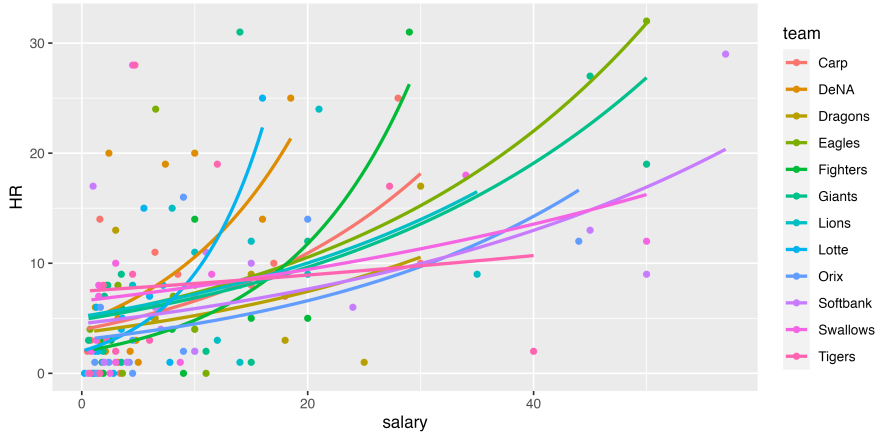


図 12.6　チームごとにバラバラの回帰分析を実施する。できなくはないけど，要するにどういうこと，と言いたくもなる。

のに，分析結果を細かくみてくれなかったら悲しい。かといって「で，結局どういうことなの」と言われた時に，「えー，第 1 球団の切片と傾きは･･･で，第 2 球団の切片と傾きは･･･で，第 998 球団の切片と傾きは･･･」と読み上げることもないでしょう。ではどうするか。傾きと切片の平均と SD で報告するんではないでしょうか。傾きの平均はこれぐらいです，球団による違いは ± これぐらいです，といえば，1000 球団の情報を要約して言えることになります。それが $u_{0j} \sim N(0, \tau_0)$ や $u_{1j} \sim N(0, \tau_1)$ が言っていることなんですね。散らばりの平均は γ_{00}, γ_{10} であり，それが τ_0, τ_1 ぐらい散らばりますよ，ということです。

これは見方を変えると，階層線形モデルを使う場合は，階層性をまとめる必要があるかどうかを考えてからでも構わないということです。12 球団しかなくて，それぐらいならじっくりみましょうというのであれば，無理に階層モデルにする必要はありません。また 100 球団，1000 球団ある世界であっても，ほとんど球団ごとの違いがないようであれば，全部まとめて分析してしまっても構いません。階層線形モデルで表現する場合は，ネストされているデータに一定の傾向はあるものの，考慮する必要がある程度に散らばりがある場合，ということになります[*11]。

またさらに別の角度から考えてみましょう。今度はこの線形モデルの，設計図を書いてみるのです。図 12.7 にそれを書いてみました。

図 12.7 では，話を簡単にするために係数の共分散を想定しない ($\tau_{10} = \tau_{01} = 0$)，1 次元正規分布から係数が来ているモデルにしています。これをみると，

1. データはポアソン分布からきている；$Y_{ij} = poisson(\lambda_{ij})$
2. ポアソン分布は線形モデルから逆リンク関数で接続されてきている；$\lambda_{ij} = exp(\beta_{0j} + \beta_{1j} X_{ij})$
3. 線形モデルの切片が別の分布からきている；$\beta_{0j} \sim N(\gamma_{00}, \tau_{00})$
4. 線形モデルの傾きも別の分布からきている；$\beta_{1j} \sim N(\gamma_{10}, \tau_{11})$
5. γ_{00} がどういう数字かわからないので，確率分布 (事前分布) でそのわからなさを表現；$\gamma_{00} \sim N(0, 10)$
6. γ_{10} がどういう数字かわからないので，確率分布でそのわからなさを表現；$\gamma_{10} \sim N(0, 10)$
7. τ_{00} がどういう数字かわからないので，確率分布 (事前分布) でそのわからなさを表現。SD なので

[*11] これも分散分析的発想で，群内の類似性と群間の類似性の比率から，十分に大きな群間変動があるようなら階層モデルにするべき，と判断する基準があります。この比率のことを**級内相関 (intraclass correlation coefficient)** と言います。詳しくは清水･荘島 (2017) を参考にしてください。

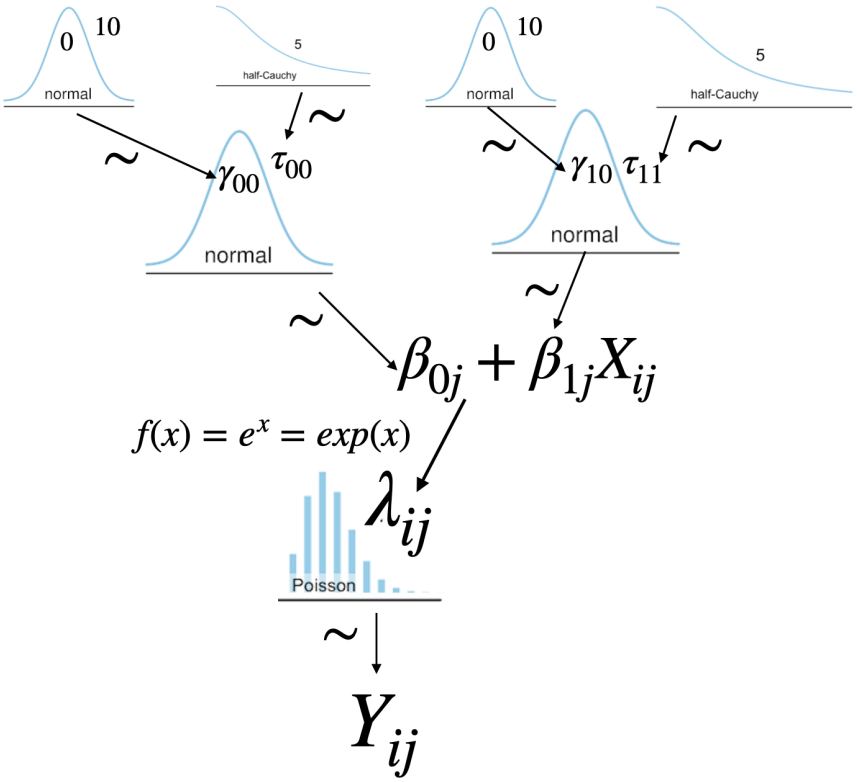


図 12.7 チームごとにバラバラの回帰分析を実施する

cauchy 分布にしてみた; $\tau_{00} \sim cauchy(0, 5)$

8. τ_{11} がどういう数字かわからないので, 確率分布 (事前分布) でそのわからなさを表現。SD なので cauchy 分布にしてみた; $\tau_{11} \sim cauchy(0, 5)$

という順番で読み解くことができます。データに分布を仮定する, これがベイズ推定のスタートでしたが, 仮定された分布のパラメータに, 線形モデルという数学的**構造**が入っています。このパラメータに数式を入れることを**モデリング (modeling)** というわけです。そしてそのモデルに含まれている未知数 (パラメータ) に, さらに別の分布が入っています。分布に分布が入っているから, 混合 mix しているわけです。しかもこの混合加減は, 上に上にとレベルが上がっていきますので, 階層的なモデリングになっています。パラメータのパラメータは, **ハイパーパラメータ (hyper parameter)** と呼んだりします。かっこいい。

さて, これをみればわかるように, 階層線形モデルは, 12 の球団にある違いに正規分布というカバーをかけ, 平均とそこからの散らばりという形で表現することでもあります。このように, 係数たちに正規分布というカバーを上からかけると, 12 球団バラバラで推定した時よりも, その係数が少し小さくなることがあります。上位階層の正規分布の平均値に, 少し引っ張られて推定されてしまうわけです。これを係数の**縮小 (shrinkage)** と言います。そうした問題はありますが, 事前分布や階層性のおかげで, 下位レベルの推定値が安定して算出できることにもなります。

さあ設計図が書けたら, もうコードに落とすことはできますね。ポアソン分布を使った階層線形モデルのコード例を Code::12.3 に挙げておきます。

code : 12.3 階層ポアソン回帰モデル

```
1  data{
```

```
 2    int L;
 3    int G;
 4    array[L] int Gindex;
 5    array[L] real X;
 6    array[L] int Y;
 7  }
 8
 9  parameters{
10    array[G] real beta0;
11    array[G] real beta1;
12    real gamma0;
13    real gamma1;
14    real<lower=0> tau0;
15    real<lower=0> tau1;
16  }
17
18  transformed parameters{
19    array[L] real<lower=0> lambda;
20    for(l in 1:L){
21      lambda[l] = exp(beta0[Gindex[l]] + (beta1[Gindex[l]] * X[l]));
22    }
23  }
24
25  model{
26    // model
27    for(l in 1:L){
28      Y[l] ~ poisson(lambda[l]);
29    }
30
31    for(g in 1:G){
32      beta0[g] ~ normal(gamma0,tau0);
33      beta1[g] ~ normal(gamma1,tau1);
34    }
35
36    //prior
37    gamma0 ~ normal(0,10);
38    gamma1 ~ normal(0,10);
39    tau0 ~ cauchy(0,5);
40    tau1 ~ cauchy(0,5);
41  }
```

■コード解説

data **ブロック**　データは整然データの形で渡します。データの長さ L, 説明変数 X, 被説明変数 Y の他に, 群の数 G, どの群に属するのかを表すインデックス変数 G[L] を用意しています。

parameters **ブロック**　群の数だけ係数 β_0, β_1 が必要です。加えて, これら切片と傾きの平均値 γ と散らばり τ を用意しています。

transformed parameters **ブロック**　`poisson_log` を使ってもいいのですが, 話をわかりやすくするためにいったん λ_{ij} を作っています。係数 `beta0[g]` のグループを表す g のところは, L 行目のデータの所

属先を表すインデックス，`Gindex[l]` で代入しています。`beta1` についても同様です。

model ブロック　データの各行は，行ごとの線形モデルで推定されますのでスッキリしたものです。`beta0[g]` や `beta1[g]` はハイパーパラメータを持つ上位の正規分布にカバーされています。パラメータの事前分布は設計図通りです。

これを使って，年俸でホームランの数を予測する階層線形モデルを推定させましょう。結果を出力 12.1 に示してあります。

MCMC の結果 12.1: ハイパーパラメータの推定値

```
# A tibble: 4 × 7
name          EAP       MED       MAP        SD       L95       U95
<chr>   <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 gamma0     1.446     1.448     1.449     0.120     1.200     1.680
2 gamma1     0.046     0.046     0.044     0.010     0.026     0.068
3 tau0       0.367     0.348     0.324     0.114     0.201     0.642
4 tau1       0.033     0.031     0.028     0.010     0.018     0.057
```

これをみると，12 球団全体で考えると，平均的に年俸が 1000 万上がると $exp(0.046) = 1.047074$，つまり 1 本ホームランが増えるわけですね。ただ年俸の効果は球団ごとの違いがありますから，$\pm$1SD の範囲だと係数は $0.046 + 0.033 = 0.0793$ から $0.0461 - 0.033 = 0.0127$ までぶれることがあるようです[*12]。それよりは切片，平均 4000 万ではありますが，2900 万から 6000 万までの差がありますので，やはりどの球団に属するかの方が効果が大きいかなあ，と思ったりもするわけです。

最後にこの結果をプロットしてみました。球団ごとの 50% 確信区間も合わせてプロットしてあります。これをみると傾きや切片の球団ごとの違いがイメージしやすくなるかと思います。

今日は分布を混ぜるというところから，線形モデルを階層的に組み上げていくことを説明してみました。数式的には難しいように感じるかもしれませんが，設計図を書いてコードにできれば，分析イメージは掴めるのではないでしょうか。データがどういうところから出てきて，どういう潜在的な影響があるのか，個人差や群の差はどの程度あるのか，といった状況に応じて，モデルを柔軟にカスタマイズしていくことができます。よりデータに適切なモデルへ，より柔軟で複雑な表現を目指すことは，とてもクリエイティブで楽しいことではありませんか！

12.4 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの　分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名　`chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと　提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方　小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問

[*12] 傾き β_1 の平均が γ_1，標準偏差が τ_1 であり，それぞれの **EAP 推定値**を見ています。γ_1 の SD，0.0120 は推定値の振れ幅，すなわち標準誤差ですのでご注意ください。

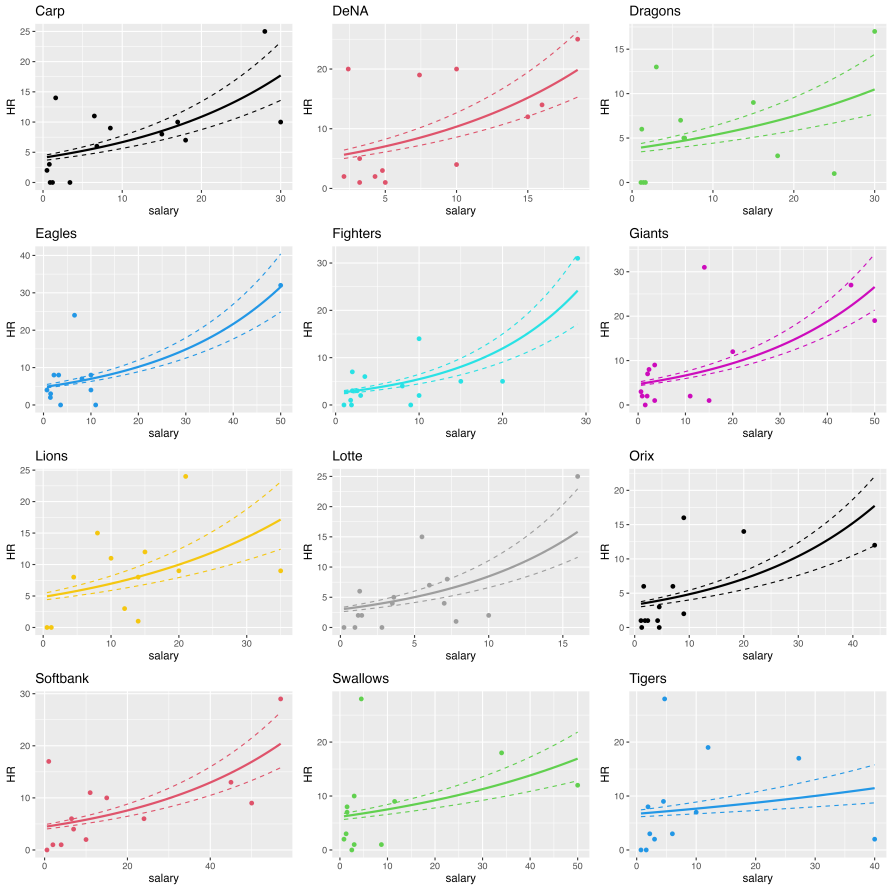


図 12.8　階層線形モデル;チームごとのポアソン回帰分析

1 の答えは ans1, 小問 2 は ans2, 小問 3 は ans3 です。複数の数値を答えるときは, ans2 <- c(mu_eap = 9.7, mu_map = 9.9) のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は, **計算した値をそのまま**にしてください。round や signif で丸めると, 正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが, 提出するオブジェクトの中身は丸めないこと。

わからないところがあれば, 曜日別 TA か小杉までメールで連絡し, 指導を受けてください。

■大問；球団を階層に持つモデル　野球のデータで, 年俸でピッチャーの勝利数を予測する階層線形モデルを考えます。ピッチャーの勝利数はポアソン分布に従うと考えられますので, コード 12.3 が参考になるでしょう。12 球団それぞれに切片を持たせ, その切片が全体平均のまわりに散らばっているという構造にします。データ加工のコードは伴走サイトで提供しますが, 自分で書いても構いません。

1. 球団を上位階層に持つ階層線形モデルを Stan で書いてください。本講の書き方にならい, 母数は球団ごとの切片 beta0 と傾き beta1, その上位にある全体の切片 gamma0 と傾き gamma1, および球団間のばらつき tau0, tau1 とすること。
2. このモデルでベイズ推定を実行し, gamma0 と gamma1 それぞれについて EAP 推定値と 90% 確信区間の下限と上限を, tau0 については EAP 推定値を報告してください。ans2 の

4239 要素名は `gamma0_eap`, `gamma0_lower`, `gamma0_upper`, `gamma1_eap`, `gamma1_lower`,
4240 `gamma1_upper`, `tau0_eap` とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり
4241 5000 サンプル，ウォームアップを 1000 として得ること。
4242 3. MCMC サンプルの 1 つ 1 つについて，12 球団の切片 `beta0` のうちどれが最大かを調べ，球団ご
4243 とに「その球団が最大になった割合」を求めてください。そのうえで，最も割合の高かった球団の番
4244 号と，その割合を報告すること。球団の番号は，データ加工のときに `as.factor` で並べた順としま
4245 す。`ans3` の要素名は `which_best`, `p_best` とすること。得られた割合の大きさと，小問 2 で求めた
4246 `tau0` の値から，球団による違いをどの程度はっきり言えるかを判断してください。

第 13 章

混合分布モデル

さて，これまでは正規分布のもとで生じたデータを使って，線型モデルを扱ってきました。線型モデルは最も単純でわかりやすいのですが，複雑なデータを直線で説明しようとしますから，どうしても無理があるというか，ぴったり当てはまることが少なくありません。しかしこの講義では，Stan をつかって確率分布をいろいろ扱えるようになっているわけですから，なにも正規分布だけにこだわる必要はありません。

今回は，複雑なデータに対してモデルを混ぜ合わせるという方法でアプローチしたいと思います。分布の混ぜ方は，パラメータの構造が複雑化するのではなく，そもそも違うパラメータの分布が混ざり合っているというパターンです。具体的な例から見ていきましょう。

13.1　混合分布モデル

図 13.1 を見てください。これは野球選手データから Tigers の年俸をヒストグラムにしたものです。年俸のデータはそのままだと左に歪んでいるので，対数をとったものを表示しています。ヒストグラムを見ると，どう

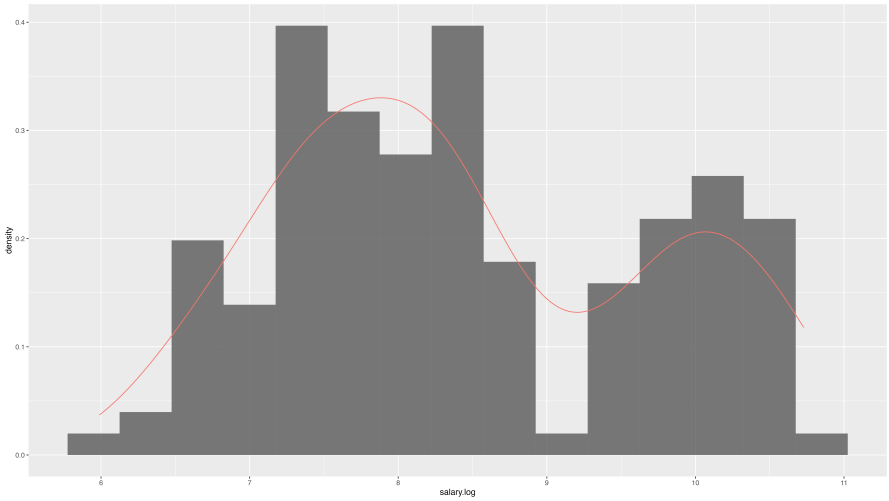


図 13.1　Tigers の選手年俸 (対数) の分布

も 2 つの山があるように見えますね。赤いラインは密度のカーブを書いたものですが，これはどう見ても 2 つのピークを持っています。このようなデータに対して，正規分布のモデルを当てはめるのはおかしなことです。**正規分布**はご存知の通りベルカーブ，すなわちピークは 1 つ (単峰) で，両裾になだらかに下がっていくものですから，このデータと違う形になっていることは間違いありません。

赤い密度カーブの方が暗示するように，この分布は 2 つの正規分布が混じり合って出てきているのではないか，と思われます。つまり，出どころや種類の違う分布があるわけです。これまで分析してきたデータは，基本的に同じ分布からデータが出てきたと考えられてきたわけですが，そうでもない分布の場合はどうするか，という問題です。

こうしたモデルを考えるのが**混合分布モデル**と呼ばれるものです。ここでは複数の正規分布を混ぜ合わせた**正規混合モデル (normal mixture model)** を扱います。

データが複数の出自を持つという時に，その違いを示す変数が明確なのであれば困ることはありません。たとえば男性と女性で分布が違うとか，東日本と西日本で分布が違うという既有知識があって，それがデータに含まれていれば，その変数でデータセットを分割して階層モデルにすれば良いだけです。しかし今回のように，阪神の選手という意味で同じ種類のメンバーであるはずなのに，データの表れ方が違う場合，「この 2 つの分布に分かれる潜在的な変数は何か」ということを探す必要があります。言い換えれば，データの表面的な特徴からグルーピングをしなければなりません。これはデータの類似性だけを用いて，外的基準を持たずして分類する，**クラスタリング (clustering)** という分析方法と関係します。そこで少し寄り道になりますが，クラスタリングの手法について見ておきましょう。

13.1.1 クラスタリングいろいろ

クラスタリングという分析方法は，先に述べたように別の変数 (外的基準) を使わずに[*1]，データの分類を行う方法です。分類は科学の基本で，まず同じものか違うものか，何が同じで何を違うものと考えるか，ということがすべてのスタートになります。クラスタリングではこれをデータから行う必要があり，データの類似性をもとに分類することが基本です。

データの**類似度**あるいは同じことですが**非類似度**を表す指標の基本は**距離 (distance)** です。距離と聞くと普通は 2 点間を直線で結んだ**ユークリッド距離 (Euclidean distance)** を想像されると思いますが，実は距離には他にもいろいろな種類があります。たとえば**相関係数**も絶対値を取ればデータの類似度を表す距離情報の一種と考えることができます。この距離を使って，距離が短い＝類似している＝同じクラスター，という考え方から分類していくことになります。

こうした分類，クラスタリングの手法はいくつかあって，大きく分けると**階層的クラスタリング (hierarchical clustering)** と**非階層的クラスタリング**の二種類に分かれます。階層的というのは HLM の発想と同じで，クラスターが積み重なっていくようなイメージです。たとえば要素 $\{a, b, c, d, e\}$ をクラスタリングする時，距離が近いものを使ってまず $\{a\}, \{b, c\}, \{d, e\}$ と分割したとします。次に $\{b, c\}$ を 1 つの新しいデータ点だと考えて，a と $\{b, c\}$，$\{b, c\}$ と $\{d, e\}$ の距離を考え，近い方をさらに次のクラスターにまとめます。すなわち $\{a, \{b, c\}\}, \{d, e\}$ というようにです。それをさらに上位階層でまとめ $\{a, \{b, c\}, \{d, e\}\}$ とすべてが 1 つのクラスターにまとめられたら終わり，とより包括的なクラスターへ，より上位のクラスターへとまとめていくのが階層的クラスタリングという手法です。この方法で分類した例を図 13.2 に示します。階層的クラスタリングは，階層が上がる時にクラスター化されたもの同士の距離をどう考えるかによって，いくつかの方法があります。先ほどの例ではまず $\{b, c\}$ が 1 つのクラスターになりましたが，これを A とすると，a と A の距離をどう考えるか，がいろいろあるわけです。大文字の A の方は，2 つの要素から合成されたもの ($A = \{b, c\}$) ですから，a と b の距離を取るのか，a と c の距離を取るのか，はたまた b, c の平均的な値と a との距離を取るのか等々，基準が必要なわけです。複数の要素を持つクラスター同士の距離の考え方として，その最大値を取る**最大距離法**，最小値を取る**最小距離法**だけでなく，データの重心を取る**セントロイド法**，平均を取る**平**

[*1] **機械学習**の文脈では，こうした基準となる変数を置かずに分析する手法のことを**教師なし学習**，と呼びます。

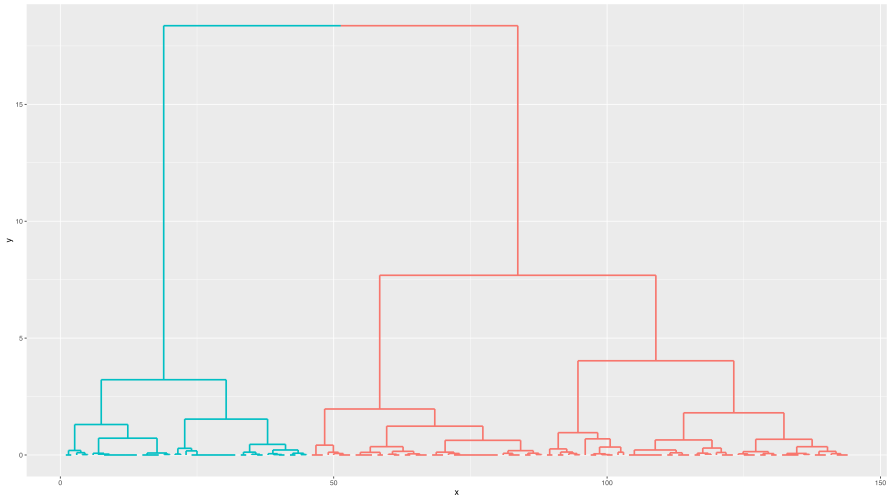


図 13.2　Tigers の選手を年俸で階層的クラスタリング (ウォード法による)

均法, クラスター内分散とクラスター間分散の差が最小となるように取る**ウォード法 (Ward's method)** などがあります[*2][*3]。最後のウォード法は弁別後の結果がわかりやすいため, よく用いられます。

一方, 階層的でないクラスター分析で有名なのが **K-平均法 (k-means 法)** です。これは変数間の距離に k 個の基準点をランダムに置き, その点に近いものをクラスターとします。次に作られたクラスターの重心を新しい基準点として, 再度分類します。それの重心を次の基準点とアップデートして再分類 $\cdots$ と繰り返して, 分類結果が変わらなくなる前で何度も反復するのです。だいたい数回の反復で分類できること, データのサイズが大きくても効率よく部類できることからよく利用される方法の 1 つです。最初に幾つのクラスターに分類するか, 明確な基準がないことが欠点でしたが, その後のモデルの展開でクラス数の適合度などを算出して適切なクラス数を求めるようになりました。

クラスター分析のもう 1 つの分類基準が, クラスターの境界強度によるものです。**ハードクラスタリング (hard clustering)** では, ある個体がどのクラスターに含まれるかがはっきりしています。この個体はクラスター A, この個体はクラスター B, と決められたらそれで決まりです。これに対して**ソフトクラスタリング (soft clustering)** は, ある個体がクラスター A に含まれる確率が XX%, クラスター B に含まれる確率が YY%, のように分類が確率的で固定的ではない方法です。

このように, データの特徴から分類をしていくクラスター分析はいろいろな手法があるのですが, 今回扱う混合分布モデルはモデルに基づいたクラスタリング, Model based clustering とも呼ばれる手法です。すなわち, 各個体が潜在的な正規分布のどちらに含まれる可能性があるか, その確率を考えながらソフトに分類していく非階層的な方法になります。

[*2] 厳密に書くと, クラスター P に含まれる要素と重心との距離の二乗和を $E(P)$ とすると, 二つのクラスター P,Q を併合して作られる新しいクラスター T を考えた時, $E(T) - E(P) - E(Q)$ を P,Q の距離と考える方法です。

[*3] R では標準関数 `hclust` で階層的クラスター分析ができ, ウォード法を使うときは `method='ward.D2'` とします。この `.D2` にはいわれがあって, かつては `ward.D` でウォード法の指定ができたのですが, この関数にバグが含まれていることが判明, 修正版を `ward.D2` としたのです。R は無償のフリーソフトウェアですから, バグが含まれているなんてやっぱり使い物にならない！という批判もあるのですが, オープンなソフトウェアであるからこそソースコードが誰にでも見られて指摘することができ, よくなっていくものでもあります。バグがあったこと, それを自浄作用でフィックスしたことを記録するために, `2` を残すことになりました。プロプライエタリな商用ソフトの場合, 間違いがあってもこっそり修正して, 同じ関数名で提供されてしまえば, ユーザは分析が間違っていたことに気づくことすらできません。科学技術に関するソフトウェアはオープンであるべきだ, という R の思想がここに現れていると言えるでしょう。

13.1.2 混合分布モデルの考え方

混合分布モデルの考え方は，アルクラスターに含まれるかどうかが確率的に決まるというものです。たとえばコイントスをして，表が出たらクラスター A，裏が出たらクラスター B，というように分類するようなものです。コイントスはベルヌーイ分布に下がいますから，確率 θ でクラスター A，確率 $1-\theta$ でクラスター B，と言っていることと同じです (図 13.3)。

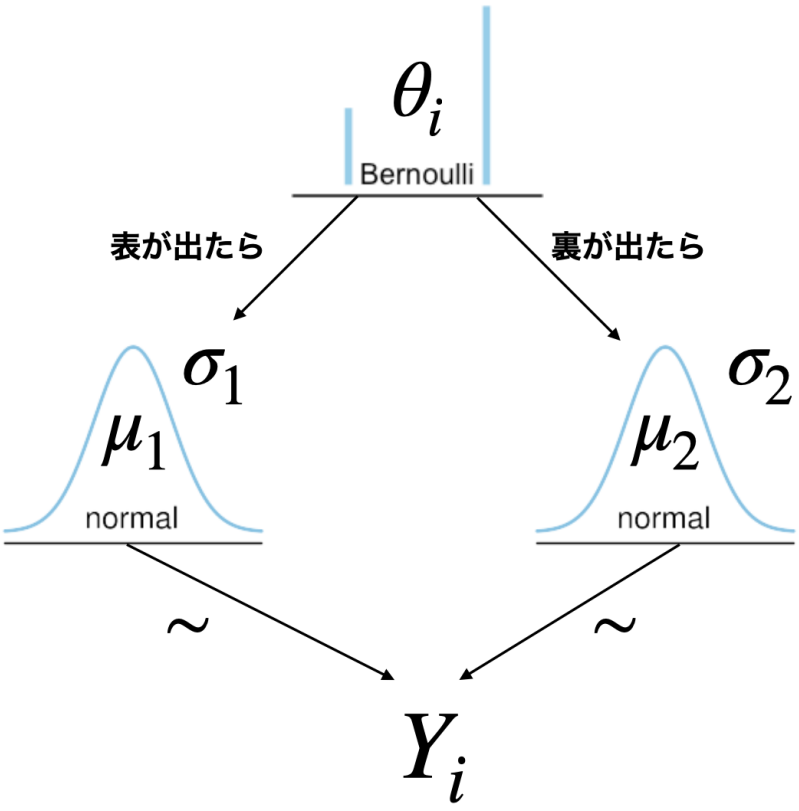


図 13.3 二つの正規分布からデータが出てくるモデル

もちろん 3 つ以上のクラスターに分類することもあるでしょう。この場合は**カテゴリカル分布**[*4]に従う確率変数 z を考えることになります。

データが K 個の正規分布，いずれかから得られていると考えるとしましょう。ここで各正規分布はそれぞれの位置母数 μ_k とスケール母数 σ_k を持っているとします。個々のデータ i がどの正規分布からきているかを表す混合確率 λ_{ik} を考えると，$\lambda_{ik} \geq 0$ で $\sum_{k=1}^{K} \lambda_{ik} = 1$ になります。つまり，i ごとの K-simplex ベクトルを考えることになります。個々のデータ Y_i の出自は，$z[i] \sim Categorical(\lambda_i)$ で決まり，データ $Y_i \sim N(\mu_{z[i]}, \sigma_{z[i]})$ からきている，と考えることになります (図 13.4)。

[*4] カテゴリカル分布は，複数のカテゴリから 1 つを選ぶ試行の確率分布を表現する離散確率分布です。例えば，サイコロを振って 1 から 6 の目が出る確率や，アンケートで「とても良い」「良い」「普通」「悪い」「とても悪い」のいずれかが選ばれる確率を表現できます。カテゴリカル分布は多項分布の特殊な場合で，試行回数が 1 回の場合に相当します。ベルヌーイ分布は，カテゴリ数が 2 つの場合のカテゴリカル分布と見なすことができます。

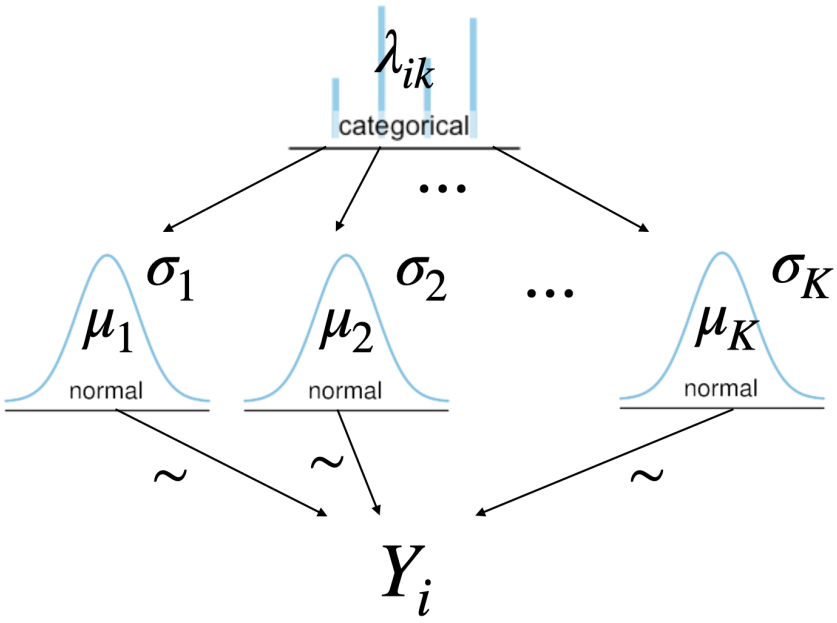


図 13.4　K 個の正規分布からデータが出てくるモデル

データ生成メカニズムの設計図は図 13.3，13.4 の通りです。具体的なプログラミングに進む前に，イメージをしっかり掴んでおきましょう。これらの設計図の一番上にある，どのクラスターからデータが出てくるかを確率的に決めるというところがポイントです。ですがこのモデルを実装できるようになれば，データ発生の条件分岐を確率モデルで表現できるようになるのです。確率 θ で左，$1-\theta$ で右のルートを取る，というように，データが出てくるメカニズムの背後に潜在的な条件を設定できますから，分析の可能性がグッと広がることになります。実践的にはたとえばマーケティングにおいて，顧客がどういうクラスター (主婦層，独身者，大家族 etc...) に属するかはわかりませんが，買い物の傾向から分類するというようなことができます。あるいは心理学的なシーンでも，潜在的に異なる種類 (抑うつ傾向があるタイプ，ないタイプなど) が，反応傾向の違いから読み取れるようになるかもしれません。

クラスター分析はデータから分類するので，分類後にそれがどう言った変数と関係しているのか，あるいは分類されたクラスターにはどういう特徴があるのかを探索することが一般的です。それでもデータだけから分類できるのは魅力的な手法であると言っていいでしょう。

13.2　ターゲット記法と周辺化消去

さてこの条件分岐をどのようにコードに書いていくか，というところに話を進めましょう。ここで新しく，2 つの技術を導入する必要があります。1 つはターゲット記法，もう 1 つは**周辺化消去 (marginalizing)** です。

13.2.1　ターゲット記法

これまで Stan で，確率変数に従う分布の記法は ~ を使って表現していました。すなわち，コード 13.1 のような書き方です。

code : 13.1 sampling 記法

```
1  for(i in 1:N){
2      Y[i] ~ normal(mu,sigma);
3  }
```

これと同じ動きをする，別の表記方法があります。それが**ターゲット記法**というもので，コード 13.2 のように書きます。

code : 13.2 target 記法

```
1  for(i in 1:N){
2      target += normal_lpdf( Y[i] | mu, sigma);
3  }
```

何やら奇妙な書き方に見えますが，順番に見て行きましょう。まず +=の右側，noraml が norma_lpdf になっているところに注目してください。ここで lpdf は log probability density function，すなわち対数確率密度関数という意味です。normal というのがついていますから，正規分布の対数確率密度関数ということになりますね。ちなみに離散変数の場合は**確率密度 (probability density)** ではなく**確率質量 (probability mass)** になりますから，lpmf，すなわち log probability mass function(対数確率質量関数) になります。具体的には，bernoulli_lpmf とか poisson_lpmf となりますので注意してください。

さて対数確率密度関数ってのがなぜ出てきたのでしょうか。ここで**尤度 (likelihood)** のことを思い出してほしいのですが，尤度とはデータが**確率分布**から出てくるときの尤もらしさ，出てきやすさのような指標なのでした。確率分布はパラメータがわかっている時にデータが出てくる確率を記述する関数ですが，同じ関数でパラメータが未知，データが既知の場合 (多くの研究シーンはこちらですが) に，それを尤度関数とよぶのでした。データが得られた時の尤度を計算し，その尤度が最も大きくなるところをパラメータの推定値とするのが**最尤推定法 (maximum likelihood method)** であり，尤度を使って事後分布を算出，事後分布の形から推定値を考えるのが**ベイズ推定法 (Bayesian inference method)** なのでしたね。つまり MCMC で計算するにはデータ全体の尤度を考えているのです。そしてデータ全体尤度は各データ点の尤度の総積 $\prod$ で考える必要がありますが，確率の掛け算は数字が小さくなるので**対数尤度 (log-likelihood)** の総和で計算するのでした。

具体的に書くと，データ $\boldsymbol{Y} = Y_1, Y_2, Y_3$ がパラメータ θ のベルヌーイ分布から出てきている場合，データの尤度は

$$L(\boldsymbol{Y}|\theta) = \prod_{i=1}^{3} bernoulli(Y_i|\theta) = bernoulli(Y_1|\theta) \times bernoulli(Y_2|\theta) \times bernoulli(Y_3|\theta)$$

であり，対数尤度は

$$LL(\boldsymbol{Y}|\theta) = \sum_{i=1}^{3} \log(bernoulli(Y_i|\theta)) = \log(bernoulli(Y_1|\theta)) + \log(bernoulli(Y_2|\theta)) + \log(bernoulli(Y_3|\theta))$$

となるわけです。ここで $bernoulli(Y|\theta)$ の縦棒 | は，θ が与えられた時の Y の出る確率，という条件付き確率の記号になります。

プログラミング的には，$Y_1, Y_2, \cdots, Y_n$ は for(i in 1:N){ ~ }という形で 1 から N までの繰り返し計算を意味しますから，コード 13.1 や 13.2 でやっていることは，各データ点についての対数尤度を足し合わせて行っている作業そのものを意味しています。

ここでターゲット記法にある +=ですが，これはプログラミング独特の表記方法で，数学表記ではありません。プログラミングでは代入を使って，たとえば変数 A に 1 を加えたものを新しい A と

する (上書きする), ということを `A = A + 1` と表記できます。この `+=`はこうした上書きを一言で書く記号で, この A の例だと `A += 1` とすれば 1 を加えて上書き, と同じ意味になります。これを踏まえてターゲット記法を見ると, `target += normal_lpdf( Y[i] | mu, sigma)` ですから, `target = target + normal_lpdf( Y[i] | mu, sigma)` としていることと同じです。つまり, 対数尤度を足しあげて行ってね, ということを明示的に書いていることになります。`target` は足し合わされたもの, というだけで「目標はこれ」を意味するぐらいの予約語だと思ってください。

サンプリング記法とターゲット記法は, 書き方が違えど同じ働きをするのですからどちらでもいいじゃないか, と思うかもしれませんが, ターゲット記法でないと表現できないようなことがあるので, このような代替手法が用意されているのだと思ってください。ターゲット記法でないと表現できないようなこと, というのはもちろん今回の, 分布を混ぜ合わせる時がそれです[*5]。

13.2.2 周辺化消去

さてここまでさまざまな問題を解決してくれた Stan ですが, Stan には離散型のパラメータを扱うことができない, という欠点があります[*6]。今回は $z[i] \sim Categorical(\lambda_i)$ と, どの目が出たかというパラメータが離散的ですから, このままではコード化できません。ではどうするか。仕方がないので, 「起こりうる可能性をすべて数え上げて考える」という方法を取ることにします。最終的な事後分布は対数尤度関数を足し合わせたものになりますから, 条件分岐したルートすべてについて対数尤度関数を考えて, それを足し合わせてやれば良いのです。

簡単な例として図 13.3 にあるような, 2 つの正規分布モデルを考えてみましょう。コイントスで表が出れば $N(Y_i \mid \mu_1, \sigma_1)$, 裏が出れば $N(Y_i \mid \mu_2, \sigma_2)$ のルートに行けば良いのです。表が出る確率は θ ですから, 表が出るルートを通ってデータ Y_i が出てくる確率は $\theta \times N(Y_i \mu_1, \sigma_1)$ になります。同じく裏が出るルートを通ってデータが出てくる確率は, $(1-\theta) \times N(Y_i \mid \mu_2, \sigma_2)$ ということになります。

ですから, 最終的にデータ Y_i が出てくる確率は $\theta \times N(Y_i \mid \mu_1, \sigma_1) + (1-\theta) \times N(Y_i \mid \mu_2, \sigma_2)$ だ, ということができるわけです。

さて, これらは確率をそのまま使った表現になっていますが, Stan の中では対数で考えるのでした。つまり表が出るルートの対数尤度は

$$\log(\theta) + \log(N(Y_i \mid \mu_1, \sigma_1))$$

ですし, 裏が出るルートもまた同様に log をとって考える必要があります。対数を取ると積が和になるので, $\times \rightarrow +$ に変わっていることに注意してください。

ところがこの表ルート, 裏ルートは足し合わせないといけません。対数をとったものを直接足し合わせるわけにはいきませんから, 対数尤度をただの尤度に戻す, つまり log の逆関数である exp 関数を通す必要があります。ややこしいですが,

$$\exp\left(\log(\theta) + \log(N(Y_i \mid \mu_1, \sigma_1))\right) + \exp\left(\log(1-\theta) + \log(N(Y_i \mid \mu_2, \sigma_2))\right)$$

が尤度であり, これの対数をとったものを `target` に加える必要があるのです。

このように, exp を通して, 足し合わせ ($\sum$) て, さらにその log をとるという作業をひとまとめにした関数が, Stan にはあります。それが `log_sum_exp` 関数です。この関数は, `log_sum_exp(x,y)` とすると $\log(\exp(x) + \exp(y))$ という操作をします。この関数に渡すものがベクトルであれば,

[*5] そのほかにもターゲット記法は, サンプリング記法では省略されている事後対数確率も記録するという特徴があり, ブリッジサンプリングによる事後対数尤度の計算などにはこちらの表記が必要になってきます。もっとも非常に専門的なことなので, ここでは条件分岐の際に必要な記法, と割り切っていただいても構いません。

[*6] JAGS という確率的プログラミング言語でしたら, これは可能です。

$$\texttt{log_sum_exp}\left(\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}\right) = \log\left(\sum_{i=1}^{n} exp(x_i)\right)$$

という操作をすることになります。

これを使って混合分布モデルの実装をしていきましょう。

13.2.3 混合分布モデルのコード

先ほどの，阪神タイガースにおける選手の年俸分布を 2 つの正規分布から出てきているもの，と考えてコード化したものは，Code13.3 のようになります。

code : 13.3 混合正規モデル

```
1  data {
2    int<lower=1> K;
3    int<lower=1> L;
4    array[L] real Y;
5  }
6
7  parameters {
8    array[L] simplex[K] theta;
9    ordered[K] mu;
10   array[K] real<lower=0> sigma;
11 }
12
13 transformed parameters{
14   array[L] vector[K] lp;
15   for (l in 1:L) {
16     for (k in 1:K) {
17       lp[l,k] = log(theta[l,k])+ normal_lpdf(Y[l]|mu[k],sigma[k]);
18     }
19   }
20 }
21
22 model{
23   for(l in 1:L){
24     target+=log_sum_exp(lp[l]);
25   }
26   sigma ~ cauchy(0,5);
27   mu ~ normal(0,10);
28 }
29
30 generated quantities{
31   array[L] vector[K] prob_class;
32   array[L] int<lower=1,upper=K> pred_class;
33   for(l in 1:L){
34     prob_class[l] = softmax(lp[l]);
35     pred_class[l] = categorical_rng(prob_class[l]);
36   }
```

```
37  }
```

■コード解説

data **ブロック**　クラスター数 K, データ長 L, アウトカム変数 Y を宣言しています。

parameters **ブロック**　合計が 1 になる K の長さを持つベクトルである，`simplex` 型ベクトルを，データ数だけ用意します。また，K 個の正規分布からデータが出てきているのですから，平均と標準偏差も K 個用意します。ここで大事なポイントとして，平均 μ が `ordered` 型としてあるところです。このベクトルは，要素 $\mu_1 > \mu_2 > \cdots > \mu_k$ と，要素が大きさの順に並んでいることです。このような制約をかけておかないと，K 個の正規分布がどの位置にあるのか判然としな苦なるという問題が生じます。たとえばあるチェインで $\mu_1 > \mu_2$ となっていて，データ X_k が 1 番目の正規分布から出てきていると考えられたとしましょう。しかし別のチェインで $\mu_1 < \mu_2$ となっていれば，X_k は平均値が大きい方から出てきているのですから，こちらのチェインでは 2 番目の正規分布から出てきたものとして，MCMC サンプルが進むことになります。複数のチェインから得られた MCMC サンプルは最終的に統合されますから，このようになっていると X_k は正規分布 1 からも 2 からも同じように出てきていることになります。各チェインでは正しくサンプリングできているのですが，チェインごとに分布のラベルが違うので，統合すると訳がわからなくなる，この問題をとくに**ラベルスイッチング (label switching)** 問題といい，混合分布モデルではよく生じる問題なのです。そこで分布の位置母数に順番の制約を入れて，ラベルの変動を止める必要があります。

transformed parameters **ブロック**　長さ K のベクトルをデータの数 L だけ用意した `lp` 変数を作りました。これは log-probability の略でこのような名前にしましたが，任意の名前で結構です。l 行目のデータがクラスター k に含まれる可能性は，$\theta_{lk} \times N(Y[l] \mid \mu_k, \sigma_k)$ ですから，それの対数をとった形で表現しています。

model **ブロック**　モデル尤度のところは，対数 → exp 関数で確率の形 → ベクトルの要素をすべて足し合わせる → log をとって対数尤度にする，という計算をまとめて `log_sum_exp` 関数で行い，それをすべて `target` に追加していくという方法で書いています。事前分布のところは普通のサンプリング記法で書きました。ちなみに μ, σ はそれぞれ K 個ずつありますが，Stan は添字の省略を許してくれます。

generated quantities **ブロック**　久しぶりに生成量ブロックの登場です。ここでは分析結果を使って，l 行目のデータが結局何番目の正規分布から出てきたのかを逆算して出しています。まず，対数尤度で表現されている K 個の要素を持つベクトルを，確率の形に計算したものを `preb_class` 変数にしています。ここで使われている `softmax` という関数は，

$$softmax(x) = \frac{exp(x)}{\sum_{k=1}^{K} exp(x_k)}$$

という計算をするものです。分母はベクトルの総和で分子がその要素ですから，要するに各クラスに入る確率を表すベクトルになっている訳です。これを使って `pred_class` 変数を作っています。今回のデータから予想される所属クラスなので，K 面サイコロを振った出目という形で表現します。`categorical_rng` はカテゴリカル分布からの乱数発生で，その発生確率が先ほどの `prob_class` になっているので，この乱数で出た目が予測されるクラスということです。

これを実行するための R コードをコード 13.4 に用意しました。

code : 13.4 混合分布モデルのコード

```
1  dat <- read_csv("BaseballDecade.csv")
2  dat.tmp <- dat %>%
3    filter(position != "投手") %>%
4    filter(Games > 50) %>%
5    filter(team == "Tigers") %>%
6    mutate(salary.log = log(salary)) %>%
7    dplyr::select(salary.log, Name) %>%
8    rowid_to_column("ID")
9
10 dataSet <- list(K = 2, L = NROW(dat.tmp), Y = dat.tmp$salary.log)
11 model <- cmdstanr::cmdstan_model("latent.stan")
12 fit <- model$sample(data = dataSet, chains = 4, parallel_chains = 4)
```

■コード解説

1 行目　データファイル BaseballDecade.csv を読み込みます。

2-8 行目　分析用のデータに加工しています。

3 行目　フィルターをかけてデータを野手だけのものにします。

4 行目　フィルターをさらにかけて，50 試合場出ている選手に限定します。

5 行目　チームが Tigers の選手だけに絞っています。

6 行目　年俸のデータは歪んでいるので，対数をとって正規分布の形に近似させました[*7]。

7 行目　使う変数だけに絞っています。

8 行目　行番号を ID という変数名にして追加しました

10 行目　データセットを作っています。

11 行目　cmdstanr でコンパイルしています。

12 行目　cmdstanr のサンプリング関数です。

推定した結果を見てみましょう。私の環境では出力 13.1 のようになりました。

MCMC の結果 13.1

```
# A tibble: 4 × 7
  name           EAP       MED       MAP        SD       L95       U95
  <chr>    <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu[1]        7.624     7.624     7.638     0.107     7.415     7.826
2 mu[2]        9.702     9.699    10.017     0.284     9.164    10.138
3 sigma[1]     0.656     0.653     0.646     0.063     0.544     0.788
4 sigma[2]     0.762     0.786     0.901     0.237     0.380     1.192
```

対数をとった時の平均値が 7.624 と 9.702 ですから，指数関数を入れて読み直せば $\exp(7.624) = 2046.733$,$\exp(9.702) = 16350.28x$ となります。2000 万円クラスの選手と，1 億 6500 万円クラスの選手の 2 つのグループに分かれることが見て取れますね。事後予測分布から，所得平均の高い「超一流」選手と「一

[*7] データを変形させずに確率分布の方で対応するのがよい，と常々話をしてきましたが，ここではデータを変形させています。実は左に歪んだような分布は対数正規分布というのがあり，そこからのデータ生成を考えても良いのですが，対数正規分布はデータの対数が従う正規分布というだけなので，データの方を変えてしまった方が説明しやすいのです。比率や度数など，データの生成メカニズムがそもそも違うばあいは，データを変換せずに使うのが正しいやり方です。

般的」な選手とに分けて分布を色分けしてみました (図 13.5)。データから，この選手がどちらのクラスに所属しているかも想像できるところがおもしろいですね。

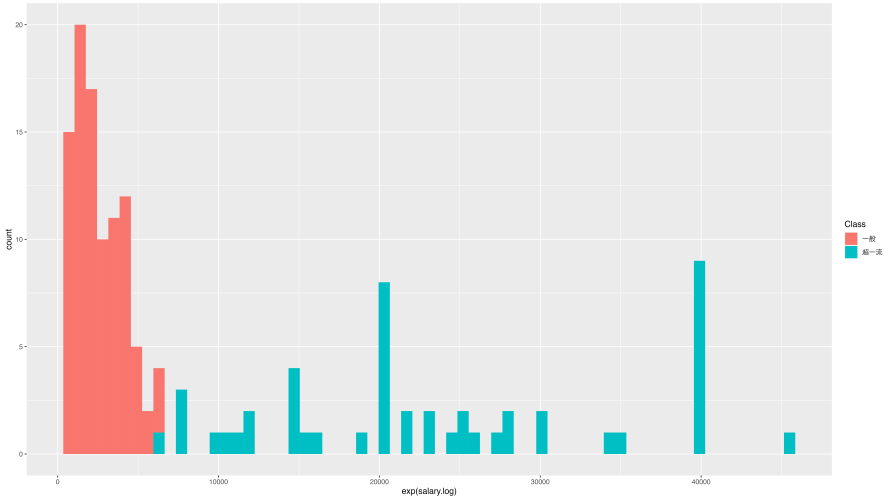


図 13.5　選手のクラスタリング

13.3　ゼロ過剰ポアソン分布モデル

さて，異なる分布の混ぜ合わせ，離散確率分布による条件分岐の例として，もう 1 つ別の例を示しましょう。次の図 13.6 を見てください。

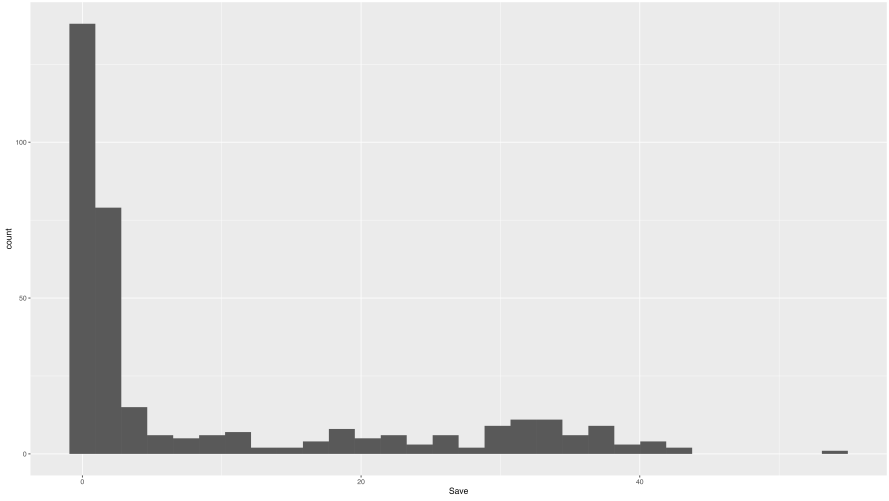


図 13.6　投手のセーブ数

これは投手のセーブ数の度数分布です。野球は 9 回のターン制で，1 回の最初から投げる投手は先発投手と言います。先発投手が 9 回まで投げ切れればいいのですが，疲労も溜まるし戦術によっては投手の打席に代打を投入することもあります。投手が降板した場合は，次の投手が代わって続きから投げますが，「勝っている試合で後半まできたので，最後の 1，2 回は確実に勝ち逃げできるようにしたい」というときに投入されるのが抑え (クローザー) と呼ばれる投手です。最近のプロ野球は分業が進んでいますから，先発投手が

完投することは少なく，100 球程度で交代し，中継ぎ – 抑えと投手リレーしていくのが一般的な戦術です。さて，先発投手は最初から出てきて 6，7 回まで投げますから (途中でボロボロに打たれたり怪我で降板などがなければ，ですが)，毎日のように登板することはなく，一度登板すると 3，4 日休んで次の試合にでる，ということになります。日本プロ野球は年間 140 試合近くありますが，通年で 2-30 回も登板することはありません。一方，抑えの投手は最後の 8，9 回を投げる程度で，しかも負けている試合などでは出番がありません。必然的に中 4 日の休憩なども入りませんから，試合数はたくさん出ることができます。そして抑えに成功するとセーブポイントというのがつきますが，このセーブポイントは優秀なクローザーであれば年間数十ポイント取ることができる訳です。

さて，図 13.6 にあるようにこのセーブポイントを見てみますと，ほとんどの投手がゼロになっています。以下データをみますと，1 セーブが 53 人，2 セーブが 26 人･･･となって，20〜40 セーブある投手が 10 人弱ずつぐらいいる，という分布をしています。これは先ほども言った分業制のせいで，先発投手にセーブポイントがつくことはありませんし，先発投手は 1 チームに 10 人弱ぐらいは揃えているでしょうから，投手全体で見るとセーブポイントは 0 の人が多いのです。

このセーブポイント，0 以上の整数を取りますからポアソン分布に従うと考えられるのですが，ポアソン分布のパラメータ λ をどう変えても (図 13.7)，こんな形にはなりません。

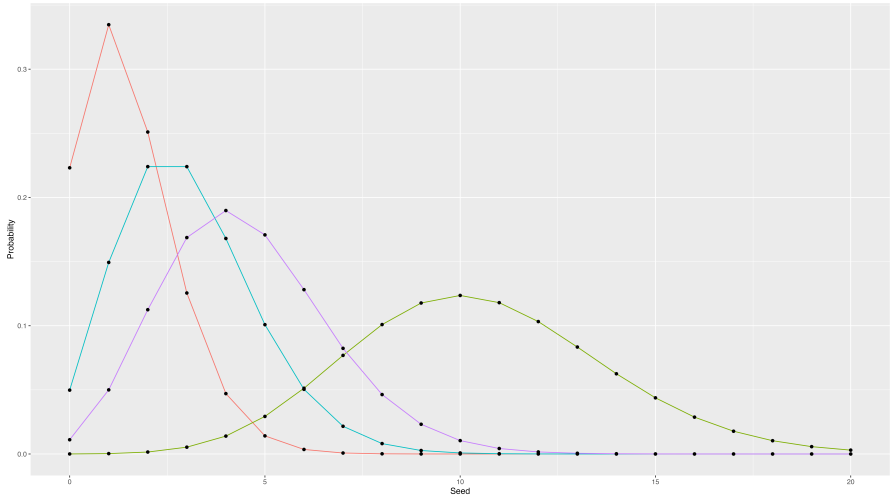


図 13.7　さまざまなポアソン分布。λ は 1.5，3，4.5，10.5 で描画

そこで混合分布です。まずセーブ数が 0 かどうかを考えます。セーブ数 0 というのは，そもそもクローザーではないか，クローザーなのに 0 セーブポイントのへっぽこか，の二択です。セーブ数が 0 でない限り，ポアソン分布からセーブ数が生成される，と考えるのです。設計図のイメージは図 13.8 の通りです。

このモデルがちょっと特殊なのは，ご覧の通り 0 というデータの値一箇所に極端な偏りがあるところです。この分布のことを**ゼロ過剰ポアソン (zero-inflated Poisson)** といいます。

注意してほしいのですが，ポアソン分布から 0 の度数が出てくることもあり得ます。逆に考えると，データ $Y_i = 0$ であれば，これはクローザーでないからそうなっているのか，クローザーなのにヘッポコでセーブ数がつかなかったのかのどちらか，という判断をしなければならないということです。つまり，データからの分岐を考えることになります。クローザーかどうかは確率 θ で決まるとして，θ ならクローザー，$1-\theta$ ならクローザーじゃないとすると，

$$\begin{cases} \text{データが 0 ではない} & \to \theta \times Poisson(Y_i \mid \lambda) \\ \text{データが 0 だ} & \to 1-\theta \text{ or } \theta \times Poisson(0 \mid \lambda) \end{cases} \tag{13.1}$$

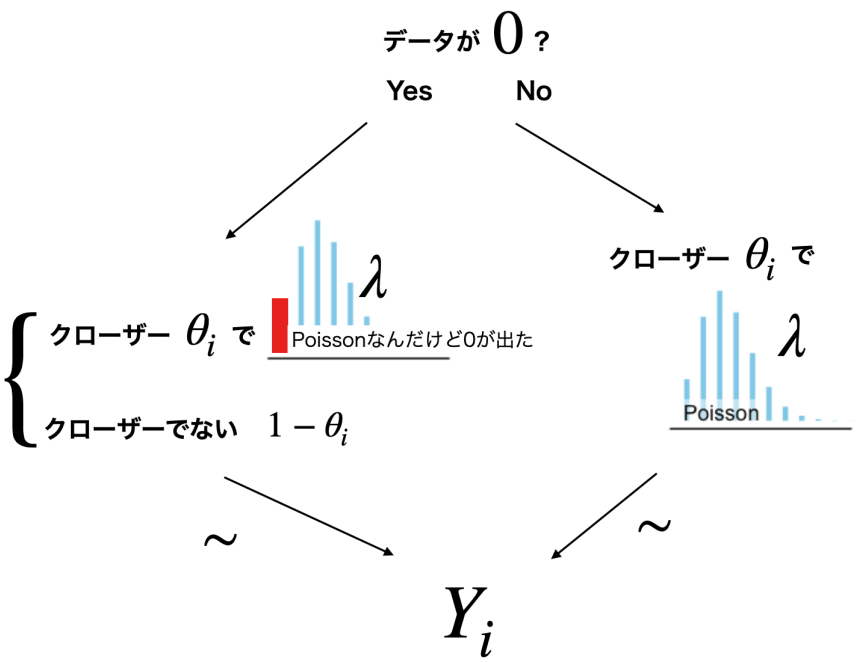


図 13.8 ポアソン分布との混ぜ合わせ

このデータが 0 のときに生成メカニズムが条件分岐していますから，ここに log_sum_exp 関数を入れる必要があります。

$$\texttt{log_sum_exp}\begin{pmatrix}\log(1-\theta)\\ \log(\theta)+\texttt{poisson_lpmf}(0\mid\lambda)\end{pmatrix}$$

このことを念頭において，コードを書いてみましょう (コード 13.5)。

code : 13.5 ゼロ過剰ポアソン分布

```
data{
  int L;
  array[L] int Y;
}

parameters{
  real<lower=0,upper=1> theta;
  real<lower=0> lambda;
}

model{
  for(l in 1:L){
    if(Y[l]==0){
      target += log_sum_exp(log(1-theta),log(theta)+poisson_lpmf(0|lambda));
    }else{
      target += log(theta) + poisson_lpmf(Y[l]|lambda);
    }
  }
}
```

■コード解説

data ブロック　データ長 L, アウトカム変数 Y を宣言しています。

parameters ブロック　クローザーかどうかを決める θ と, ポアソン分布のパラメータ λ がパラメータです。

model ブロック　データの中で, プログラミング言語としての `if` 文による条件分岐が出てきています[*8]。データが 0 であれば, クローザーでないか, ポアソン分布からゼロが出ているかです。そうでなければ, 確率 θ 経由でのポアソン分布です。

これを実行するための R コードはコード 13.6 のようになります。

code : 13.6　ゼロ過剰ポアソンモデルのコード

```
dat.tmp <- dat %>%
  dplyr::filter(position == "投手") %>%
  dplyr::filter(Games > 50) %>%
  dplyr::select(Save)

model <- cmdstan_model("ziPoisson.stan")
dataSet <- list(L = NROW(dat.tmp), Y = dat.tmp$Save)
fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4
)
```

推定した結果を見てみましょう。私の環境では出力 13.2 のようになりました。

MCMC の結果 13.2

```
# A tibble: 2 × 7
  name          EAP       MED       MAP        SD       L95       U95
  <chr>   <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 lambda     14.666    14.665    14.661     0.261    14.154    15.185
2 theta       0.606     0.606     0.608     0.025     0.556     0.655
```

これを見ると，6 割の確率でクローザー判定され, クローザーであれば $\lambda = 14.7$ ぐらいのポアソン分布でデータが生成されるということですね。事後予測分布のように, この確率モデルで出てくるデータを図 13.9 に示してみました。単一の分布では表現できないモデル化ができているのではないでしょうか。もっとも, 今回は 1，2 回セーブポイントがついている人のところをうまく表現できませんでした。しかしコードを書き換えることで, セーブ数 0 の先発投手と, 本職のクローザーに加え, セーブ数が一桁ぐらいの中継ぎ投手の三種類に分割する, ということもできるでしょう。

ここでみた分布の混ぜ方は, たとえば「お店に初来店した人とリピーターの来店数」とか,「20 歳までの恋愛経験の数」のようなデータがあったときに, その特徴から分割してモデルを調整できます。研究のシーンでも, たとえば「質問紙で何も考えずにどちらとも言えないと答える人と, ちゃんと考えてどちらとも言えないに丸をつけた人」というように回答者のバイアスを除去してその特徴を考えるなど[*9], 積極的な分析をできるのです。

[*8] プログラミングでの条件分岐については第 2 講, セクション 2.1.3，Pp.21 でやりましたね。忘れた人は戻って再確認しましょう。

[*9] この場合は中点の 3 が多い, というようなケースでしょうから，3 過剰正規分布とでも言えば良いでしょうか。応用例として清水 (2018) などがあります。

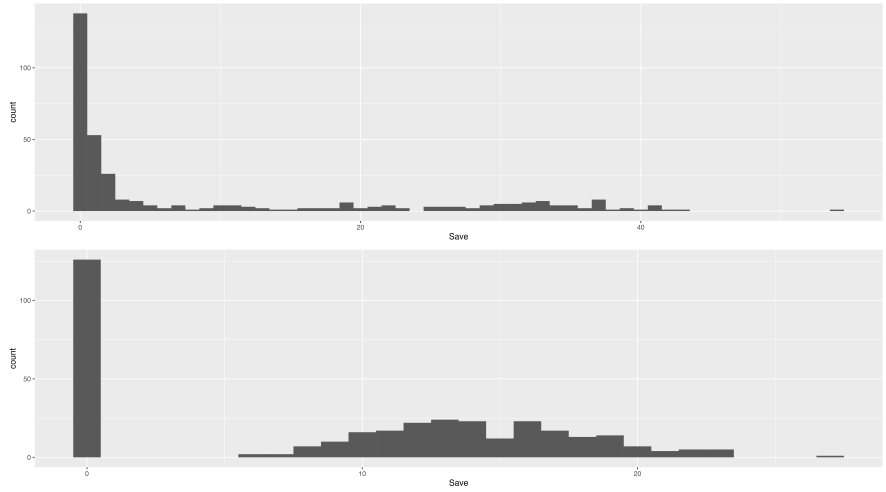


図 13.9　ゼロ過剰ポアソン分布による当てはめ。上がロウデータ, 下が事後予測分布

さらに言えば, このポアソン分布の λ に線形モデルを入れて, ゼロ過剰ポアソン回帰分析にすることだってできますね。たとえばクローザーのセーブ数が年俸によって説明される, というモデルにするなら, コード 13.7 のようにすれば良いのです。

code : 13.7　ゼロ過剰ポアソン回帰

```
data{
  int L;
  array[L] int Y;
  array[L] real X;
}

parameters{
  real<lower=0,upper=1> theta;
  real beta0;
  real beta1;
}

transformed parameters{
  array[L] real<lower=0> lambda;
  for(l in 1:L){
    lambda[l] = exp(beta0 + beta1 * X[l]);
  }
}

model{
  for(l in 1:L){
    if(Y[l]==0){
      target += log_sum_exp(log(1-theta),log(theta)+poisson_lpmf(0|lambda[l]));
    }else{
      target += log(theta) + poisson_lpmf(Y[l]|lambda[l]);
    }
  }
```

```
28  }
```

これは一般化線形モデルの応用ですから，ポアソン分布に限らず使える技であることは，すぐにお分かりいただけることと思います。

データの分布や特徴を考えて，それにあった分析モデルを作っていくのは，モデラーとしての階段を一歩登ったことになります。図 13.10 には統計分析家の成長ステップの模式図を描いてみました[*10]。最初はデータの記述や線形モデルの当てはめなど，得られたデータに振り回されるような形でモデルを作り上げてきましたが，今回のように分布を混ぜ合わせると，さらにその表現力が豊かになることが実感できたのではないでしょうか。図にあるように，確率モデルのパラメータが心理事象の (数学的) 記述から導出され，それがデータと合致する可動かで検証するようになれれば，さらにもう一段階研究のレベルが上がることになるでしょう。最終的には，心理学理論の方から新しい確率分布が導出される日が来るかもしれませんね。

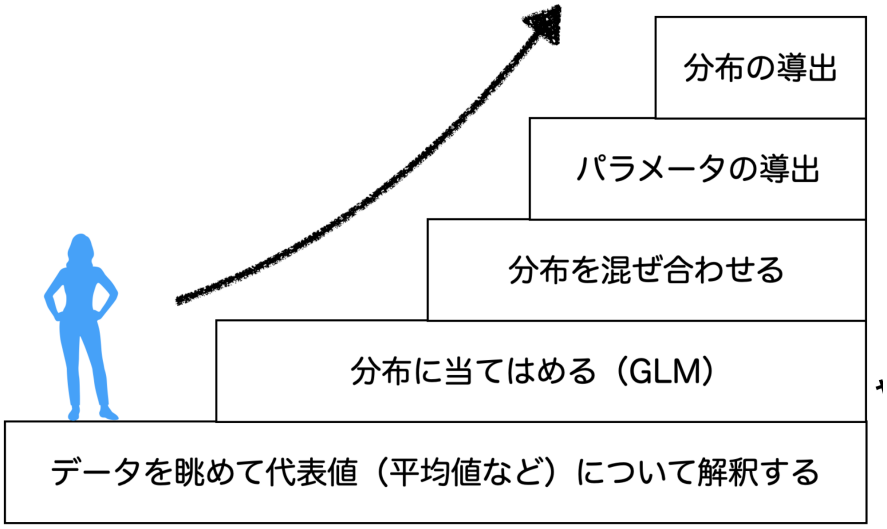


図 13.10　統計モデラーの道。浜田宏先生 (東北大学) のアイデアをもとにイラスト化したもの

13.4　課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの　分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名　`chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと　提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方　小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，

[*10] この図は東北大学大学院文学研究科の浜田宏先生が，専修大学社会知性開発研究センター主催の研究会 (2019.2.26 開催，於 専修大学サテライトキャンパス) にてご講演いただいた際の，資料をもとに作図したものです。

`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；年俸の分布を 2 つに分ける　本講では阪神 Tigers のデータで混合分布モデルを扱いました。今度は西武 Lions のデータで同じことをします。2020 年度の Lions の選手のうち，年俸が記録されている 51 名について，年俸の対数をとったものを使ってください。まずヒストグラムを描いて，2 つの山が見えることを確かめてから分析に進むとよいでしょう。

1. 2 つの正規分布の混合モデルを Stan で書いてください。本講の書き方にならい，ラベルスイッチングを避けるため 2 つの平均は `ordered` 型の `mu` として宣言し，成分ごとの標準偏差 `sigma`，所属確率 `theta` とあわせて推定すること。周辺化消去には `log_sum_exp` を使います。
2. このモデルでベイズ推定を実行し，2 つの成分それぞれの平均 `mu` と標準偏差 `sigma` について EAP 推定値を報告してください。あわせて `mu` については 90% 確信区間の下限と上限も示すこと。`ans2` の要素名は `mu1_eap`，`mu1_lower`，`mu1_upper`，`mu2_eap`，`mu2_lower`，`mu2_upper`，`sigma1_eap`，`sigma2_eap` とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. 各選手について，年俸の高い方の成分 (`mu[2]` の側) に属する確率を求めてください。そのうえで，その確率が 0.5 を超える選手が何人いるかと，年俸で並べてちょうど中央にあたる 26 番目の選手がその成分に属する確率を報告すること。`ans3` の要素名は `n_high`，`p_median_player` とすること。中央の選手の確率がどのような値になるかを見て，混合分布モデルが個人をどう扱っているのかを考えてみてください。

第 14 章

確率的プログラミング；項目反応理論

さてここまで，線形モデルを中心に**モデリング**のステップを一段一段登ってきました。この後は，これまでの技術や考え方を応用し，より実践的なプログラミングへと進んでいきます。さまざまなモデルの確率的表現や，そのコーディング技法を学ぶことで，ご自身の研究にも応用できる技術的ヒントが得られるかもしれません。

今回は項目反応理論を，Stan を使って実装する例を見ていきたいと思います。項目反応理論は，テストや質問紙の各項目に対する回答から，受検者の能力や特性を推定する統計的な理論です。詳しくは，データ解析応用 1 の方で論じておりますので，そちらのテキストをご覧ください。

14.1 ロジスティックモデルの復習

14.1.1 パラメータモデル側から

ここで簡単に前期の復習をしておきます。

目に見えないものを測定するという意味では，心理尺度や学力検査は同じ技術であるというところから，測定モデルの話を導入しました。とくに学力検査で想定されている測定対象，学力は，正規分布していると考えられること，累積的な能力の蓄積の程度を測定していると考えられることから，正答率が累積正規分布をつかった潜在能力 (学力) の関数と考えられるというところからモデル化がすすみました。累積正規分布を直接扱うモデルもありますが，一般にはそれによく近似する**ロジスティック関数**，すなわち

$$f(x) = \frac{1}{1 + exp(-1.7x)}$$

を使って近似することで，項目の特徴を描写することが考えられたのでした。

x 軸が潜在能力 θ を表していると考えれば，縦軸は能力に応じた**通過率**になると考えられます。この関数を左右に動かす位置パラメータ，b_j を加えたモデルが **1 パラメータ・ロジスティックモデル (one parameter logistic model)** であり，次の式 14.1 で表現されるのでした。

$$p_j(\theta) = \frac{1}{1 + exp(-1.7(\theta - b_j))} \tag{14.1}$$

ここで $p_j(\theta)$ は項目 j に対する θ の通過率であり，b_j は**困難度 (difficulty)** 母数と言われるものです。

さらに項目を特徴づけるために，傾きの大きさを表すパラメータ，a_j を加えたモデルが **2 パラメータ・ロジスティックモデル (two parameter logistic model)** です。モデル式は式 14.2 で表されます。

$$p_j(\theta) = \frac{1}{1 + exp(-1.7a_j(\theta - b_j))} \tag{14.2}$$

a_j はのことをとくに**識別力 (discriminant)** と呼ぶのでした。a_j の値によって傾きがどのように変わるかを，図 14.1 の二段目をみて確認しておきましょう。

最後に，3 つ目のパラメータである c_j，**当て推量母数 (guessing parameter)** を入れたモデルも紹介しておきます。モデルは式 14.3 のように表されます。

$$p_j(\theta) = c_j + \frac{1 - c_j}{1 + exp(-1.7a_j(\theta - b_j))} \tag{14.3}$$

ここには切片のように定数 c_j が入っていますから，関数のベースラインが上に上がるイメージです。これはつまり，学力 θ にかかわらず一定の正答率を有するということで，「適当に答えても当たる確率」ということから当て推量とよばれています。

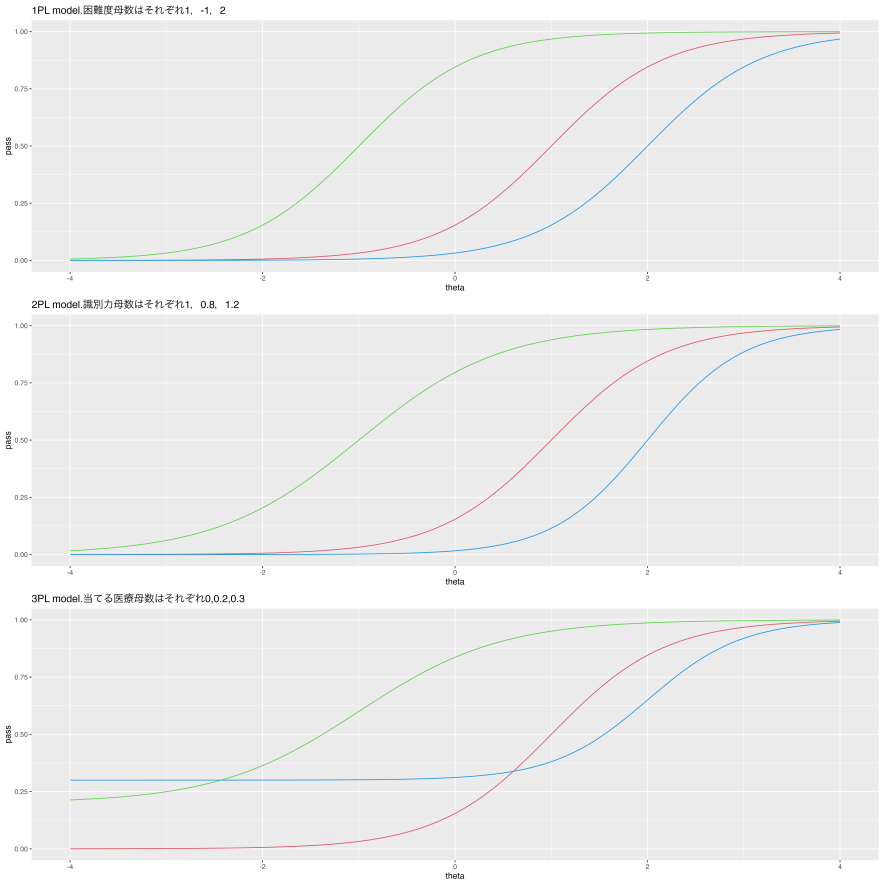


図 14.1　上から順に，1，2，3 パラメータロジスティックモデル。どこがどう変わっていくか確認しておこう。

14.1.2　確率モデル側から

さて今度は同じモデルをデータとの対応の観点から見てみましょう。テストのデータは 0/1 の二値であり，ベルヌーイ分布から出てきていると考えることができます。またロジスティックモデルはその言葉の通りロジスティック関数を使って，$-\infty$ から $+\infty$ まであり得る θ の値を，0 から 1 の範囲に入るように変換しているわけです。これは一般化線形モデルの文脈でいうところの，**ロジスティック回帰**分析であり，**リンク関数**がロジット関数，**逆リンク関数**がロジスティック関数になっているモデルだと考えることができます。

データ生成のモデル設計図として考えると，図 14.2 のようになります。パッケージを使っての分析の時は**最尤法**による推定結果でしたが，ここで被検者母数に標準正規分布を，項目母数に正規分布を事前分布としておいただけで，ベイズ推定モデルに置き換えることができました。

これまで利用してきた技術の応用で考えることができますから，すぐにでも実装できそうですね！

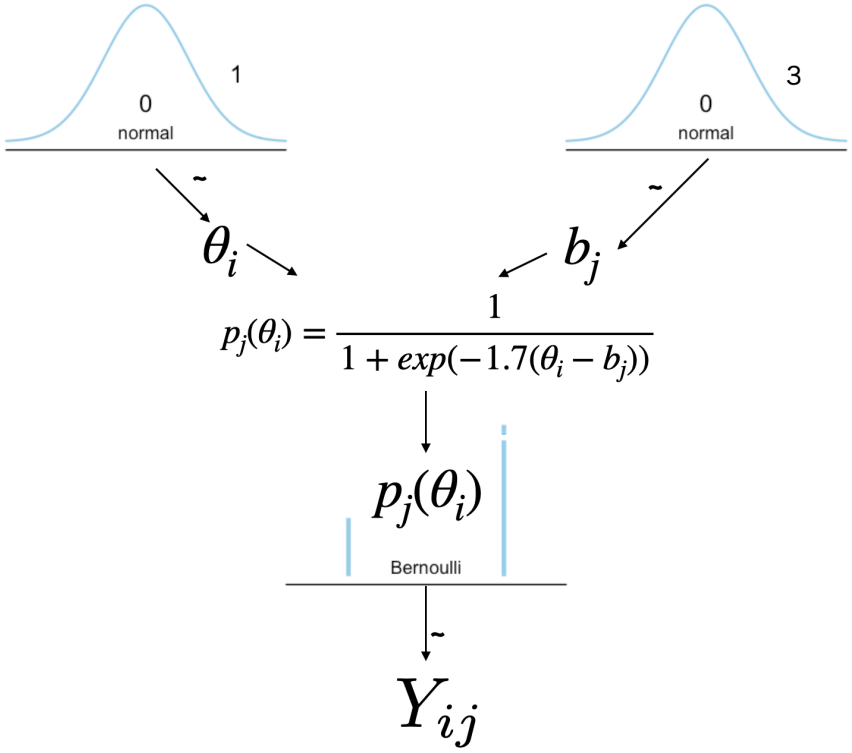


図 14.2　確率モデルとしての設計図。ロジスティック関数とベルヌーイ分布の組み合わせでテスト理論のモデルが表現できる。

14.2　ロジスティックモデルでの実装

それでは設計図をもとにモデルをコードにしていきましょう。

項目数が M，被検者数が N の $N \times M$ サイズの行列でデータが与えられていると考え，行列の各要素に対して通過率 p_{ij} を計算し，それをベルヌーイ分布に当てはめるという形で実装したのがコード 14.1 になります。

code : 14.1　1PL モデル

```
1  data{
2    int<lower=0> M;
3    int<lower=0> N;
4    array[N,M] int<lower=0,upper=1> resp;
5  }
6
7  parameters{
8    array[M] real<lower=-5,upper=5> b;
9    array[N] real theta;
```

```
}

transformed parameters{
  array[N,M] real<lower=0,upper=1> prob;
  for(n in 1:N){
    for(m in 1:M){
      prob[n,m] = inv_logit(1.7*(theta[n]-b[m]));
    }
  }
}

model{
  for(n in 1:N){
    for(m in 1:M){
      resp[n,m] ~ bernoulli(prob[n,m]);
    }
  }
  //prior
  b ~ normal(0,3);
  theta ~ normal(0,1);
}
```

今回はわかりやすくするために，transformed parameters ブロックを使いましたが，直接 model ブロックに書き込んでも構いませんし，bernoulli_logit 関数を使えばさらに高速に安定した推定結果が得られます。

transformed parameters ブロックにパラメータを追加するだけで，2PL,3PL モデルに拡張することも容易にできます。2PL モデルに拡張した例がコード 14.2，3PL モデルに拡張した例がコード 14.3 です。

code : 14.2 2PL モデル

```
...(前略)...
parameters{
  array[M] real<lower=0> a;
  array[M] real<lower=-5,upper=5> b;
  array[N] real theta;
}

transformed parameters{
  array[N,M] real<lower=0,upper=1> prob;
  for(n in 1:N){
    for(m in 1:M){
      prob[n,m] = inv_logit(1.7*a[m]*(theta[n]-b[m]));
    }
  }
}
...(後略)...
```

code : 14.3 3PL モデル

```
...(前略)...
parameters{
  array[M] real<lower=0> a;
```

```
  array[M] real<lower=-5,upper=5> b;
  array[M] real<lower=0,upper=1> c;
  array[N] real theta;
}

transformed parameters{
  array[N,M] real<lower=0,upper=1> prob;
  for(n in 1:N){
    for(m in 1:M){
      prob[n,m] = c[m] + (1-c[m])*inv_logit(1.7*a[m]*(theta[n]-b[m]));
    }
  }
}
...(後略)...
```

サンプルデータをつかって，このモデルで推定するためのコードがコード 14.4 です (cmdstanr での例)。

code : 14.4　IRT のコード

```
dat <- read_csv("IRTsample.csv")
model_1pl <- cmdstan_model("oneParameter.stan")
model_2pl <- cmdstan_model("twoParameters.stan")
model_3pl <- cmdstan_model("threeParameters.stan")

dataSet <- list(N = NROW(dat), M = NCOL(dat), resp = as.matrix(dat))
fit1 <- model_1pl$sample(data = dataSet, chains = 4, parallel_chains = 4)
fit2 <- model_2pl$sample(data = dataSet, chains = 4, parallel_chains = 4)
fit3 <- model_3pl$sample(data = dataSet, chains = 4, parallel_chains = 4)
```

分析結果を見てみましょう。**EAP 推定値**はそれぞれ表 14.1，図 14.3 のようになりました。

表 14.1　それぞれのパラメータ推定値

	1PL	2PL		3PL		
Qid	b_j	a_j	b_j	a_j	b_j	c_j
1	0.727	0.347	1.716	2.865	1.567	0.221
2	-1.605	0.724	-2.140	1.818	-0.761	0.557
3	-1.175	1.124	-1.229	2.900	-0.626	0.338
4	-1.077	0.955	-1.222	2.033	-0.535	0.362
5	0.577	0.691	0.771	2.451	0.946	0.159
6	1.271	0.694	1.742	1.100	1.659	0.050
7	0.564	0.692	0.753	1.027	0.941	0.099
8	1.590	0.360	3.650	1.581	3.355	0.084
9	0.514	0.484	0.893	0.836	1.205	0.142
10	-0.864	1.214	-0.887	2.547	-0.395	0.298

14.3　整然データでの分析

このように，IRT モデルを簡単に実装できました。

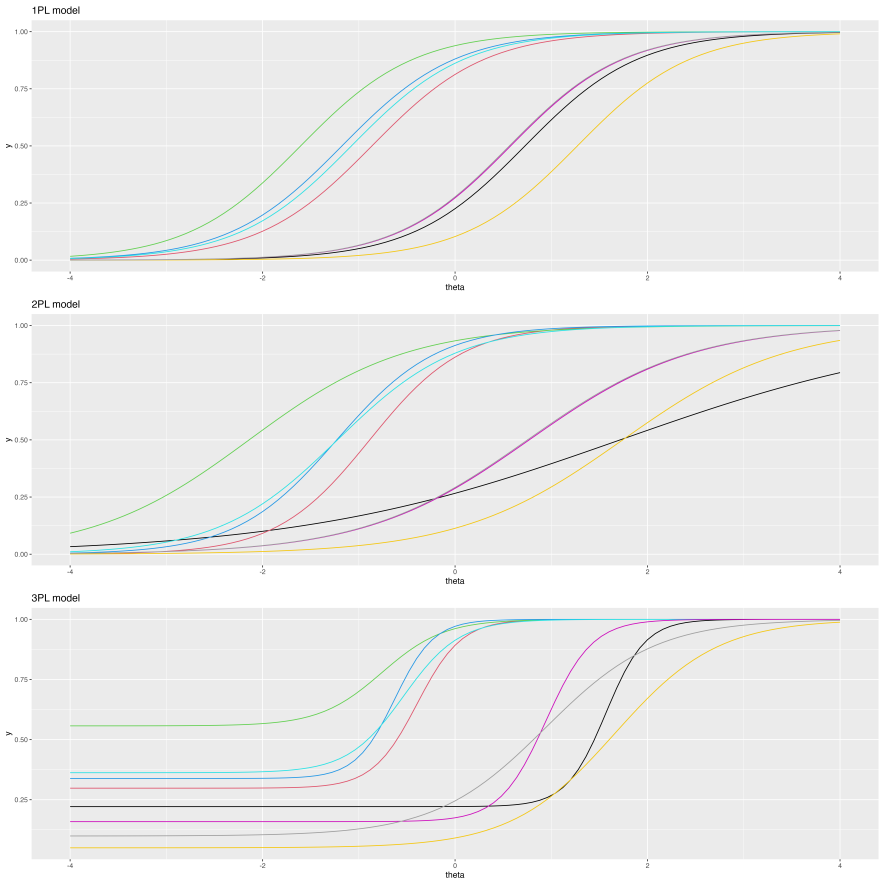


図 14.3 推定値を使って描いた各モデルの ICC

ところで, このモデルをもう少し使いやすくするために, データを**整然データ (tidy data)** にすることを考えてみましょう。というのも, 今はデータを行列形で渡していますが, これだと欠測値があった場合にうまく機能しないからです。Stan はデータに欠測値を取ることができず, 与えられるデータに `NA` が入っていることは許されません。

整然データは, 1 行に 1 つの数字が入っているようなデータで (セクション 8.3.1, Pp.101 参照), その行を見るだけですべての情報が手に入るようなデータです。行列型のデータは, 人の ID を行ラベルで, 変数の ID を列ラベルでみなければなりませんので, 参照方向が 1 つに定められていません。具体的には次のような形のデータになります (`R` **の出力** 14.1)。この形式ですと, 一行で誰の (`Pid`), どの問いに対する (`Qid`) 反応か (`value`) ということがわかりますね。

R の出力 14.1: 整然データになったテストデータ

```
# A tibble: 20 × 3
     Pid   Qid value
   <int> <dbl> <dbl>
 1     1     1     0
 2     1     2     0
 3     1     3     1
 4     1     4     0
 5     1     5     0
 6     1     6     0
 7     1     7     0
 8     1     8     0
 9     1     9     0
10     1    10     0
11     2     1     1
12     2     2     1
13     2     3     0
```

もし欠測値があればその行を削除してしまえば，完全データになりますから，Stan に欠測値を与えなくて済むことになります。

もちろんこれに対応する形で，Stan のコードを書き換える必要があります。こんどはデータも長くなりますから，何行目に誰のど問いに対する反応が入っているかを，識別変数を使いながら指定することになります。具体的には次のようなコード例になるでしょう (コード 14.5)。

code : 14.5 Tidy Data に対応した 2PL モデル

```
1  data{
2    int<lower=0> L;
3    int<lower=0> N;
4    int<lower=0> M;
5    array[L] int<lower=0,upper=N> Pid;
6    array[L] int<lower=0,upper=M> Qid;
7    array[L] int<lower=0,upper=1> resp;
8  }
9
10 parameters{
11   array[M] real<lower=0> a;
12   array[M] real<lower=-5,upper=5> b;
13   array[N] real theta;
14 }
15
16 transformed parameters{
17   array[N,M] real<lower=0,upper=1> prob;
18   for(n in 1:N){
19     for(m in 1:M){
20       prob[n,m] = inv_logit(1.7*a[m]*(theta[n]-b[m]));
21     }
22   }
23 }
24
```

```
25  model{
26    for(l in 1:L){
27      resp[l] ~ bernoulli(prob[Pid[l],Qid[l]]);
28    }
29    //prior
30    a ~ normal(0,3);
31    b ~ normal(0,3);
32    theta ~ normal(0,1);
33  }
```

■コード解説

data ブロック　データ長 L, 最大被検者数 N, 最大項目数 M, 被検者識別変数 Pid, 項目識別変数 Qid, L 行目の反応 resp としてデータを受け取ります。

parameters ブロック　これまでと同じです。

transformed parameters ブロック　ここも手を加える必要がありません。

model ブロック　L 行目の反応に対して, 識別変数で特定された確率をつかってモデルを当てはめます。

これの結果はこれまでのモデルと変わりませんが, 欠測値に対応できたはずですので, 元の完全データに技と欠測値を与えて改変し, 結果がどう変わるかを確認してみましょう。

code : 14.6　tidy データにして技と欠測値を与える

```
1  # 配布データは500名ぶんある。そのまま推定すると被験者母数が500個になり
2  # 時間がかかるので，ここでは先頭200名だけを使う
3  dat <- head(dat, 200)
4
5  dat.tmp <- dat %>%
6    rowid_to_column("Pid") %>%
7    pivot_longer(-Pid) %>%
8    mutate(Qid = str_extract(name, pattern = "\\d+") %>% as.numeric()) %>%
9    dplyr::select(Pid,Qid,value)
10
11 # わざと欠測値を与える
12 dat.tmp$value[1] <- NA
13 dat.tmp$value[11:13] <- NA
```

■コード解説

1 行目　dat に入っている行列型･完全データを変形していきます。

2 行目　一行ごとに被検者の反応が入っていますので, 行番号を被検者識別変数 Pid として作ります。

3 行目　データを縦長にします。その時のキーとなるのは, 先ほど作った Pid で, この変数は除いて縦長にする関数が pivot_longer です。

4 行目　少し技巧的ですが, ここまでの段階で変数は Pid,name,value という 3 つになっています。なかでも name 変数には元データの変数名が入っていますので, これを加工して問題番号を取り出します。str_extract 関数の str とは string, すなわち文字列を扱う関数であるという意味です。str_extract は文字列から条件に合ったものを抜き出す extract というもので, 条件を pattern で指定しています。ここで\\d+ とあるのは**正規表現 (regular expression)** というもので, 文字列を一

定の規則に従って特定の文字列を表現する方法です。ここでは数字だけを抜き出しています[*1]。また，このようにして取り出せた数字は文字列としての数字ですので，これを `as.numeric` 関数に送ることで，数値であることを教えています。

5 行目　被検者識別変数，項目識別変数，反応の値だけにデータを限定しています。

7 行目　ここでこのデータセットの 1 行目に欠測値 `NA` を上書きしています。わざと欠損させたのです。

8 行目　同じく 11 行目から 13 行目までも欠測値に上書きしてしまいました。

こうしてできたデータセットは次のようになります (**R の出力** 14.2)。

R の出力 14.2: 整然データになったテストデータ

```
# A tibble: 20 × 3
     Pid   Qid value
   <int> <dbl> <dbl>
 1     1     1    NA
 2     1     2     0
 3     1     3     1
 4     1     4     0
 5     1     5     0
 6     1     6     0
 7     1     7     0
 8     1     8     0
 9     1     9     0
10     1    10     0
11     2     1    NA
12     2     2    NA
13     2     3    NA
14     2     4     0
15     2     5     0
16     2     6     0
17     2     7     0
18     2     8     0
19     2     9     0
20     2    10     1
```

確かに数カ所，欠けているところができましたね。これをそのまま渡すことができませんから，欠損のあるところは削除してデータセットを作り，推定してみましょう。

code : 14.7　データセットを作って推定する

```
1  # 欠測値を消したデータセットにする
2  dat.tmp <- na.omit(dat.tmp)
3
4  dataSet <- list(
5    L = NROW(dat.tmp), N = max(dat.tmp$Pid), M = max(dat.tmp$Qid),
```

[*1] 少し丁寧にいうと，`\d` は正規表現で数字を意味する記号です。ただ，バックスラッシュ (`\`) がそのままでは正規表現の記号だと認識されず特殊文字だという宣言をしている (エスケープシーケンス，といいます) だけになってしまいます。そこで，「特殊文字のバックスラッシュだよ」を表すために重ねて`\\`としています。また，`+` は正規表現でいうところの「直前の文字が 1 回以上繰り返される」という意味です。今回は 10 という数字が出てくる可能性があり，`\\d` だけだと 1 しか抜き出せないので，数字が連なっていたら全体を取り出すようにしています。要するに，数字を全部取り出しましょう，が正規表現では`\\d+` になるわけです。

```
  Pid = dat.tmp$Pid, Qid = dat.tmp$Qid,
  resp = dat.tmp$value
)

model_2pl_ver2 <- cmdstan_model("twoParameters2.stan")
fit2.2 <- model_2pl_ver2$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4
)
```

欠損があっても，データを整形して欠損を取り除いた形で分析し，問題なく推定できたと思います。欠損が含まれているから，そのデータはすべて使い物にならないと考えるのではなく，データの存在するところ・利用できるところは利用しつくすという有効活用ができたと思います。

最後に，推定結果を確認しておきましょう。1 人目の被検者は 1 つ，2 人目の被検者は 3 つの欠損がありました。つまり他の被検者は 10 問分の情報を持っているのに，この 2 人はそれぞれ 9 問，7 問分しか情報が得られなかったことになります。そのことが結果の推定値にどう変わるのかというと，出力 14.1 の通りです。

MCMC の結果 14.1

```
# A tibble: 3 × 7
  name           EAP       MED       MAP        SD       L95       U95
  <chr>    <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 theta[1]    -1.269    -1.241    -1.159     0.488    -2.301    -0.384
2 theta[2]    -0.869    -0.874    -0.947     0.580    -2.008     0.275
3 theta[3]    -0.770    -0.786    -0.779     0.474    -1.663     0.218
```

これを見ると，10 問分の情報を持っている 3 人目の被検者の能力値が $\theta_3 = -0.770$ と推定されていますが，その SD が 0.474 です。これに比べて，9 問しか情報のない被検者 1 の SD は 0.488，7 問しか情報のない被検者 2 の SD は 0.580 と，情報が少なくなると SD が大きくなっていくことがわかります。推定値の SD が大きいということは，幅が広い，すなわちわからないことがより多くあることを意味します。得られる情報が少なければ，絞り込みが難しくなるというのがデータにも表れていることがわかりますね。

14.4 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；2 パラメータロジスティックモデル　本講で扱った項目反応理論のモデルを，整然データに対応した形で実装します。データと R コードは伴走サイトで提供しています。ただし配布データ `IRTsample.csv` は 500 名ぶんあり，そのまま推定すると母数が 500 個を超えて時間がかかります。この課題では**先頭 200 名**だけを使ってください (`dat <- head(dat, 200)` のようにします)。

1. 2 パラメータロジスティックモデルを Stan で書いてください。整然データ (1 行が「誰が・どの項目に・どう答えたか」) を受け取る形にし，母数として識別力 `a`，困難度 `b`，被験者母数 `theta` を宣言すること。
2. このモデルでベイズ推定を実行し，10 項目それぞれの識別力 `a` と困難度 `b` の EAP 推定値を報告してください。`ans2` の要素名は `a1_eap` から `a10_eap`，`b1_eap` から `b10_eap` の 20 個とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. 最も困難度が高い項目と最も低い項目の番号を答えてください。あわせて，MCMC サンプルの 1 つ 1 つについて 10 項目の困難度のうちどれが最大かを調べ，小問 2 で「最も困難度が高い」とした項目が実際に最大になった割合を求めること。`ans3` の要素名は `hardest`，`easiest`，`p_hardest` とすること。その割合を踏まえて，項目の難しさに順位をつけてよいかどうかを判断してください。

第 15 章

確率的プログラミング；変化点と折線回帰

さて今回は，今までの線形モデルとはちょっと違うモデリングになります。その名も変化点検出，そして折線回帰です。タイトルだけでも面白そうではありませんか？

さらに今回は次のようなデータを扱います (図 15.1)。何を隠そうこのデータは，私の体重の推移のデータなのです。私のモーニング・ルーティンとして，朝目が覚めるとトイレに行きます。そして体重計にのり，体重と体脂肪を測定します。その後顔を洗って[*1]，計測値を iPhone のアプリに書き込むというのがあります[*2]。このルーチンも早いもので 10 年近く続けていることになります (一時期やめていたこともありましたが，最古のレコードは 2012 年 06 月 08 日です)。ともかく，このデータを使って，いろいろ遊んでみようと思います。

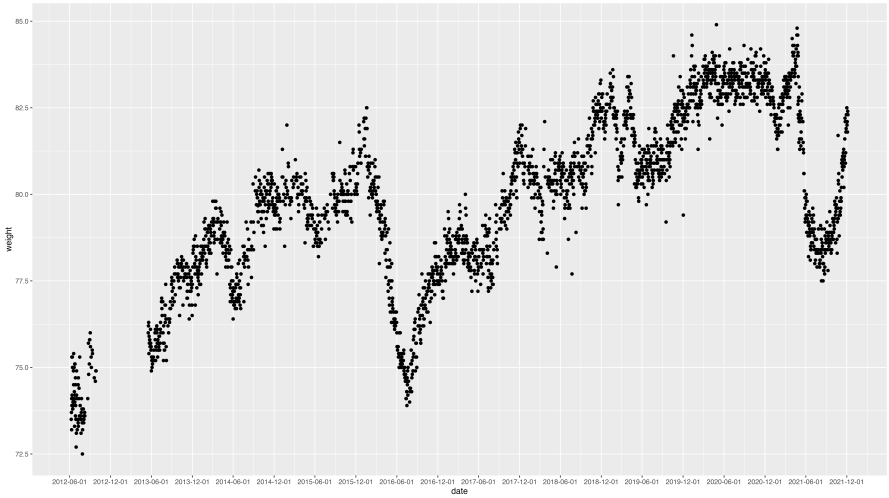


図 15.1　体重の時系列的推移

[*1] 体の内側の水分を出し，表皮に水分をつけないようにすることで，しっかりと肉の量を測ろうとしているのでこの順番になります。

[*2] ついでにいうと，書き込んだ結果はツイートします。フォロワーの中には私の体重の推移を楽しみにしている人がいるのです。おかしな世の中です。

15.1 混合分布モデルの応用

データが 10 年分というのはちょっと多いですから，少し時期を区切ることにします。2019 年から 2020 年の 2 年間に限定したデータが，図 15.2 になります。

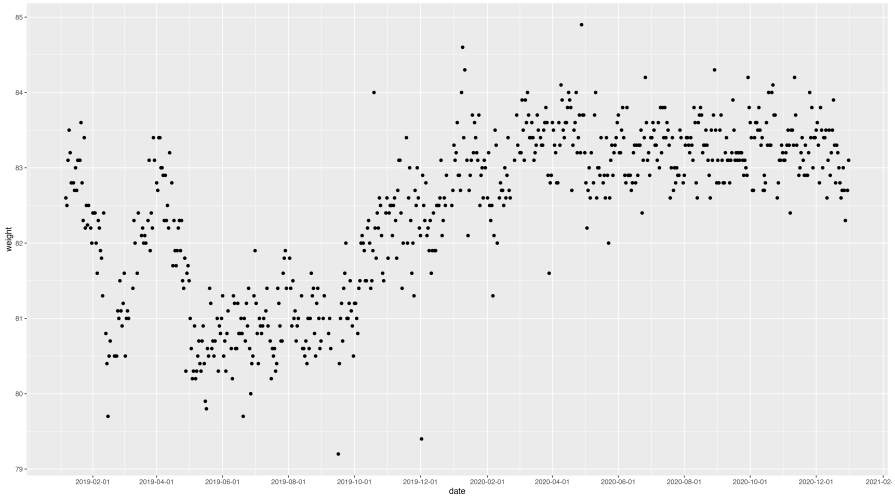


図 15.2 2019-2020 年のデータ

これをみると，うーん残念なことに，2020 年になると体重が増えてしまっているようですね。2019 年は 81kg 台をうろうろしていたようですが，2020 年になると 83kg 台になっているようです。毎回の体重の計測に偶然の測定誤差がついているとしても，19 年と 20 年とではそもそも体重が違ってしまっているようです。これは平均値の違う正規分布が混合している，混合分布モデルを考えてみることができそうですね。

第 13 講を参考に，混合分布モデルを考えてみましょう。筆者の体重には 2 つの状態があり，軽い方の状態なのか重い方の状態なのかは，確率 θ で変わると考えてみます。するとデータは，確率 θ で μ_1 を平均とする正規分布から，確率 $1-\theta$ で μ_2 を平均とする正規分布から得られるわけですから，確率モデルは次のようになります。

$$p(W_j) = \theta_j \times N(\mu_1, \sigma) + (1-\theta_j) \times N(\mu_2, \sigma)$$

ここで θ_j としたのは，観測時点 j ごとに θ が変わるという想定です。どちらのモードになるのかは一定の数字というより，毎回違っているように思えたのでそのようにしてみました。されてこを Stan で実装するにはどうすれば良いのでしょうか。そう，`log_sum_exp` 関数を使って，2 つの状態をベクトルで表記し，それを足し合わせる必要があるんでしたね。実際にコードにしてみたのがコード 15.1 です。

code : 15.1 2 つの体重モードモデル

```
1  data{
2    int L;
3    array[L] real W;
4  }
5
6  parameters{
7    array[L] real<lower=0,upper=1> theta;
```

```
  ordered[2] mu;
  real<lower=0> sigma;
}

model{
  for(l in 1:L){
    target += log_sum_exp(
      log(theta[l]) + normal_lpdf(W[l]|mu[1],sigma),
      log1m(theta[l]) + normal_lpdf(W[l]|mu[2],sigma)
    );
  }

  mu ~ normal(80,10);
  sigma ~ cauchy(0,5);
}
```

これを使って，体重のデータを分析してみましょう。コード 15.2 を実行し，図 15.3 のような結果を得ます。

code : 15.2 混合分布モデルのコード

```
dat1 <- dat %>%
  dplyr::filter(date > "2019/01/01") %>%
  dplyr::filter(date < "2021/01/01")

model <- cmdstanr::cmdstan_model("changePoint1.stan")
dataSet <- list(L = NROW(dat1), W = dat1$weight)
fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  seed = 12345)
```

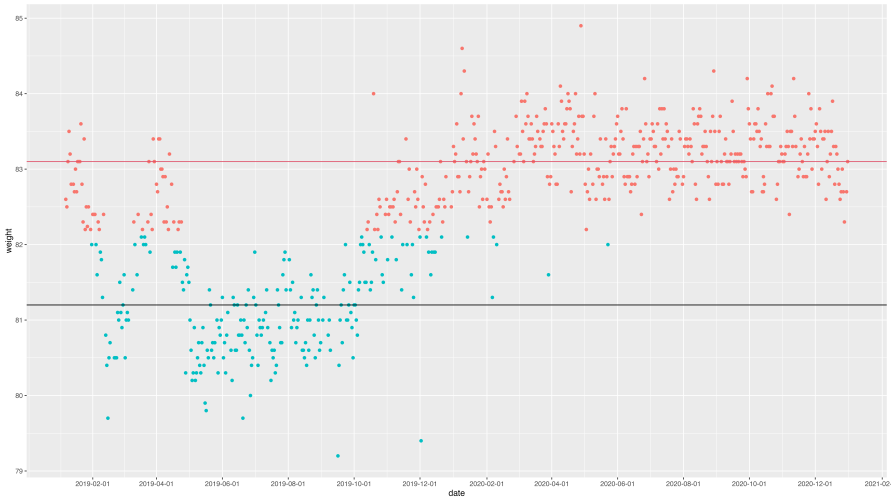


図 15.3 2 つの体重のモード

結果の図を見ると，どうやら 2019 年の初期にも 2 回ぐらい平均値が高い，「重いモード」が混在してお

り[*3]，2019 年の 10 月ごろからそちらの比率が増えています。もっとも，2020 年の 1 月にも「軽いモード」に入っている点はあるようですね。

とまあ，このような分析結果になったわけですが，これをみるとデータが 82kg ぐらいのラインを超えたかどうかで分割されているな，というのがわかります。当然，平均値の違う 2 つの分布を混ぜたわけですから，平均値の違いによって分かれるわけです。それにしても 2019 年の初期はどうしたんでしょうね。1 月や 4 月は重いモード，2-3 月と 5 月以降は軽いモードと，コロコロ入れ替わっています。ここでは 2 つのモードのどちらからデータが出てきているのか，ということだけを考えているのでこれでいいのですが，体重というのはそもそも時系列的な変化をするものですから，どこかで重くなった，どこかで軽くなった，というような時系列的なつながりについての情報が，うまくモデル化されていないように思えます。そこで，横軸が時間的な連続であるということに注意して，今度は 2021 年 1 月から 11 月までのデータを見てみましょう。その区間を取り出したのが図 15.4 になります。

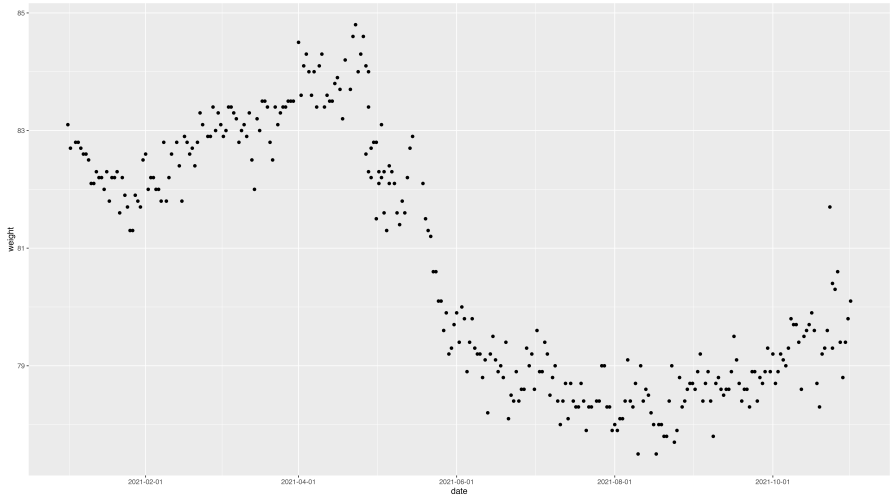


図 15.4　大きく変化したデータ

これは 2 つのモードがあるというより，途中で大きく変化したというべきではないでしょうか。とくに 2021 年 5 月ごろから 4kg ほど体重がぐぐぐっと下がることがあり，6 月以降は下がった体重のレベルに落ち着いているようです。このように，横軸が時系列だと考えると，5 月に何か変化があったのではないか，ということが推察されます。これをモデルで表現してみましょう。

15.2　変化点検出

何かのきっかけでデータの様相がガラリと変わってしまった，その変化したところを変化点と呼び，ここでの課題はその変化点を見つけ出す**変化点検出 (change point detection)** ということになります。たとえばセンサーが検出するデータの針が急に違うレベルに変化すると，測定している対象の状態がガラリと変わったのではないか，と考えることができますね。

他にもたとえば，心理学の応用領域として，科学捜査研究所が担当する**ポリグラフ検査**というのがあります[*4]が，これなども針が大きく触れたことで変化をみることになります。

[*3] 誰がデブモードやねん

[*4] 皮膚電気活動や心拍，呼吸など複数の整理指標を同時に測定するので**ポリ**グラフであり，熟練の検査官が反応パターンから被疑者の特別な反応を検出するものです。嘘発見器と呼ばれることがありますが，正確には**虚偽検出**と読んだほうが良いでしょう。

変化点検出はその名の通り「いつ」変化したのかを検出するものです。今回のデータも，だいたい 5 月ごろに大きな変化があったというのはわかるのですが，いつなのかを特定するのは難しいところです[*5]。これをデータとモデルから明らかにしようというのです。

ベイズ統計はわからないことを確率で表現し，データでその確率情報をアップデートしていくというものです。今回はいつ変化したのがわかりませんから，その変化した時期を τ とし，データ区間の中にその日があると考えて，その区間の一様分布を事前分布とします。その時期 τ を境に，データが出てくる分布の位置パラメータがずれると考えるのです。設計図は次のようになります (図 15.5)

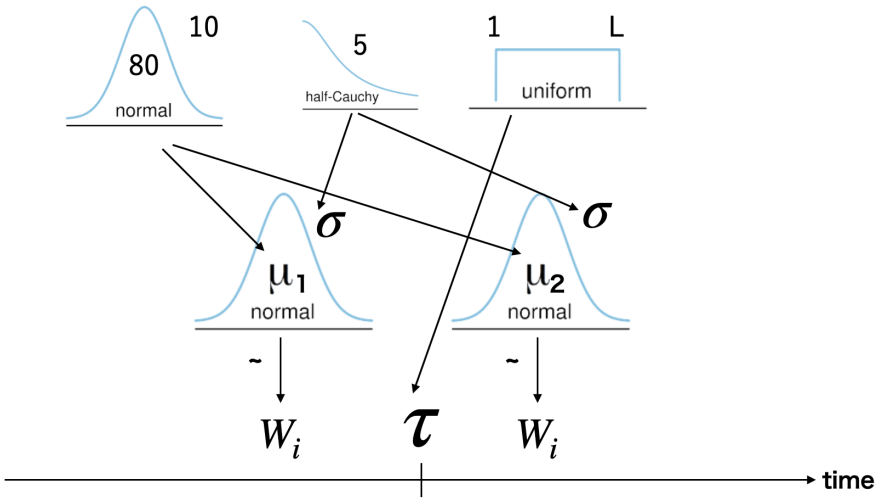


図 15.5 変化点検出のモデル設計図

設計図をもとに，コードにしてみたのがコード 15.3 になります。

code : 15.3 変化点検出のモデルコード

```
1  data{
2    int L;
3    array[L] real W;
4  }
5
6  parameters{
7    real<lower=1,upper=L> tau;
8    ordered[2] mu;
9    real<lower=0> sigma;
10 }
11
12 model{
13   for(l in 1:L){
14     if(l < tau){
15       W[l] ~ normal(mu[2],sigma);
```

最近では**隠匿情報検査 (concealed information test)** ということもあります。ちなみに科学捜査研究所，通称科捜研は，心理の他にも物理，化学，法医，文書 (筆跡鑑定) などの専門領域があり，各都道府県に 1 つずつ設置されています。

[*5] とはいえこのデータは筆者自身の体重変化ですので，何があったのかは実はわかっています。5 月 11 日，筆者が帰宅途中に自転車で転倒する事故を起こし，右肩の鎖骨を骨折しました。17 日に手術した後，利き手が使いにくいものですから食事の量が減り，結果的に体重 (筋肉?) が落ちたというのが真相です。

```
16      }else{
17        W[l] ~ normal(mu[1],sigma);
18      }
19    }
20
21    tau ~ uniform(1,L);
22    mu ~ normal(80,10);
23    sigma ~ cauchy(0,5);
24  }
```

日々のデータが順に 1 から L 行目まで並んでいるとします。パラメータ tau があり，1 行目のデータが変化点 tau より前にあるときは μ_2, σ の正規分布から，tau より後になれば，μ_1, σ の正規分布からデータが出てくると考えるのです。今回は後半の平均値が小さいことが明らかですから，μ のベクトルを ordered 型で宣言してあります。ベクトル要素の小さい順に並びますから，前半が μ_2，後半が μ_1 としています。

これを実行して，変化点がいつになるのかを推定してみましょう。推定結果は出力 15.1 のようになりました。

MCMC の結果 15.1

```
# A tibble: 4 × 7
  name        EAP       MED       MAP        SD       L95       U95
  <chr> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 mu[1]    82.770    82.769    82.762     0.062    82.648    82.893
2 mu[2]    78.891    78.892    78.896     0.060    78.775    79.008
3 sigma     0.750     0.749     0.747     0.031     0.693     0.812
4 tau     144.642   144.578   144.431     0.646   143.321   146.190
```

データの 145 行目がちょうどその変化点だといって，ほぼ間違いないようですね。145 行目はといいますと，データから 5 月 23 日だったことがわかります。

R の出力 15.1: いつでしょう

```
> dat2[145, ]
# A tibble: 1 × 3
  date                weight bodyFat
  <dttm>               <dbl>   <dbl>
1 2021-05-23 06:50:57   80.6      26
```

違いがわかるようにデータとモデルの推定値を合わせたのが，図 15.6 です。このようにして，ここで様相が変わったのだなということを，データから見出すことができました。

15.3 折線回帰

先ほどは，変化点を機に平均値が変わる，というモデルでした。では次のような分布の場合はどうなるでしょうか。今度は 2016 年に注目してみました (図 15.7)。この年はダイエットを志した時期で，最初の半年ぐらいでぐんぐん体重が落ちていってるのがわかります。そして残念なことに，8 月ごろでしょうか，ダイエットが

図 15.6　検出された変化点と平均値の違い

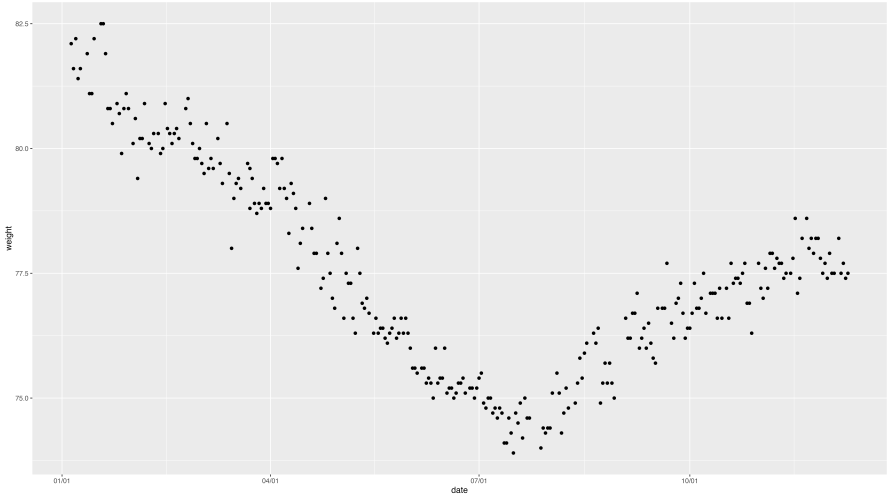


図 15.7　2016 年の変化

終わりリバウンドが始まったのがよくみて取れますね[*6]。

さてこのデータに対して，先ほどの変化点検出もできるのですが，変化点の前後で平均値が違う，というような簡単な話ではなさそうです。変化点の前はどんどん数字が減っていき，変化点のあとはどんどん数字が大きくなっていく，というのが実態です。回帰分析をしたら，変化点前は負の傾きが，変化点後は正の傾きが推定されそうな，そんなデータになっていますね。ということで，そのイメージができたのであれば，そのままモデルを描いてしまいましょう。設計図が少しごちゃっとしてしまいましたが，ポイントは傾きの係数を前半は負，後半は正に限定しているところでしょうか (図 15.8)。また，変化点は大体このあたり･･･ということでデータの 100 行目 (2016/04/29) から 250 行目 (2016/10/15) の間に置いてみました。

[*6] このときのダイエット法は，1. ラーメンのスープは飲まない，2. 夕食と晩酌は分ける (食べながら飲むのではなく，食べ終わって，お酒だけ飲む),3. お酒を飲むときにおつまみは食べない，という 3 か条を守るというものでした。お酒が飲みたいので食事の量を減らし，すぐに飲むことにしたので体重が減り，食事の前に飲んでしまえばいいんじゃないかという裏技を見つけてダイエットが崩壊したのを覚えています。

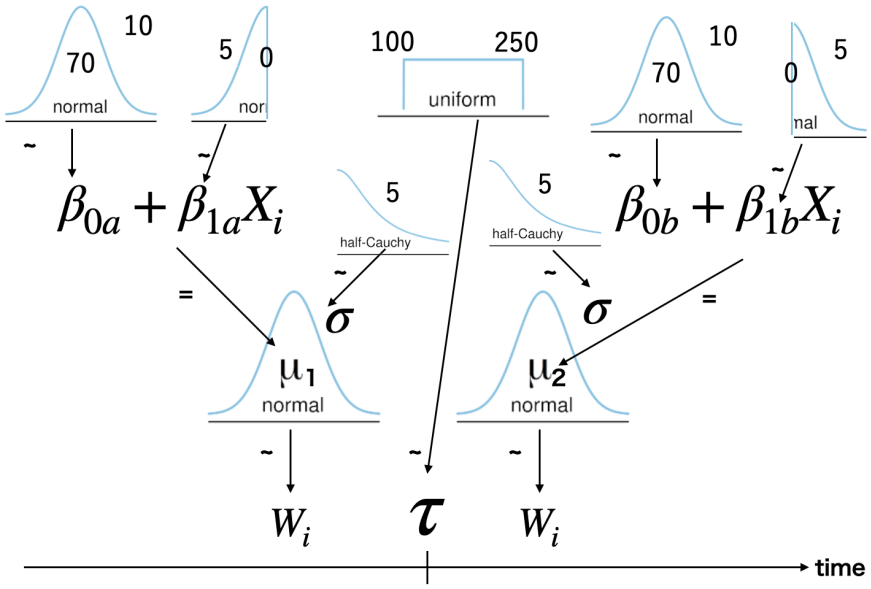


図 15.8　折線回帰モデル設計図

設計図をもとに, コードにしてみたのがコード 15.4 になります。

code : 15.4　折線回帰のモデルコード

```
1  data{
2    int L; // data length
3    array[L] real W;
4    array[L] real X;
5  }
6
7  parameters{
8    real<lower=100,upper=250> tau;
9    array[2] real beta0;
10   real<upper=0> beta1a;
11   real<lower=0> beta1b;
12   real<lower=0> sigma;
13 }
14
15
16 model{
17   for(l in 1:L){
18     if( l < tau ){
19       W[l] ~ normal( beta0[1] + (beta1a * X[l]),sigma);
20     }else{
21       W[l] ~ normal( beta0[2] + (beta1b * X[l]),sigma);
22     }
23   }
24
25   beta0 ~ normal(70,10);
26   beta1a ~ normal(0,5);
```

```
27    beta1b ~ normal(0,5);
28    sigma ~ cauchy(0,5);
29  }
```

回帰係数を正・負に限定するところは，パラメータの宣言で<lower=0>や<upper=0>としていることで表現しています。あとはデータが出てくる正規分布の平均に，線形モデルが入っているだけですね。これを実行すると，出力 15.2 のような結果が得られます。データの 187 行目 (2016/08/01) あたりが変化点ですかね。95% 区間で言うと 185 行目 (2016/07/30) から 188 行目 (2016/08/02) の間に変化点があると言えそうです。

MCMC の結果 15.2

```
# A tibble: 6 × 7
  name           EAP       MED       MAP        SD       L95       U95
  <chr>    <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!> <num:.3!>
1 beta0[1]    81.871    81.872    81.874     0.075    81.721    82.014
2 beta0[2]    70.548    70.551    70.549     0.380    69.793    71.274
3 beta1a      -0.043    -0.043    -0.043     0.001    -0.044    -0.041
4 beta1b       0.026     0.026     0.026     0.002     0.023     0.029
5 sigma        0.499     0.498     0.497     0.021     0.461     0.543
6 tau        187.135   187.326   187.460     0.832   185.057   188.215
```

そして回帰係数をプロットしてみました (図 15.9。なかなかデータにフィットしていそうです。

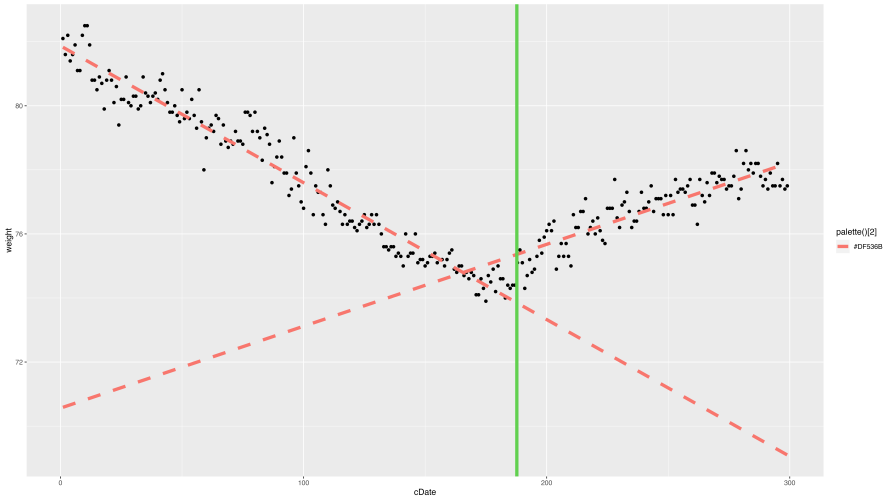


図 15.9 折線回帰モデル推定結果

このままでもよいのですが，変化点と折れたポイントが合致しないのはなんだか気持ち悪いですね。これを合わせることを考えてみたいと思います。変化点 τ のあるところで 2 つの回帰線は交わる，つまり同じ値になるはずですから，$\beta_{0a} + \beta_{1a}\tau = \beta_{0b} + \beta_{1b}\tau$ という式が成り立つはずです。ここから逆算して，

$$\beta_{0a} + \beta_{1a}\tau - \beta_{1b}\tau = \beta_{0b}$$

と考えることができますから，推定するパラメータを 1 つ減らすことができます。

code : 15.5 折線回帰のモデルコード 2

```
1  ...(前 略)...
```

```
 2  parameters{
 3    real<lower=100,upper=250> tau;
 4    real beta0a;
 5    real<upper=0> beta1a;
 6    real<lower=0> beta1b;
 7    real<lower=0> sigma;
 8  }
 9
10  transformed parameters{
11    real beta0b;
12    beta0b = beta0a + ((beta1a-beta1b) * tau);
13  }
14  ...(後略)...
```

このコード 15.4 にあるように，beta0b を計算式から出すようにして，改めて推定をした結果が図 15.10 になります。これだと変化点が前にずれて，データの 169 行目 (2016/07/10) で心がポッキリ折れていることがわかります。

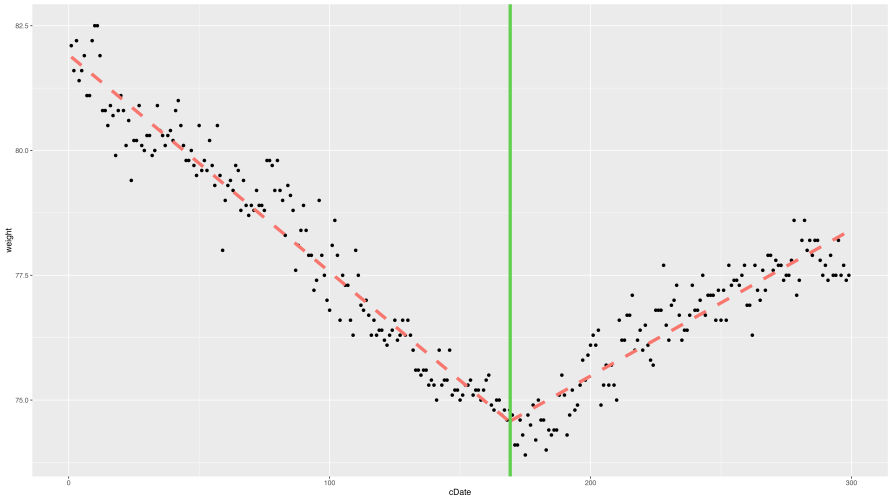


図 15.10　変化点を合わせた折線回帰モデル

このように，モデリングの力を借りるとわからなかった変化点がどこにあるのか，そしてその変化点の前後で予測モデルを変えるといったようなことが簡単に表現できます。ただの回帰分析であっても，データにあったモデルを考えることでさまざまな応用可能性が出てくるのではないでしょうか。

ところで，今回は時系列的なデータに対して回帰分析を行いました。この時の説明変数 X_i は日付，あるいは「何日目か」というデータだったわけです。しかし回帰分析には重要な仮定として，各データ点は (同じ分布から) 独立に得られていると言うものがありました。たとえば身長と体重の回帰分析，という話をするときに，各データ点にあたる個々人の身長・体重は，互いに影響し合わない独立だったはずであり，だからこそ尤度の計算の時に各データの尤度を掛け合わせていくことができたのでした。それに対し，体重のデータは明らかに違います。すなわち，今日の体重が明日の体重に影響している，それどころか今日の体重は明日の体重と大いに関わっているはずですし，明日明後日とその後の日々にも少なからず影響しているはずなのです。

このように考えると，独立していないデータ点，時系列的なデータに対しては異なる分析をする必要がありそうです。次回はこの時系列的なデータを扱うモデルを紹介していくことになります。

15.4 課題

今回の課題は大問 1 つ, 小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と, モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`, `chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは, こちらの環境でそのまま実行して確認します。提出する前に, 新しいセッションを開いて最初から最後まで通して実行し, エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は, 指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`, 小問 2 は `ans2`, 小問 3 は `ans3` です。複数の数値を答えるときは, `ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない オブジェクトに入れる数値は, **計算した値をそのまま**にしてください。`round` や `signif` で丸めると, 正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが, 提出するオブジェクトの中身は丸めないこと。

わからないところがあれば, 曜日別 TA か小杉までメールで連絡し, 指導を受けてください。

■大問；体重の変化点を見つける 筆者の体重データについて, 2016 年 3 月 1 日から 2016 年 9 月 30 日にかけて, 体重が減少から増加に転じたところがありそうです。この期間を対象に, 変化点を含む折れ線回帰を考えます。この期間には 189 件の記録があります。

1. 変化点の前後で傾きが変わる折れ線回帰のモデルを Stan で書いてください。本講の書き方にならい, 変化点の位置は `tau` とし, あわせて変化点前後の傾き `beta1a`, `beta1b` と切片 `beta0` を母数として宣言すること。ただし `tau` の範囲は, 本講のコードのように`<lower=100,upper=250>`と決め打ちにせず, `<lower=1,upper=L>`としてデータ数 `L` から決まるようにすること。この期間のデータは 189 件なので, 決め打ちのままだとデータの外側に変化点を置けてしまいます。
2. このモデルでベイズ推定を実行し, 変化点前後の傾き `beta1a`, `beta1b` の EAP 推定値を報告してください。あわせて, 変化点 `tau` の EAP 推定値が何番目の観測にあたるか, それが何月何日かも示すこと。`ans2` の要素名は `beta1a_eap`, `beta1b_eap`, `tau_eap` とすること。なお MCMC サンプルは, 4 チェインで 1 チェインあたり 5000 サンプル, ウォームアップを 1000 として得ること。
3. MCMC サンプルの 1 つ 1 つについて, 変化点 `tau` が何月にあたるかを調べ, 最も多くのサンプルが入った月を答えてください。あわせて `tau` の EAP 推定値と 90% 確信区間の下限･上限も報告すること。`ans3` の要素名は `which_month`, `tau_eap`, `tau_lower`, `tau_upper` とすること。区間の幅が日数にしてどれくらいあるかを見て, 変化点が何月に起きたといえるか, また日付まで特定できるかどうかを判断してください。

第 16 章

確率的プログラミング；状態空間モデル

前回は体重の変化データを例に，データの平均点が変わるモデル，変化点を検出したり変化点を境に傾きが変わる線形モデルをみてきました。

同じように，今回も時系列的なデータを扱います。前回の最後に案内したように，時系列データに普通の回帰直線をそのまま当てはめることは適切ではありません。どのような注意点が必要なのかについて，少し考えてみましょう。

16.1　時系列データの特徴

時系列的なデータはどのようなときに得られるでしょうか。前回導入した時のように，誰か 1 人が日々の生活を記録し続けている，それも立派な時系列データです。心理学には**日誌法**と呼ばれるデータ収集法があります。一言で行ってしまえば日記なのですが，日々多くの出来事や感情が到来するのを後から振り返って考えるのでは，正しく思い出せなかったり記憶が歪んでしまうこともあります。日々の状態を細かく記録し積み重ねることは，数字になっていなくても貴重なデータなのです。また最近は，**経験サンプリング**と呼ばれる調査法もあります。これは決まった間隔で質問紙がスマートフォンなどに送られてきて，定期的に心情を心理尺度に反映させていく，というものです。こうしたことができるようになったのには，私たちの周りに電子的なデバイスがたくさんあることが理由の 1 つです。さらに最近ではウェアラブル端末といって，身につける電子端末がありますね。Apple Watch など携帯電話と同じような機能を持ったこれらのデバイスは，GPS 機能がついていたり，万歩計による歩数の計測，血圧・脈拍のリアルタイムな計測が行われていたりします[*1]。これを蓄積したデータとすると，かなりの時系列的なデータが手に入ることになります。

あるいはまた，社会的なデータも時系列的に得られるものが多いです。一番わかりやすいのは株価の指標などでしょうか。日経平均株価など，時事刻々と変化する株価がグラフになって表されているのをみなさんもみたことがあると思います。また，Twitter などの SNS で，今どのようなキーワードが使われているかといったことが，リアルタイムに計測されます。これらの指標は個々人の特徴というより，社会全体のうねりのようなものを表現していることになりますが，これを研究することも立派な社会心理学的テーマになり得ます。時系列データは，人文社会科学領域ではこれまで経済学やマーケティングなどが専門的に扱ってきたところがありますが，これからは心理学の領域でもこうしたデータを分析することが流行してくるかもしれません[*2]。

さて時系列的なデータはどういった特徴があるかと言うと，端的に言えば**自己相関 (auto correlation)** がある，ということでしょう。すなわち，ある時点のデータはその前の時点と相関している，ということです。体重のデータの場合でもそうですが，明日の体重は突然ランダムな値になるのではなく，今日の体重に応じて変

[*1] こうしたデータは**ライフログ (life-log)** と呼ばれることもあります。

[*2] もちろん生理心理学や動物心理学では，古くからこうしたデータを扱ってきています。

化するはずだからです。また，データの変動に周期的なパターンが存在することもあります。脳波などのデータは基本的に波ですから，大きな波，小さな波がパターンを描いています。株価など社会的なデータも，季節による一定の変動など全体的な揺らぎがある上で，目的とする意味のある変化を見つけ出さなければならないという課題に取り組んでいます。たとえば周波数のデータの平均はゼロになりますから，基本的な記述統計では太刀打ちできないところがあります。回帰分析もその前提として，各データが独立に得られていると言うものがありますから，自己相関するデータは当然この仮定に違反していることになります。こうしたデータを分析するためには，周波数の大きさをスペクトル解析するとか，多次元の行列であるテンソルを使ってさまざまな要素を分解する，といった特殊な応用数学を駆使する必要があります。

今回はこうしたさまざまなモデルの中でも，**状態空間モデル (state space model)** を紹介します。

16.2 状態空間モデル

時系列的なデータは，自己相関すなわち前の時点の状態が，次の時点に影響しているという特徴があり，これをうまく表現するのが状態空間モデルです。状態空間モデルはこれまでの**モデリング**の技法を使うと簡単に実装できます。というか，これまでの時系列的なデータ解析は，自己相関を減らすために一時点前のデータとの差分をとってそれをモデルにする，時点ごとの変化をスムージングするなど，さまざまな工夫の上に成り立っていました。状態空間モデルはそれを非常にすっきり表現してくれているので，それまでの分析ノウハウがなくとも誰でも簡単に利用できるモデルだといえるかもしれません。

状態空間モデルは，まず観測されたデータと，その背後にある**状態**を区別します。体重変化のデータの例で考えてみましょう。計測された体重は，当然体の重さを反映したものですが，測定の時に多少の誤差が生じるでしょう。例に使っている筆者のデータでも，毎朝全裸で計測しているわけではありませんから，季節の変わり目で薄いパジャマからジャージにしたり，冬用の厚手のパジャマに変えたりすると，それだけで計測された数字には違いができます。本来知りたいのは，体の重さ，肉の重さそのものなはずですね。ここでいう体重そのもの，計測誤差のない肉の重さが，ここでいう状態のことになります。

そして t 時点の状態 μ_t は，次の時点の状態に影響します。服の重さなどを取り除いた t 時点の肉の状態に，食事や運動といった変化が加わって明日 $t+1$ 時点の肉の状態になるわけですから，$\mu_{t+1} = f(\mu_t)$ と考えることができるわけです。

もし翌日の体重が，今日の体重にちょっとした誤差がつくような変動しかしない，と言うのであれば $\mu_{t+1} \sim N(\mu_t, \tau)$ と表すことができるでしょう。あるいは，運動量 P_t の関数だというのであれば，$\mu_{t+1} \sim N(\mu_t + \beta_1 P_t, \tau)$ のように回帰分析の確率モデルのように表現すれば良いでしょう。もちろん説明変数が増えたり，一次関数ではないものを考えても構いません。

一方，計測値 W_t は，この μ_t に誤差 σ がついて得られる，すなわち $W_t \sim N(\mu_t, \sigma)$ という関係になります。これらの関係を表現したのが，図 16.1 になります。このようにして，状態が時間 $t, t+1, t+2, \cdots$ を通じてつながっており，状態に応じて観測値が得られると考えます。観測値を手に入れる際には偶然誤差 σ が生じますし，状態の変化は確率的に変わるのであれば幅 τ で，系統的に変わるのであればそれをモデリングしてやれば良いことになります。

この発想に基づいて，設計図を書いてみましょう。ここでは体重の変化に特別な傾向を考えず，ただ幅 τ で確率的にノイズが加わる**ホワイトノイズモデル (white-noise model)** で考えてみることにします。少しイメージしにくいかもしれませんが，たとえば次のように表現できます (図 16.2)。

それではこれをコードにしてみましょう。少し長くなりましたが，ホワイトノイズのモデルはコード 16.1 のようになります。

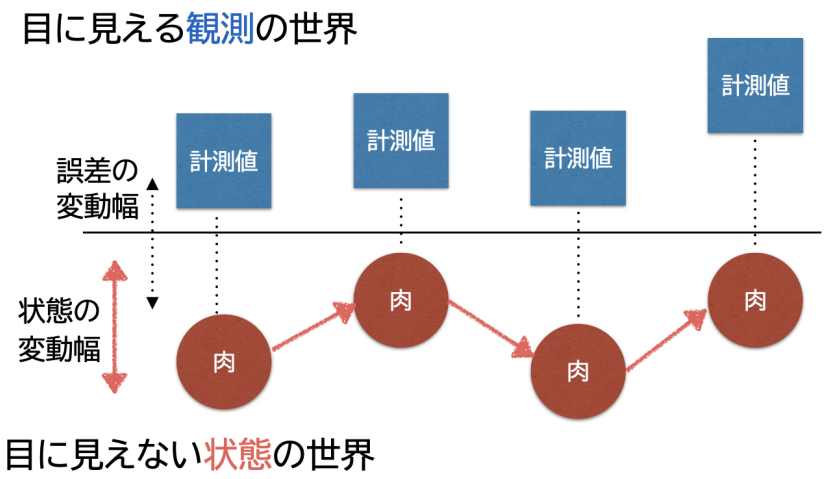


図 16.1 状態空間モデルのイメージ (体重変化の例)

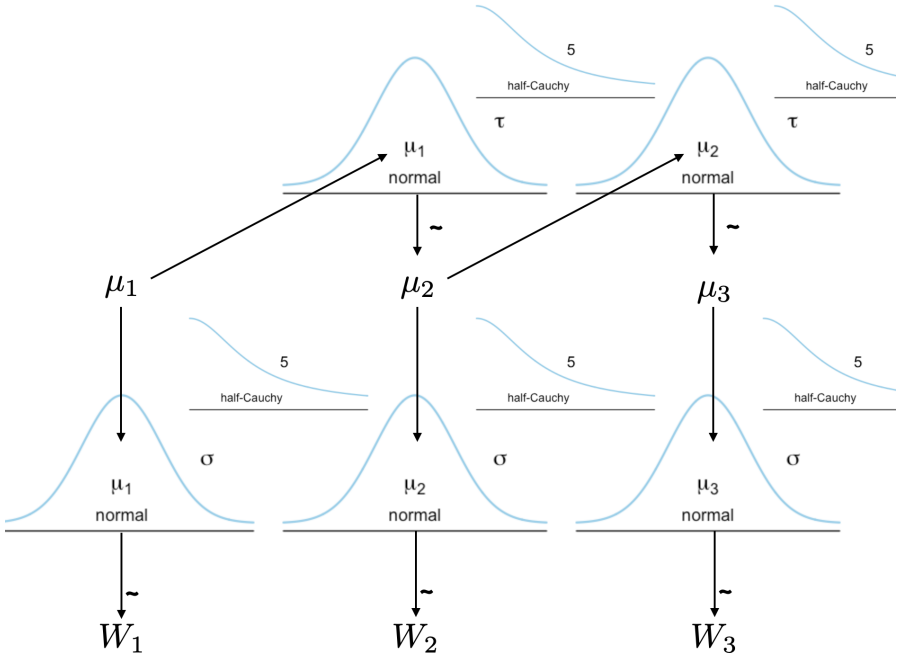


図 16.2 状態空間モデルの設計図

code : 16.1 ホワイトノイズモデル

```
1  data{
2   int L;
3   array[L] real W;
4  }
5
6  parameters{
7    real muZero;
8    array[L] real mu;
```

```
 9    real<lower=0> sig;
10    real<lower=0> tau;
11  }
12
13  model{
14    mu[1] ~ normal(muZero, tau);
15
16    for(l in 1:L){
17      W[l] ~ normal(mu[l],sig);
18    }
19
20    for(i in 2:L){
21      mu[i] ~ normal(mu[i-1],tau);
22    }
23
24    muZero ~ normal(80,10);
25    sig ~ cauchy(0,5);
26    tau ~ cauchy(0,5);
27  }
```

■コード解説

data ブロック　データ長 L, 各時点の体重 W がデータになります。

parameters ブロック　データ長と同じだけの状態を表す mu と, 状態の変動幅 tau, 状態に付加される計測誤差の変動幅 sig を宣言しています。それに加えて, 最初の状態 μ_0 をパラメータとしました。これは 2 時点目の状態 μ_2 は $\mu_2 \sim N(\mu_1, \tau)$ で, 3 時点目は $\mu_3 \sim N(\mu_2, \tau)$ で･･･と表現できるのに対し, 最初の点は $\mu_1 \sim N(\mu_0, \tau)$ となって, 計測される前の状態を考えなければならないからです。わからないものは確率で表現する, というベイズの流儀に則って, これはパラメータとして推定してやることにします。

model ブロック　まず 1 時点目の状態は, 0 時点目の状態から出てくるものですので, $\mu_1 \sim N(\mu_0, \tau)$ をモデル化しました。次に計測された値 W_l は状態 μ_l から出てくるものですから, 1 時点目からデータ長 L まで順に $W_l \sim N(\mu_l, \tau)$ として尤度を書きます。次に状態のモデルですが, 2 時点目以降は前の時点からの影響で表現できるので, $\mu_i \sim N(\mu_{i-1}, \tau)$ としています。最後に事前分布として, これまでの経験から $\mu_0 \sim N(80, 10)$ とし, 変化の幅はいずれも SD ですから半コーシー分布で表現しました。

これを使って, 体重のデータを分析してみましょう。データは 2020 年からのものを使います。図 16.3 をみると, 2021 年まではあまり変化がなく, 21 年初頭に少し体重が下がって, また上がって, その後 5 月から謎の下降[*3], そしてその反動[*4], とおそらく線形モデルでは表現できないような複雑な動きをしています。

さて, コード 16.2 を実行し, 図 16.4 のような結果を得ます。複雑な動きについても, モデルがしっかりと予測しているのがみて取れますね。状態空間のパワフルな表現力をお楽しみいただけましたでしょうか。

code : 16.2　状態空間モデルのコード

```
1  dat1 <- dat %>%
2    filter(date > "2020/01/01") %>%
```

[*3] 自転車事故による鎖骨骨折と, それに伴ってお箸持ちにくくて食事量が減る, 運動も減るといったのが原因でしょう。
[*4] 悔しいです。

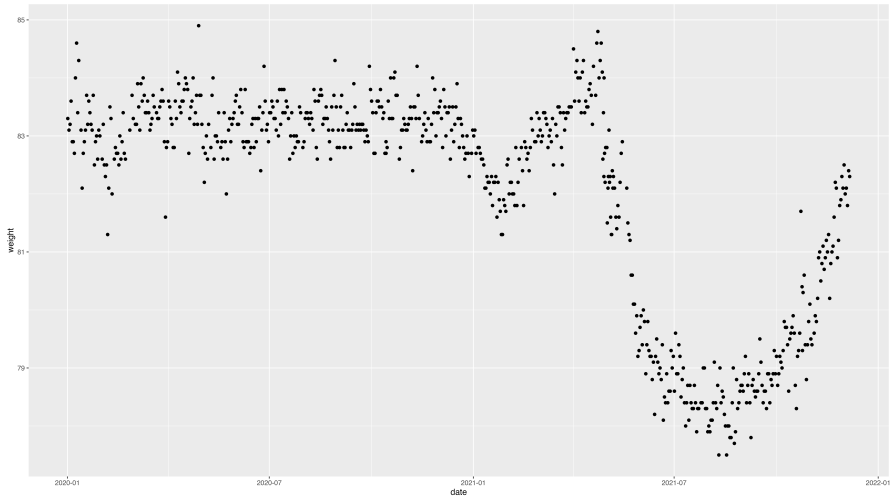


図 16.3 2020 年の体重変化

```
  mutate(date = as.Date(date))

model <- cmdstan_model("StateSpace.stan")
dataSet <- list(L = NROW(dat1), W = dat1$weight)
fit1 <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4
)

fit1.df <- fit1$draws() %>%
  posterior::as_draws_df() %>%
  as_tibble() %>%
  select(-.chain, -.iteration, -.draw) %>%
  rowid_to_column("iter") %>%
  pivot_longer(-iter, names_to = "Varname") %>%
  group_by(Varname) %>%
  summarise(
    EAP = mean(value),
    MED = median(value),
    MAP = map_estimation(value),
    SD = sd(value),
    L95 = quantile(value, probs = 0.025),
    L50 = quantile(value, probs = 0.25),
    U50 = quantile(value, probs = 0.75),
    U95 = quantile(value, probs = 0.975)
  )

Est1 <- fit1.df %>%
  dplyr::filter(str_detect(Varname, "mu")) %>%
  dplyr::mutate(ID = str_extract(Varname, pattern = "\\d+") %>%
  as.numeric()) %>%
  arrange(ID)
```

```
36
37  g <- dat1 %>%
38    rowid_to_column("ID") %>%
39    left_join(Est1, by = "ID") %>%
40    ggplot(aes(x = ID, y = weight, ymin = U95, ymax = L95)) +
41    geom_point() +
42    geom_point(aes(x = ID, y = MAP), color = palette()[2]) +
43    geom_ribbon(fill = palette()[3], alpha = 0.2)
44  plot(g)
```

■コード解説

1-3 行目　データファイルから該当する日程だけ抜き出します。日付を Date 形式に変換しています。

5-11 行目　cmdstanr でコンパイルしてサンプリングするところです。

14-28 行目　MCMC サンプルを集計し，**EAP**，**MED**，**MAP** 推定値，SD, 確信区間などを出しています。

30-34 行目　推定された状態 μ_i を取り出しています。

36-43 行目　もとのデータと推定値を合体させ, プロットしています。

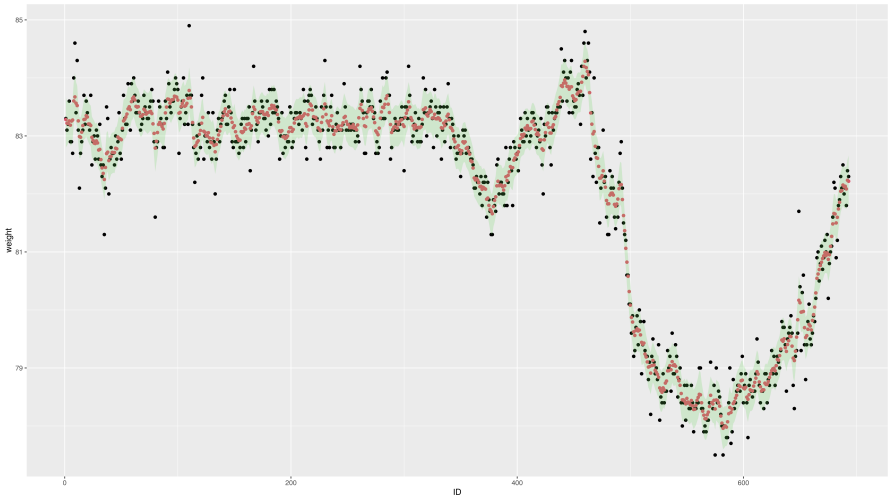


図 16.4　2020 年の体重変化をモデルで追跡してみた結果

16.3　欠測値の補間

ところで, データをよくみてみると, 実は毎日のデータになっていないところがあることに気づきます。いや, 気づかないかもしれないですね。次のコード 16.3 を実行してみてください。

code : 16.3　日付は連続かな?

```
1  dat1 %>%
2    mutate(lag = lag(date)) %>%
3    mutate(date = as.Date(date), lag = as.Date(lag)) %>%
4    mutate(FLG = date - lag) %>%
5    dplyr::filter(FLG > 1)
```

■コード解説

1 行目 先ほどの推定に使ったデータファイルです。

2 行目 lag 関数で，データを 1 行ずらした列を作ります。

3 行目 体重を記録した日の変数 date と，先ほど作った一行ずらした変数 lag はいずれも日付に関する変数ですので，日付型に変更します。

4 行目 日付変数の引き算をして，一行下のデータと何日ずれていたのか算出しています。

5 行目 1 日以上ずれている日をフィルタリングで抜き出しています。

これをみると，ちょくちょく抜けていて，時には 4 日も空いていたりすることがわかります[*5]。ともかく，これでは正確にデータを分析できているとは言えません。

R の出力 16.1: 抜けている日

```
# A tibble: 21 × 5
   date       weight bodyFat lag        FLG
   <date>      <dbl>   <dbl> <date>     <drtn>
 1 2020-01-13   83.1    26.8 2020-01-11 2 days
 2 2020-02-01   82.6    26.7 2020-01-30 2 days
 3 2020-02-12   82.6    26.7 2020-02-10 2 days
 4 2020-02-26   83.1    26.8 2020-02-22 4 days
 5 2020-02-28   83.7    27   2020-02-26 2 days
 6 2020-03-02   83.2    26.9 2020-02-29 2 days
 7 2020-03-21   83.4    26.9 2020-03-19 2 days
 8 2020-05-09   82.8    26.8 2020-05-07 2 days
 9 2020-05-18   82.8    26.7 2020-05-16 2 days
10 2020-07-31   82.8    27.7 2020-07-29 2 days
# … with 11 more rows
```

しかし計測はできていなくても，出張中にも状態の変化は続きいているはずで，次の計測時点はその日の状態の関数になっているはずです (図 16.5)。

この欠損した状態のデータをどのように考えれば良いでしょうか。実はこのことのヒントはすでに，最初のコードの中に入っています。状態空間モデルの 1 時点目，μ_1 は μ_0 から影響されているはずですが，データから μ_0 はわかりません。しかしわからないことは確率で表現するのがベイジアンのやり方です。今回も，欠損している測定値はわからないものとして確率で表現し，推定してやれば良いのです！このようにして間を埋めることをとくに**補間 (interpolating)** と言います。

補間をするためには，R コードの方でも Stan コードの方でもすこし工夫が必要です。まず R のほうで，データを加工するところから見てみましょう。Stan には欠測値 NA を与えることはできませんから，欠損しているところには特別なあり得ない数字，そうですね，999 という数字でも入れておきましょう。

code : 16.4 連続したデータを作り欠測値に借りの値を代入する

```
1  fullDays <-
2    data.frame(date = as.Date("2020/01/01"):as.Date("2021/12/01")) %>%
3    mutate(date = as.Date(date, origin = "1970-01-01")) %>%
```

[*5] 2020 年 2 月 22 日から 24 日は岡山県にベイズ塾合宿 (出張) でした。コロナがまだ豪華客船の中だけに抑え込まれている時期で，ギリギリ出張できた時期です。このあと大学の卒業式がなくなったり，前期オンライン授業になったりと言う，波乱の 2020 年度が始まったのでした。

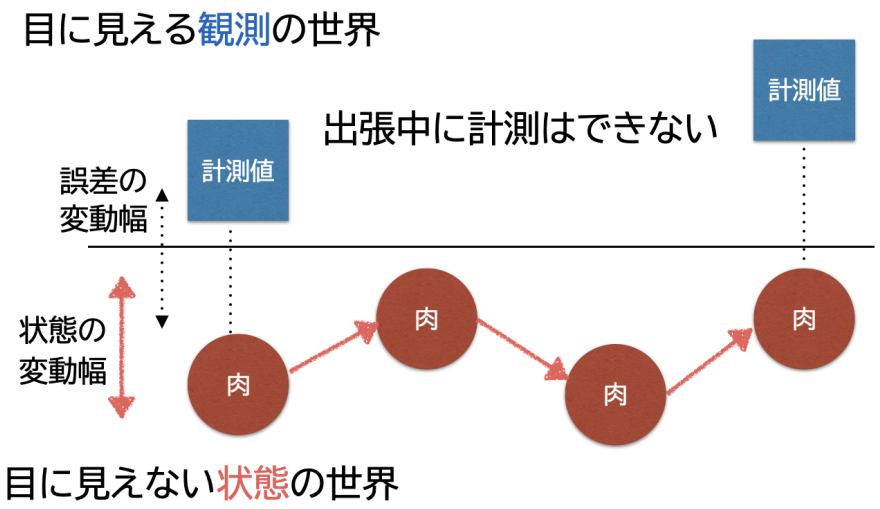


図 16.5　計測できない時点も変化は続く

```
4    left_join(dat1, by = "date") %>%
5    tidyr::replace_na(list(weight = 999, bodyFat = 999))
```

■コード解説

2 行目　2020 年 1 月 1 日から 2021 年 12 月 1 日までにデータを限定します。この期間，連続した日付を date 変数に追加します。

3 行目　日付変数を日付型に変更しています。

4 行目　もとのデータを日付変数に結合しています。left_join 関数は左にもとのデータセットを置き，右から dat1 を引っ付けていきます。結合する際は，変数 date をキーにし，キー変数が同じものは同じ行に合わせるという湯やり方です。このやり方だと，該当する行がない場合 (計測データセットの日付がない場合)，もとのデータセットは残して，体重や体脂肪変数は NA になります。

5 行目　replace_na 関数で，欠損のところに 999 という数字を入れています。

このようにすることで，次のようなデータセットができました (出力 16.2)。

R の出力 16.2: 抜けている日を無くした完全データセット

```
> fullDays
          date weight bodyFat
1   2020-01-01   83.3   26.70
2   2020-01-02   83.1   26.80
3   2020-01-03   83.2   26.90
4   2020-01-04   83.6   27.00
5   2020-01-05   82.9   28.70
6   2020-01-06   82.9   26.80
7   2020-01-07   82.7   26.70
8   2020-01-08   84.0   27.10
9   2020-01-09   84.6   27.30
10  2020-01-10   83.4   26.90
11  2020-01-11   84.3   27.20
12  2020-01-12  999.0  999.00
13  2020-01-13   83.1   26.80
14  2020-01-14   82.1   65.00
15  2020-01-15   82.7   26.70
```

これをみると，2020 年 1 月 12 日は欠損だったのですが，データ上は 999 という数字が入って完全データのように見えます[*6]。このデータを Stan に与えてやり，体重が 999 というあり得ない数字の場合は別の処理をする，という 0 過剰ポアソンの技術を応用します (セクション 13.3，Pp.173)。

code : 16.5　欠測値補間のコード

```
1  data{
2   int L;
3   array[L] real W;
4   int<lower=0> Nmiss;
5  }
6
7  parameters{
8    real muZero;
9    array[L] real mu;
10   array[Nmiss] real<lower=0> Miss_W;
11   real<lower=0> sig;
12   real<lower=0> tau;
13 }
14
15 model{
16   mu[1] ~ normal(muZero, tau);
17
18   {
19     int j = 0;
20     for(l in 1:L){
21       if( W[l] != 999){
22           // こっちは尤度
23           W[l] ~ normal(mu[l],sig);
```

[*6] ちなみに 2020/01/11 から 12 にかけては，犬会という研究会があったため関西に出張していました。宿に泊まったので，12 日の朝のルーティンができなかったのです。

```
24        }else{
25          j = j + 1;
26          // こっちはパラメータ
27          Miss_W[j] ~ normal(mu[l], sig);
28        }
29      }
30    }
31
32    for(i in 2:L){
33      mu[i] ~ normal(mu[i-1],tau);
34    }
35
36    muZero ~ normal(80,10);
37    sig ~ cauchy(0,5);
38    tau ~ cauchy(0,5);
39  }
```

data **ブロック**　データ長 L, 各時点の体重 W がデータです。加えて, 推定すべき欠測値の数もデータとして受け取ります。

parameters **ブロック**　データ長と同じだけの状態を表す mu と, 状態の変動幅 tau, 状態に付加される計測誤差の変動幅 sig を宣言しています。また最初の状態 μ_0 を表すパラメタと, 欠損の数だけある推定用パラメータ Miss_W を用意しました。

model **ブロック**　先ほど同様, 0 時点目の状態から出てくるものですので, $\mu_1 \sim N(\mu_0, \tau)$ をモデル化しています。次に各回のデータについて, W_l が 999 でなければ (同じでない, という論理式は!=で表現します), 普通の状態空間モデルのように尤度として計算します。そうではない, すなわちデータ $W_l = 999$ である, つまり欠測値であるはずということなら, $W_{miss} \sim N(\mu_l, \sigma)$ で推定してやります。ここで特殊な変数 j が出てきています。これは「何番目の欠測値か」を表す変数です。ブロックの中の一部だけで変数を宣言するため, まず for 文全体を中括弧で括り, はじめに $j = 0$ という数字を宣言しています。ここで欠測値推定のシーンになると, j のカウターを 1 つ増加させ, j 番目の欠測値の推定をさせる, というやり方をしています。欠測値の数と行番号が合致しないので, こうした工夫が必要なのですね。その後のコードは先ほどと同じです。

このようにして, 推定してみましょう。Stan に与えるデータセットは次のようにして作ります。

code : 16.6　Stan に与えるデータセット

```
1  dataSet <- list(L = NROW(fullDays), W = fullDays$weight,
2                  Nmiss = sum(fullDays$weight == 999))
```

欠測値の数は, オブジェクト fullDays の weight 変数が 999 かどうかを判定させ (==でイコールかどうかという論理判断になります), 条件が合致した数を数え上げるという方法をとっています。

コードが走り出すと, 推定は順調に進むと思います。が, 結果の出方が少しややこしいです。状態 μ_i はすべての日程について推定されますが, これに加えて欠損があったデータも推定されていきますので, 描画する際は「実測値なのか推定値なのか」,「補間した欠損の推定値は何番目の欠測値だったか」を判断しながら結合していき, データにする必要があるからです。Stan の中で作ったカウント変数 j を R の方でも考えてやらないといけないわけですね。

code : 16.7　プロット用にデータを整形する関数とプロットのコード

```
Est2 <- fit2.df %>%
  dplyr::filter(str_detect(Varname, "mu")) %>%
  dplyr::filter(!str_detect(Varname, "muZero")) %>%
  dplyr::mutate(ID = str_extract(Varname, pattern = "\\d+") %>%
  as.numeric()) %>%
  arrange(ID) %>%
  select(ID, MAP, U95, L95)

Est2miss <- fit2.df %>%
  dplyr::filter(str_detect(Varname, "Miss_W")) %>%
  dplyr::mutate(ID = str_extract(Varname, pattern = "\\d+") %>%
  as.numeric()) %>%
  arrange(ID) %>%
  select(ID, MAP, U95, L95)

### plot用の関数を準備
plotFunction <- function(fullDays, Est, MissEst) {
  tmp <- fullDays %>%
    rowid_to_column("ID") %>%
    left_join(Est, by = "ID") %>%
    rowwise() %>%
    mutate(FLG = if (weight != 999) {1} else {2})
  misJ <- 1
  tmp$weight2 <- NA
  tmp$weight2U <- NA
  tmp$weight2L <- NA
  for (i in 1:NROW(tmp)) {
    if (tmp$FLG[i] == 2) {
      tmp$weight2[i] <- MissEst$MAP[misJ]
      tmp$weight2U[i] <- MissEst$U95[misJ]
      tmp$weight2L[i] <- MissEst$L95[misJ]
      misJ <- misJ + 1
    } else {
      tmp$weight2[i] <- tmp$weight[i]
      tmp$weight2U[i] <- tmp$weight[i]
      tmp$weight2L[i] <- tmp$weight[i]
    }
  }
  return(tmp)
}

plot.tmp <- plotFunction(fullDays, Est2, Est2miss)
g <- ggplot(data = plot.tmp) +
  geom_point(aes(x = date, y = weight2)) +
  geom_errorbar(aes(x = date, y = weight2, ymin = weight2L,
                    ymax = weight2U, color = palette()[2])) +
  geom_point(aes(x = date, y = MAP, color = palette()[3])) +
  geom_errorbar(aes(x = date, y = MAP, ymin = L95,
                    ymax = U95, color = palette()[4])) +
  scale_x_date(date_breaks = "1␣month",
```

```
51                    limits = as.Date(c("2020-01-01", "2020-05-01"))) +
52     theme(legend.position = "none")
```

■プログラム解説

Est2 を作るブロック　MCMC サンプルをデータフレーム化したものの中から，状態 μ に関するデータだけを抜き出したものを作る。

Est2miss を作るブロック　欠測値を補間するための推定値を，MCMC サンプルから取り出した欠測値補間だけのデータセットを作る。

plot 用の関数　プロットはこの後にも行いますので，もう関数を作ってまとめてしまうことにします。もとのデータセット `fullDays` と，状態の推定値データセット `Est`，欠測値の推定値データセット `MissEst` を引数にとる関数です。

関数 3 行目　まずもとデータに行番号を `ID` として変数化します。

関数 4 行目　状態の推定値は，すべての行について推定しているはずですので，行番号をキーに結合 `left_join` します。

関数 5 行目　もとの観測値が実測値なのか欠測値なのかを表現する変数 `FLG` を用意しておきます。これらはすべて，一時的なオブジェクト `tmp` に納められます。

関数 6 行目　欠測値のインデックスを表す変数 `misJ` をつくり，カウントを 1 に設定しておきます。

関数 7-9 行目　先ほどの一次オブジェクトにいまから体重の推定値を入れていきますが，この推定値は実測値で得られている時は不要で `NA` になるはずですので，いったんすべてに `NA` を代入しています。

関数 10-21 行目　オブジェクト `tmp` を一行ずつ見ていって，もし `FLG` 変数が欠測値であることを教えてくれたら，`misJ` 番目の推定値を代入します。代入が終わったらカウンタを 1 つ追加しておきます。欠測値でなければ，もとの体重をそのまま移し入れます。ちなみに U や L がついているのは 95% 区間の上限と下限なのですが，実測値の場合は推定ではありませんのですべて同じ数字になります。

関数 22 行目　作ったオブジェクト `tmp` を戻り値として返します。

関数の結果を受けとる　`plot.tmp` は描画用の一時データオブジェクトです。ここに全期間のデータと，今回の状態推定値，欠損推定値を入れておきます。

描画する　x 軸は全日程です。まずは体重 W_j を `geom_point` と `geom_errorbar` でプロットします。欠損の場合は推定値が入っています。次のポイントとエラーバーの geom は，状態 μ のものになります。

さあどうでしょう。コードはともかく，図 16.5 を見てみましょう。実際に観測しているのは黒点，状態の推定値は緑点と赤いエラーバーで表示されています。青いバーは欠損した時の推定値で，値がないのですから当然触れ幅は大きいですが，おそらくこの体重が測定できなかった日はこれぐらいだったんじゃないか，という値が推定されているわけです。数字で見ると，たとえば 2020 年 1 月 12 日の MAP 推定値は 83.4kg，95% 区間でいうなら [82.6,84.2] です。前日が 84.3kg で翌日が 83.1kg ですから，そんなもんかなあという気もします。

ところで，測定していないはずの日も補間できるよ，ということを考えてみると，これはすごいことだと思いませんか。そうです，未来の値ももちろん「測定していない日」ですから，このモデルを使うと未来予測ができるのです！すごい！

図 16.6　欠損していても推定できる

早速同じコードで，日程を未来に飛ばしてみましょう。データは 2021 年 12 月 06 日までしかありません[*7]から，2021 年 12 月 31 日までデータを伸ばし，推定してみようではありませんか。コードは省略しますが，実行した結果は次のような図 16.7 として得られます。

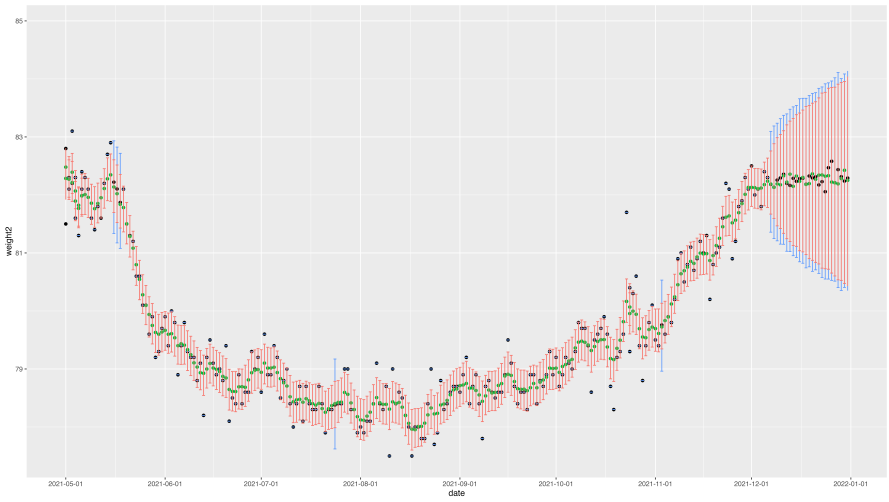


図 16.7　未来もある意味欠測値

これをみると，データが得られなくなった次の日から，確信区間が日を追うごとに広がっていく様がみて取れます。当然そうですね。μ_{t+1} は μ_t にプラスかマイナスか，いずれとも言えない振幅幅で揺れ動くのですから，先に行けば行くほどその可能性は不確かなものになっていくのです。ちなみに描画用のデータから 2021 年最後の数日間の予測値を見てみると，次のようになっています。

code : 16.8　予想される 2021 年末の体重 (一部略)

```
1  > plot.tmp %>% tail
2  # A tibble: 6 × 11
```

[*7] この原稿は 2021 年 12 月 10 日 17 時に執筆しています。

```
# Rowwise:
     ID date        weight bodyFat   MAP   U95   L95   FLG weight2
   <dbl> <date>     <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>
   738 2021-12-26    999      NA  82.2  83.8  80.5     2    82.1
   739 2021-12-27    999      NA  82.3  83.9  80.4     2    82.1
   740 2021-12-28    999      NA  82.2  83.9  80.4     2    82.2
   741 2021-12-29    999      NA  82.2  84.0  80.4     2    82.0
   742 2021-12-30    999      NA  82.3  84.0  80.3     2    82.3
   743 2021-12-31    999      NA  82.3  84.0  80.3     2    82.2d
```

未来の状態 μ は 82.2 のあたりでほぼ固定，それに伴って想定される体重 weight2 も 82.2 程度ですが，幅がどんどんと広くなっていってます。可能性として，80kg に落ちているかもしれませんし，84kg になっているかもしれない，というのが現時点での予測になります。

予測ができるとは言え，なんだかほとんど変わり映えしない数字で面白くないですね。それは当然，「明日の体重は今日の体重 + 誤差だろう」という，とくに何らメカニズムや仮定を入れなかったものですから，ほぼ横一線の推定にしかならないわけです。

では少し欲張って，構造を入れみましょうか。ここまでは，μ_t は μ_{t-1} にのみ影響される，というモデルになっていましたが，もう 1 日前の影響が入っていることを考えます。たとえば体重が増加傾向にあるとか，減少傾向にあるといった，変化を考えてみることにしましょう。すなわち，$\mu_t - \mu_{t-1} = \mu_{t-1} - \mu_{t-2} + \varepsilon$ です。これは昨日から今日への変化 ($\mu_t - \mu_{t-1}$) は，一昨日から昨日への変化 ($\mu_{t-1} - \mu_{t-2}$) と同じようなものだろう (誤差 ε はあるけど)，と考えていることになります。このようなモデルを **2 階差分のトレンド**と呼びます。

この式を変形すると，

$$\begin{aligned}\mu_t &= \mu_{t-1} + \mu_{t-1} - \mu_{t-2} + \varepsilon \\ &= 2\mu_{t-1} - \mu_{t-2} + \varepsilon\end{aligned}$$

となることはすぐにわかりますね。これを確率モデルで表現するなら，

$$\mu_t \sim N(2\mu_{t-1} - \mu_{t-2}, \tau)$$

と考えていることと同じですから，そのように Stan のコードも変形します (コード 16.9)。

code : 16.9　2 階差分トレンドのコード

```
...(前略)...
model{
  mu[1] ~ normal(muZero, tau);
  mu[2] ~ normal(mu[1] , tau);

  {
    int j = 0;
    for(l in 1:L){
      if( W[l] != 999){
          // こっちは尤度
          W[l] ~ normal(mu[l],sig);
      }else{
        j = j + 1;
        // こっちはパラメータ
        Miss_W[j] ~ normal(2*mu[l-1]-mu[l-2], sig);
      }
    }
```

```
18    }
19
20    for(i in 3:L){
21      //2階差分
22      mu[i] ~ normal(2*mu[i-1]-mu[i-2],tau);
23    }
24
25    muZero ~ normal(80,10);
26    sig ~ cauchy(0,5);
27    tau ~ cauchy(0,5);
28  }
```

コードは 2 階差分になっているので，状態は 3 時点目からしか推定できません。最初の 2 行で μ_1, μ_2 をメカニズムに沿って推定し，変化が取れるようになってから，は 2 階差分のコードで推定します。`for` 文が 3 から始まっていることに注意してください。

このコードと年末までのデータ，そしてプロット関数を使って推定した結果が次の図 16.8 になります。

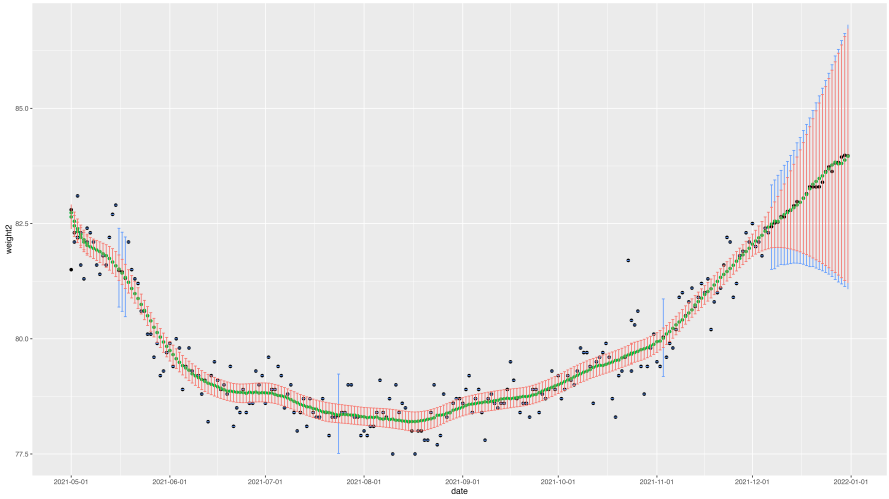


図 16.8　2 階差分トレンドの予測

これを見ると，それまでの増加傾向を反映して，年末には 84kg，最悪の場合 86.6kg まで増えている可能性があることがわかります[*8][*9]。

code : 16.10　2 階差分トレンドで予想される 2021 年末の体重 (一部略)

```
1  > plot.tmp %>% tail()
2  # A tibble: 6 × 11
3  # Rowwise:
4     ID date       weight bodyFat   MAP   U95   L95   FLG weight2
5  <dbl> <date>     <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>
6    738 2021-12-26    999      NA  83.9  85.6  81.5     2    83.6
7    739 2021-12-27    999      NA  83.9  85.8  81.5     2    83.8
8    740 2021-12-28    999      NA  84.0  85.9  81.4     2    83.8
```

[*8] なんてこった，まだ執筆中は答えが出ていませんが，この予測通りになったらとんでもないことですよ。だって私は 20 代 66kg だったんです。あの頃から 20kg も増えている自分の未来なんて想像したくないですよ。またダイエット始めるかなぁ。

[*9] 2021/01/15 追記。2021/12/31,6:07AM の記録は 82.3kg，体脂肪率 26.6% でした。ちゃんと確信区間の中に入ってますね！

```
 9    741 2021-12-29    999       NA  84.1  86.1  81.4     2    83.9
10    742 2021-12-30    999       NA  84.2  86.3  81.3     2    84.1
11    743 2021-12-31    999       NA  84.2  86.5  81.2     2    84.0
```

今回は 2 階差分のトレンドを入れての予測となりましたが，この他にも周期的に影響してくる季節項をいれるとか，たとえば「週末は体重が増加するだろう」と考えて週末効果の項を入れるとか，さまざまな工夫を思いつくことができるのではないでしょうか。

16.4 状態空間モデルの展開

最後になりましたが，状態空間モデルは表面に現れている値の背後に潜在変数を仮定し，それを滑らかに繋ぎ合わせた**スムージング (smoothing)** の技術であるとも考えることができます。時系列を通じて，その前後の「状態」を，柔らかなバネで結合し，データ全体を通じて変化する関数として相互に調整させあっている，と考えることもできるでしょう。

また，時間というのは過去から未来へ進む，1 次元の道のりです。ところで私たちは 2 次元の地図を見たり三次元の空間を見たりしていますよね。たとえばこの状態空間が，時系列的一次元ではなく，X 軸と Y 軸に広がる網のような二次元結合していることを考えると，空間的な統計分析が可能になります。たとえば地価というのは，駅からの距離や大型商業施設，コンビニエンスストアなどの利便性に応じて変化しますが，おそらくそれらは連綿と繋がっていて，徐々に変化していくものであるはずです。であれば徐々に変化する要素をつなぎ合わせて，面の状態空間モデルを作ってやることもできるわけです。さらにその地価の変動を考える，つまり二次元平面 × 時系列に潜在変数をつなげてスムージングしたモデルを作れば，時系列的な地価の変動を考えたり予測したりできるかもしれません。

心理学はこれまで，調査法で一時点を切り出してその構造を把握する，というアプローチが多くありました。時系列を扱うとしても，プレ・ポストの変化を見るとか，あまり長期にわたって追いかけるようなことはしてきませんでした。しかし冒頭でお話ししたように，これからの心理学データは時間的にも空間的にも広がるものを計測し，利用できるようになるかもしれません。そのためにはより深い人間の観察や推察によるモデルの作り込みと，状態空間モデルとベイズ推定のような数理モデルと強力な推定法のタッグが，必要になってくるでしょう。

16.5 課題

今回の課題は大問 1 つ，小問 3 つです。次の要領で提出してください。

提出するもの 分析を実行する R コード (`.R`) と，モデルを記述した Stan コード (`.stan`) の 2 つです。

ファイル名 `chNN_学籍番号_氏名.R`，`chNN_学籍番号_氏名.stan` としてください。NN は回の番号です。

動くこと 提出されたコードは，こちらの環境でそのまま実行して確認します。提出する前に，新しいセッションを開いて最初から最後まで通して実行し，エラーが出ないことを確かめてください。動かないものは未提出扱いになります。

答えの書き方 小問で求めた数値は，指定された名前のオブジェクトに入れてください。小問 1 の答えは `ans1`，小問 2 は `ans2`，小問 3 は `ans3` です。複数の数値を答えるときは，`ans2 <- c(mu_eap = 9.7, mu_map = 9.9)` のように名前つきベクトルにまとめること。要素の名前はそれぞれの課題文に書いてあります。

数値は丸めない　オブジェクトに入れる数値は，**計算した値をそのまま**にしてください。`round` や `signif` で丸めると，正しく計算できていても不正解になることがあります。画面に表示するときに丸めるのは構いませんが，提出するオブジェクトの中身は丸めないこと。

わからないところがあれば，曜日別 TA か小杉までメールで連絡し，指導を受けてください。

■大問；筋肉量の変化を追う　体重変化のデータには体脂肪の記録もありますので，体重 ×(1-体脂肪率) から筆者の筋肉量を計算できます。**2021 年 1 月 1 日から 4 月 10 日までの 100 日間**を対象に，筋肉量の変化を状態空間モデルで推定しましょう。この期間には実測が 96 日ぶんあり，残る 4 日は欠測です。本講の書き方にならい，日付を 1 日きざみで並べた 100 行の系列を作り，記録のない日は `999` で埋めてください。なお本講のコードは体重を扱っていたので `muZero` の事前分布が `normal(80,10)` になっていますが，今回は筋肉量 (60kg 前後) を扱いますので，中心をそれに合わせて変えること。

1. 2 階差分トレンドモデルを Stan で書いてください。本講の書き方にならい，状態 `mu`，状態の変化の大きさ `tau`，観測誤差 `sig` を母数として宣言すること。予測は本講と同じやり方で行います。すなわち，最終日の翌日から 30 日ぶんの行を系列の末尾に足し，その値をすべて `999`(欠測) としておくのです。未来の日も「測定していない日」ですから，欠測値補間の仕組みがそのまま予測になります。系列の長さは 100 日 + 30 日で 130 になります。
2. このモデルでベイズ推定を実行し，データの最終時点における状態 `mu` の EAP 推定値と 90% 確信区間の下限と上限を報告してください。あわせて `tau` と `sig` の EAP 推定値も示すこと。`ans2` の要素名は `mu_last_eap`，`mu_last_lower`，`mu_last_upper`，`tau_eap`，`sig_eap` とすること。なお MCMC サンプルは，4 チェインで 1 チェインあたり 5000 サンプル，ウォームアップを 1000 として得ること。
3. 予測値を使って，30 日後の筋肉量がデータ最終日の観測値を上回っている確率を求めてください。あわせて，30 日後の筋肉量の EAP 推定値も示すこと。`ans3` の要素名は `p_increase`，`pred30_eap` とすること。その確率から，筋肉量が増加傾向にあるといえるかどうかを判断してください。

第 17 章

モデル比較

さて，今回でこの授業も最終回となりました。今回は授業全体のまとめとして，これまでの流れを俯瞰的に捉え直すとともに，今後みなさんがこの講義を通じて学んだことが，どのように利用できるのかに言及していきたいと思います。

本書の前半，心理学データ解析応用 1 として前期にお話ししてきた内容は，心理学で利用されるデータ，とくに心理尺度を用いたものが，どういう根拠で「心」を測定していると言えるのかについて解説してきました。後半，心理学データ解析応用 2 の方は，プログラミングやベイズ統計といった目新しい研究手法の方が目についたかもしれませんが，その背後にあったメッセージは，どういうメカニズムでデータが生成されてきたのかという，**データ生成メカニズム**というアイデアをもつことでした。その意味で，本書全体のメッセージは一貫しています。すなわち，心理学者がデータを取るという時の，データに込められた意味や想定されるメカニズムをどのように表現し抽出し解釈するかという，心理学的営みそのものをお伝えしたかったのです。ここに無知・無自覚なまま，心理尺度で何かが測定できると考えたり，検定の結果から意味を解釈したりできるはずがないのですから。

そのための準備として，確率統計に関する理論や，コンピュータ，プログラミングに関する技能などの習得が必要だったわけです。心理学を学ぼうと思って入ったはずの大学で，どうして数学やプログラミングをしているのか，こんなはずじゃなかったと思った人もいるかもしれませんが，逆にどうして既知の知識や技術だけで，この摩訶不思議な心，人間，社会のありようが理解できると思っているのかと言いたいほどです。もちろん他のアプローチや，他にも学ぶべきことがたくさんありますし，時間は有限ですから苦手な手法は後回しにしたくなる気持ちはわかりますが，実は他の教養を身につけるよりもこちらの方が直接的で近道なルートだといえるかもしれません。

17.1　ベイジアンモデリング

Stan を使った統計的アプローチは，**ベイジアンモデリング (Bayesian modeling)** と呼ばれることがあります。Stan が事後分布からの乱数発生機であり，事前分布と尤度を記述するだけで事後分布を得ることができるというところが，ベイズの定理に根拠を持ったベイズ統計学ですから，「ベイジアン」という冠がつくのは当然です。しかし後半の「**モデリング**」については，なにもベイズ統計学に限った話ではありません。モデルを立てて考えるということだけを考えれば，広い意味では心理学だって「心」というモデルを立てて考えているのですから。もっというと，学問というのは一般的に抽象化 (あるいは単純化) した理想の世界での論理的構造を扱うものですから，すべてモデリングアプローチだと言えるかもしれません。

心理学はその研究方法として，調査，実験，観察，介入といったものがあります。これらを通じて心とはなにかということを考えていくわけです。とくに実験や介入においては，条件の制御や統制を行って，それに伴って

生じた結果の違いを**効果**と考えるのでした。そのことと**帰無仮説検定**は非常に相性が良かった事は，想像に難くないでしょう。帰無仮説検定は，出てきたデータと変数の関係について，機械的に判断を下すことができる方法論でした。帰無仮説検定が農学から生まれてきたことからもわかるように，その方法はどの研究分野に対しても適用できる，共通のルール・御作法なのです。心理学者は実験計画に心理学的要素を埋め込み，結果を統計的に判断することで，心理学的な結果の解釈を進めるてきました。いいかえれば，実験の計画を立てることそのものが心理学のエッセンスであったのです。

しかし皆さんはすでに，実験計画が統計的に見れば**線形モデル**にすぎないことを知っていますね。そしてごく単純な線形モデルによってしか，心理学のエッセンスを表現する方法がなかった時代にあっては，想定しうる心のメカニズムが平均値の差として出てくるように置き換えるほかなかったのです。方法論に自由がない分，その他のところで工夫するしかなかったとも言えます。そうした工夫の中には芸術的なまでに作り込まれているものがあったりしますから，そこに痺れる憧れる，という人も少なからずいるのは当然ともいえるでしょう。

線形計画の結果は，操作/介入の効果があったかなかったかという判断になりがちです。結果がそれしか示しておらず，その結果を生むための工夫はすでになされており，得られるデータの精度も平均値差を示すことができれば良い程度にしかない，ともいえるかもしれません。しかし測定のツールは時事刻々と進歩してきましたので，より精緻に違いを見て，よりリッチな解釈を許す意味合い豊かなデータも得られるようになってきました。そのような時，伝統的な方法論だけで立ち向かうのはもったいないのではないでしょうか。方法論の呪縛から自由になり，さまざまな表現ができるようになれば，心理学ももっと発展させることができるでしょう。

モデリングは，変数間関係を数式で表現することでもあります。「心」や「気持ち」といった表現を始め，「幸せ」とか「恋愛感情」といった日常用語で考えるべきところを，$X_i, Y_{ij}, \alpha, \beta ...$ といった記号で置き換えて表現するのには慣れが必要です。ひょっとしたら抵抗を感じる人もいるかもしれません。しかし，数式で表現する事は，他の解釈を許さない一意的表現をすることでもあります。人間って色々だよね，当てはまることもあれば当てはまらないこともあるよね，という日常的な理解で留まりたいのであれば – そしてそこで止まっていられるのは幸せなことでもあるのですが – それでいいのですが，学問を進める以上はより明確に，より誤解のない表現をしなければなりません。個別のケースにしか当てはまらないこと，その程度を確率で表現して，理論的に全体的な傾向を「こうなっているに違いない」と厳密に表現するには，日常用語ではなく数式用語が必要なのです。モデリングを進める上で，そうした表現ができるようにトレーニングする必要はありますが，使えるようになると折線回帰や状態空間モデルのように，さまざまな表現ができることも見てきた通りです。

この授業を受けてきたからと言って，さあ今すぐ自分で心理学的モデルを作れと言われても難しい，と思うかもしれません。実際これまで私が教えてきた中で，よく質問されるのは「やり方はわかるけれども，自分でやれる気がしない。どうやってモデルを思いついたらいいのか」ということです。これに対する答え方は 2 つあって，1 つは「色々なパターンを見て模倣する」です。「学ぶ」は「真似ぶ」からきているとも言われたりしますが，どのような表現の可能性があるのかについては，実際の応用パターンをさまざま見るのが早いでしょう。馴染みのない料理や調理法があれば，レシピ本を買って応用パターンをいろいろ仕入れますよね。そうすることで，「こんな工夫ができるのか。じゃあ自分はこうしてみよう，ここをちょっと変えてみよう」と前に進むことができるのですから。こうしたモデルがたくさん載っているものとして，リー・ワゲンメーカーズ (2017)や豊田 (2017, 2018, 2019)などがありますので，パラパラっとみながら応用例を広く眺めてみるといいでしょう。

もう 1 つの答えは，「紙とペンをつかって設計図を書く」というものです。この方法は，この講義の中で再三お伝えしてきたやり方になります。データを見た時に，どういうメカニズムでデータが得られたかを考えるところにこそ，心理学者の本当の興味関心があるはずです。このデータは心のこういう仕組みを反映しているのではないか。データの背後には変数同士のこういう関係が潜んでいるのではないか，ということを考えていくわけです。わからないところは確率で表現しつつ，変数間関係を関数で記述する。そのための設計図です。

設計図と Stan があれば答え (事後分布) は得られるのですから，データ生成の仕組みを作り込むことに我々は全力を傾けることができます。複雑な関数関係をいきなり思いつくのが難しい，ということもあるでしょう。その場合はデータを可視化し，データに合うような関数はどんな形なのかを考える，あるいはいったん，単純な線形モデルを当てはめて，それを徐々に複雑にしていく，というやり方が良いでしょう。クルシュケ（2017）は心理学者が書いたベイズ統計の本ですが，その後半は線形モデルです。一般化線形モデルや階層モデルについて，とくに複雑な前置きなく「こういう仕組みになってるんだから，とりあえずそれを反映して，関係は線形でも大体うまくいくでしょ」という感じで話が進んでいきます。簡単なモデルから徐々に複雑にしていく，というのがモデリングの王道でもあります。完成品だけをみるととても複雑に思えますが，設計図片手に紐解いていけば，意外とわかりやすいかもしれません。

モデリングは自由に設計図を書くことができるので，**分散分析**や **t 検定**など平均値差だけにこだわっている方法をみると，そちらがいかにも窮屈そうにしていると思えるかもしれません。講義を通じて得た知識や技術で，自由にデータを調理していただければと思います。

17.2 帰無仮説検定の代案

ベイジアンモデリングは，自由に書いた設計図から**確率的プログラミング言語 (probabilistic programming language)** をつかって答えを得る方法です。プログラミング言語が使えるようになるの大変です。これまでの一般的な統計ソフトでは，ボタン 1 つで答えを出してくれたのに，と思う人もいるかもしれません。しかしそれは，従来の分析方法がさまざまな仮定や前提に基づいた，マニュアル的アプローチを許す範囲のものに限定されていたからですし，「簡単，楽ちん」ということが「知らなくていい」ということだとすると，その考え方がさまざまな問題を引き起こしてきたことを知らなければなりません。これは**心理学における再現性問題**池田・平石（2016）として取り沙汰されており，心理学そのものに疑問符を投げかけるような事態を招いています。その原因の 1 つが，統計的技術の誤用・悪用にありました。

心理学ではながらく，帰無仮説検定によって理論を積み重ねてきました。帰無仮説検定では p 値が 5% より小さければ良し，という「ここだけ見ておけば良い」というようなマニュアル的基準があり，差があればすごい効果が発見されたぞ！と喧伝するということで進んできた側面があります。すでに述べたように，この**有意差**を出すために要因計画の方にこそ心理学者は注力してきたのですが，$p < 0.05$ になりさえすれば良いという表面的な理解しかしていないマニュアル人間ができあがってしまうことが，問題を大きくしたのかもしれません。

ベイズ統計によるアプローチの場合，データがどのように出てきたのか，事前分布はどうなのか，といったことを考えないわけにはいきません。自由に設計図を描くことができるというのは，裏を返せば，最初は白紙でしかないということであり，きちんと動く統計ツールを作るためには細部まですべて記述しなければならないのです。帰無仮説検定では，極端な場合「データ生成には正規分布が仮定されている」ということを知らなくても，p 値がどうなるかだけをみる事はできてしまうのに，です。このように，ベイズ統計を使う利点の 1 つは，前提や仮定についてすべて自覚的に，明示的に記述しなければならないというところにあり，これが悪用・誤用を生みにくくする安全装置となっている面があります。

最近では JASPJASP Team (2021) という，GUI でベイズ分析もできるソフトウェアが出てきました。帰無仮説検定と同じような分析を，ベイズでやったらどうなるか，すぐに試すことができます。この便利なアプリでは Stan や JAGS などとは違って，どのような前提があるかを自覚しなくても，分析結果を得る事はできます。ですから，今挙げた「安全装置」としてのベイズ統計の利点は，簡単に外れてしまう日が来るかもしれませんし，その方がユーザにとってはいいかもしれません。この講義では第 6 講から第 9 講まで，帰無仮説検定の代わりにモデリングアプローチをする例を紹介してきました。さて，ではこうした帰無仮説検定の代わりとし

て，ベイズ統計を使う利点はどこにあるでしょうか。

もちろんいくつか考えられますので，箇条書き的に解説してみたいと思います。

■幾重にも組み合わさる仮定や補正からは無縁 たとえば二群の平均値の差を検定する場合，データが正規分布に従っているかどうかを検定し，また二群の分散が等しいかどうかも検定して，問題なければ t 検定に進みます。もし分散が等しくないということになれば，Welch の補正をしなければならない，とマニュアルにはあります。またたとえば，Within の ANOVA をするときも，球状性の検定を行なってデータが分析の前提となる仮定にあっているかどうかを判定しなければなりません。主効果が見られた，交互作用が見られたとなれば，今度はどこに差があるのかをみるために – 検定の繰り返しを避けるために，下位検定を行います。

このように，仮説検定をする場合は条件にあっているかどうかを検定し，あっていなければ補正をするなどの手続きが必要です。同時に，仮説検定を同一のデータに何度も繰り返す事は，5% 水準という**タイプ 1 エラー**を生じる確率が適切に制御できなくなるため，本来なら実験単位ではなく「一連の分析単位」でこれを調整しなければなりません。正直なところ，ここまで厳密な調整が，あらゆる心理学研究できちんと行われているかと言われれば，その答えは No なのですが。

ベイズ統計の場合，こうした問題は非常に軽微なものに変わります。たとえば二群の分散が等しいかどうかについては，「等しくないモデル」を考えれば良いのです。具体的にいうと，二群の分散が等しいモデルは，

$$X_i \sim N(\mu_1, \sigma), Y_i \sim N(\mu_2, \sigma)$$

であり，等しくないモデルは

$$X_i \sim N(\mu_1, \sigma_1), Y_i \sim N(\mu_2, \sigma_2)$$

とするだけです (σ に添字がある，すなわち違う大きさとして推定する)。球状性の検定についても分散共分散行列の要素それぞれを推定するだけであり，その数値に事前に定められた制約はありません。

またたとえば，タイプ 1 エラーの制御についても，そもそもタイプ 1 エラーという発想がありませんから存在しないのです。データがあって，平均構造を線形モデルで表現したのであれば，そこから出てくる事後分布は 1 つしかありません。その単一のパラメータ同時事後分布をどのように切り分けようとも，ある判断のエラーに確率が伴う事はないのです。

帰無仮説検定を厳密に行うには，下位検定を事前に計画してどこにどのような差があると考えるかも準備しておく必要があります。帰無仮説検定は勝敗を決める手法ですから，ゲームのルールは事前に決めなければならないのです。ゲームの設定如何によっては，判定結果が変わってくることがあるからです。これに対してベイズ統計的アプローチであれば，結果 (事後分布) は決まっているので，下位検定を事前に計画する必要はなく，事後分布をどう切り分けようと分析者の自由です。

■停止規則やサンプルサイズの問題が生じない 同様のことは，サンプルサイズの設計についても言えます。帰無仮説検定の場合は，ゲームのルールを事前に決める必要があると言いました。このルールの中には，サンプルサイズも含まれています。サンプルサイズが大きくなればなるほど，有意であると判定される可能性は上がっていきます。帰無仮説検定における，最もやってはいけない研究実践のひとつは，データを取りながら検定を繰り返し，有意になったらデータを取るのをやめる，というものです。検定を繰り返すと，誤った判断をしてしまう可能性が増えますし，サンプルサイズが大きくなると有意になりやすくなります。つまり「勝つまでゲームをやめない」というのは卑怯な方法なのです。

しかし実際は，数人相手にデータをとってみたけど有意差はでなかった，でも惜しいから頑張ってデータを増やした，結果としてある日有意な差になったので，サンプルサイズをちゃんと記載して論文として提出した，ということがあるわけです。努力して真実に辿り着くのはいい話ですが，この例は努力したことが結果の捏造になっていたかもしれないという，より悲しい話になってしまいます。

帰無仮説検定は事前にサンプルサイズを決めなければいけません。このサイズ，この基準で勝負するぞ，というルールがあってからの一発勝負なのです。どこでデータの収集を止めるかというのは，**停止規則 (stopping rule)** といって，これまた事前に決めておかねばならないことなのです。事後的に，勝負に勝ったからこのやり方でよかったんだろう，と考えるのは間違っているのです。

これに対して，ベイズモデルの場合はこうした問題が生じません。最後が確率的な判断にならないので，こうした問題が生じないのです。データが蓄積されていくことは，徐々に確信度が高まっていくことでもあります。差があるのかないのか，という判断をするにあたって，よりその違いが明確になっていくだけなのです。

■より柔軟な仮説を立てることができる　帰無仮説検定は，帰無仮説と対立仮説という 2 つの仮説の比較判断になります。ここで帰無仮説は「差がない」「相関がない」といったものになります。たとえば二群間の母平均を比較する場合，帰無仮説は $\mu_1 = \mu_2$ になります。しかし実際には，差がないということを主張したい場合もあるかもしれません。帰無仮説に「差がある」というのをおくことができないのは，「あるというのは，どれぐらいあればいいのか」という問題がすぐに浮上するからです。たとえば母平均の差が $\mu_1 - \mu_2 = 0.1$ なのか，$\mu_1 - \mu_2 = 0.11$ なのか，$\mu_1 - \mu_2 = 0.111$ なのか･･･言い出すとキリがないですね。p 値は帰無仮説のもとで計算されるものですから，差があるという仮説は無限に作れてしまうので，検証すべき帰無仮説も無限に膨れ上がってしまいます。差がないという状態は $\mu_1 - \mu_2 = 0$ と一意に定まるので，帰無仮説を設定することが可能なのです。

では帰無仮説を積極的に支持すればよいではないか，と思われるかもしれませんが，それができません。判定結果の p 値は帰無仮説のもとで計算していますから，「帰無仮説のもとで考えるとおかしな結果になった」という背理法が成り立たないのです。「帰無仮説のもとでおかしな結果にならなかった」というのは，帰無仮説以外の可能性を除外できるものではありません。

このように，帰無仮説検定では「効果がなかった」とか「関係がなかった」とは言えず，せいぜい「効果がないとは言えない」というにとどまるのでした。これに対して，ベイズ統計では**モデル比較**の観点から検証することになります。帰無仮説は $\mu_1 = \mu_2$ というモデル，対立仮説は $\mu_1 \neq \mu_2$ というモデルです。言い換えれば，帰無仮説は

$$X_i \sim N(\mu_1, \sigma_1), Y_i \sim N(\mu_1, \sigma_2)$$

というモデルであり，対立仮説は

$$X_i \sim N(\mu_1, \sigma_1), Y_i \sim N(\mu_2, \sigma_2)$$

というモデルだとしているに過ぎないからです。

これまでみてきたように，ベイズ統計のアプローチで平均値の差を考える場合は，$\mu_1 - \mu_2$ のような差の分布を**生成量**でつくり，その大きさをそのまま吟味できます。どうしても「差があるのか，ないのか」という判断がしたければ，**実質的に等価な範囲 (region of practical equivalence; ROPE)**(→ セクション 6.2, Pp.74) を事前に設定し，その区間に差の分布が入ってくるかどうかを判断するのでした。そもそも差も分布しているわけですから，どれぐらい重複しているかという幅で考えるほかないわけです。帰無仮説検定の文脈においても，**効果量 (effect size)** とよばれる**標準化された差**の大きさを算出して考えますが，このように「どれぐらい違いがあれば差があると言えるか」という程度問題が前面に出てくることが重要なのです。5% 水準のように機械的に判断するのではなく，領域固有の知識 (ドメイン知識) に基づいて，実質的な差の大きさをケースバイケースで判断することが重要です。もちろん機械的に手続きを進められないことは不便かもしれませんが，不便だからといって真実を曲げるようでは本末転倒です。

また生成量を使ったアプローチのところで解説したように，パラメータについて考えるだけでなく，そのモデルから生成されるであろうデータについて仮説を立てたり検証したりできるのも，ベイズ統計的アプローチの

利点です。実際どの程度の差があると意味があるのかについて，具体的な数値シミュレーションで考えることができるからです。パラメータが δ 以上の差があれば良いとか，二群のデータが D 以上違ってくる確率はどれぐらいか，といった検証の仕方は，仮想空間上の p 値よりも具体的で意味のある情報をもっています。パラメータやデータについて，「同じかどうか」以上の情報を検証できることは，ベイズ統計の長所といえるでしょう。

17.3 モデル比較

先ほど，**モデル比較**ということに言及しました。このことについて，最後に少し触れておきたいと思います。ベイジアンモデリング，すなわちモデルをたててベイズ統計学的に推定するなかで，そのモデルの優劣を考える方法があります。

もう一度ベイズの公式を見てみましょう。ベイズの公式は次のようなものでした。

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}$$

ここで右辺の分母は $p(D)$，つまりデータの確率をあらわすもので**周辺尤度 (marginal likelihood)** と呼ばれます。これはどのように計算されるのでしょうか。尤度は $p(D|\theta)$ で表されますが，$P(D)$ はこのパラメータが無くなったものになります。どのようにしてパラメータをなくすかというと，パラメータのありそうな可能性をすべて足し合わせることで**周辺化消去**すれば良い，ということになります。

たとえば性別と学年のクロス表があったとして，ランダムに選んだ 1 人の学生が女性である確率は，すべての学年の女性の確率を足し合わせたものになるわけです。記号で考えると，条件付き確率 $P(D|\theta)$ において，θ の可能性を全部考えれば $P(D)$ だけになるということです。これはパラメータ θ が離散的な例ですが，連続的な場合は

$$p(D) = \int p(D|\theta)d\theta$$

のように積分で考えればよいでしょう。

これはモデルに含まれるあらゆるパラメータのパターンを網羅して，データが得られる確率を計算したことになりますから，モデルが表現できる世界の中でどれぐらいデータを捕まえることができたか，ということを表しているとも言えます。ここでモデルが複数ある場合を考えましょう。あるモデルを $\mathcal{M}$ と表現することにして，第一のモデルを $\mathcal{M}_1$，第二のモデルを $\mathcal{M}_2$，と表したとします。すると事後分布も「あるモデルを仮定した時の事後分布」ということになりますから，次のようにすべて「モデルのもとでの」という条件付き確率で表現できます。

$$p(\theta|D, \mathcal{M}_i) = \frac{p(D|\theta, \mathcal{M}_i)p(\theta|\mathcal{M}_i)}{p(D|\mathcal{M}_i)}$$

ここで右辺の分母を見ると，$p(D|\mathcal{M}_i)$，つまり「モデル i のもとでデータが得られる確率」を表していることになります。もしこの確率が高ければ，このモデルはこのデータを生む確率が高いわけです。逆にもしこの確率が低ければ，このモデルはこのデータを生む確率が低いということになります。この周辺尤度は，モデルがデータを支持する度合いであり，証拠 (**エビデンス**) の強さとも言えるわけです[*1]。

その上で，次のような数字を考えます。

[*1] これを考えると，どんなデータでも作れる幅広いモデルを考えておけばいいじゃないか，と思うかもしれませんが，モデルを広く考えすぎるとそこから出てくるデータはそのごく一部でしかなく，かえってエビデンスとしては弱くなってしまいます。なんでも予測できるモデルは何も予測していないことと同じなのです。

$$BF_{12} = \frac{p(D|\mathcal{M}_1)}{p(D|\mathcal{M}_2)}$$

この数字は**ベイズファクター (Bayes factor)** と呼ばれるもので，モデル 2 に比べモデル 1 がどれほどデータに支持されているかを示す数字です。この数字が 1.0 より大きければ，モデル 1 はモデル 2 に比べて相対的に強くデータに支持されているということになります。逆に 1.0 を下回ると，モデル 2 のほうがモデル 1 よりもいいね，ということです。モデル同士の比による表現ですので，BF_{21} としても構いません。ここでモデル 1 が対立仮説，モデル 2 が帰無仮説だとすると，どちらがよりデータをうまく表しているか，どちらがよりデータに支持されているかがわかるわけです。最近では，帰無仮説検定の文脈でも BF を併記して，どちらのモデルが良いかを示すこともありますし，この方法ですと帰無仮説 (というか差がないというモデル) を積極的に支持する，ということもできます。

この手法はベイズ推定するものであれば一般的に通用する考え方ですので，1 つのデータに対していろいろなモデルを考えついたとしても，ベイズファクターを計算して相互に比較できるわけです。すなわち，ベイズのモデル比較はベイズファクターという共通の基盤を手にしたとも言えます。そんな優れた方法があるのであれば，なぜもっと早く教えてくれなかったのか，という声が聞こえてきそうです。実は理屈上ではこの通りなのですが，この方法には実践上の問題が残されているのです。それは，周辺尤度の計算が難しいというものです。すべてのパラメータについて，考えられるすべての可能性を計算して足し上げるわけですから，それはそれは大変な計算になります。有効な近似計算の方法として，全パラメータの全確率空間を計算し尽くすかわりに，確率が高そうなところを効率よくサンプリングして推定値を得る**ブリッジ・サンプリング (bridge sampling)** という手法が考えられており，その計算をする R のパッケージ Gronau et al. (2020) が提供されていますから，今後こうしたモデル比較も増えてくるでしょう。

またモデル評価の方法として，**情報量規準**によるアプローチも考えられています。これは「良いモデルとは良い予測をするモデルである」と考えるところから始まっており，平均的に良い予測をするモデル程度を指標化するものです。ベイズの定理を基本としながら，データを真のモデルから生成されるサンプルと考えたり，確率分布同士の近さを**カルバック・ライブラー情報量 (Kullback-Leibler divergence)** で表現するなど，非常に高度な数学的体系が確立されています。詳しくは浜田他 (2019)など丁寧な解説書があり，R パッケージなどで簡単に計算できるようになっています Vehtari et al. (2017)。数学的にやや複雑な内容を含んでいますので，本講義の中では扱いきれんませんが，より進んだ研究や利用のために言葉の紹介だけはしておきます。

17.4 おわりに

本講ではここまで，ベイズ統計と従来の統計的分析を比較して議論してきました。ベイズ統計の方がメリットが大きいような解説をしてきましたが，これは筆者の好みもかなり含まれるものです。帰無仮説検定はなにも悪いものではなく，前提や仮定をしっかり守れば，一般的なソフトウェアで誰にでも結論を出すことができるという点で，とても良いものなのです。前提や仮定が多くて面倒だから，マニュアル人間ができてしまうというのは，あくまでもユーザ側の問題であって方法論の問題ではありません。ベイズ統計だって，モデルが間違っていれば結論は間違ったものになりますし，なによりデータに合わせてモデルを逐一設計図から書き起こすのは，大変コストがかかることでもあるからです。本来，これら 2 つの流儀は相反するものではなく，それぞれ依って立つ理論や考え方があって，必要に応じて選べるようになるのが一番です。

またベイズ流の研究アプローチ，とくにモデル比較については，まだまだ議論が活発に行われている段階であり，とくに心理学研究にこうしたアプローチがどれほど有用かについては，いまだに結論が出ていないところでもあります。パラメータの推定値で考えるのが良いのか，モデル比較をして議論を進めるのかについても，

まだまだ定番の方法というのは見つかっていません。

そもそも心理学の知見を数式に落として**モデリング**できるようにするためには，心理学そのものの理論的発展や，人間についてのより深い理解も必要です。人間の行動と予測が完全にできるようなモデルや一貫した理論体系は，まだまだ遥か先の目標に過ぎないというのが現状でしょう。それでも有用なツールを武器に，少しずつ知見を積み重ねていくことができれば，心理学をより楽しく学べるのではないかとおもいます。Enjoy!

付録 A

よくある質問とミスの例

A.1 Frequently Miss and Comments

Rmd でレポートを提出したのに，なんだか中身の問題じゃないのに突き返された，中身を見てくれよ！と思う人もいるかもしれません。テキストでは R や Rmd での課題を提出するよう求めているところがありますが，その際よく見られる学生さんのミスとその対応についてのコメントをここにまとめておきます。

A.1.1 FMC1；そもそもファイルの書式が違う

Rmd で提出してください，R で提出してください，という指示に対して，違うものが提出されてくることがあります。書式があっていない，というのは些細なことのように思えるかもしれませんが，学術論文は書式に捉われず内容に集中するためにも，書式は整えられたものである必要があります。学会誌に掲載されている論文も，みなさんが書く卒業論文も，レポートに至るまで，書式や指示に沿ったものを準備する必要があります。書式があっていない場合は，門前払いになっても文句が言えないのがアカデミックの世界です。

ということですので，R や Rmd で提出してください，という指示があれば，R や Rmd で提出してください。ファイルの種類は拡張子で分類され[*1]，R ファイルは `.R`，Rmd ファイルは `.Rmd` という拡張子になっています。たまに `.Rproj` というファイルを提出してくる人がいますが，これは R プロジェクトのファイルで，これには R スクリプトも文書も含まれておらず，みなさんの計算環境情報が少し書いてあるだけです。また，`.Rmd` だけ入っていれば良いかというとそうではありません。まれに `Filename.Rmd.R` というようなファイルを送ってくる人がいます。これはファイル名にピリオドが含まれているだけで，ファイルの種類を識別する拡張子は `.R` ですから Rmd ファイルではありません[*2]。そもそもファイル名には 2 バイト文字はもちろん，!や?などの記号を含めるべきではありません。ピリオドも当然記号の一種ですから，ファイル名にするのは不適切です[*3]。

ではどうやって適切なファイル形式にするか，ということですが，最も素直な方法としては RStudio で新しくファイルを開くときに，R markdown 形式 (略して Rmd) を選ぶようにしましょう。もし間違って，R script 形式 (`.R` 形式) で開いてしまった，というときは，そのファイルを破棄して新しく作り直すのが一番ですが，エディタペインの右下にあるファイル種類表示 (図 A.1) をクリックして修正することもできます。

[*1] 拡張子についてはセクション B.5,Pp.248 参照

[*2] 最近の OS は拡張子を表示しない設定になっているものも少なくないので，このようなミスが生じます。

[*3] 長いファイル名などの場合，空白を入れるのも適切ではありません。できなくはないのですが，やるべきではないのです。どうしても空白を入れたければ，アンダースコアなどで区切ると良いでしょう。

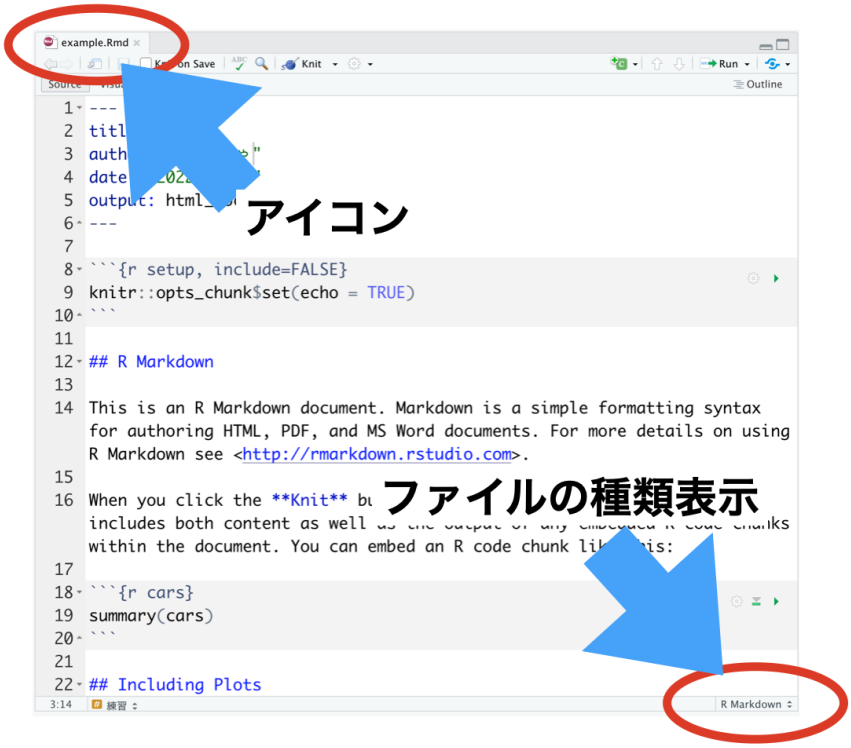


図 A.1 ファイルの種類を判別する方法

A.1.2 FMC2；やっぱりファイルの形式が違う

Rmd 形式のファイルは，拡張子だけで決まるものではありません。図 A.2 にあるように，Rmd ファイルは冒頭の 6 行 (正確には---で囲まれた領域) が YMAL と呼ばれるところで，文書全体の設定をしています。その下に，R のコードを実行する部分 (**チャンク (chunk)**) や文章の領域があります。文章のところは#記号で見出しを作ったりできます。

この YMAL 部分が壊れている，あるいはチャンクが正しく記述されていない場合，拡張子が.Rmd であっても適切な Rmd ファイルにはなっていません。YMAL 部分の書き方はよくわからない，という人も多いと思いますので，RStudio で Rmd ファイルを作ったときの状態をなるべく変更しないように注意すると良いでしょう[*4]。

チャンクは R のコードを書くところで，バッククォーテーション 3 つでくくるのが決まりです。チャンク領域が始まるところに{r}とかいて「ここが R で計算するところですよ」というのを指定するわけです[*5]。このバッククォーテーション 3 つ (```) が全角だったり (｀)，ダブルクォーテーションだったり (") すると，機械は正しくチャンクであると認識しません。フォントなどの見せかけ上は，微妙な違いのように見えますが，機械にとって違う文字列は違う意味を持ちますので注意してください[*6]。RStudio で編集する場合，チャンクの領域はやや

[*4] Rmd ファイルを新しく開くときに，文書タイトルや著者名，日付，出力ファイル形式などを設定することができるウィンドウが開きますので，そこに必要な情報を書くと自動的にそれを使った YMAL が生成されます。また，本文として幾つかのサンプルコードが最初から含まれていますが，これらに関してはサンプルをそのまま使うことはないので，全て削除してしまって結構です。

[*5] ほかにも Python や Julia など他の計算言語を混ぜることもできます。

[*6] 記号の名称や入力については，セクション D,Pp.259 を参照してください。

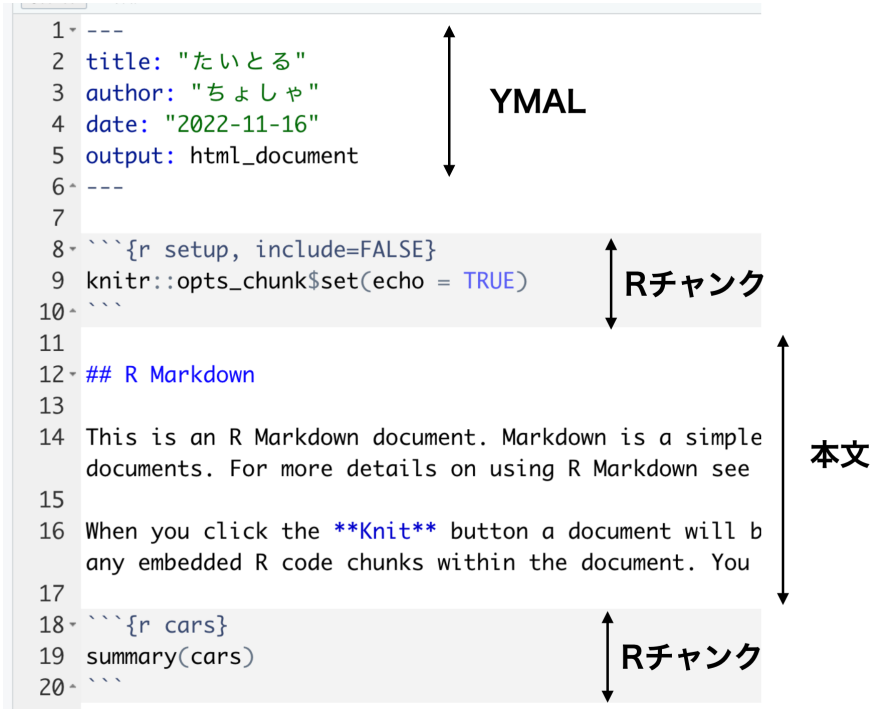


図 A.2　Rmd ファイルの中身における書式

灰色がかった強調表示がされますので，どこにチャンクがあるかわかると思います。もし強調表示されていないようであれば，チャンクとして認識されていない可能性を疑った方が良いでしょう。

チャンクはバッククォーテーション 3 つで開き，おわったら同じくバッククォーテーション 3 つで領域を閉じます。閉じなければずっと R の計算領域が続くと解釈されますし，最後までチャンクが閉じられないと Rmd ファイルとして正しくない書式ということになります。チャンクが正しく入力できているかについて，常に注意を払っておく必要があります。また，自分でバッククォーテーション 3 つを使ってチャンク領域を開いたり閉じたりするのが面倒だ，という人は RStudio のチャンク挿入ボタンから挿入すると間違いないでしょう。

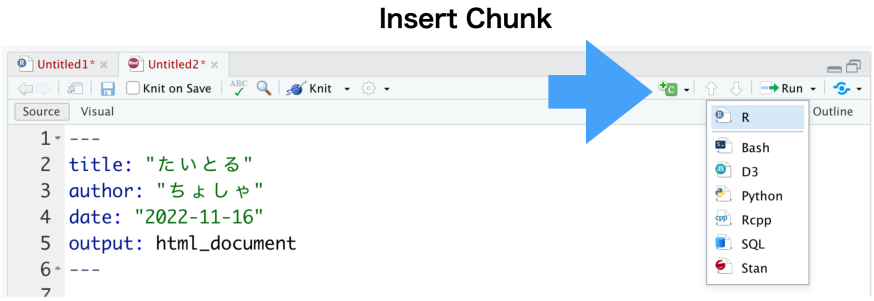


図 A.3　RStudio のチャンク挿入ボタン

A.1.3　FMC3；Rmd が knit できない

Rmd は文書作成に加えて，R での計算がセットになったファイル形式であり，文中の数字や分析結果は「書き写す」のではなく「その場で生成する」ものです。生成する，というのは Rmd ファイルを変換して，

HTML や DOCX，PDF 形式のファイルにすることを指します[*7]。このファイル変換を**ニット (knit)** といいます。編み物を編むようなイメージですね。このときに，タイトルをつけ，見出しのサイズを変え，R の計算をして結果を埋め込む作業をするわけです (図 A.4)。こうすることで，結果のコピペを避けること，再現性を担保することができるようになるわけです。すでに説明したように，チャンクを使って計算に必要な指示を Rmd ファ

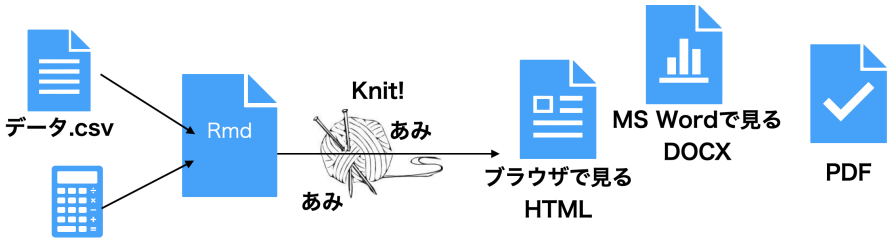


図 A.4 knit してレポートが完成する

イルに書きます。この指示はエラーのないコードである必要があります。当然ですが，スペルミスや間違った R コードは「実行できない」というエラーになるわけです。その場合，knit は中断され結果のファイルが出力されません。提出されたレポートは knit して出来上がったもののことを指しますから，knit できない Rmd ファイルを提出されてもレポートが提出されたことにはならないわけです。

knit できない Rmd ファイルになっていないか，というのはご自身の RStudio 環境で knit して確認できることです。提出前に一度，きちんと機能する Rmd ファイルになっているかどうか確認するようにしてください。以下で述べるように，自分の環境で knit できても，提出先の (私の) 環境で knit できないということもあり得ます。しかし，自分の環境で knit できないのに提出先でできる，ということはあり得ませんので，「knit できませんよ」というコメントをつけて返される前に自分で確認するようにしてください。

A.1.4 FMC4；外部環境を参照してしまう

さて，R で行う作業の中には，csv ファイルを読み込むといった「外部のファイルを使う」指示もありえます。例えば次のようなコードです。

code : A.1 Rmd 中の R コードが外部環境を参照してしまう

```
1  dat <- read.csv("social_effects.csv")
2  dat <- read.table("clipboard")
```

このコードでは，上の行は `social_effects.csv` というファイルを，下の行はクリップボードの内容を読み込むようになっています。ファイルを読み込む指示は，ファイルがなければ当然エラーになりますから注意が必要です。レポート等で Rmd ファイルを提出するとき，Rmd ファイルの他に必要な読み込むべきファイルがあれば一緒に提出するようにしてください。また，クリップボードの内容は再現できません。クリップボードとは，コピーアンドペーストのコピーを行ったとき，PC の内部で一時的にコピー内容を覚えておく場所のことです。つまり，みなさんがみなさんの環境で，クリップボード上にデータを一旦保持している場合は，このコードでエラーが生じることはありません。しかし，レポート提出先の (私の) 環境で，事前にデータのコピー作業を行なっているわけではないのですから，提出先でエラーが発生します。

[*7] HTML は Hyper Text Markup Language の略で，ブラウザで開くファイル形式です。DOCX は Microsoft Word のファイル形式です。PDF は Portable Document Format の略で，データを紙に印刷した状態のようにサイズを固定して出力したファイル形式であり，OS がもつビューワーや Adobe Acrobat Reader などで見ることができます。

そもそも Rmd ファイルは，作業の再現性を担保するためのファイルになっているわけですから，クリップボードの利用のような「自分の環境だけで可能な記録されない作業」をそのファイルに含めるのは適切ではありません。Rmd ファイルは Rmd ファイルだけで分析作業が完結するように，必要な記録は全て記載されている必要があります。同様に，分析作業に関係のない冗長な指示や無駄な指示は Rmd ファイルに含めるべきではありません。

A.1.5 FMC5；提出先の環境を変更してしまう

R はさまざまなパッケージを使って分析環境を拡張することができます。パッケージは **CRAN** を通じてインターネット経由で配布されますから，ネット環境があれば誰でも最新のパッケージをとってくることができます。みなさんが Rmd ファイルの中で使う R の関数の中には，パッケージの関数も含まれているでしょう。パッケージがなければ関数が動きませんから，パッケージをインストールする作業も Rmd に書いておきたいと思うかもしれません。しかし，これは推奨できません。

パッケージのインストールは，実行環境の準備にあたる作業です。Rmd ファイルを使ってコードのやり取りをするとき，提出先の環境で分析することになりますが，提出先の環境にどのようなパッケージをいつ入れるかは，提出先の環境の管理人が判断すべき問題です。提出する Rmd ファイルにパッケージをインストールする関数，すなわち `install.packages()` 関数が含まれているというのは，相手の環境を勝手に操作してしまうことと同じであり，セキュリティ的にも適切な発想ではありません。

環境の準備は提出先の管理人が管理すべき問題であり，またすでにパッケージが入っている場合は無駄な上書き作業をさせることにもなります。また `install.package` 関数は CRAN のサーバを参照したりしますから，適切な設定がなければ R チャンク実行時にエラーが発生します。いずれにせよ，R チャンクの中に `install.package` 関数を含めないようにしましょう。

以上が Rmd ファイルや R ファイルでレポートを提出するときの留意点です。その他にも R や RStudio を使うときによくある質問がありますので，それらも一問一答型で紹介しておきましょう。

A.2 Frequently Asked Questions；よくある質問と答え

Q. A.2.1: テキストを参考にパッケージをインストールしようとしたところ，エラーが発生しました。

A. 質問ではなくて報告ですね。お返事としては，「わかりました。エラーが発生して大変ですね。」としか言いようがありません。どの環境で，何をしようとして，どのようなエラーが出たのか，明示してください。メールのやり取りで指示が明確になるように，テキストを準備しています。何章，何ページの，どの文章を参考にしたのかも教えてください。

Q. A.2.2: 添付のようなエラーが発生しました (図 A.5)。対応策を教えてください。

A. エラーの発生画面を送ってこられていますが，この画面に写っていない上の方でエラーが発生していますから，これではどのエラーなのかわかりません。スクリーンショットを送ってこられることはよくありますが，ほとんどの場合，適切な箇所が写っていません。複数枚添付してこられる人もいますが，画面を拡大しながら読むのも難しいので，**R コードそのものを送ってきてください**。そうすると，何行目のどこにエラーがあるかが明確になり，対応に関係ない無駄なやり取り (ex.「もう少し上を写してください」「違う，もっと上」) が減ります。

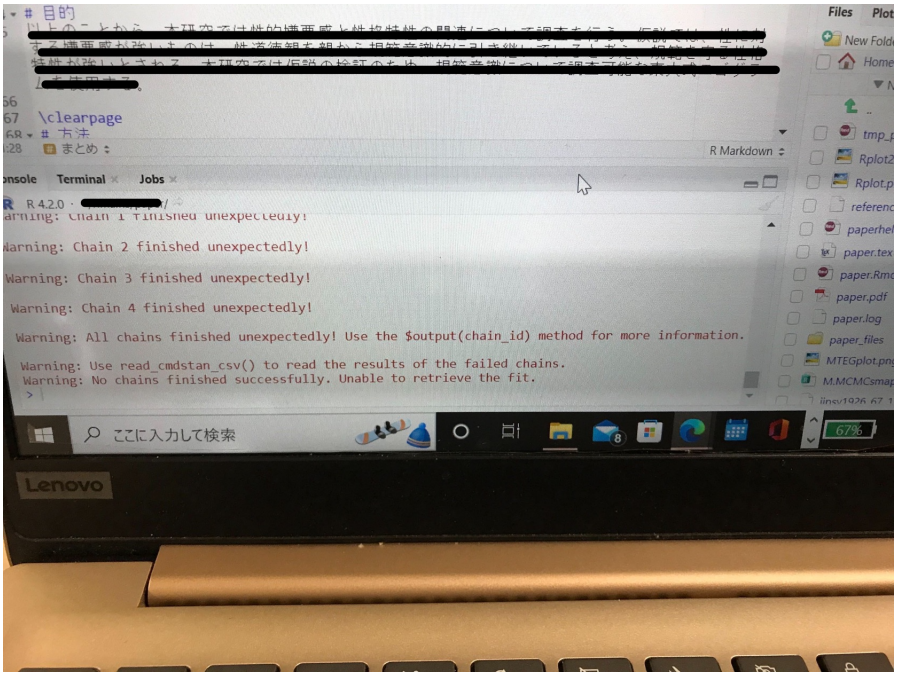


図 A.5　添付されてくるスクリーンショットの一例

Q. A.2.3: ファイルを読み込もうとすると次のようなエラーが出ます。どうすれば良いですか。

`'xxxxx'` に不正なマルチバイト文字があります

A. 文字化けの原因は，たとえば UTF-8 形式で提供されているファイルを，Windows 標準の CP932 形式で読み込もうとする，という文字コードの不一致です。ですからそれをオプションで指定してあげれば問題解決です。`read.csv` 関数を使っている場合，オプションの指定は `read.csv("foo.csv",fileEncoding="UTF-8")` のようにします。`tidyverse` の `read_csv` 関数を使っている場合，locale オプションで，locale 関数の encoding を明示的に UTF-8 とします。`read_csv("foo.csv", locale=locale(encoding = "UTF-8"))` のようにします。

Q. A.2.4: ファイルを読み込もうとすると次のようなエラーが出ます。どうすれば良いですか。

`'xxxxx'` に不正なマルチバイト文字があります

A. 文字化けの原因は，たとえば UTF-8 形式で提供されているファイルを，Windows 標準の CP932 形式で読み込もうとする，という文字コードの不一致です。ですからそれをオプションで指定してあげれば問題解決です。`read.csv` 関数を使っている場合，オプションの指定は `read.csv("foo.csv",fileEncoding="UTF-8")` のようにします。`tidyverse` の `read_csv` 関数を使っている場合，locale オプションで，locale 関数の encoding を明示的に UTF-8 とします。`read_csv("foo.csv", locale=locale(encoding = "UTF-8"))` のようにします。

Q. A.2.5: パッケージを読み込もうとすると次のようなエラーが出ます。どうすれば良いですか。

`library(tidyverse)` でエラー；`'tidyverse'` というパッケージはありません。

A. パッケージがないので，インストールしてください。

Q. A.2.6: パッケージをインストールしようとすると次のようなエラーが出ます。どうすれば良いですか。

```
Warning in install.packages:
    'lib = "C:/Program Files/R/R-4.0.5/library"' is not writable
```

A. ユーザがファイルに書き込みをする権限を持っていないので，(パッケージファイルをドライブに) 書き込みできないというエラーです。RStudio を実行する時に，「管理者として実行」を選びましょう。

Q. A.2.7: パッケージをインストールしようとすると次のようなエラーが出ます。どうすれば良いですか。

```
Warning in install.packages:
    ディレクトリ 'C:〜(任意の文字列)〜\????' を作成できません。理由は
```
```'Invalid argument' です。```

A. ユーザ名が全角文字を含んでいるため, 文字化けして R 側から操作ができません。解決する方法は二つあります。

ユーザを作り直す方法 ユーザ名に全角文字を含まない, 新しいユーザを作成します。設定＞アカウントから新しいアカウントを作りましょう。

インストールフォルダを指定する R のコンソールで `.libPaths()` と実行すると, パッケージをインストールするフォルダが出てきますが, ここを変更する方法です。次の 2 ステップで対応できます。

- まず C ドライブのすぐ下にインストール先のフォルダを作ります。`myLib` としましょう。
- 次に R のコンソールで `.libPaths("C:/myLib")` 書いて実行します。

これでインストール先が変わりますので, 書き込み・インストールができるようになります。老婆心ながら付け加えますと, フォルダ名はなんでも構いませんが, 全角文字ではいけません。また場所も C ドライブのすぐ下である必要はありませんが, 全角文字や空白を含むフォルダの下に入れてしまってはいけません。OndDrive のようなクラウドを指定するのも良くありません (探しに行った時にオフラインだとまたエラーになります)。
```

Q. A.2.8: ファイルが読み込めません

```
file(filename, "r", encoding = encoding) でエラー:
  コネクションを開くことができません
 追加情報:  警告メッセージ:
 file(filename, "r", encoding = encoding) で:
  ファイル 'foo.csv' を開くことができません: No such file or directory
```

A. 指定された場所にファイルがないので，読み込むことができないエラーです。確認すべきは，「プロジェクトを開いているか」，「プロジェクトフォルダの中に当該ファイルはあるか」，「ファイル名のミススペルはないか」です。プロジェクトってなんだという人は，RStudio の基本に立ち戻り，プロジェクトでファイルやフォルダを管理するようにしてください。プロジェクト管理については，Pp.15 にもその説明があります。

プロジェクトによる管理とは要するに，R が今見ているフォルダの場所を固定する方法です。プロジェクトを開くと，プロジェクトフォルダが「今見ているフォルダ (ワーキングディレクトリ)」になりますので，その中のファイルを参照することになります。プロジェクトフォルダの中に当該ファイルがないと読み込むことができませんので[a]，当該ファイルをプロジェクトフォルダ内に移してきてください。

Rmarkdown や Quarto ファイルの場合も，必ずプロジェクトフォルダの中に置くようにしてください。これらは Knit するときはファイルの置かれた場所を WorkingDirectory にしますので，プロジェクトフォルダの中にないとパスが通らない問題が頻発します。

[a] フォルダの位置を相対的・絶対的に指定してやれば，どこにおいていても読み込むことはできますが，この問題で悩んでいる人は相対・絶対パスの指定というところでさらに疑問が深まることになると思いますので，気にしないでくれて結構です。相対・絶対パスが知りたい人は，付録 B,251 でファイル場所とは何かを再確認してください)。

Q. A.2.9: ANOVA 君が読み込めません

```
file(filename, "r", encoding = encoding) でエラー:
  コネクションを開くことができません
 追加情報:  警告メッセージ:
 file(filename, "r", encoding = encoding) で:
  ファイル 'http:/riseki.php.xdomain.jp/index.php?plugin=attach&refer=
  ANOVA 君&openfile=anovakun_486.txt' を開くことができません: Invalid argument
```

A. ファイルを読み込みにいく先が URL，すなわちインターネット上になっています。ANOVA 君のファイルを一度手元の PC のフォルダにダウンロードし，そのローカルのファイルの位置を指定して読み込むようにしてください。

Q. A.2.10: 効果量として Hedges の g を算出してください。という指示について実行すると g ではなく d が出てしまうようなのですが，どうすればよいでしょうか。コードは次の通りです。

```
cohen.d(value ~ condition, data = dat, hedges.crrection = T)
```

A. Hedges の補正 (correction) のオプションが通っていません。スペルミスです。オプションのスペルが間違っているので無視されたので，関数名通り Cohen の d が算出されます。

Q. A.2.11: 描画の際に次のような注意が出てきます。これはどういう意味ですか。

```
Removed 671 rows containing missing values (geom_point).
```

A.「671 件の行で欠測値が含まれています」ということです。つまりデータセットの中に欠測値 (観測されていない，数値が入っていない行) が 671 件あったので，それは表示できませんでしたよ，という意味です。警告が嫌だということであれば，データセットの中で欠けているものを除外する必要があります。R の関数では，`na.omit` で欠測値を除外することができます。

Q. A.2.12: lm 関数を実行するコードでオブジェクトがないと言われます。

```
result <- lm( weight - height, data = dat)
```

A. 従属変数と独立変数とをつなげるのはチルダ (~) という記号です。そこがハイフン (-) になっています。スペルミスの一種です。

Q. A.2.13: lm 関数を実行するコードでオブジェクトがないと言われます。

```
result <- lm( weight - height )
```

A. データを与えていないので，`weight` というオブジェクトを探しに行って，見つからないというエラーです。`data` オプションでデータを与えてあげてください。

Q. A.2.14: 因子分析の Robust 法での RMSEA の p 値って超えてたらまずいですか。Standard(DWLS) の方だけで報告はだいじょぶでしょうか。

A. 非常に専門的な質問をしておられますが，まず，「因子分析」「Robust 法」「RMSEA」など，個々の用語の意味はわかっているでしょうか。キーワードによる検索で，「ロバスト法とは」「Robust の意味」などは答えが出てくると思いますが，これらを分析法の体系的な文脈の中に位置付けないと，答えられない種類の問題です。この例をもとに説明すると，Robust を辞書で引くと「壮健」「たくましいこと」などと出てきますが，もちろんそういう意味ではありません。ロバスト法を検索すると，「統計学の分野でロバスト推定法というやり方がある」「観測値に外れ値が含まれている可能性を考え，その影響を抑えることを目的とした手法」などの説明が出てきます。ロバスト推定法は因子分析に限らず，回帰分析など他の手法でも使われる考え方なので，このような一般的な解説になります。ですがここで知りたいのは「因子分析におけるロバスト推定」ですから，因子分析でロバスト推定するとはどういうことか，因子分析の観測値における外れ値とは何か，因子分析における推定とは何を推定するのか，そもそも因子分析とは何を目的としているのか，なぜ自分は因子分析方を使うのか，さらに何故因子分析の中のロバスト推定を使いたいと思っているのか，といったことがわかっていないと，この質問に正しく回答することができないのです。このように，複合的な要素について一回で質問しても，適切な答えに辿り着けないことがあります。聞くことは恥ずかしいことではありません。知らないことを知っているふりをすることが恥ずかしいことであり，知らないまま「みんなそうやっているから」「テキスト/ネットに書いてあったから」と看過してしまうことこそ恥ずべきことなのです。ちなみに，質問に対する回答を間違えることも恥ずかしいことではありませんから，「正しく答えられなければ恥をかく (から質問しない)」というのも同様に恥ずべきことです。こうした恥ずかしさ (保身) から，「専門用語を使ってそれっぽく質問してわかった感じになろう」というのは，かえって遠回りになります。実は回答よりも，質問の仕方でその人の理解度が明らかになってしまっています。このように書くと，なんでも反射的に「わかりません，教えてください」という人もいますが，それも適切な質問方法ではありません。教員がアドバイスできるのは，「答え」ではなく「理解」が欲しい人に対してだけなのです。質問する場合は，自分は何がわかっていて何がわかっていないのか，何が知りたいのかを明確に言語化して質問するようにしてください。

付録 B

電子計算機のイロハ

B.1 前置き

このセクションは心理統計ではなく，コンピュータについての四方山話をダラダラと書いています。そんなの聞かなくてもわかってるよ，という人もいるかもしれませんし，知らなくてもみなさんはきっとスマートフォンやタブレットを使えていることと思います。しかし知らずに使うことと，知ってて使うことには大きな違いがありますし，今後大学でレポートや論文を書いたり，それに必要な統計処理をするためにも，計算機の基本的な特徴を知っておくと，トラブルに出会した時に「あぁこれってひょっとして」というヒントが得られたり，納得できるようになるかもしれません。知らなければ「何だかわからないけどパソコンが壊れた」というか，「パソコン運が悪い」「自分はパソコンが苦手なのだ」と間違えた帰属をしてしまうことになります。

コンピュータ関係の授業で聞いたことがある話，これから聞く話もあると思いますが，もし苦手意識を持っている人がいたらこれを機に再入門するつもりで読んでください。

B.2 コンピュータの基礎

21 世紀に生きる私たちは身の回りをコンピュータに囲まれて生きています[*1]。それは携帯電話の形をしていたり，ノートパソコン，デスクトップパソコンの形をしていたりします。また時々テレビなどで報道されますが，気象予報や飛沫がどのように飛び散るかをシミュレーションする大型計算機「富嶽」などもコンピュータですね。これらは形は違いますが，いずれも電子計算機であり，電子計算機には次の 5 つの装置があります。

入力装置　キーボードやマウス，タッチパネルなどを使って情報を取り込むデバイス (装置)

出力装置　モニタやプリンタ，タッチパネルなどを経由して情報を出力するデバイス

演算装置　プログラムの命令に従って計算処理 (四則演算や論理演算) をする装置。一般に電子計算機の中央で一括して処理するので，Central Processing Unit(CPU) と呼ばれるものです。

制御装置　演算結果に従って他の装置に指示を出す装置のこと。演算装置とまとめて CPU に実装されています。

記憶装置　計算結果などの情報をいったん保持しておく装置のこと。コンピュータの内部にあって一時的な

[*1] 私は 1976 年生まれですが，生まれた頃は周りにコンピュータなんかありませんでした。小学生の頃にマイコン (マイクロコンピュータ，小さなコンピュータという意味でもありますが，My Computer，私のコンピュータという意味でもあります。つまり個人単位でコンピュータが使えるようになった，というだけでも大きな出来事だったのです。) という言葉が出てきて，なんかかっこいいなと思った記憶があります。私が 10 歳になったころ，ビデオゲーム (テレビゲームでやるゲームが家庭でできるようになり，それをこのように呼びました。) が身の回りに出てきました。ファミリーコンピュータ，とくに「スーパーマリオブラザーズ」によって日本中の子供たちが熱狂したのが 11 歳の頃，「ドラゴンクエスト」によって社会問題になったのが 12 歳の頃になります。ともかくこの頃は，コンピュータといってもゲーム機のような扱いでした。

計算に使われる一次記憶装置，ハードディスクドライブ (Hard Disk Drive,HDD) やソリッドステートドライブ (Solid State Drive,SSD) などの二次記憶装置など。

最後の記憶装置については，一次記憶装置，二次記憶装置と種類が分かれていますがこの区別は簡単で，電源を落とした時に記憶が消えてしまうのが一次記憶装置，電源を落としても記憶が消えないのが二次記憶装置です。たとえば $12+38=$ という計算をするとき，頭の中で「えーっと一の位が 2 と 8 だから 10 になって繰り上がるから･･･」と考えてから，ノートに $=50$ という答えを書くと思います。次の問題に進むと，先ほどの「1 繰り上がるから･･･」という情報は忘れてますよね。でもノートに書いた $12+38=50$ というのは残っています。このノートがいわば二次記憶装置であり，頭の中で一時的に保持していた情報が一次記憶装置ということになります。一次記憶装置は RAM(Random Access Memory) とも呼ばれます[*2]。

ところで，コンピュータがやっているのは計算だけです。私はこの資料を PC に向かって書いており，キーボード (入力装置) を叩きながら，画面 (出力装置) を見て文字を連ねています。これも「キーが押されたら文字を表示させ，その文字列を記録する」という処理を機械が淡々とこなしているに過ぎません。あるいはマウスやトラックパッドで，アイコンを指し示し，カチリと押す[*3]ことで選択し，押し込んだまま移動させ (ドラッグ)，離すことで別の場所に置いたりします (ドロップ)。トラックパッドの場合は 2 本指で同時に押したりしますし，タッチパネルの場合は二本指を広げたり (ピンチアウト)，逆に二本指を狭めたり (ピンチイン)，3 本以上の指でファサーっと触って場所を広げたりします。たとえば「ファイルを掴んでゴミ箱に捨てる (削除する)」というのが我々にとっての操作ですが，コンピュータの内部では実はこんなことをしていません。ファイルは (二次) 記憶装置に書き込まれた情報です。記憶装置は原稿用紙のように小さなマス目がたくさんあって，ファイルとはそのマス目の XXX 番目から YYY 番目までの情報，ということです。このファイルを削除するというのは，記憶装置のある場所 (アドレス) に「削除されたものなので画面に表示しない」という情報を書き込む，という操作をしているだけです。じゃあなぜ私たちは「ゴミ箱にドラッグ＆ドロップ」なんてするのでしょう？ それはそのほうがわかりやすいからですよね。「ファイルを削除する」というのは，「メモリアドレスの XXX 番地に別の情報を書き込む」という操作だと言われてもピンとこないので，コンピュータが人間にとってわかりやすい表現をして見せてくれているのです。このユーザにとってわかりやすい幻を見せてその気にさせてくれるというデザインのことを，ユーザーイリュージョンと言います。ともかくこういう「画面で見ながら操作する」ことをグラフィカル・ユーザ・インターフェイス (Graphical User Interface,GUI) といいますが，これのおかげでコンピュータの操作は随分楽になっています[*4]。

B.3 コンピュータの歴史

コンピュータの装置，すなわちハードウェアについての解説につづいて，ソフトの側面についても解説を加えようと思うのですが，そのためには少し歴史的な流れを説明したほうがわかりやすいかもしれません。

コンピュータの発展の歴史は，小型化の歴史でもあります。最初にできたコンピュータは ENIAC といいます。1946 年の話です。この ENIAC は 27 トン，広さにして倉庫 1 つ分 ($167m^2$，90 畳以上の広さ) が必要なもので，真空管を使った計算機でした。軍事的な計画のために開発されたオーダーメイドのものですから，一

[*2] 実はこのように人間を 1 つのコンピュータに喩えて，そこではどのように計算がされているのか，人間の記憶装置や演算装置，入出力装置の特徴はどうなっているのか，というのを研究するのも心理学の仕事です。記憶や演算，制御については認知心理学や学習心理学，入出力については知覚心理学や生理心理学が専門的に扱っています。そういう意味でもコンピュータの登場は心理学に大きな影響を与えているのですが，それはまた別の講釈で。

[*3] 押すときの音から，この操作をクリック click と言います。2 回続けて押すことをダブルクリックと言います。

[*4] 実は人間の意識もこのユーザーイリュージョンのようなもので，実際の体の動かし方や感覚情報の受け止め方，処理の仕方は別ですよね。すべての情報を意識に上げるのではなく，「私は XXX をしている」と身体がそれっぽい幻想を投影して見せてくれているのが意識の正体ではないか，という議論があります。興味のある人はノーレットランダーシュ (2002)を読んでみてください。

般人が触れるはずがないものです。時代が下がって真空管が半導体，IC チップになった頃，やっとサイズが小さくなって，家庭用・個人用のコンピュータというのができるかもという時代が来ました。Apple コンピュータの創始者，スティーブ・ジョブズとスティーブ・ウォズニアックが最初のパソコン (Personal Computer,PC)，Apple I を発売したのが 1976 年。爆発的に売れた Apple II が発売されたのが 1977 年です。AppleII はブラウン管表示装置とキーボードを持っていましたので，今の PC の原型とも言えるかもしれません[*5]。PC を作っている会社は Apple だけでなく，IBM や DEC などがありましたが，まだこの頃はパーソナルなレベルのものよりも大型計算機の開発が進んでいました。IBM が PC を作ったのは 1980 年で，この頃から小型化が進められていきます。

私事で恐縮ですが，1976 年に生まれた私が初めて PC を手にしたのは 1991 年，高校入学のお祝いで買ってもらった Fujitsu の FM-Towns という機体でした。この頃は「マルチメディア」という名前もなく，「ハイパーメディア」と読んで売り出していました。この機体は他の PC(NEC の PC-9801 や Sharp の X68000 シリーズが有名でした) とは違って，CD-ROM ドライブをつけていたことが画期的だった時代です。その後 1994 年に大学生になりましたが，この頃は連絡を取り合うツールはポケベルが主流であり，携帯電話 (や PHS) のような個人端末は高級品という時代でした。大学に入るとコンピュータを学ぶ授業があり，アカウントをもらったりするのですが，それは大学が持っている大型計算機に端末からアクセスするためのものでした。いろんな部屋にあるのは「端末」で，それほど機能の優れた PC ではなく，複雑な計算 (統計的な計算など) は大型計算機に仕事を依頼しその返信を待つ，というスタイルでした。関西私立のマンモス校でしたので学生数は非常に多かったのですが，多くの学生が一度にアクセスしても，大型計算機はものすごくものすごーく計算が早かったので，瞬時に回答をもたらしてくれるものでした[*6]。つまり，まだ「専門的な計算は大型計算機」という時代であり，パーソナルなコンピュータになるにはもう少し時間が必要でした。

どれぐらいの時間が必要だったかというと，実はその次の年なのです。1995 年，Windows 95 という OS が発売されました。これを機に日本でも PC がどんどん浸透していくことになります。Windows95 は物凄いんだぞ，と発売前からテレビでも散々とりあげられ，発売日には行列ができて真夜中のカウントダウンと同時に大フィーバー，という売れ行きでした。当時のそれは何が凄かったのでしょう？ コンピュータにはそれ動かす基本ソフトが必要です。HDD にデータを書き込み，キーボードからの入力をディスプレイに表示する，といったごく基本的な装置を統括し，メモリ番地をファイルという単位で扱うと言った基本的な操作は OS いうソフトウェアが担当します (図 B.1)[*7]。この OS，大型計算機は Unix と呼ばれるものを使っていましたが，各企業が個人向けにコンピュータを売り始めるときにも当然必要で，各社で開発もしていましたが，PC の共通規格をつくることで OS 部分は共有できるようになりました。そこを提供したのが Microsoft 社のビル・ゲイツです。どんなパーツで作られた PC であっても Windows という OS が共通のフィールドを用意してくれるので，ユーザは Windows で動くアプリケーションを選ぶだけで良い，ということになったのです。

そして Windows95 は，GUI，つまり「ファイルを掴んでポイ」といった直感的な操作で使えることも大きな特徴でした。大型計算機で使われている Linux は基本的に Command User Interface,CUI で，黒い画面にプログラムを書いて実行するといった手法で，初心者には人気がなかったのです。GUI については，その頃 Apple を追放されていたスティーブ・ジョブズが，NeXT という会社で GUI を備えた OS を開発していました。この NeXT はその後 Apple に買収され，ジョブズは Apple に戻って活躍することになります。その頃から巷では，Windows の GUI は Apple の OS を真似したものだと非難されていたのですが，商業的には Windows が大勝利，というわけです。ともかくこれを機に企業などはもちろん一般家庭でも PC を使うような

[*5] Mac の歴史についてはアイザクソン (2011)が読み物としておもしろいですよ。

[*6] Time Sharing System,TSS，時分割システムとよばれる機構です。命令を小さな単位に分割し，それを順次捌いていくという方法でした。

[*7] 物理的な機構と OS との間に Basic Input/Output System,BIOS というのが入りますが。

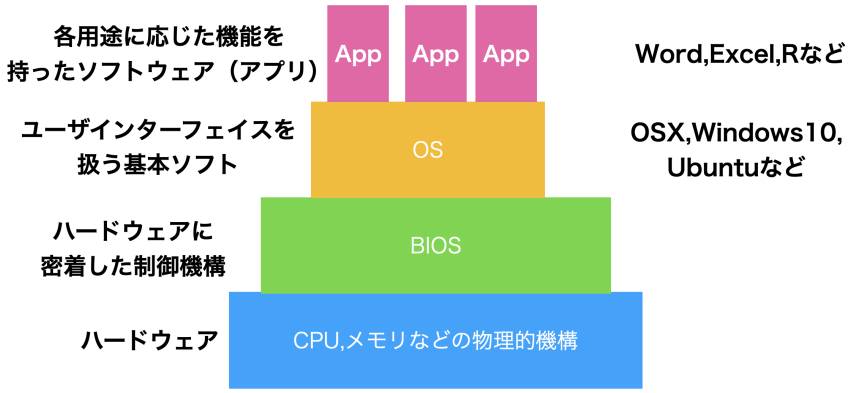


図 B.1 コンピュータのハードウェアとソフトウェアの関係

時代になりました。

ちなみにインターネットが広まったのもこの頃です。私は大学 3 年生の時，大学の授業で初めてインターネットを介して世界の情報を得る，という経験をしました[*8]。その頃から徐々に，大型 PC のアカウントではなくインターネットで使えるメールアドレスというのを個々人が持つようになりました。携帯電話も廉価な PHS が広まり，メッセージだけでなく徐々に音声，写真，短い動画が遅れるようになっていきます。当初は当然画質・音質も今とは比べものにならないのですが，それでもインターネットを経由してつながるという経験は想像を超えたものでした。

皆さんはすでに，携帯電話やインターネットがある時代に生まれた世代だと思います。こんな話はすでに過去のものであり，不便な頃の話を聞かされても困る，と思うかもしれません。私の世代は，幸いにもこのようにちょうどコンピュータが使われはじめ，広がり，高性能になっていくにつれて育ってきていますので，そのぶん機械の根本的な理解に直結しやすい社会環境にあったのです。みなさんは，便利な時代ではありますが，言い換えると「初心者が苦労しないように補助輪をつけておく」「何もしなくてもできているような感覚が得られるようなイリュージョンを見せておく」という状態におかれているので，補助輪が対応できない道に進んだり，多くの人とは違う使い方をするときに，急にサポートが外れどこで困っているかわからなくなる，ということになりかねません。非常に大雑把な Historical Review ではありますが，理解の一助になればと思います。

ところで，先ほどサポートという言葉を使いましたが，このサポートを商売にしているのが Microsoft や Apple という IT 企業です。これらの会社は，ユーザが使いやすいようにソフトウェアを提供してくれますが，有償ですし想定外の使用をするユーザに対してはサポートをしてくれません。逆にいうと，「決められた路線を走らなければ面倒を見ない」ということでもあり，これはユーザに不自由を強いているとも言えます。また，どのような仕組みで動いているのかを尋ねても，企業秘密と言って答えてくれません。コンピュータは誰でも自由に使えるべきものであり，勝手にユーザの情報を盗んだりしていないか，と言ったチェックをするためにもオープンであるべきではないか。そういう考え方に基づく，フリーソフトウェアというソフトウェアのあり方があります。Unix という OS を PC 用にした Linux がそうであり，オフィスソフトの LibreOffice，統計環境の R などもこの精神に賛同するものです。フリーソフトウェアは自由であり，無償です。お金と秘密を払ってサポートを受けるのではなく，ユーザが相互に助け合ってオープンで自由な世界を広げていこうという活動です。急に何の話なんだ，と思うかもしれませんが，心理学ひいては科学的活動すべてにおいて重要な問題であるこ

[*8] 教職関係の科目で，教育技術として今後使われるだろうということで担当教員が実演してくれました。隣の部屋から電話線を延長し，モデムというパソコンの信号を音情報に帰る機器を繋げて，NASA の Web ページを Netscape というブラウザでみたのが初めての体験でした。

とをご理解いただきたいと思います。

B.4 情報の単位

コンピュータを取り巻く世界の話はこれまでにして，ソフトの側面についての解説に入りましょう。

コンピュータは文字，音，絵，動画ファイルいずれについても，すべて 0 か 1 のデータとして管理します。0/1 の 1 つの単位を 1 bit(ビット) と言います。1 bit であれば Yes か No か，という二択の情報しか提供できませんが，これが 7 つあれば $2^7 = 128$ ですから，これで 128 種類の状態を表現できます。コンピュータは一般に，8bit で 1 つの単位として計算します。8bit のことを 1 byte(バイト) と言います。この 1 byte が 1024 集まったものを 1kb(キロバイト) といいます[*9]。1kb の次は，1024kb=1Mb(メガバイト) です。さらに 1024MB=1GB(ギガバイト) で，1024GB=1TB(テラバイト)，1024TB=1PB(ペタバイト)，1024PB=1EB(エクサバイト) と続きます。K,M,G,T,P,E といった名称は 1000 倍ごとに変わる大きさの桁をあらわしているのであって，「ギガが減る」というのは本来意味をなさない表現です[*10]。

このビット・バイトは情報に関する基本単位なのであちこちに使われます。記憶装置について使われるとき，一次記憶装置も二次記憶装置も同じ単位なので混同するかもしれませんが，2021 年現在では二次記憶装置の単位は GB から TB が使われます。USB フラッシュメモリーや外付け HDD などは数 TB の容量が一般的でしょう。これに対して，一次記憶装置は 4〜64GB ぐらいが相場かと思います。一次記憶装置は暗算の途中経過のように一時的に記憶する場所に過ぎないので十数 G でも問題ありませんが，二次記憶装置は結果の記録なので大きければ大きいほど余裕が持てますね。たとえば数 TB でもテレビドラマや映画を何本も記憶できるのですから，PB や EB なんて使うのかな，と侮ってはいけません[*11]。すぐにそれぐらいのサイズが必要な時代が来ることでしょう。

実際，それぞれ単位ではどれぐらいの情報が記録できるのでしょうか。1 byte は 128 文字表現できるので，英語のアルファベット 26 文字に加え，数字や簡単な記号であれば 1 byte で表現できます。たとえば A という文字は 01000001，B という文字は 01000010，小文字の a は 01100001，と言ったように 0/1 の文字列 8 個と一対一対応させて考えるのです。日本語は 1 byte では足りませんので，一文字あたり 2byte が割り当てられます。また 1KB(=1024byte) は 500 文字ですから，原稿用紙一枚ぐらいになります。1MB(=1024KB) は文字だけだと新聞一紙 (朝刊の 40 ページ分) ぐらいで，昔の記録媒体であるフロッピーディスク一枚に保存できるのがちょうど 1MB でした[*12]。ちなみに「カメラ映像 + 音声」のオンラインビデオ会議を 1 時間やると 200〜300MB ぐらいの容量をやりとりしていることになります。ビデオ会議では文字 (チャット) だけでなく，画像 (動画) や音声も送り合いますね。実は画像や音声も，0/1 に置き換えています。音声の場合，1 秒間を短い間隔にくぎります。この区切りのことをサンプリング周波数といい，たとえば 44.1 kHz という単位は 1 秒間に $44.1 \times 1000 = 44100$ 点のデータの採取をします。この 6 データ点において，音の振動幅を区切ります。ビットレートと言いますが，たとえば 16bit で区切る場合は $2^16 = 65,536$ 段階で区切り，その高さの音がある (1) かない (0) かで表すわけです。このように時間と音階を細かく区切り，その目に情報があ

[*9] キロメートルやキログラムのように，キロは 1000 の単位を表す言葉ですが，コンピュータは 2 進数なので 1000 ちょうどではなく 1024 で 1 つ上の位に上がることになります。

[*10] 同様に「USB を紛失する」というのもよく聞くおかしな表現です。USB は Universal Serial Bus の略で，データ転送規格のことを指します。なくすことができるのはそれにつなげるメモリースティックなどです。

[*11] 私事ですが，私が 1991 年に生まれて初めて手した PC，FM-Towns はハードディスクが 40MB，RAM は 2MB で，CD-ROM がついていましたが，それは 360MB の容量でした。購入するときに，「ハードディスクが 40MB もあって何を記録するんです? CD-ROM なんか情報が詰まり過ぎてますよ！」と店員さんに笑われたのを今でも覚えています。数年後，PC の動きが遅くなったので，2 万円で 2M の RAM を追加したのも良い思い出です。今でこそ，2MB なんて USB フラッシュメモリーでも売ってないほど微小なサイズですが。

[*12] 正確には 1.2MB 入る規格 (2DD) と 1.44MB 入る規格 (2HD) とがあります。

るかないかを積み重ねてデータとするわけです。テキストよりも圧倒的に情報が多くなるのが分かりますね。画像も同様に，図面を細かい単位 (ピクセルなど) で分けて，そこの色合いを色々な段階で区切ります。色は R(赤)G(緑)B(青) の組み合わせで表現でき，それぞれを 8bit$= 2^8 = 256$ 段階で表現したりします。一点一点にその情報がありますから，図の情報も非常に多くなるのがわかると思います。動画はその画像が時系列的に細かく分割されたものと思ってください。このように分解しますので，テキスト，音声，図，動画の順にデータサイズが大きくなります。通信機器が最初ポケベル＝数 byte の情報しか送れなかったものから，徐々に絵文字，ショートメッセージ (音声)，写真[*13]，動画が送れるように発展していきました。今では町中のあちこちで，誰もが手軽に動画をモバイル端末で見られるようになっています。あらためて，すごい進歩ですね[*14][*15]。

ところで，1 byte は 256 種の情報が記録できるので，英語のアルファベットや数字は 1 byte あれば十分だが，日本語や中国語など，英語以外の言語は文字種が多いので，2byte で一文字を表すという話をしました。この 2 バイト文字も，たとえば「あ」とか「亜」という文字に 111000111000000110000010 とか 111001001011101010011100 という文字列を割り当てるのですが，言語ごとによってどの数字をどの文字に割り当てるかという対応表が変わってきます。これを文字コードと言います。日本語はかつて Shift-JIS というコードで変換していましたが，今は世界のあらゆる言語に対応している共通規格である，UTF-8 という文字コードで変換することが一般的です。ところがなぜか，日本の Windows OS だけ Shift-JIS をいまだに使い続けており，他の PC とファイルをやり取りするときに文字コードの変換エラー問題が起きます。ファイルを開いて文字化けをしたりとか，プログラムが実行される際に「ファイルにアクセスできない」というエラーが生じたりするのは，この文字コードの問題が大きいのです[*16]。受身的な対策法になってしまいますが，PC でつかうユーザ名やファイル名などは半角英数文字を使い，短くした方がこうしたエラーに出くわしにくくなります。逆に，全角文字やスペース (空白) などを含んだファイル名，やたらと長いファイル名を使っていると，こうした問題に出くわしやすくなるということです。

B.5 ファイルの種類と拡張子

ここまで述べてきたように，計算機というのは基本的に物理的実体 (記録装置，記憶装置) の上で 0/1 のデータをやり取りしているだけです。記録 (記憶) 装置上に置かれている情報のセットは「ファイル」という形で記録されています。スマートフォンやタブレットは，ユーザの利便性のためにファイルの存在を意識しなくても良いようになってはいますが，バックエンドでは実行されるアプリケーションもファイルですし，開かれる音声や動画もファイルです。とくにパソコンでは，どの媒体，どのアプリで使うどういうファイルかを識別するために，拡張子 (かくちょうし) と呼ばれる識別記号をファイルの後ろにつけています。拡張子はファイル名の背後にピリオドで区切って追記されています。ついてないように見えても，OS がそれを表示させない設定にしてあるだけであることに注意してください。代表的な拡張子と，それに対応づけられているアプリケーションは次のようなものがあります。

[*13] できた当時は写真を撮ってメールができることをとくに「写メールする」と言い表したほどです。

[*14] 実は音でも画像でも，分割してそのままデータにしてしまうと膨大になりすぎるので，人間が気付きにくい周波数や色合いなどは削除して作ります。これを非可逆圧縮処理と言います。一度落としてしまった情報は戻らない，という意味です。ライブや生きている人間が処理している情報は，携帯の画面から得られるものの何億倍もの情報量なんですよ!

[*15] デジタル化のすごいところは，こうした文字，音，図版，動画といったものを bit という共通の単位に落とし込んだことです。こうすることですべて一元的 (bit という共通次元) で処理することができるようになったのです。メディアの違いが問題にならなくなり，0/1 の情報であれば複写も簡単にできてしまいます。情報化社会においては情報に特別性はなく，情報があるかないか，それを生み出せるかどうかこそが重要なのです。

[*16] Windows だけ世界標準から外れているので，早く修正して欲しいのですが，歴史的な経緯からユーザ数が多くて切り替えられないでいることと，こうした違いがあることをユーザに説明しない (素人は知らなくていい，と馬鹿にされているようなものです) ので，問題が解決される日はまだ先になりそうです。

.docx　マイクロソフト社の文書作成アプリケーション，Word で使うファイル

.xlsx　マイクロソフト社の表計算アプリケーション，Excel で使うファイル

.pdf　Adobe 社が開発した Portable Document Format 形式。OS が違っても同じレイアウトで文書を表示できるのが利点で，PDF 形式を読むことができるアプリケーションは多数。

.txt　シンプルな文字だけのテキストファイル。文字の飾りやレイアウトなどの情報がない最もプレーンな形式なので，OS が違っても文字コードさえ合っていれば読むことができる。

.mp3　音楽，音声のファイル。音声データには他の種類もあります。

.jpg　画像のファイル形式の一種。

.png　画像のファイル形式の一種。

.csv　comma/character separated variables ファイル。変数をカンマ (,)，あるいはタブ，半角スペースなど文字コードで区切ったファイルという意味で，中身は.txt と同じく装飾のない文字/数字だけであり，文字コードさえ間違えなければ OS を問わずに読み書きできる。データのやり取りはこの形式で行われることが多い。

.zip　圧縮ファイルの一種。1 つまたは複数のファイルをパックして圧縮してあるもの。ファイルの冗長な部分をうまくまとめてコンパクトにまとめ上げるため，ファイルサイズが小さくなるし，複数のファイルもひとまとめにできる。また圧縮の際にパスワードをかけることもできるため，メールなどに添付する場合はこの形式にまとめられることが多い[*17]。可逆圧縮であり，圧縮されたファイルは展開する (解凍するともいう) ことでパッキングを開封できる。zip ファイルの圧縮/展開は各種 OS が標準的に対応している。

`.txt` や `.csv` といった形式は，「装飾のない，文字だけの」ファイルです。こうした種類のことを ASCII ファイルと言います。メモ帳などのエディタと呼ばれるプログラムで読み書きできます。逆に `.docx` など特定の会社が提供するアプリケーションに対応しているファイルは，アプリの中でのさまざまな操作・装飾を暗号化して保存しており，メモ帳などで読んでも意味がわかりません。対応しないアプリでは開くこともできません。こうした形式は ASCII ファイルに対してバイナリファイルと言います。

OS は拡張子を見てファイルの種類を判別し，そのファイルを開くのに適したアプリケーションを自動的に起動し，開いてくれます[*18]。`.csv` ファイルは Excel などのアプリケーションで開くことも当然できますが，その際文字コードのエラーが生じたり，保存するときに文字コードを変えたりして，形式・内容が気づかずに変わっていることがあります。Windows も良かれと思ってやっていることなのですが，処理が徹底してないのか，かえって不便になってしまっています[*19]。

B.6　クラウドとは

すでに述べたように，計算機は基本的に 0/1 データのやりとりであり，それを保存してあるのがファイルとよばれるものです。ファイルは HDD や USB メモリ，SSD に保存することができます。

ところで，最近はこうした手元の物理的実体にファイルを置くことに加えて，クラウドに保存することも少な

[*17] **PPAP** というとピコ太郎の楽曲を思い出す人もいるかもしれませんが，圧縮ファイルの文脈では “「Password つき zip ファイルを送ります。Password は次のメールで送ります」 Angoka(暗号化) Protocol の略です。つまりメールでパスワード付きのファイルを送り，そのファイルを開くためのパスワードをまたメールで送るという，日本でよく見られるおかしな風習です。おかしな，というのは，メールがハッキングされていたらパスワードもどうせバレるわけで，同じメールに書いてあるのはもちろん馬鹿馬鹿しいですが，すぐ次のメールに書いてあるのも同じぐらいに馬鹿馬鹿しいことです。情報セキュリティ対策手法のつもりで行われる慣習が広まっていますが，PPAP の標語のもと，馬鹿馬鹿しいのでやめましょうという風潮になってきました。

[*18] 見たことのない拡張子の場合は，どのアプリケーションで開くべきか尋ねてくるでしょう。

[*19] 実際，この授業での課題データを UTF-8 形式の csv ファイルで提供しても，Excel で開いたばっかりに文字化けして分析できなくなる，という相談がこれまで多く寄せられています。根本的な解決策として，Windows を使うのをやめることをお勧めします。

くありません。クラウドとは雲という意味で，インターネットの向こう側のどこか，ということを意味します。ですが，基本的にはインターネットで繋いだその先にも電子計算機があるのです。たとえばパソコン A とパソコン B をケーブルで繋ぐと，パソコン A からパソコン B のファイルにアクセスできます[*20]。このケーブルをどんどん伸ばすと，遠く離れていてもこの操作ができます。このケーブル網を世界レベルに広げているのがインターネットです[*21]。

ここで覚えておいて欲しいのは，当たり前のようですが，ネットといっても基本的には電子計算機と電子計算機を繋げている実体がどこかにあって，ファイルのやりとりをしているだけだということです。ブラウザはウェブサイトの情報を書いたファイルを取り込んでホームページを見せてくれていますし[*22]，Youtube は動画のファイル，Instagram は画像のファイルへのアクセスをして見せてくれているのです。最近ではクラウドサービ

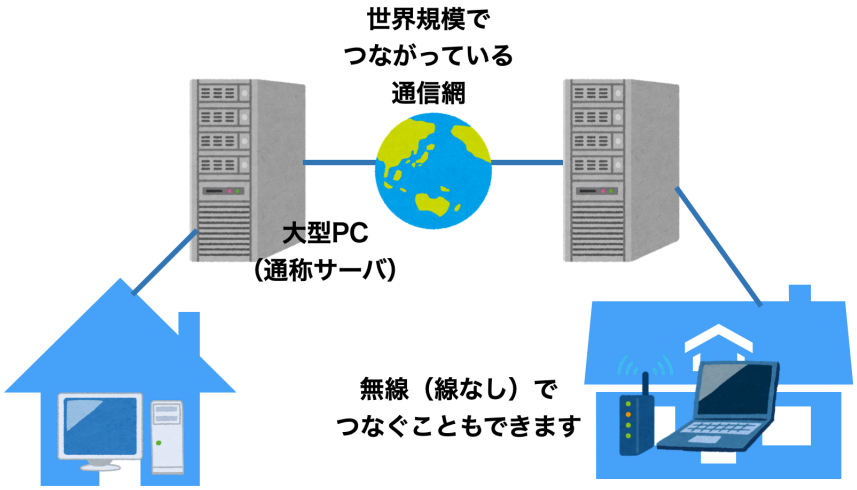


図 B.2 インターネットの世界

ス，というのがよくあります。中でも Dropbox とか OneDrive，iCloud などが有名です。これらは，ファイルをインターネットを介した別の大型電子計算機に保存 (アップロード) し，インターネットを介して読み込み (ダウンロード) して使う，というものです。手元の電子計算機の中に記録されているファイルのことを「ローカル」，サーバなど遠隔地にある大型機に記録されているファイルのことを「サーバ」「クラウド」と呼んで区別しますが，このローカルとサーバのファイルを常に同じものに同期しておく便利なシステムです。こうしたクラウドサービスを使うと，たとえば大学で作業して保存したファイルを，USB メモリに保存して自宅のパソコンにコピーする，という操作をしなくても，大学でクラウドサービス上のフォルダ内に保存しただけで，自宅のパソコンにそのファイルがコピーされているため[*23]，意識せずにそのまま作業を続けられるということです。自動的にバックアップをとってくれているとも言えるので便利ですね。

もちろん注意すべきこともあります。「他の人に見られたら困るファイル」，「アプリケーションの実行ファイ

[*20] もちろんファイルの情報をどのように信号に変えて送受信するか，アクセス権限はどうするかといった細々したことを調整しなければなりませんが，そのあたりの仕事をやってくれるのが OS のありがたいところです。

[*21] インターネットは軍事通信網として始まりましたが，それを電話回線や企業内通信網，大学間通信網などと接続しあって世界中に広げています。網と網が相互に (inter) 繋がっているので inter net であり，大学内・企業内のネットワークのことはイントラ (intra) ネットと言います。ちなみに一般用語としてのインターネットは Internet と大文字で書き始めます。

[*22] ホームページとは本来ブラウザが最初に開くページのことを指し，企業や個人が情報発信しているページのことはウェブサイトというのが適切です。

[*23] これはユーザが特段の指示をしなくても，アプリケーションがユーザの見えないところで (バックグラウンドで) アップロード，ダウンロードの作業を進めているからです。パソコンはシャットダウンしていなければ，裏でこうした作業を淡々とこなし続けてくれています。

ル」などは通信網の向こうではなく，手元 (ローカル) に置いておきましょう。個人情報・機密情報などを，クラウドドライブに保存すると，悪意を持った人が大型計算機に攻撃を仕掛けて情報を盗んでいく可能性があるからです。情報化の怖いところは，取られても気づかない (コピーすればいいだけで元ファイルになんの影響もない) ところにあります。加えて，取られたものをばら撒かれる＝インターネットを介して誰でもアクセスし保存できるようにされると，すべてを回収できなくなるのも問題です。失言が記録されて拡散されると大変なことになるのは，みなさんもこの時代に生きる人間のマナーとして色々見聞するところと思います。また，クラウドサービスで自動的に同期されるといっても，アプリケーションの実行ファイルなどはローカルに保存するべきです。アプリケーションは実行に際してさまざまな関連ファイルにアクセスしますので，1 つでも場所が違うところがあるとエラーになって動かなくなります。同じ OS でも，です。インターネットからとってきたソフトウェアをうっかり OneDrive に保存してしまうと，アクセスできないエラーで起動しない，ということもありますので注意してください[*24]。

余談ですが，R などプログラミングを推している時に最も頻発する質問が「ファイルがありません/読み込めません」です。その理由は大きく 2 つあって，1 つはスペルミスです。ファイル名の大文字・小文字の違いということもありますし，拡張子のまえに半角スペースが入っている，というよくみてみないと気づかないようなものもあります[*25]。2 つめの問題は，パスが通らないことです。デスクトップに置いたのに見つからない，ということもよくあります。教員や TA を悩ませるタイプの問題として，同じファイル名・フォルダ名が別の場所にあることがあります。フォルダ A の中にファイルを置いたのに見つかりません，という質問に対応していて，確かにフォルダ A を開いているのにな，と思ったら別の場所のフォルダ A だった，ということがあるのです。パスは隅々まで同じでなければ意味がないので，自分がどこに何を置いたのかをしっかり把握しておきましょう[*26]

B.7 ファイルの位置の指定

ここでファイルとそのパスについての話をしておきたいと思います。

計算機が情報を 0/1 で管理し，それらがファイルとなってどこかに保存されている，ということでした。私たちは Finder や Exploer などファイルブラウザをつかって，実行したいファイル，参照したいファイルを探していきますね。ファイルはまとめてフォルダの中に含まれていますし，フォルダの中にフォルダがあるといった，階層状態になっていることも少なくありません。ちなみに**フォルダ**と同じ意味で**ディレクトリ**という言葉が使われることもあります。

B.7.1 相対パスと絶対パス

普段 PC を使っているときは気にすることがありませんが，R や RStudio などプログラミング言語をつかっているときは，「今どこで作業しているか」という現在地が重要になってきます。たとえば，RStudio で

[*24] マイクロソフト社は，これまたユーザのためを思ってやっているのかもしれませんが，デフォルトで OneDrive に保存させようとします。それでうまくインストールできなかったという相談も多々寄せられています。抜本的な解決策として，Windows を使わないことをお勧めします。

[*25] これはたとえば，`sample.csv` というファイルをダウンロードしてね，と指示した時に，複数回同じ場所に保存してしまって，機械が，`sample (1).csv`，`sample (2).csv` などと名前をつける時に起こります。この `(1)` を削除してもその前に半角スペースが入ってたりするのです。まったく Windows は！

[*26] Windows は One-Drive にもデスクトップがあります。同じ環境をローカルと同時にクラウドにもあって常に同期していればいいのですが，必ずしもそうではないので (OneDrive の設定によります)，`C:\Users\Username\Desktop` と `C:\Users\Username\OneDrive\Desktop` のどちらに保存したのかが分かりにくくなっています。デスクトップに保存したのに，と言われてもどっちのデスクトップなのか，という問題が生じるわけです。さらに専修大学では 2023 年から仮想デスクトップ環境 (VDI) を使うようになりました。これで 3 つ目のデスクトップができたことになるわけです。なんてこった！まったく Windows は！

C:\User\kosugitti\Document\kiso1\というところでプロジェクトを開いているとします。スラッシュ (\) はフォルダ，コロン (:) はドライブを表す記号です。プロジェクトフォルダは，C ドライブの User フォルダの下にある，kosugitti フォルダの下にある，Document フォルダの下にある，kiso1 というフォルダということになります (プロジェクト名が kiso1 だとそうなります。)。この kiso1 フォルダが現在地です。

このフォルダの中で，Rmd ファイルや R スクリプトファイルを使って，他のファイルを参照するようなコードを書くとしましょう。たとえば script1.R というファイルに read_csv 関数を書いたとします。読み込みたいファイルは，同じフォルダの中にある，sample.csv とだとします。このとき，read_csv の書き方は次のようになるでしょう。

code : B.1 相対パスで読み込む

```
1 dat <- read_csv("sample.csv")
```

しかし別の書き方もあります。たとえば code:B.2 のような書き方でも問題ありません。

code : B.2 絶対パスで読み込む

```
1 dat <- read_csv("C:\User\kosugitti\Document\kiso1\sample.csv")
```

後者 code:B.2 の書き方は，ファイルの場所を全部書いてありますから，確実にその場所が特定できます。それに比べて前者 code:B.1 の書き方は，なぜファイルを書いただけでいいのでしょうか。これは，このコードを実行している現在地と同じフォルダの中に sample.csv ファイルがあるからです。プログラムは，命令を受けるとファイルを探しにいきますが，現在地と同じフォルダの中を探すことになっているのです。この現在地，すなわち現在作業しているフォルダのことを**作業フォルダ (working directory)** と言います。

では作業フォルダと別のフォルダの中にファイルがあれば，アクセスできないのでしょうか。そんなことはありません。code:B.2 の書き方を使えば，作業フォルダがどこにあっても位置を特定できますから，作業フォルダを問わずに書くことができます。ちょっと長くて面倒ですが，確実にある場所を指定しているからです。この書き方のことを**絶対パス**による指定といいます。一方，code:B.1 の書き方は，今の作業フォルダから見た場所，という相対的な書き方になっています。この書き方のことを**相対パス**による指定，といいます。相対パス指定で，違うフォルダにアクセスする場合には，次のようにします (code:B.3)。ここでは，Document フォルダの中に，kiso1,kiso2 フォルダがあり，kiso1 フォルダの中で作業している時に kiso2 フォルダの sample2.csv ファイルを読み込む例を書いています。

code : B.3 絶対パスで読み込む

```
1 # 絶対パス指定
2 dat <- read_csv("C:\User\kosugitti\Document\kiso2\sample2.csv")
3 # 相対パス指定
4 dat <- read_csv("..\kiso2\sample2.csv")
```

絶対パスはそのままなのですが，相対パスは..\という記号になっていますね。このピリオドを 2 つ打つ方法で，「ひとつ上のレベルの」という意味になります。このように，現在地からの相対的な位置関係で，ファイルを指定することもできます。

絶対パスと相対パスのどちらが良いのか，というのは一概には決められません。絶対パスは，PC が変わったりフォルダの構造が変わったりすると役に立ちませんから，使い勝手が悪いと言えなくもないですが，確実に指定できる方法です。相対パスは，PC が変わったりしても「現在地から相対的に見てどこか」という話ですから，たとえばこの例で kosugitti フォルダごと別の場所に移しても (たとえば D:\Univ\Classes\kosugitti\kiso1 のように)，コードはエラーなく動きます。kiso1 フォルダ，kiso2

フォルダの相対的な位置関係が変わらなければいいのですから。バックアップを取ったり，複数の環境で同期しながら作業する場合などは相対パスの方がいいでしょうね。

いずれにせよ，現在どこで作業しているかということ，すなわち作業フォルダの場所を，意外と意識しておかなければならないということには注意が必要です。ファイルをどこに置いたか，どんなファイルを置いたか，自分はどこにいるのか，これが変わってくると「ファイルが見つかりません」というエラーになるのです。言い方を変えると，**ファイルが見つかりませんエラーの原因は，この 3 つのどれかである**ことがほとんどです。

B.8 ファイルのバージョン管理

これからみなさんは大学生活の中で，たくさんのファイルを生み出していくことになるでしょう。たとえば 4 年生の時に卒業論文を書くことになりますが，データファイル，分析ファイル，図を書いたファイル，引用文献リストを書いたファイル，卒論本文などなど，1 つの研究でも複数のファイルが作られることはよくあります。さらに，これらのファイルは日々加工されますから，その度に上書き保存することになります。いわば，ファイルがバージョンアップしていくのです。

卒論などの場合はとくに，「途中で保存しておく」ことが重要です。途中まで書いていた時に，横に置いていたマグカップが倒れて PC にコーヒーがかかり，変な音を立てて PC が壊れてしまった，ということがあるかもしれませんからね。紙に書いていた時代は，その手のハプニングがあってもせいぜい原稿用紙数枚がダメになっただけで，「ちくしょう，やりなおしかぁ」で済んだのですが，電子データの場合は電子の藻屑になると復元させることができません。ですから**バックアップは非常に大事**なのです[*27]。

バックアップの基本は，「別の場所に」「別のファイル名で」というものです。同じ名前で上書きすると元に戻ることができませんから，面倒でもコツコツと違う名前をつけましょう。そうするとよくあるのが， `soturon1.docx`,`soturon2.docx`,`soturon3.docx`,`soturon3(修正).docx`,`soturon3(最新).docx`,`soturon3(最新)(修正).docx`,`soturon3(最新)(修正)(提出版).docx`,`soturon3(最新)(修正)(提出版2).docx`.... というようになっていくやつで，「どれが最新版だっけ...」と書いてる本人でも探すのに苦労することになります。

この問題の解決策として，`soturon1001.docx`，`soturon1005.docx` のように日付を入れるというものがあります。10 月 1 日分，10 月 5 日分，としていけば「いつまで戻れば良いか」もわかるのでいいやり方ですね。日付の数字をファイルの先頭につけておくと，並べ替えも簡単です。この日付をつけて保存するというのを習慣化し，1. 昨日までのファイルを開いて今日の日付で別名保存する，2. 作業を進めて，時々上書き保存，最後にも上書き保存，3.PC に USB メモリをさして，バックアップ保存して作業終了，というルーティンを作っておくと，確実に記録が残って良いでしょう。

ただし，この方法の問題は，ファイルサイズが大きくなりすぎることです。図表などを含めたファイルが数百 MB になることは少なくありません。それを次々複製していくわけですから，大容量の USB メモリでも限界が来るかもしれません。これは「丸ごとコピー」していることが原因で，たとえば昨日は 10 行目まで書いた，今日は 11 行目から 14 行目まで書いた，というのであれば，この増えた 4 行分 (差分) だけを追加保存すればいいのに･･･と思いませんか。

こうしたバックアップやバージョン管理をやってくれる仕組みとして，`Git` というものがあります。`Git` は作業フォルダの中身の差分だけを記録し，必要であれば過去のバージョンに戻すこともできるシステムです。毎回上書き保存 (`commit` する，といいます) のたびに「どこを変更したか」というメモを付けて保存しておけば，

[*27] ちなみに私の経験上，レポートが電子の藻屑になったので助けてください！と言われることがよくあり，4 年間の学生生活の間では平均 10–15 名に 1 人の割合で発生することのようです。

そのメモを見ながら「ここの時点まで戻ろう」という使い方をすることができます。ファイル名は変更する必要なく，同じファイル名で進めていけますから，同じようなファイルがたくさんあって訳がわからなくなるということもありません。さらに保存先をクラウドにした GitHub というものもあり，これを使うとクラウド上に追記していくことができます。この GitHub は IT 企業などでプログラムをチームで進めていく時にも使われている技術で，全体のプログラムに個別の機能を複数人が追加，管理者が必要なものだけ取り入れる，というように使われています。国里ゼミや小杉ゼミでは，卒論を GitHub で管理し，学生が書いた分を commit し，それを hub 上にアップロード (push といいます) する，というようにします。教員の方は学生の進捗が管理できますし，どこがどう変わったかが分かりやすく，バージョン管理と同時にバックアップもできるという便利な仕組みです。GitHub は無料でアカウントを作ることができますから，興味があれば皆さんもチャレンジしてみてください[*28]。

さてここで，ひとつ注意をしておきます。卒論の原稿やプログラムは日々変化するものですからバージョン管理が必要ですが，データファイルはアップデートする必要がありません。いや，アップデートしてはおかしいのです。たとえば 100 人分のデータを取って分析をしていて，後で「やっぱりこのデータを削ろう」というのは，研究不正が疑われかねません。自分に都合の良いデータだけで議論し，都合の悪いデータは削除して統計的に有意な結果が出るように細工しよう，なんてことがあれば困るのです。データファイルはバージョンアップせず，それを加工，計算するプログラムがバージョンアップしていく。そしてその加工プロセスは誰でも見ることができるように公開されている。少なくとも，科学的な営みをする上では，そうしたやり方が必要なのです。自分だけのデータで自分だけの分析方法で，良い結果だけ示すというのは適切な方法ではありません。

Open Science Framework(https://osf.io/) はこうした「オープンな科学」にむけた取り組みの一種です。このアカウントは誰でも無料で作ることができ，ここにファイルをアップロードしたり，分析計画を事前に記録しておくことができます。何も難しいことではなくて，クラウド上のファイル置き場だ，というぐらいに思っていただければ結構です。ここにおかれたファイルは自動的にバージョン管理され，同じファイル名のものがアップデートされてもその記録 (ログ) が残ります。最近はこの OSF をつかって，論文化されたデータやプログラムを公開するという取り組みも進んでいます。

クラウド，バックアップ，オープンサイエンスといった新しい研究方法は日々生まれてきています。皆さんも便利な機能はどんどんキャッチアップしていきましょう！

B.9 おわりに

古臭い話をしてしまうのは，私が歳をとった証拠でしょうか。皆さんはこんな話を知らなくても，スマホやタブレットを使いこなしていることと思います。細かいことを知らなくても，ユーザとして利用するだけなら知らなくて良い話なのかもしれません。私はここにも書いたように，高校生の頃からコンピュータの発展と一緒に大人になってきましたから，学ぶともなく学んできたところがあります。皆さんは生まれた頃からコンピュータやがあったネイティブ・デジタル・カウボーイですから，苦労なんかする必要なかったわけです。

しかし細かい仕組みを知らないということは，問題が生じた時に「何か・誰かが，どこかでどうにかなって，今私が困っている」という状況になります。問題を特定できないと，解決することもできません。コンピュータは文房具に過ぎませんから，それを使いこなせないほうが格好悪いのです。しかも今後ますますコンピュータに囲まれた世界になっていくのは自明ですから，ここに学習コストをかけない方が勿体無い。幸い，わからないことに対して，自ら調べて学んだ利する時間と環境が用意されているのが大学という世界なのですから，今

[*28] このテキストやシラバスも GitHub で管理していますし，公開されているサイトも GitHub 上のものです。これからは教科書も日々成長していくものになるかもしれません。

のうちにしっかり基礎固めをしておきましょう。

ここのくだらない懐古的エッセイのような文章が, 何かの足しになれば幸いです。

付録 C

ギリシア文字一覧

ギリシア文字ってかっこいいけど，読み方わからない･･･という人のための一覧。ついでに TEX 表記も。

表 C.1　ギリシア文字とその読み方

読み方	大文字	小文字	英語表記	TEX 表記
アルファ	A	α	alpha	`\alpha`
ベータ	B	β	beta	`\beta`
ガンマ	Γ	γ	gamma	`\gamma`
デルタ	Δ	δ	delta	`\delta`
エプシロン	E	ε	epsilon	`\varepsilon`
ゼータ	Z	ζ	zeta	`\zeta`
イータ	H	η	eta	`\eta`
シータ	Θ	θ	theta	`\theta`
イオタ	I	ι	iota	`\iota`
カッパ	K	κ	kappa	`\kappa`
ラムダ	Λ	λ	lambda	`\lambda`
ミュー	M	μ	mu	`\mu`
ニュー	N	ν	nu	`\nu`
クサイ	Ξ	ξ	xi	`\xi`
オミクロン	O	o	omicron	`\mathrm{o}`
パイ	Π	π	pi	`\pi`
ロー	R	ρ	rho	`\rho`
シグマ	Σ	σ	sigma	`\sigma`
タウ	T	τ	tau	`\tau`
ウプシロン	U	υ	upsilon	`\upsilon`
ファイ	Φ	ϕ	phi	`\phi`
カイ	X	χ	chi	`\chi`
プサイ	Ψ	ψ	psi	`\psi`
オメガ	Ω	ω	omega	`\omega`

付録 D

記号の入力とキーボードの場所

プログラミングのミスでよくあるのが打ち間違い，スペルミスです。X と x，S と s など大文字と小文字で形が同じものや，l(エルの小文字) と 1(数字のイチ) の違いなどは，プログラミング用フォントにして違いがわかるようにするとか，文字列の意味から類推する (Normal とあればノーマルであって，ノーマ・イチではないと察する) など工夫が必要かもしれません。

また，理由はよくわからないのですが頻発するスペルミスは，データ (data) をデート (date) と書いてしまうことです。データはラテン語の datum(与えられたもの) の複数形なのですが，最近のデータサイエンスの文脈では data も単数形と考えるようです。ともかく，日付を表す date とは由来も意味もスペルも全て違うので，気をつけましょう。

さて，これらはまだ序の口。プログラミングのコードを読んでも，日本語の五十音に入っていない記号の違いがわからない，どこでそれが入力できるかわからない，質問しようにも読み方がわからないといったものもあります。ここではこれらをまとめて解説します。記号の上では微妙な違いですが，当然形が違うのでプログラミング上は違う文字として扱われますので，形の細部までよくみてください。なお，プログラミングでつ変わる時は言語に依存することもありますので，ご注意ください[*1]。

特に目立つのはハイフンとチルダ，アンダースコアの入力ミスです。それぞれ文字を書く領域における位置が違ったり，形が違ったりするのでよくみてください。

ハイフンは真ん中	A-B
アンダースコアは下	A_B
オーバーバーは上	A¯B
チルダはニョロ	A~B

これを踏まえて，そのほかの記号の名称や意味を表 D.1 で確認しましょう。

一般的な日本語キー配列の場合，1 つのキーに 4 つの文字・記号が割り当てられていますが，英語入力モードの場合はキーの左側，日本語入力モードはキーの右側を見ることになります。キーを押すと下の段の文字が入力されますが，シフトキーを押しながらキーを押すことで上の段の文字が入力されることになります (図 D.1)。

これを踏まえて，日本語キーについては D.2，US キーについては図 D.3 に代表的な記号とキー配列の位置を示しました。入力に困った場合は一度図を見て確認してください[*2]。

[*1] たとえばコメントアウトは C 言語では\\，R では#，TeX では % など，それぞれ異なります。

[*2] US キー配列はキートップに一種類の文字しかなく，美しい配置なのでおすすめです。

表 D.1　記号と読み方

記号	読み方	解説
:	コロン	英文中では「すなわち」などの意味。セミコロンと間違えないように
;	セミコロン	英文中では文章の区切り, 接続詞のようにつかう
.	ピリオド	英文の終わりを意味する。日本語で言う句点
,	カンマ	英文の区切りを意味する。日本語で言う読点
@	アットマーク	メールアドレスに用いられることで有名
$	ドルマーク	米国の通貨単位。R では変数名指定のときにもちいる
/	スラッシュ	割り算の記号
*	アスタリスク	掛け算の記号
+	プラス	足し算の記号
-	マイナス	引き算の記号
^	ハット	累乗の計算の記号
=	イコール	等号。プログラミングでは==で一致しているかどうかの判定にも
!	エクスクラメーション	強調。プログラミングでは否定 (NOT) の意味になることも
_	アンダースコア, アンダーバー	位置に注意。ハイフンではなく文字領域の下の線 変数名をつなげる時 (ex. `snake_case`) に使ったりする
~	チルダ	R では独立変数と従属変数をつなぐときに使う ハイフンやオーバーライン, アンダースコアと間違えられる率高め
‾	オーバーライン, オーバーバー	アンダースコアの逆。滅多に使わないが。文字化けを直したときにみられる。
%	パーセント	プログラミングではコメントアウトの時などに使われたりする
&	アンパサンド	プログラミングにおける AND(論理積) の記号など
|	縦棒	プログラミングにおける OR(論理和), 条件付き確率の記号にも
\	バックスラッシュ	プログラミングではコメントアウトの時などに使われたりする
"	ダブルクォーテーション	文字列の開始･終了を表す。同じ記号で閉じる
'	シングルクォーテーション	文字列の開始･終了を表す。同じ記号で閉じる
`	バッククォーテーション	文字列の開始･終了を表す。同じ記号で閉じる
[]	大括弧	プログラミングでは配列を意味することがある
{}	中括弧	プログラミングではブロックの開始と終了を意味することがある
()	小括弧	数式のまとまりを表す

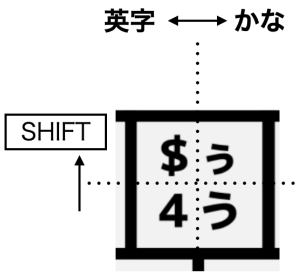


図 D.1　日本語キーで入力する場合

図 D.2　代表的な記号と日本語キー配列

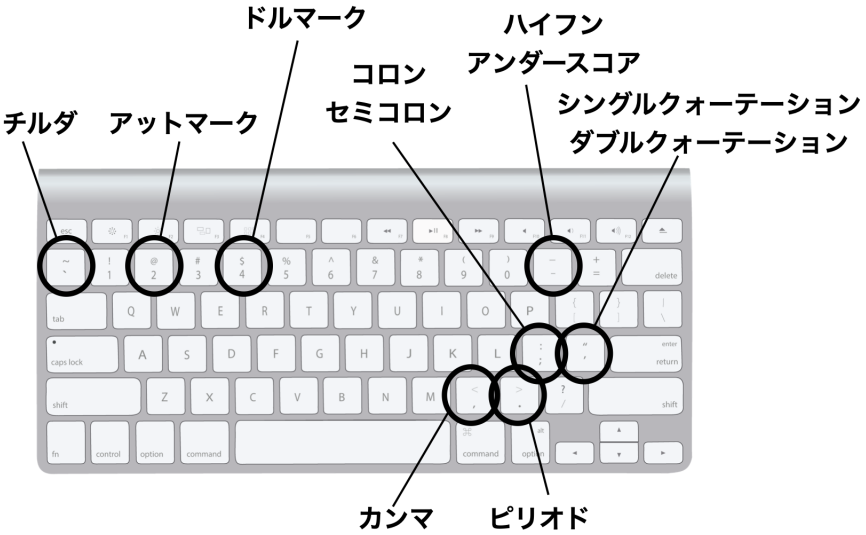


図 D.3　代表的な記号と US キー配列

引用文献

Amrhein, V., Greenland, S., & McShane, B. (2019). Scientists rise up against statistical significance. Nature, 567, 305–307. https://doi.org/10.1038/d41586-019-00857-9

Gelman, A. (2006). Prior distributions for variance parameters in hierarchical models (comment on article by Browne and Draper). Bayesian analysis, 1 (3), 515–534.

Gronau, Q. F., Singmann, H., & Wagenmakers, E.-J. (2020). bridgesampling: An R Package for Estimating Normalizing Constants. Journal of Statistical Software, 92 (10), 1–29. https://doi.org/10.18637/jss.v092.i10

南風原 朝和・芝 祐順 (1987). 相関係数および平均値差の解釈のための確率的な指標 教育心理学研究, 35 (3), 259–265. https://doi.org/10.5926/jjep1953.35.3_259

浜田 宏・石田 淳・清水 裕士 (2019). 社会科学のためのベイズ統計モデリング 朝倉書店

清水 裕士(2018). 阪神ファン – 巨人ファンの 2 大精力構造は本当か 豊田 秀樹(編) 北大路書房

池田 功毅・平石 界 (2016). 心理学における再現可能性危機:問題の構造と解決策 心理学評論, 59 (1), 3–14. https://doi.org/10.24602/sjpr.59.1_3

アイザクソン W. (2011). スティーブ・ジョブズ 講談社(Original work published 1995, JSTOR)

JASP Team (2021). JASP (Version 0.16)[Computer software].

小杉 考司 (2019). R でらくらく心理統計 : RStudio 徹底活用 講談社

小杉 考司・紀ノ定 保礼・清水 裕士 (2023). 数値シミュレーションで読み解く統計のしくみ～R でためしてわかる心理統計 技術評論社

クルシュケ J.K. (2017). ベイズ統計モデリング: R, JAGS, Stan によるチュートリアル 原著第 2 版 共立出版 (Original work published 2014, Elsevier)

久保 拓弥 (2012). データ解析のための統計モデリング入門——一般化線形モデル・階層ベイズモデル・MCMC—— 岩波書店

リー M.D ・ワゲンメーカーズ E-J. (2017). ベイズ統計で実践モデリング: 認知モデルのトレーニング 北大路書房(Original work published 2013, Cambridge University Press)

松浦 健太郎 (2016). Stan と R でベイズ統計モデリング 共立出版

ノーレットランダーシュ T. (2002). ユーザーイリュージョン——意識という幻想—— 紀伊國屋書店 (Original work published 1999, Penguin)

清水 裕士 (2016). フリーの統計分析ソフト HAD : 機能の紹介と統計学習・教育, 研究実践における利用方法の提案 メディア・情報・コミュニケーション研究, 1, 59–73.

清水 裕士・荘島 宏二郎 (2017). 社会心理学のための統計学——[心理学のための統計学 3]: 心理尺度の構成と分析—— 誠信書房

清水 裕士 (2021). 心理学統計法 放送大学教育振興会

豊田 秀樹 (2017). 実践 ベイズモデリング——解析技法と認知モデル-—— 朝倉書店

豊田 秀樹 (2018). たのしいベイズモデリング——事例で拓く研究のフロンティア—— 北大路書房

豊田 秀樹 (2019). たのしいベイズモデリング 2——事例で拓く研究のフロンティア—— 北大路書房

豊田 秀樹 (2020). 瀕死の統計学を救え！ 朝倉書店

Vehtari, A., Gelman, A., & Gabry, J. (2017). Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. Statistics and Computing, 27, 1413–1432. https://doi.org/10.1007/s11222-016-9696-4

山田 剛史・村井 潤一郎 (2004). よくわかる心理統計 ミネルヴァ書房

シ (2016). 計算機言語のまとめノート 暗黒通信団

索引

記号／数字

1 パラメータ・ロジスティックモデル 181
2 階差分のトレンド 218
2 パラメータ・ロジスティックモデル 181

B

BUGS 42

C

Cohen の d 77
CRAN 235

E

EAP 73, 81, 82, 210
EAP 推定値 136, 159, 185

I

int 型 60

J

JAGS 42

K

K-平均法 165

M

MAP 82, 210
MAP 推定値 53, 140
matrix 型 60
MCMC 32, 141
MED 210
MED 推定値 136

P

p 値 70

R

real 型 60
Rhat 87
Rhat 64

T

t 検定 69, 87, 95, 145, 225

V

vector 型 60

W

Welch の補正 73

あ

当て推量母数 182
閾上率 85
位置母数 132, 148
一様分布 32
一般化線形混合モデル 141, 147
一般化線形モデル 141, 147
一般線形モデル 131, 141, 147
インタプリタ 43
インタプリタ 15
隠匿情報検査 197
ウォード法 165
ウォームアップ 65
エディタ 43
エビデンス 31, 228
オッズ 143, 147
オッズ比 144
オブジェクト 17

か

回帰分析 31, 131

階層性 153
階層線形モデル 117, 155
階層的クラスタリング 164
カウント変数 148
確信区間 66, 82
確率質量 168
確率質量関数 121
確率的プログラミング言語 34, 41, 42, 59, 87, 225
確率分布 32, 33, 37, 58, 60, 61, 73, 82, 121, 168
確率密度 168
確率モデル 30
型 60
片側検定 78
カッパ係数 127
カテゴリカル分布 166
可読性 12
カルバック・ライブラー情報量 229
間隔尺度水準 121
観測度数 123
機械学習 42, 164
棄却 70
危険率 70
期待値 36
期待度数 123
帰無仮説 70, 97
帰無仮説検定 72, 93, 224
帰無モデル 97
逆リンク関数 147, 182
客観性 29
級内相関 156
教師なし学習 164
虚偽検出 196
距離 164
区間推定 66
クラスタリング 164
クロス集計表 121
群間計画 93, 107
群内計画 107
経験サンプリング 205
効果量 77, 88, 227
高級言語 42
交互作用 147
構造方程式モデリング 123
固定効果 117, 155
混合分布モデル 164
困難度 181
コンパイラ 43
コンパイラ 15

さ

最高密度区間 55
最小二乗法 31, 141
採択 70
最尤推定法 168
最尤法 29, 31, 135, 141, 183
作業フォルダ 252
シード値 34
識別力 182
事後確率最大値 66
自己相関 205
事後分布 31, 83, 95
事後予測分布 82, 83, 84
事前・事後デザイン 107
事前分布 31, 58
実験計画 29, 93
実質的に等価な範囲 75, 227
弱情報事前分布 58
尺度母数 132
周辺化 33
周辺化消去 167, 228
周辺尤度 31, 32, 228
縮小 157
順序尺度水準 121
状態空間モデル 206
情報量規準 229
信頼区間 66
スクリプト 43
スムージング 220
正規化定数 31, 32
正規混合モデル 164
正規表現 188
正規分布 34, 163

生成量 76, 113, 227
整然データ 101, 114, 186
正方対称行列 108
絶対パス 252
ゼロ過剰ポアソン 174
線形モデル 224
宣言 60
相関係数 164
相対パス 252
ソフトクラスタリング 165

た

ターゲット記法 168
対応のある t 検定 107
対角 108
対数尤度 63, 168
タイプ 1 エラー 94, 226
タイプ 2 エラー 91
対立仮説 70
多次元正規分布 107
多重比較 94
多変量正規分布 107
チャンク 232
通過率 181
データ生成モデリング 30, 57, 69, 77
停止規則 227
適合度 123
デザイン行列 131
点推定 136
同時確率空間 63
等裾区間 55
独立性の検定 123
トレースプロット 64, 87

な

二項分布 35, 122, 145
二次元正規分布 107
日誌法 205
ニット 234
ネイピア数 148

は

ハードクラスタリング 165
ハイパーパラメータ 157
配列 61
パラメータ 82
パラメータリカバリ 87
半コーシー分布 58
反復測定 107, 154
非階層的クラスタリング 164
標本統計量 70
ブリッジ・サンプリング 229
プログラミング言語 42
分割表 121
分散分析 93, 124, 225
ベイジアンモデリング 223
ベイズ推定 93, 141
ベイズ推定法 168
ベイズファクター 229
ベイズ法 29
ベルヌーイ分布 35, 121, 137, 147
変化点検出 196
変量効果 117, 155
ポアソン分布 145, 148
補間 211
ポリグラフ検査 196
ホワイトノイズモデル 206

ま

マルコフ連鎖モンテカルロ法 32
名義尺度水準 121, 131, 137
モーメント法 29, 66, 78, 85
モデリング 157, 181, 206, 223, 224, 225, 230
モデル比較 227

や

ユークリッド距離 164
有意差 225
有意水準 70
優越率 85
有効サンプルサイズ 65, 87
尤度 31, 168
要因計画 93

ら

ライフログ 205

ラベルスイッチング 171
ランダム化比較実験 97
離散確率分布 137
リンク関数 141, 147, 182
類似度 164
例数設計 88, 91
連言命題 125
ロジスティック回帰 140, 182
ロジスティック関数 142, 147, 181
ロジット関数 143
論理演算 21

漢字

中央値絶対偏差 51